

第一章 資料分析	4
1-1 簡介	4
各工具比較.....	4
依據用途區分常用套件.....	4
資料分析的流程.....	5
1-2 平台安裝	7
Python 與虛擬環境.....	7
第二章 機器學習與資料分析	21
2-1 Pandas 模組	21
2-2 Series.....	23
2-3 DataFrame.....	32
2-4 Matplotlib	45
長條圖.....	50
箱型圖.....	51
散點圖.....	53
圓餅圖.....	54
2-5 DataFrame 合併	58
檔名與欄位.....	58
說明每個檔案的用途：	58
第一部分：一個資料表分析.....	60
第二部分：多個資料表分析.....	62
第三部分：KMeans + PCA	68
3.1 機器學習流程（非監督式學習（Unsupervised Learning））	68
資料前處理.....	68
特徵選擇.....	69
PCA 降維	71
K-Means 分群→尋找最佳 K 值→分群結果分析→ PCA 2D 視 覺化	73
3.2 主成分分析(Principal Component Analysis，PCA)	76
目的.....	76
步驟如下.....	76
圖形說明.....	76
細部說明如下：	78
計算協方差矩陣/共變異數矩陣(Covariance Matrix):	82
以 5 位同學的身高與體重為例.....	95

PCA 實作	104
3.3 K-Means(K 平均值分群法).....	107
介紹.....	107
K-Means 實作	113
K-Means 手肘法 (Elbow Method).....	116
輪廓係數(Silhouette Coefficient)說明	118
3.4 PCA+Kmean 實際範例 1(線上購物資料分析)	121
第 1 階段：資料載入與合併.....	122
第 2 階段：合併資料.....	123
第 3 階段：統計每位顧客購買各部門的數量.....	123
第 4 階段：資料標準化.....	124
第 5 階段：PCA 降成 2 維.....	125
第 6 階段：畫 PCA 散佈圖	125
第 7 階段：K-Means 分群	126
第 8 階段：畫出 K-Means 分群結果	127
第 9 階段：分析每一群到底是什麼顧客.....	128
第 10 題：分析「購買比例」	130
第 11 題：使用 Elbow Method 與 Silhouette Coefficient 決定最佳分群數.....	134
第四章 機器學習分類器	136
4.1 機器學習演算法分類	136
監督式學習 (Supervised Learning)	136
非監督式學習 (Unsupervised Learning)	137
線性預測 (Linear Prediction)	138
線性模型特色.....	139
建議的教學順序 (適合初學者)	140
4.2 Linear Regression :	141
說明.....	141
房價預測 1 (California Housing)	146
房價預測 2 (California Housing)	149
房價預測(已訓練好模型+Django Web)	161
4.2 KNN (K-Nearest Neighbors, K 最近鄰演算)	165
說明.....	165
利用身高來判斷男生或女生，使用 knn 演算法.....	166
利用 knn 的手寫數字辨識系統(使用學者的資料庫與方法).170	
利用 knn 的手寫數字辨識系統(匯出資料庫成圖片、匯出模型、測試圖片)	171

利用 knn 的手寫數字辨識系統修改 1(重新產生與訓練資料庫)	181
利用 knn 的手寫數字辨識系統修改 2	189
利用 knn 的手寫數字辨識系統 by Django Web.....	200
4.3 CNN(Convolutional Neural Network，卷積神經網路).....	211
安裝套件.....	213
CNN 說明	213
基本知識(捲積、邊緣偵測 Prewitt、Sobel)	214
利用 CNN 判斷圖片裡是直線還是橫線	225
利用 CNN 手寫數字辨識	228
利用 CNN 人臉辨識系統	246
Fine-tuning（微調）	251
第 x 章 股票分析.....	253

第一章 資料分析

1-1 簡介

➤ 各工具比較

- Google Colab：最方便入門，不需安裝，適合新手、學生、快速測試 ML。
- Jupyter Notebook：適合研究人員，掌握本地環境，靈活但需安裝配置。
- VSCode：適合開發者，功能完整，能整合版本控制、專案管理。



一、整體快速比較表

項目	Google Colab	Jupyter Notebook	VS Code
類型	雲端 Notebook	本機 Notebook	本機整合式開發環境
是否需安裝	✗ 不需	✓ 需要	✓ 需要
是否免費	✓ (有使用限制)	✓	✓
是否需網路	✓ 必須	✗ 可離線	✗ 可離線
支援語言	以 Python 為主	多語言	幾乎所有
適合對象	初學者 / 教學 / AI	資料分析 / 教學	專業開發者
硬體資源	Google 提供 CPU/GPU	本機	本機

➤ 依據用途區分常用套件



五大分類整理的 Python 常用套件表格，每個分類列出了三個常用套件：

分類	套件名稱	主要用途說明
資料分析與處理	NumPy	高效能數值運算、陣列與矩陣處理
	Pandas	表格資料 (DataFrame) 處理與分析
	Matplotlib	資料視覺化、繪製各類圖表
Web 開發	Django	全功能 Web Framework (後端)
	Flask	輕量級 Web Framework
	FastAPI	高效能 API 開發 (支援非同步)
機器學習 / AI	scikit-learn	傳統機器學習演算法
	TensorFlow	深度學習框架 (Google)
	PyTorch	深度學習框架 (研究與實務常用)
網路爬蟲 / 資料擷取	Requests	發送 HTTP 請求
	BeautifulSoup	解析 HTML/XML
	Selenium	模擬瀏覽器操作、動態網頁爬蟲
自動化 / 系統工具	os	作業系統相關操作
	shutil	檔案與資料夾管理
	schedule	排程與自動化任務

➤ 資料分析的流程

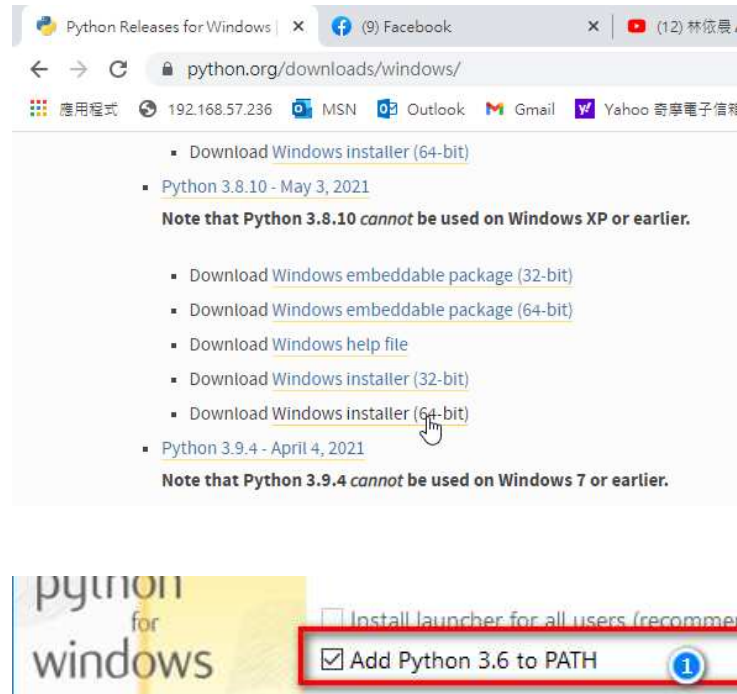
1. 定義問題 (Problem Definition)

- 明確分析目標與商業問題

- 定義關鍵指標（KPI）與成功標準
- 確認利害關係人的需求
- 2. 資料收集（Data Collection）
 - 從資料庫、API、檔案、爬蟲等來源取得資料
 - 確認資料的完整性與可用性
 - 記錄資料來源與時間範圍
- 3. 資料清理（Data Cleaning）
 - 處理缺失值（missing values）
 - 移除重複資料
 - 修正錯誤或異常值（outliers）
 - 統一格式（日期、單位、編碼）
- 4. 資料探索（Exploratory Data Analysis, EDA）
 - 計算敘述統計（平均、中位數、標準差等）
 - 視覺化資料分布（直方圖、盒鬚圖、散佈圖）
 - 找出變數間的關聯性
- 5. 資料轉換與特徵工程（Data Transformation & Feature Engineering）
 - 資料標準化 / 正規化
 - 建立衍生變數
 - 類別變數編碼（One-Hot、Label Encoding）
- 6. 建模與分析（Modeling & Analysis）
 - 選擇適合的方法（統計檢定、迴歸、分類、分群、預測等）
 - 訓練模型並調整參數
 - 驗證模型效能（交叉驗證、指標評估）
- 7. 結果解讀（Interpretation）
 - 從模型與分析中提取洞察（insights）
 - 對照原始商業問題驗證結論
 - 評估結果的可靠性與限制
- 8. 溝通與報告（Communication & Reporting）
 - 使用視覺化圖表（Dashboard、簡報）
 - 撰寫報告說明方法、發現與建議
 - 針對不同受眾調整敘述方式

1-2 平台安裝

➤ Python 與虛擬環境



```
# python --version
# python -V
C:\Users\tony>python --version
Python 3.9.7

# pip list (查看已安裝的套件)
C:\Users\tony>pip list
Package      Version
-----
pip          21.2.3
setuptools   57.4.0

# pip show virtualenv

安裝虛擬環境
# pip install virtualenv
```

```
C:\Users\tony>pip list
Package      Version
-----
distlib      0.3.6
filelock     3.9.0
pip          21.2.3
platformdirs 3.0.0
setuptools   57.4.0
virtualenv   20.19.0
```

virtualenv 是一種用來建立「虛擬環境」的工具，可在該環境中獨立安裝專案所需的套件與函式庫。由於虛擬環境與原有作業系統相互隔離，因此即使安裝不同版本的套件，也不會影響系統本身或其他專案。當專案不再使用時，只需刪除該虛擬環境即可，乾淨又安全。

此外，虛擬環境也非常有利於專案移轉至第三方平台或其他電腦，只要依照套件清單重新建立環境，便能快速重現相同的開發環境，大幅提升專案的可移植性與維護性。

以下整理常用的虛擬環境工具，以表格方式呈現，方便教學與比較：

工具名稱	適用語言 / 平台	主要用途	優點	常見使用情境
virtualenv	Python	建立獨立的 Python 虛擬環境	輕量、速度快、跨平台	一般 Python 專案
venv	Python (內建)	Python 內建虛擬環境模組	無需額外安裝、官方支援	小型或教學用專案
conda / Anaconda	Python、R	套件與環境管理	可管理非 Python 套件、適合資料科學	AI、資料分析
pipenv	Python	套件 + 環境管理	自動管理依賴、支援 Pipfile	中小型專案
poetry	Python	專案與依賴管理	版本鎖定清楚、現代化工具	正式專案、團隊開發
Docker	跨語言	容器化應用環境	環境完全一致、可部署	上線部署、微服務
Node.js nvm	JavaScript	Node.js 版本管理	可切換多版本 Node.js	前端 / Node.js 專案
Java Virtual Machine (JVM)	Java	Java 執行環境	跨平台執行	Java 應用程式

```
# pip uninstall virtualenv(刪除虛擬環境套件)
```

```
# cd c:\
```

```
# virtualenv dvds (建立一個虛擬環境 dvds)
```

```
c:\>virtualenv dvds
created virtual environment CPython3.8.10.final.0-64 in 1242ms
creator CPython3Windows(dest=C:\dvds, clear=False, no_vcs_ignore=False, global=False)
seeder FromAppData(download=False, pip=bundle, setuptools=bundle, wheel=bundle, via=copy, app_data_dir=C:\Users\tony\AppData\Local\pip\virtualenv)
added seed packages: pip==21.1.2, setuptools==57.0.0, wheel==0.36.2
activators BashActivator,BatchActivator,FishActivator,PowerShellActivator,PythonActivator,XonshActivator
```

進入虛擬環境(DOS):

```
# cd dvds
```

```
# Scripts\activate
```

```
c:\>cd dvds
```

```
c:\dvds>Scripts\activate
```

```
(dvds) c:\dvds>_
```

進入虛擬環境(PowerShell):

以系統管理身分執行(PowerShell or VSCode)



```
# set-executionpolicy remotesigned
```

```
# get-executionpolicy -list
```

說明:PowerShell 更改執行原則

Reference:

https://learn.microsoft.com/zh-tw/powershell/module/microsoft.powershell.core/about/about_execution_policies?view=powershell-7.4

```
系統管理員: Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. 著作權所有，並保留一切權利。
請嘗試新的跨平台 PowerShell https://aka.ms/pscore6
PS C:\Windows\system32> set-executionpolicy remotesigned
執行原則變更
執行原則有助於防範您不信任的指令碼。如果變更執行原則，可能會使您接觸到 about Execution Policies 說明主題 (網址為 https://go.microsoft.com/fwlink/?LinkID=135170) 中所述的安全性風險。您要變更執行原則嗎?
[Y] 是(Y) [A] 全部皆是(A) [N] 否(N) [L] 全部皆否(L) [S] 暫停(S) [?] 說明 (預設值為 "N"): y
PS C:\Windows\system32>
```

進入虛擬環境:

```
# cd dvds
# Scripts\activate
```

- Visual Studio Code

安裝 VS Code 套件

1、python

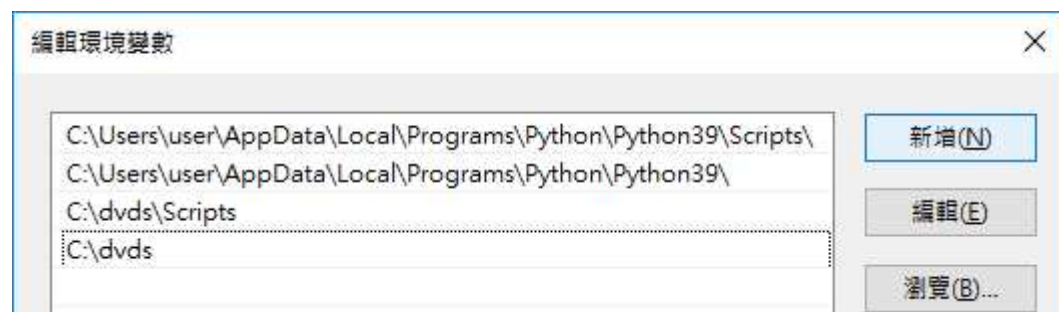
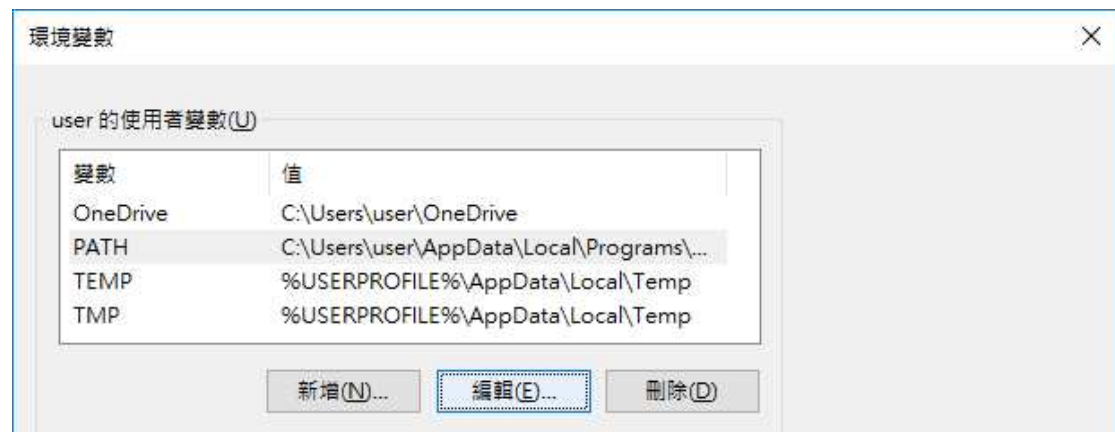


設定環境變數：

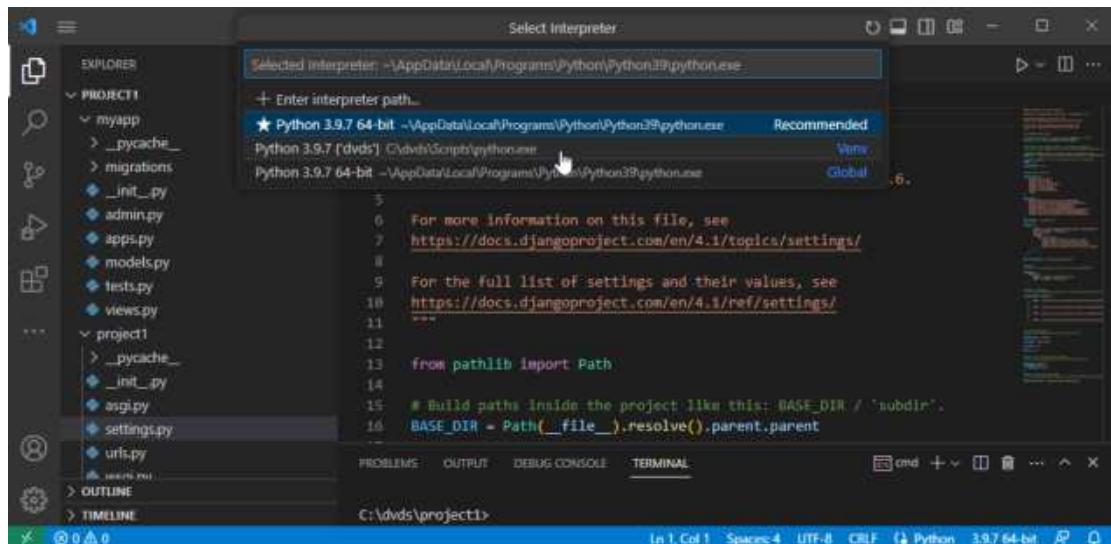
本機 > 內容 > 進階設定 > 環境變數，在 Path 變數內新增以下兩個路徑，請依據您 Python 安裝的路徑及名稱而定：

C:\dvds

C:\dvds\Scripts

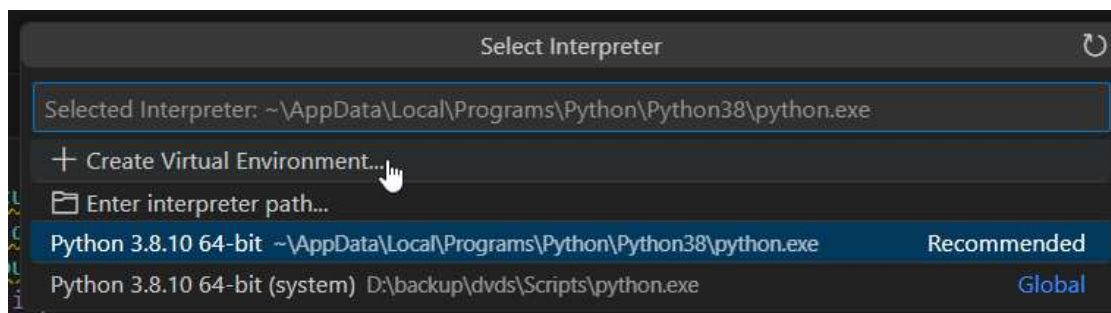


重啟 vscode 後，進入到專案，點選最下一排(左下角或右下角)python...，選擇當前專案使用的虛擬目錄，如下：

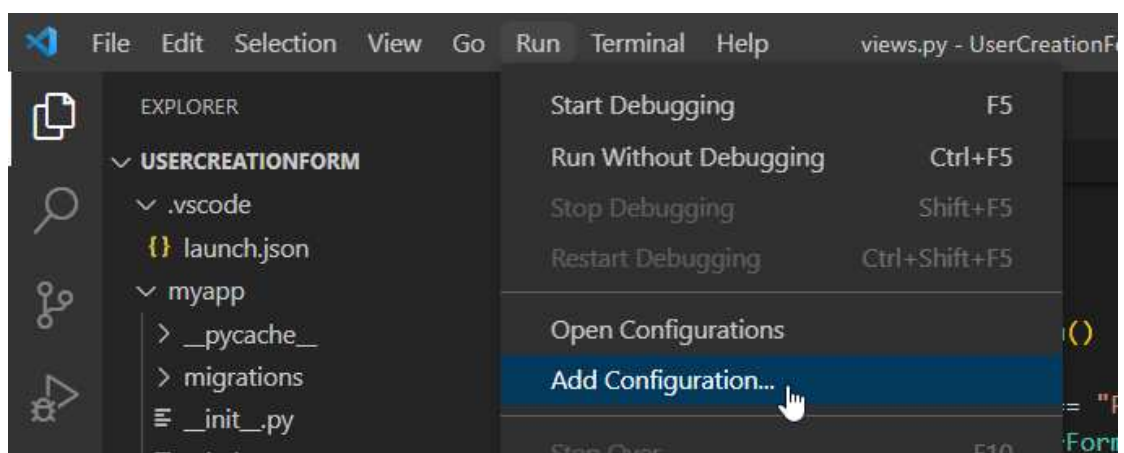


或是手動:

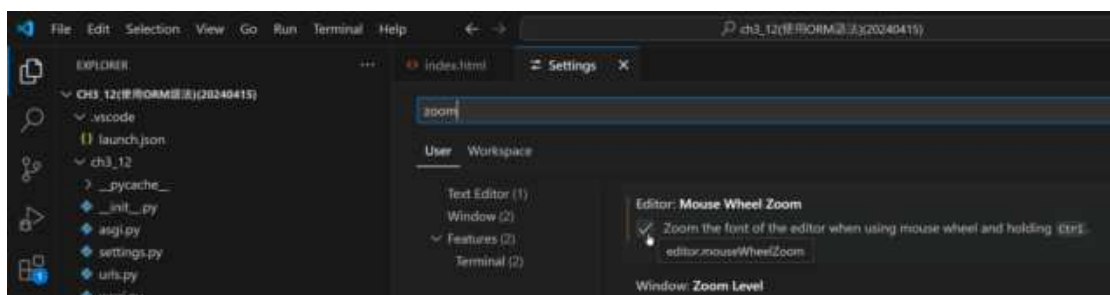
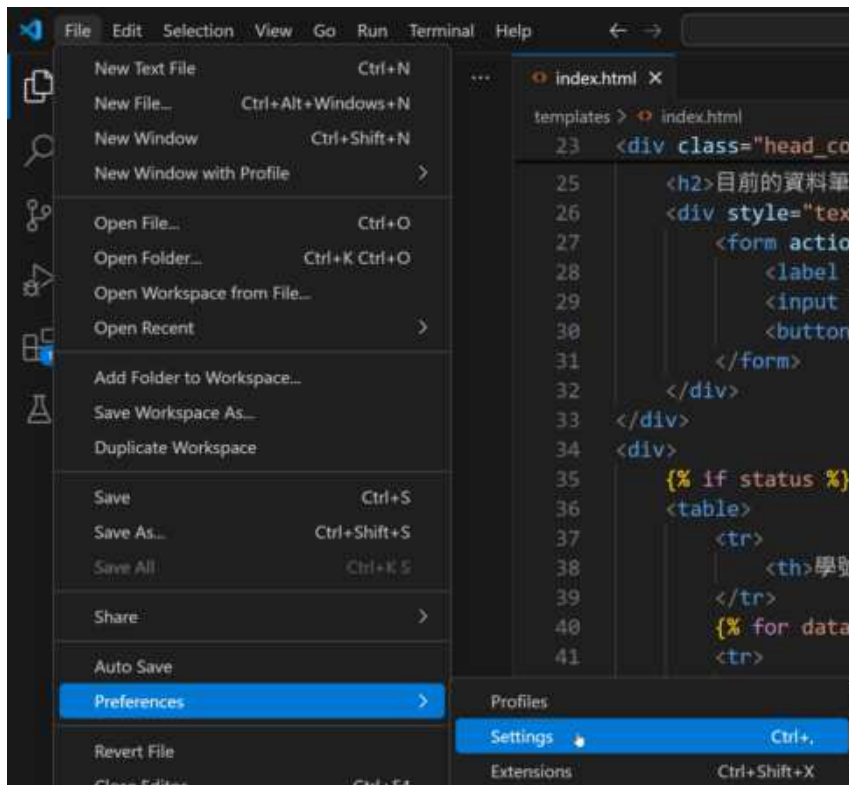
- 1、View → Command Palette
- 2、輸入「python:select interpreter」
- 3、手動新增 python 路徑



- 4、新增組態

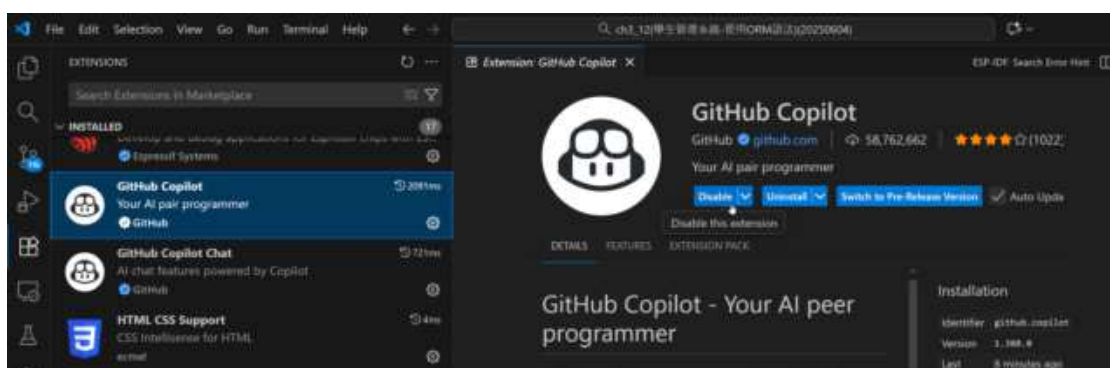


Vscode 字型大小使用滾輪:



2、Github copilot for Visual Studio Code

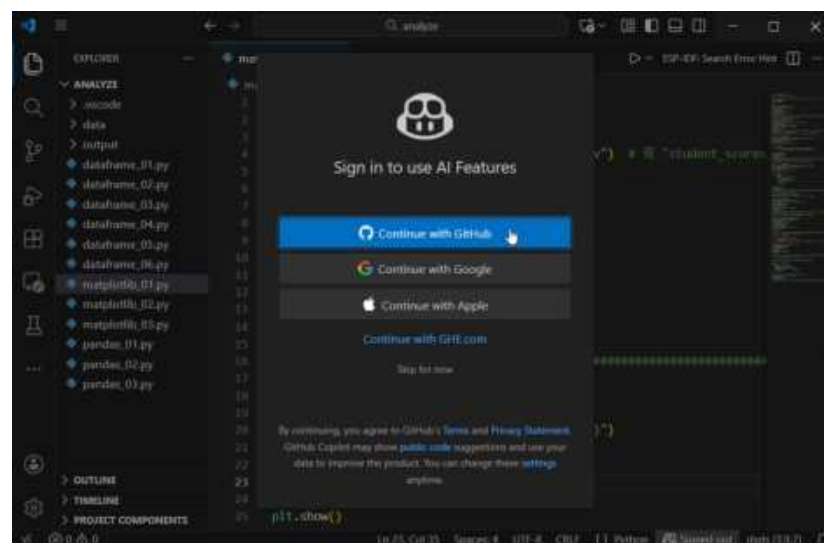
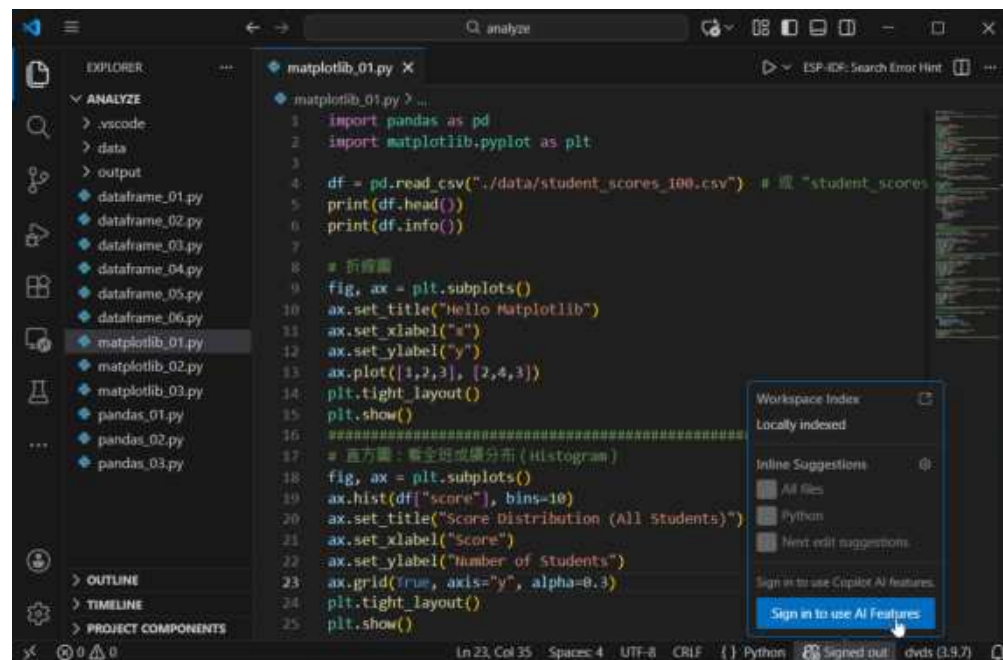
在 extensions 中，安裝 Github copilot

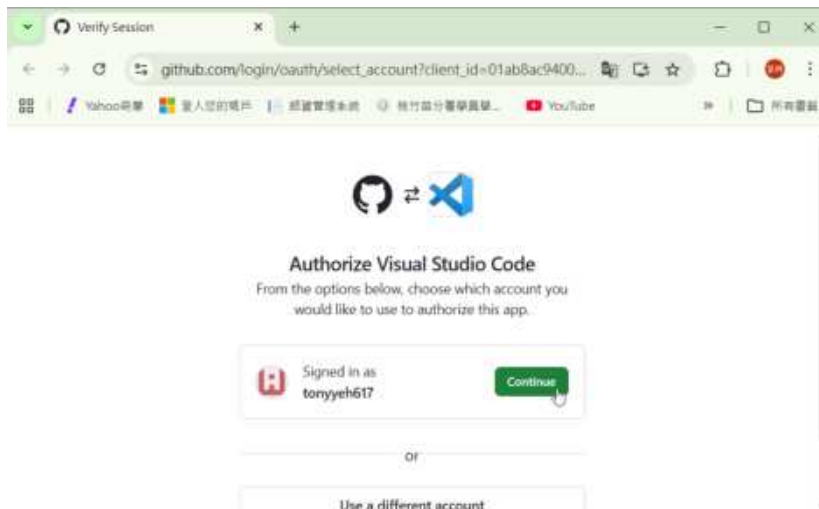


在 github 註冊

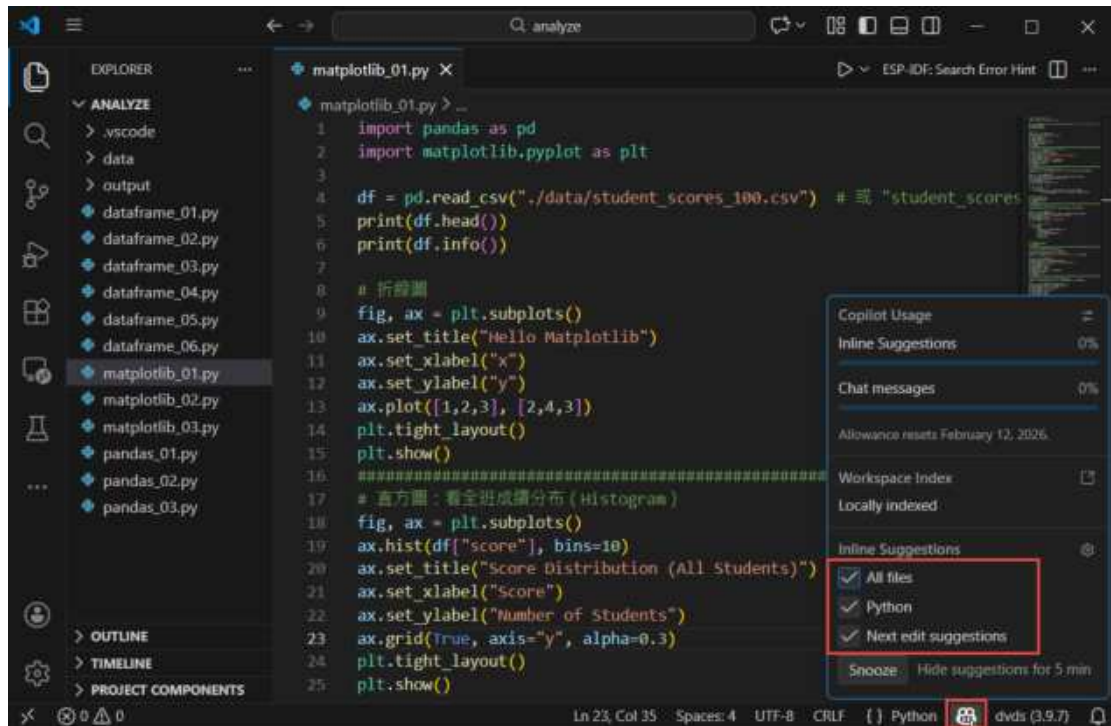
<https://github.com/>

在 vscode 註冊



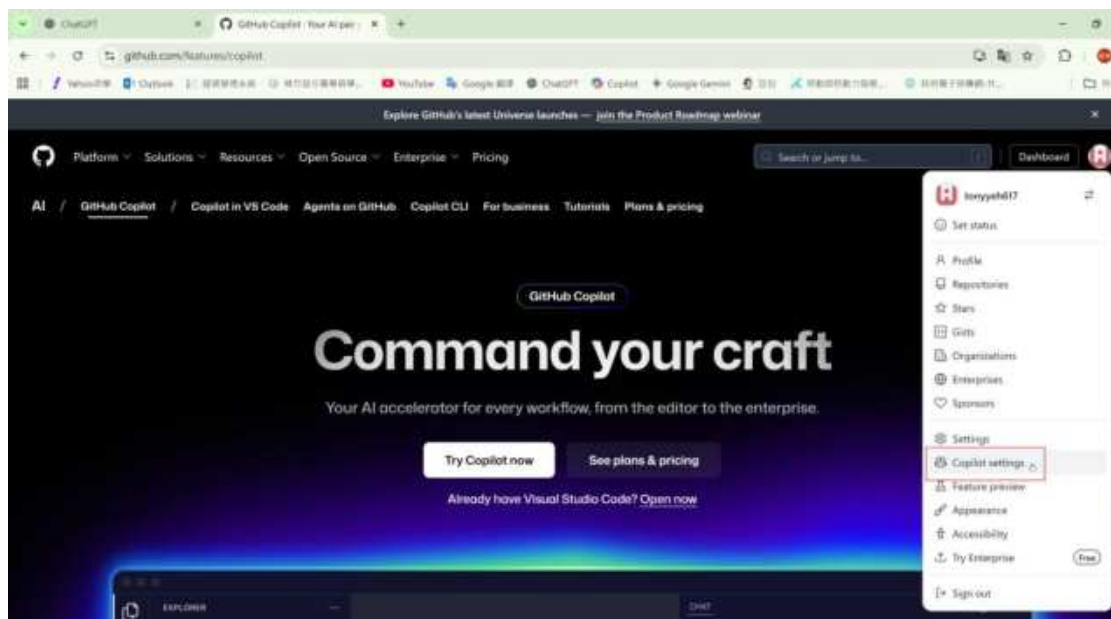


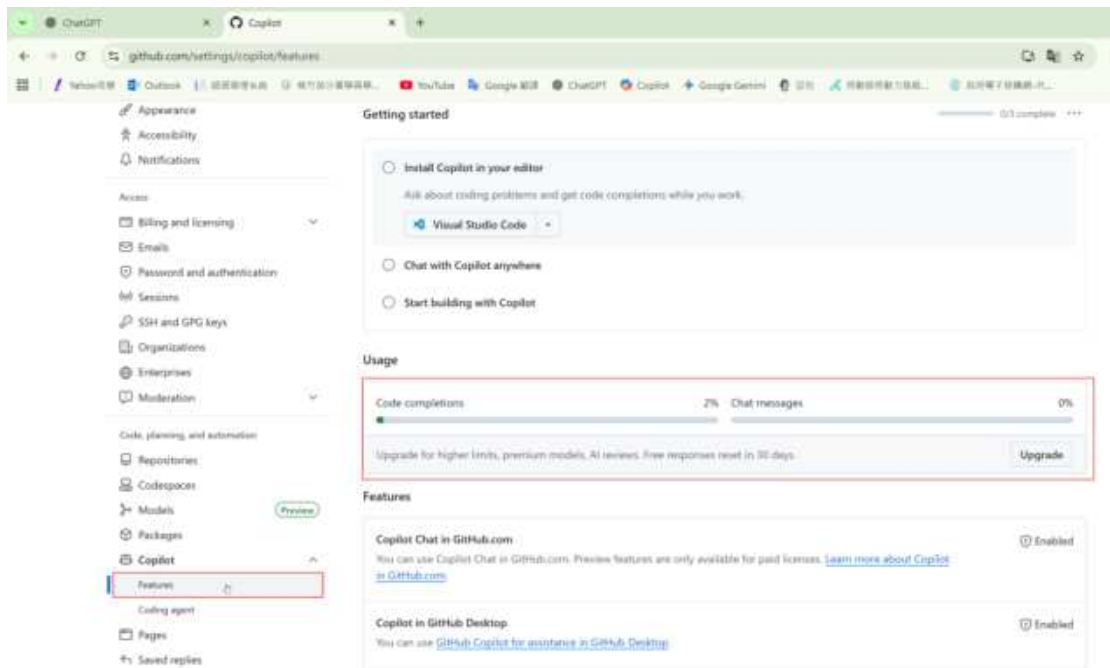
成功後，再點選「Copilot Usage」



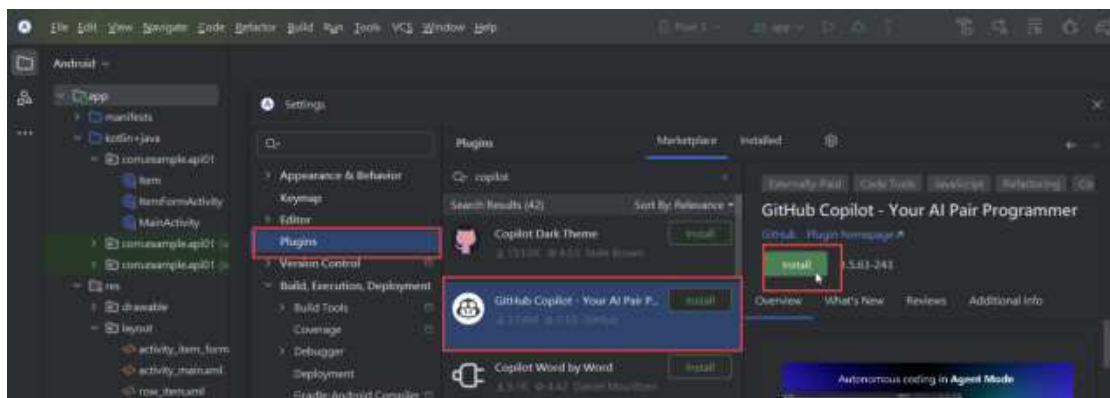
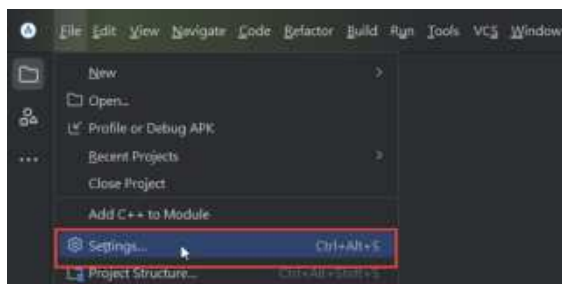
可以到 github 上檢查

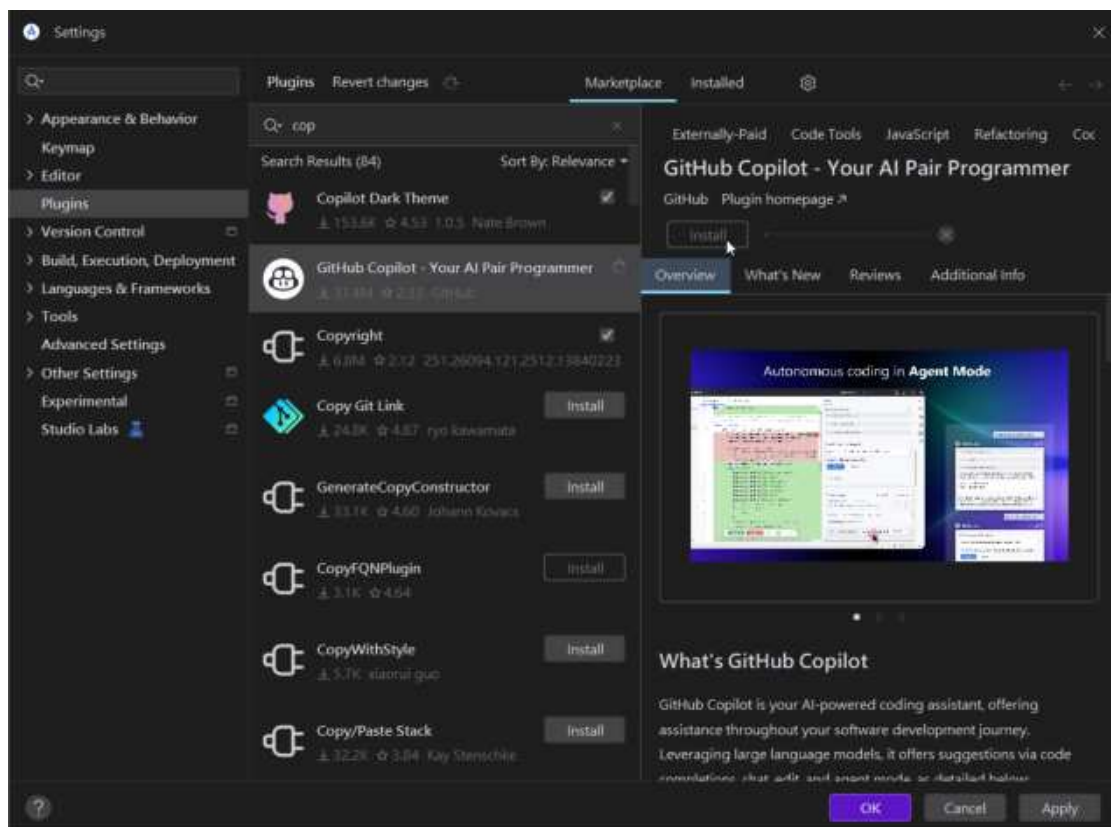
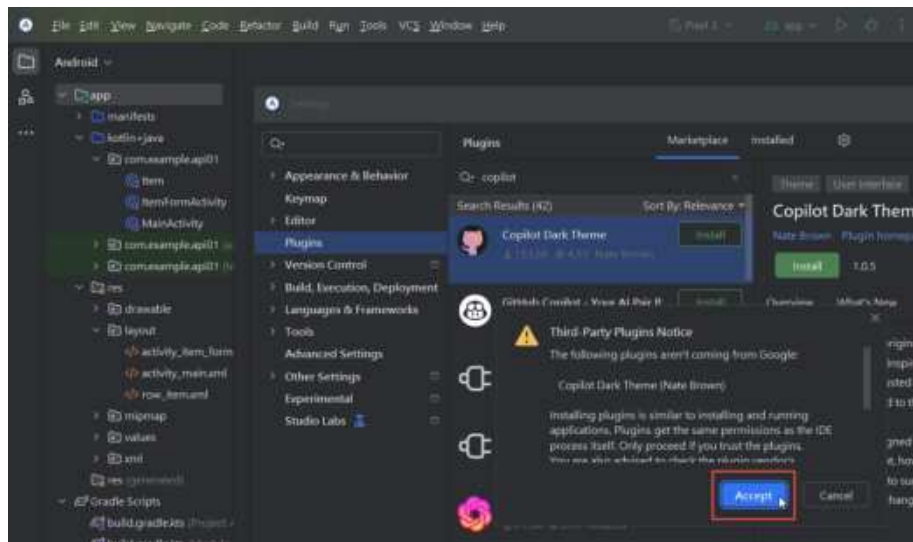
<https://github.com/features/copilot>

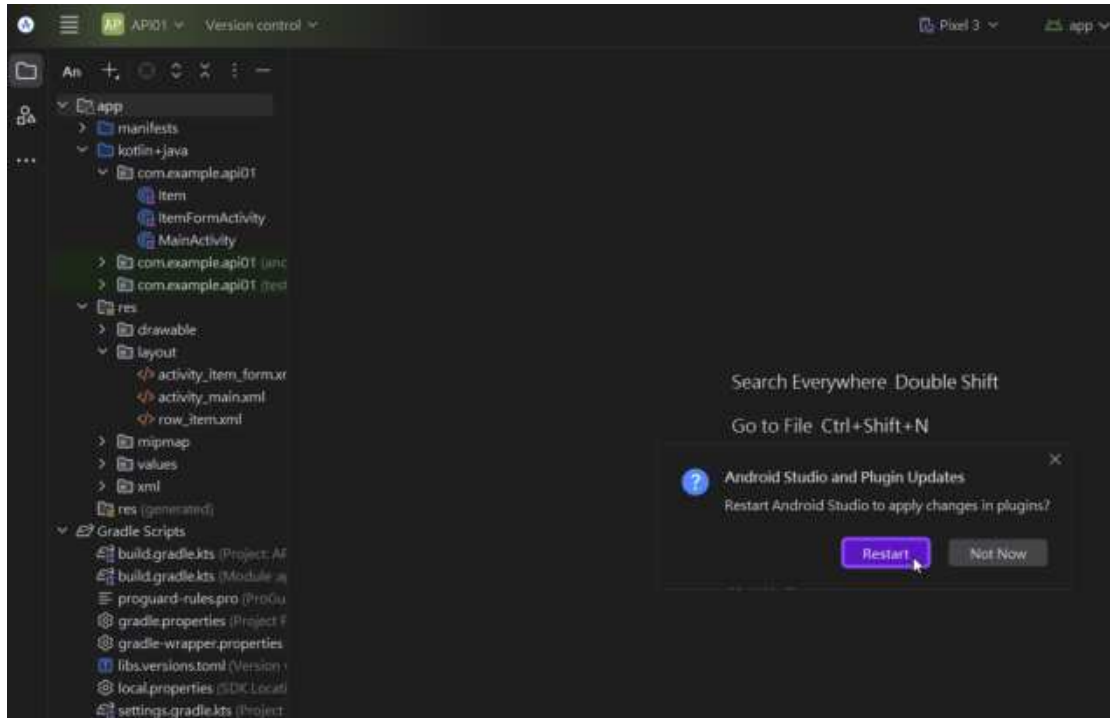
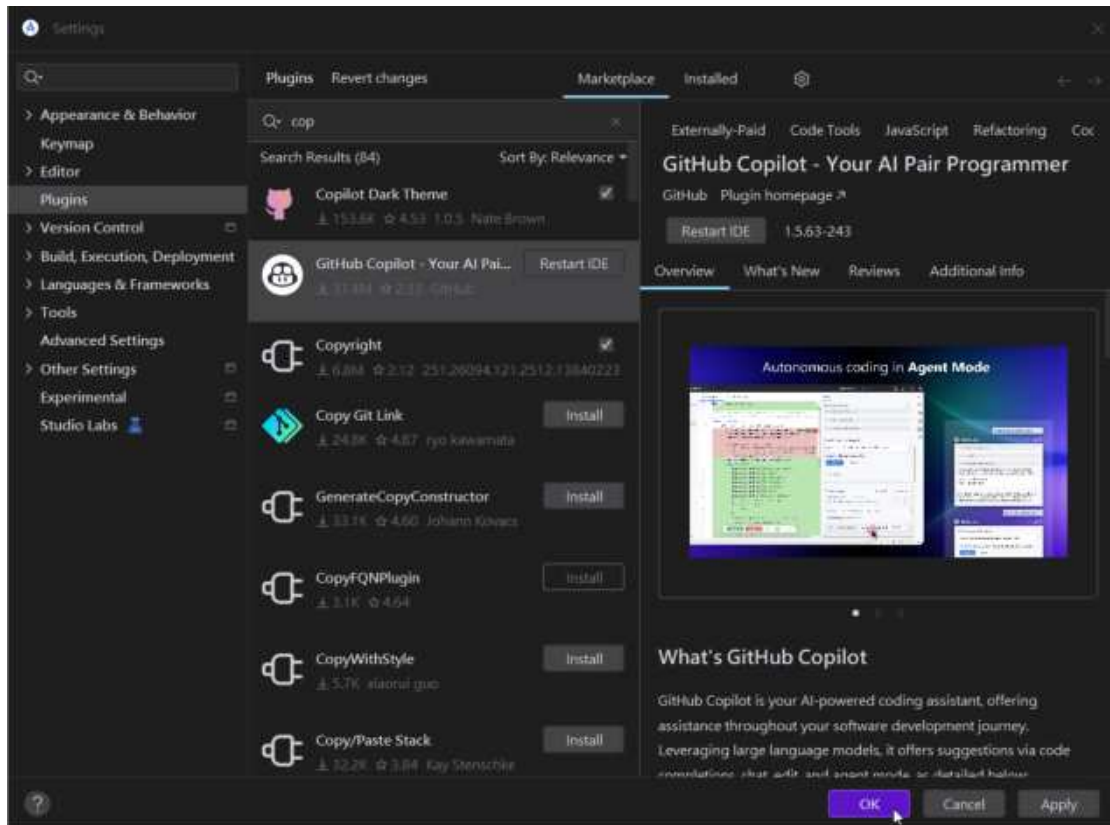


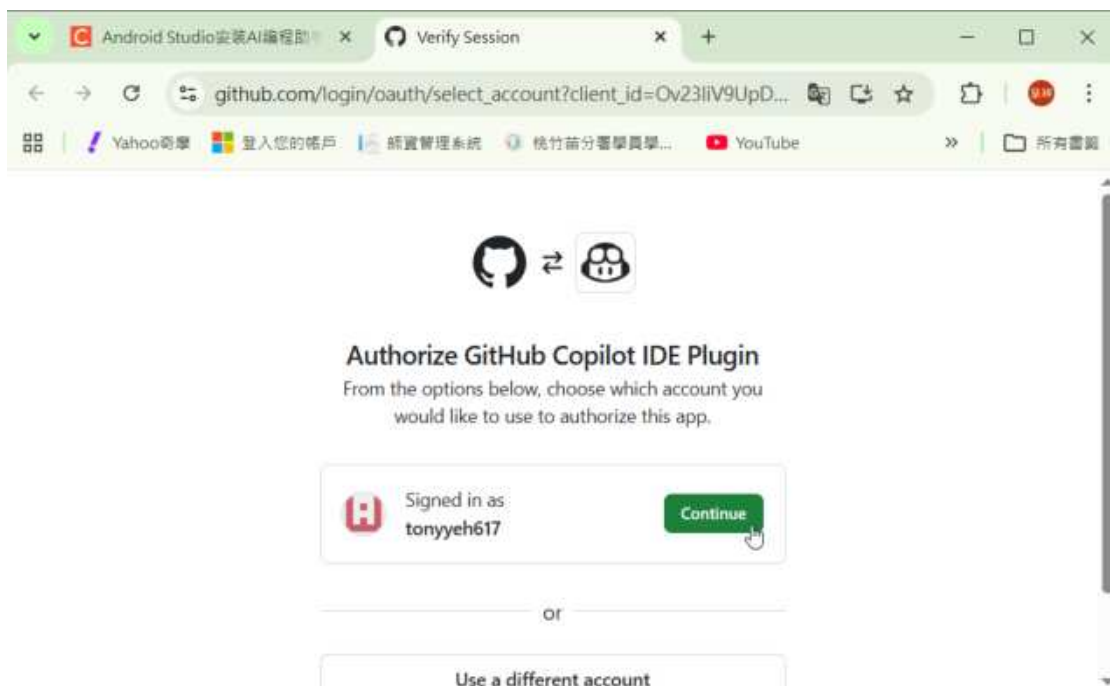
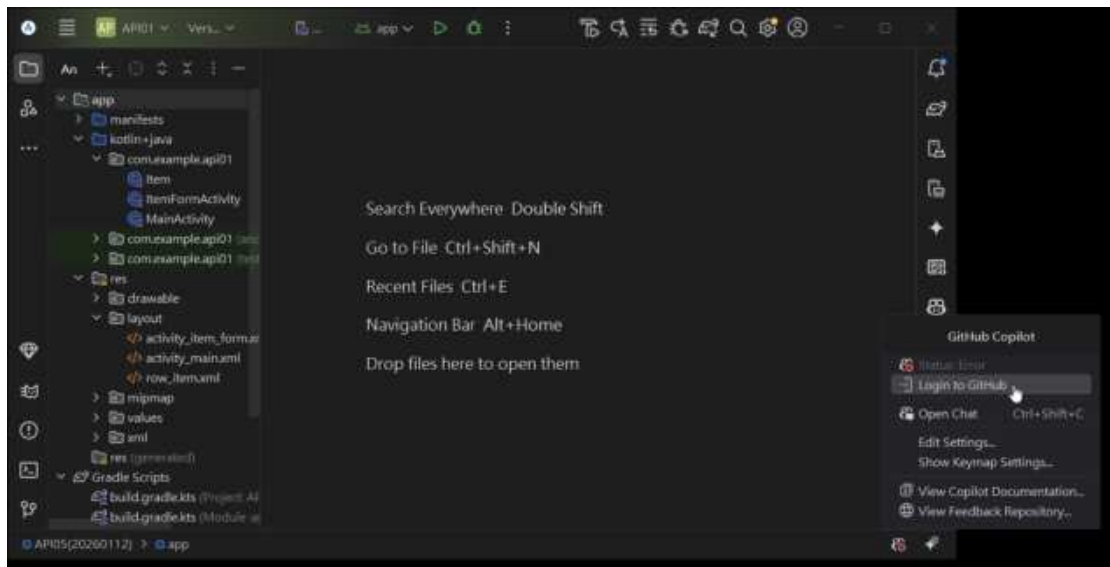


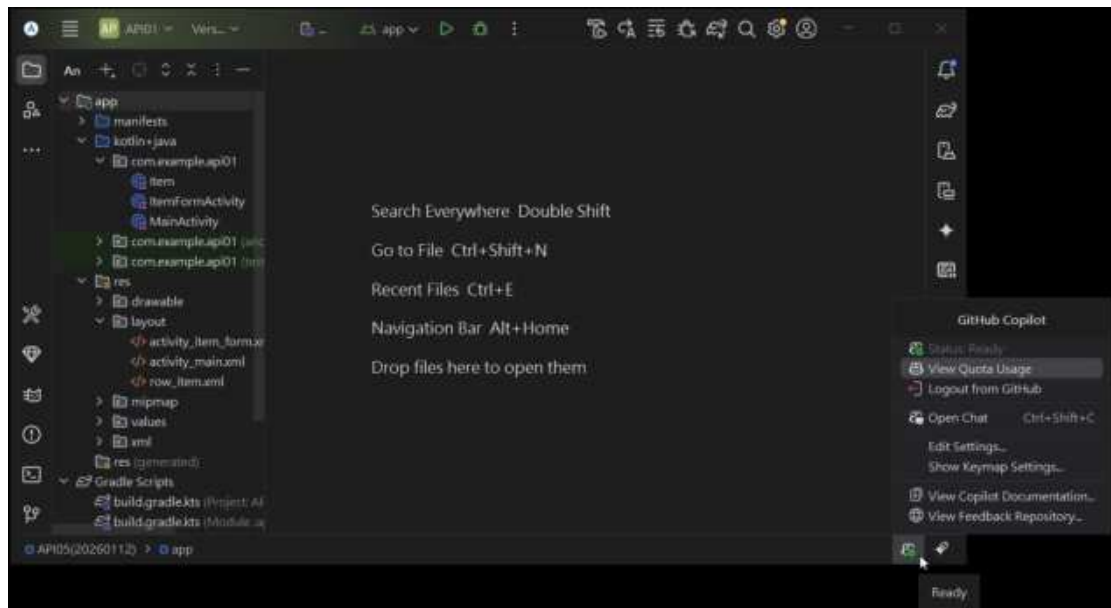
3、Github copilot for Android











第二章 機器學習與資料分析

2-1 Pandas 模組

Web:

<https://pandas.pydata.org/docs/index.html>

Pandas 是 Python 的資料分析與資料處理套件，提供高效能、易操作的資料結構（Series 與 DataFrame）。

◆ 1 Series (像一條直線)

nginx

Index	Value
0	10
1	20
2	30
3	40

◆ 2 DataFrame (像一張表格)

markdown

	姓名	數學	英文
0	小明	90	85
1	小華	80	70
2	小美	95	88

Pandas 三種資料結構使用時機比較表					
資料結構	維度	結構說明	類似概念	適合使用時機	常見應用
Series	1 維	單一欄位資料 + Index	一維陣列	單一變數、時間序列	溫度、成績、股價
DataFrame	2 維	多個 Series 組成	Excel 表格	一般結構化資料	學生成績、銷售資料
MultiIndex DataFrame	多維 (邏輯上)	具多層索引的 DataFrame	階層式表格	多條件、多層分類資料	IoT、多時間多設備

```
# pip install pandas
# pip list
```

• PS D:\backup\myExam\pandas> & D:\backup\myExam\Scripts\Activate.ps1

• (myExam) PS D:\backup\myExam\pandas> pip list

Package	Version
contourpy	1.3.0
cycler	0.12.1
fonttools	4.59.1
importlib_resources	6.5.2
joblib	1.5.2
kiwisolver	1.4.7
matplotlib	3.9.4
numpy	2.0.2
opencv-python	4.12.0.88
packaging	25.0
pandas	2.3.2
Pillow	11.3.0
pip	25.1.1
pyparsing	3.2.3
python-dateutil	2.9.0.post0
pytz	2025.2
scikit-learn	1.6.1
scipy	1.13.1
seaborn	0.13.2
setuptools	80.4.0
six	1.17.0
threadpoolctl	3.6.0
tzdata	2025.2
wheel	0.45.1
zipp	3.23.0

2-2 Series

python 複製程式碼

```
pd.Series(data, index=None, dtype=None, name=None)
```

參數說明

參數	說明
data	資料本體 (list · tuple · dict · numpy array · scalar)
index	索引 (標籤)，可自訂
dtype	資料型態 (int · float · str...)
name	Series 的名稱

Web:

<https://pandas.pydata.org/docs/reference/series.html>

Series 是一種一維的陣列資料結構，在這個陣列內可以存放整數、浮點數、字串、Python 物件(例如;字串 list 字典 dict ..) Numpy 的 ndarray、純量，...,等。雖然是一維陣列資料，可是看起來卻好像是二維陣列資料，因為一個是索引(index)或稱標籤(label)，另一個是實際的資料。

功能	語法
建立	<code>pd.Series()</code>
修改	<code>series[索引] = 值</code>
取值	<code>series[索引]</code>
新增	<code>series[新索引] = 值</code>
刪除	<code>series.drop(索引)</code>

```

1 import pandas as pd
2
3 # 建立 Series
4 print(".....1 建立 Series.....")
5 s1 = pd.Series([10, 20, 30, 40, 50]) # 預設索引
6 s2 = pd.Series([15, 25, 35], index=['a', 'b', 'c']) # 指定索引
7 print("s1")
8 print(s1)
9 print("s2")
10 print(s2)
11
12 # 修改
13 print(".....2 修改.....")
14 s1[1] = 99
15 s2['b'] = 88
16 print("s1")
17 print(s1)
18 print("s2")
19 print(s2)

```

```

pandas_01.py X
pandas_01.py > _
20
21 # 取值
22 print(".....3 取值.....")
23 print(f"s1[0] = {s1[0]}")
24 print(f"s2['a'] = {s2['a']}")
25
26 # 新增
27 print(".....4 新增.....")
28 s1[5] = 60 # 新索引 5
29 s2['d'] = 45 # 新索引 d
30 print("s1")
31 print(s1)
32 print("s2")
33 print(s2)
34
35 # 刪除
36 print(".....5 刪除.....")
37 s1_new = s1.drop(2)
38 s2_new = s2.drop('c')
39 print("s1 刪除 index=2")
40 print(s1_new)
41 print("s2 刪除 index='c'")
42 print(s2_new)

```

範例:series_01.py

◆ 1 條件篩選 (布林索引)

📌 語法：

python

複製程式碼

```
series物件名稱[條件]
```

📌 範例：

python

複製程式碼

```
s3[s3 > 30]
s4[s4 <= 40]
```

◆ 2 統計方法

📌 常用統計函數：

功能	語法
平均值	<code>series.mean()</code>
最大值	<code>series.max()</code>
最小值	<code>series.min()</code>
總和	<code>series.sum()</code>
筆數	<code>series.count()</code>

◆ 3 Series 與 Series 基本算術運算

索引相同時，會對應索引做運算

📌 語法：

python

```
series1 + series2
series1 - series2
series1 * series2
series1 / series2
```

◆ 4 Series 與數值運算 (廣播運算)

每個元素都會與該數值運算

📌 語法：

```
python  
  
series + 數值  
series - 數值  
series * 數值  
series / 數值
```

◆ 5 比較運算

📌 語法：

```
python  
  
series1 > series2  
series1 == series2  
series >= 數值
```

◆ 6 排序

📌 依「值」排序

```
python  
  
series.sort_values()
```

📌 依「索引」排序

```
python  
  
series.sort_index()
```

```
pandas_02.py X
pandas_02.py > ...
1  import pandas as pd
2
3  # 建立 Series
4  s3 = pd.Series([10, 35, 50, 20, 45], index=[0, 1, 2, 3, 4])
5  s4 = pd.Series([5, 40, 25, 60, 30], index=[0, 1, 2, 3, 4])
6  print("s3")
7  print(s3)
8  print("s4")
9  print(s4)
10
11 # 條件篩選
12 print(".....1 條件篩選.....")
13 print("s3 > 30")
14 print(s3[s3 > 30])
15
16 print("s4 <= 40")
17 print(s4[s4 <= 40])
18
19 # 統計方法
20 print(".....2 統計方法.....")
21 print("s3 平均:", s3.mean())
22 print("s4 平均:", s4.mean())
23 print("s3 最大:", s3.max())
24 print("s4 最大:", s4.max())
```

```
pandas_02.py X
pandas_02.py > ...
26 # 運算 (索引相同, 不會有 NaN)
27 print(".....3 基本算術運算.....")
28 print("s3 + s4")
29 print(s3 + s4)
30
31 print("s3 - s4")
32 print(s3 - s4)
33
34 print("s3 * s4")
35 print(s3 * s4)
36
37 print("s3 / s4")
38 print(s3 / s4)
39
40 print(".....4 「數值」運算.....")
41 print("s3 + 10")
42 print(s3 + 10)
43
44 print("s3 * 2")
45 print(s3 * 2)
46
47 print("s4 - 5")
48 print(s4 - 5)
49
50
51 print(".....5 比較運算.....")
52 print("s3 > s4")
53 print(s3 > s4)
54
55 print("s3 == s4")
56 print(s3 == s4)
```

```
pandas_02.py X
pandas_02.py > ...
57
58 print("s3 >= 30")
59 print(s3 >= 30)
60
61
62 # 排序
63 print(".....6 排序.....")
64 print("s3 依值排序")
65 print(s3.sort_values())
66
67 print("s4 依索引排序")
68 print(s4.sort_index())
```

範例:series_02.py

範例：

老師記錄 5 位學生的期中考成績

老師使用 Python 與 pandas 來管理學生的期中考成績：

s1：以「座號」記錄學生的成績（預設索引）

s2：以「學生姓名」記錄學生的成績（自訂索引）

```

pandas_02.py  pandas_03.py X
pandas_03.py > ...
5
6  import pandas as pd
7  print(".....1.....")
8  # s1：用座號記錄成績（預設索引）
9  s1 = pd.Series([60, 70, 80, 90, 100])
10
11 # s2：用學生姓名記錄成績（指定索引）
12 s2 = pd.Series([85, 90, 95], index=['Andy', 'Betty', 'Charlie'])
13
14 # 「這兩張表都是成績，只是索引代表的意義不同」
15
16 print("座號成績 s1")
17 print(s1)
18 print("\n姓名成績 s2")
19 print(s2)

```

```

pandas_03.py X
pandas_03.py > ...
20
21 print(".....2.....")
22 # 新增資料（補考 / 新轉學生）
23
24 # 新增補考學生（座號）
25 s1[5] = 60
26 s1[6] = 55
27
28 # 新轉學生（姓名）
29 s2['David'] = 45
30 s2['Eva'] = 52
31
32 print("更新後 s1")
33 print(s1)
34 print("\n更新後 s2")
35 print(s2)
36
37 print(".....3.....")
38 print("\n【條件篩選】")
39
40 print("s1 及格 (>= 60)")
41 print(s1[s1 >= 60])
42
43 print("\ns2 不及格 (< 60)")
44 print(s2[s2 < 60])

```

```

pandas_03.py
pandas_03.py > ...
44 print(s2[s2 < 60])
45 # 老師快速找出「及格學生」和「需要補救教學的學生」
46
47 print(".....4.....")
48 print("\n【統計方法】")
49
50 print("s1 平均成績:", s1.mean())
51 print("s1 最高分:", s1.max())
52
53 print("s2 平均成績:", s2.mean())
54 print("s2 最低分:", s2.min())
55
56 print(".....5.....")
57 print("\n【成績調整(運算)】")
58
59 print("s1 加 5 分")
60 print(s1 + 5)
61
62 print("s2 加 5 分")
63 print(s2 + 5)
64
65 print(".....6.....")
66 print("\n【排序】")
67
68 print("s1 成績排名 (由高到低)")
69 print(s1.sort_values(ascending=False))
70
71 print("\ns2 成績排名 (由低到高)")
72 print(s2.sort_values())

```

```
pandas_03.py X
pandas_03.py > ...
45 # 老師快速找出「及格學生」和「需要補救教學的學生」
46
47 print(".....4.....")
48 print("\n【統計方法】")
49
50 print("s1 平均成績:", s1.mean())
51 print("s1 最高分:", s1.max())
52
53 print("s2 平均成績:", s2.mean())
54 print("s2 最低分:", s2.min())
55
56 print(".....5.....")
57 print("\n【成績調整(運算)】")
58
59 print("s1 加 5 分")
60 print(s1 + 5)
61
62 print("s2 加 5 分")
63 print(s2 + 5)
64
65 print(".....6.....")
66 print("\n【排序】")
67
68 print("s1 成績排名 (由高到低)")
69 print(s1.sort_values(ascending=False))
70
71 print("\ns2 成績排名 (由低到高)")
72 print(s2.sort_values())
```

範例:series_03.py

2-3 DataFrame

Web:

<https://pandas.pydata.org/docs/reference/frame.html>

DataFrame 是一種二維的陣列資料結構，邏輯上而言可以視為是類似 Excel 的工作表，在這個二維陣列內可以存放整數、浮點數、字串、Python 物件(例如:字串 list、字典 dict ..)、Numpy 的 ndarray、純量、 ... 等。

目標：

◆ 1 建立 DataFrame

python

```
pd.DataFrame(字典資料)
```

python

```
pd.DataFrame(資料list, columns=[欄位名稱])
```

◆ 2 取值 (讀取資料)

✓ 取單一欄

python

```
df["欄位名稱"]
```

```
df.欄位名稱
```

✓ 取多欄

python

```
df[["欄位1", "欄位2"]]
```

✅ 用位置取值 (iloc)

```
python  
  
df.iloc[列索引]  
df.iloc[列索引, 欄索引]
```

✅ 用標籤取值 (loc)

```
python  
  
df.loc[列標籤, 欄位名稱]
```

✅ 條件篩選

```
python  
  
df[條件]
```

◆ 3 修改資料

✅ 修改單一值

```
python  
  
df.loc[列, "欄位"] = 新值
```

✅ 修改整欄

```
python  
  
df["欄位"] = 新值
```

✅ 條件修改

```
python  
  
df.loc[條件, "欄位"] = 新值
```

◆ 4 新增資料

✅ 新增欄位

```
python  
  
df["新欄位"] = 資料
```

✅ 新增列

```
python  
  
pd.concat([df, 新DataFrame], ignore_index=True)
```

◆ 5 刪除資料

✓ 刪除欄位

```
python  
  
df.drop("欄位名稱", axis=1)
```

✓ 刪除列

```
python  
  
df.drop(列索引)
```

✓ 條件刪除

```
python  
  
df = df[條件]
```

◆ 6 查看資料

```
python  
  
df.head(筆數)
```

```
python  
  
df.tail(筆數)
```

```
python  
  
df.info()
```

```
python  
  
df.describe()
```

◆ 7 更改欄位名稱

python

```
df.columns = [新欄位名稱]
```

python

```
df.rename(columns={"舊名稱": "新名稱"})
```

◆ 8 更改欄位順序

python

```
df = df[[欄位順序]]
```

python

```
df.reindex(columns=[欄位順序])
```

```
dataframe_01.py X
dataframe_01.py > ...
1  import pandas as pd
2
3  print(".....1.....")
4  # 建立 DataFrame
5  # 方法1
6  # 用 dict 建立 (最常用)
7  data1 = {
8      "name": ["Tom", "Amy", "John"],
9      "age": [20, 22, 25],
10     "score": [85, 90, 78]
11 }
12
13 df1 = pd.DataFrame(data1)
14 print(df1)
15
16 print(".....2.....")
17 # 方法2
18 # 用 list 建立
19 data2 = [
20     ["Tom", 20, 85],
21     ["Amy", 22, 90],
22     ["John", 25, 78]
23 ]
24 df2 = pd.DataFrame(data2, columns=["name", "age", "score"])
25 print(df2)
```

```

dataframe_01.py X
dataframe_01.py > ...
26
27 print(".....3.....")
28 # 取值 (讀資料)
29 print(df1["name"])
30 print(df1.name)
31
32 print(".....4.....")
33 # 取多種
34 print(df1[["name", "score"]])
35
36 print(".....5.....")
37 # 用 index 取值 (iloc: 位置)
38 print(df1.iloc[0])      # 第 0 列
39 print(df1.iloc[0, 1])   # 第 0 列 - 第 1 個
40
41 print(".....6.....")
42 # 用條件取值 (很重要)
43 print(df1[df1["score"] >= 85])
44
45 print(".....7.....")
46 # 用 loc (標籤)
47 print(df1.loc[1, "name"]) # 第 1 列的 name
48
49 print(".....8.....")
50 # 修改資料
51 # 改單一值
52 print(df1)
53 df1.loc[0, "score"] = 88
54 print(df1)
55

```

```

dataframe_01.py X
dataframe_01.py > ...
55
56 print(".....9.....")
57 # 改整欄
58 print(df1)
59 df1["age"] = df1["age"] + 1
60 print(df1)
61
62 print(".....10.....")
63 # 條件修改
64 print(df1)
65 df1.loc[df1["name"] == "Tom", "score"] = 59
66 print(df1)
67
68 print(".....11.....")
69 # 新增資料
70 print(df1)
71 df1["pass"] = df1["score"] >= 60
72 print(df1)

```

```

dataframe_01.py X
dataframe_01.py > ...
74 print(".....12.....")
75 # 再新增一列 (使用 concat)
76 new_row = pd.DataFrame([{"name": "Mary",
77     "age": 23,
78     "score": 88,
79     "pass": True
80 }])
81 ])
82 print(df1)
83 df1 = pd.concat([df1, new_row], ignore_index=True)
84 print(df1)
85
86 print(".....13.....")
87 # 刪除欄位
88 print(df1)
89 df1 = df1.drop("pass", axis=1)
90 print(df1)
91
92 print(".....14.....")
93 # 刪除列 (用 index)
94 print(df1)
95 df1 = df1.drop(1) # 刪除 1 列
96 print(df1)
97

```

```

dataframe_01.py X
dataframe_01.py > ...
98 print(".....15.....")
99 # 條件刪除
100 print(df1)
101 df1 = df1[df1["score"] >= 70]
102 print(df1)
103
104 print(".....16.....")
105 print(df1.head(1)) # 看前面幾筆
106 print(df1.tail(1)) # 看最後幾筆
107
108 print(".....17.....")
109 df1.info() # 看資料結構
110 df1.describe() # 看數字統計
111
112
113 print(".....18.....")
114 # 更改欄位名稱
115 print(df1)
116 # 方法1
117 df1.columns = ["姓名", "年齡", "成績"]
118 print(df1)
119
120 # 方法2
121 df1 = df1.rename(columns={
122     "姓名": "name",
123     "年齡": "age",
124     "成績": "score"
125 })
126 print(df1)

```

```

dataframe_01.py X
dataframe_01.py > ...
127
128 print(".....19.....")
129 # 更改欄位順序
130 # 方法1
131 df1 = df1[["score", "name", "age"]]
132 print(df1)
133
134 # 方法2
135 df1 = df1.reindex(columns=["name", "score", "age"])
136 print(df1)

```

範例: dataframe_01.py

範例: dataframe_02.py

目標：

Pandas Series / DataFrame 統計方法整理表

以下範例皆以 `df['score']` 為操作對象

方法	中文名稱	功能說明	備註 / 補充
<code>mean()</code>	平均值	計算所有數值的平均	常用於成績、數值分析
<code>sum()</code>	總和	將所有數值加總	
<code>max()</code>	最大值	取得最大數值	
<code>min()</code>	最小值	取得最小數值	
<code>median()</code>	中位數	排序後位於中間的數值	不易受極端值影響
<code>var()</code>	變異數 (樣本)	計算樣本變異數	預設 <code>ddof=1</code>
<code>var(ddof=0)</code>	變異數 (母體)	計算母體變異數	常用於完整資料
<code>std()</code>	標準差	變異數開根號	衡量資料分散程度

```

dataframe_03.py X
dataframe_03.py > ...
1  import pandas as pd
2
3  data = {
4      'score': [60, 70, 80, 90, 100],
5      'age': [20, 21, 22, 23, 24]
6  }
7
8  df = pd.DataFrame(data)
9
10 # 平均值 (Mean)
11 print("Mean:", df['score'].mean())
12
13 # 加總 (Sum)
14 print("Sum:", df['score'].sum())
15
16 # 最大值 (Max)
17 print("Max:", df['score'].max())

```

```

dataframe_03.py X
dataframe_03.py > ...
16 # 最大值 (Max)
17 print("Max:", df['score'].max())
18
19 # 最小值 (Min)
20 print("Min:", df['score'].min())
21
22 # 中位數 (Median)
23 print("Median:", df['score'].median())
24
25 # 變異數 (Variance)
26 print("Variance:", df['score'].var()) #預設是樣本變異數 (ddof=1)
27 print("Variance:", df['score'].var(ddof=0)) #母體變異數
28
29
30 # 標準差 (Standard Deviation)
31 print("Standard Deviation:", df['score'].std())
32

```

範例: dataframe_03.py

Pandas DataFrame 分組 (groupby) 語法整理

◆ 1 分組

python

```
df.groupby("欄位名稱")
```

◆ 2 分組後選擇欄位

python

```
df.groupby("欄位名稱")["數值欄位"]
```

◆ 3 分組後做單一統計

✓ 平均

```
python  
  
df.groupby("欄位名稱")["數值欄位"].mean()
```

✓ 總和

```
python  
  
df.groupby("欄位名稱")["數值欄位"].sum()
```

✓ 變異數

```
python  
  
df.groupby("欄位名稱")["數值欄位"].var()
```

✓ 標準差

```
python  
  
df.groupby("欄位名稱")["數值欄位"].std()
```



◆ 4 分組後做多種統計 (agg)

```
python  
  
df.groupby("欄位名稱")["數值欄位"].agg(["統計方法1", "統計方法2"])
```

◆ 5 分組後完整統計摘要

```
python  
  
df.groupby("欄位名稱")["數值欄位"].describe()
```

常用統計方法名稱

python

```
"mean"    # 平均
"sum"      # 總和
"var"      # 變異數
"std"      # 標準差
"min"      # 最小值
"max"      # 最大值
"count"    # 筆數
```

```
dataframe_04.py X
dataframe_04.py > ...
10 import pandas as pd
11
12 # 建立成績資料
13 data = {
14     'class': ['A', 'A', 'A', 'B', 'B', 'B'],
15     'student': ['Tom', 'Amy', 'John', 'Mary', 'Kevin', 'Lucy'],
16     'score': [80, 90, 85, 70, 60, 75]
17 }
18
19 df = pd.DataFrame(data)
20
21 print("=== 原始成績資料 ===")
22 print(df)
23 print()
24
25 print("=== 各班平均成績 ===")
26 avg_score = df.groupby('class')['score'].mean()
27 print(avg_score)
28 print()
29
30 print("=== 各班成績統計分析 ===")
31 stats = df.groupby('class')['score'].agg(['mean', 'sum', 'var', 'std'])
32 print(stats)
33 print()
34
35 print("=== 各班成績完整統計摘要 ===")
36 desc = df.groupby('class')['score'].describe()
37 print(desc)
```

範例: dataframe_04.py

```

dataframe_05.py X
dataframe_05.py > ...
22 import pandas as pd
23 # 讀取 CSV ( 含學號 )
24 df = pd.read_csv("../data/student_scores_100.csv")
25
26 print("=== 原始成績資料 ===")
27 print(df)
28 print()
29
30 print("=== A 班學生名單 ===")
31 class_a = df[df['class'] == 'A']
32 print(class_a[['student_id', 'student', 'score']])
33 print()
34
35 print("=== 成績 ≥ 95 分的學生 ===")
36 high_score = df[df['score'] >= 95]
37 print(high_score[['student_id', 'student', 'class', 'score']])
38 print()
39
40 print("=== 成績 ≥ 95 分學生 ( 只顯示姓名與成績 ) ===")
41 result = df[df['score'] >= 95][['student', 'score']]
42 print(result)
43 print()

```

```

dataframe_05.py X
dataframe_05.py > ...
45 print("=== B 班且成績低於 60 分的學生 ===")
46 b_low_score = df[(df['class'] == 'B') & (df['score'] < 60)]
47 print(b_low_score[['student_id', 'student', 'score', 'class']])
48 print()
49
50 print("=== 姓名包含字母 T 的學員 ( 不分大小寫 ) ===")
51 result = df[df['student'].str.contains('T', case=False)]
52 print(result[['student_id', 'student', 'score', 'class']])
53 print()
54
55 print("=== 姓名以 'T' 開頭的學員 ===")
56 result = df[df['student'].str.startswith('T')]
57 print(result[['student_id', 'student', 'score', 'class']])
58

```

```

dataframe_05.py X
dataframe_05.py > ...
60 print("=== 依照姓名排序([A → Z]) ===")
61 sorted_by_name = df.sort_values(by='student')
62 print(sorted_by_name)
63
64
65 print("=== 依照姓名排序([Z → A]) ===")
66 sorted_by_name_desc = df.sort_values(by='student', ascending=False)
67 print(sorted_by_name_desc)
68
69 print("=== 依照成績 (高→低)、學號 (小→大) 排序 ===")
70 sorted_df = df.sort_values(
71     by=['score', 'student_id'],
72     ascending=[False, True]
73 )
74 print(sorted_df)
75
76 print("=== 依照成績、學號排序 (顯示重點欄位) ===")
77 sorted_df = df.sort_values(
78     by=['score', 'student_id'],
79     ascending=[False, True]
80 )
81 print(sorted_df[['student_id', 'student', 'class', 'score']])
82
83 #####

```

範例: dataframe_05.py

範例：

題目說明（學生任務）

題目 1：建立查詢介面（使用者輸入）

請讓使用者輸入以下查詢條件（皆可留空）：

- 姓名關鍵字
- 班級（A / B / C）
- 最低成績

題目 2：依條件進行資料篩選

請依照使用者輸入的條件，逐一篩選 DataFrame：

- 若姓名關鍵字不為空，使用「姓名包含」進行篩選（不分大小寫）
- 若班級不為空，依班級篩選
- 若最低成績不為空，篩選成績大於等於該值的學生

提醒：條件需 可自由組合

題目 3：顯示查詢結果

若查詢結果為空，顯示「查無符合條件的資料」

否則顯示以下欄位：

- 學號
- 姓名
- 班級
- 成績

```
dataframe_06.py X
dataframe_06.py 2 ...
1 import pandas as pd
2
3 # 讀取 CSV
4 df = pd.read_csv("../data/student_scores_100.csv")
5
6 print("=== 成績查詢系統 ===")
7
8 # 使用者輸入 (可留空)
9 name_kw = input("請輸入姓名關鍵字 (可空白) : ").strip()
10 class_kw = input("請輸入班級(A/B/C [可空白]) : ").strip()
11 min_score = input("請輸入最低成績 (可空白) : ").strip()
12
13 # 開始查詢
14 result = df
15
16 if name_kw != "":
17     result = result[result['student'].str.contains(name_kw, case=False, na=False)]
18
19 if class_kw != "":
20     result = result[result['class'] == class_kw]
21
22 if min_score != "":
23     result = result[result['score'] >= int(min_score)]
24
25 print("\n=== 查詢結果 ===")
26 if result.empty:
27     print("查無符合條件的資料")
28 else:
29     print(result[['student_id', 'student', 'class', 'score']])
```

範例: dataframe_06.py

2-4 Matplotlib

Web:

<https://matplotlib.org/stable/api/index.html>

Matplotlib 是 Python 中最常用、最基礎的資料視覺化 (Data Visualization) 套件之一，主要用來繪製各種 2D (也支援部分 3D) 圖表，例如折線圖、長條圖、圓餅圖、散佈圖等。

它常與 NumPy、Pandas、SciPy 搭配使用，在資料分析、科學計算、工程、教育與研究領域被廣泛應用。

常見圖表類型	
圖表類型	說明
折線圖 (Line Plot)	顯示資料隨時間或順序變化
長條圖 (Bar Chart)	比較不同類別的數值
圓餅圖 (Pie Chart)	顯示比例關係
散佈圖 (Scatter Plot)	顯示兩組數據的關係
直方圖 (Histogram)	分析資料分布情形

學生成績資料分析與視覺化例子：

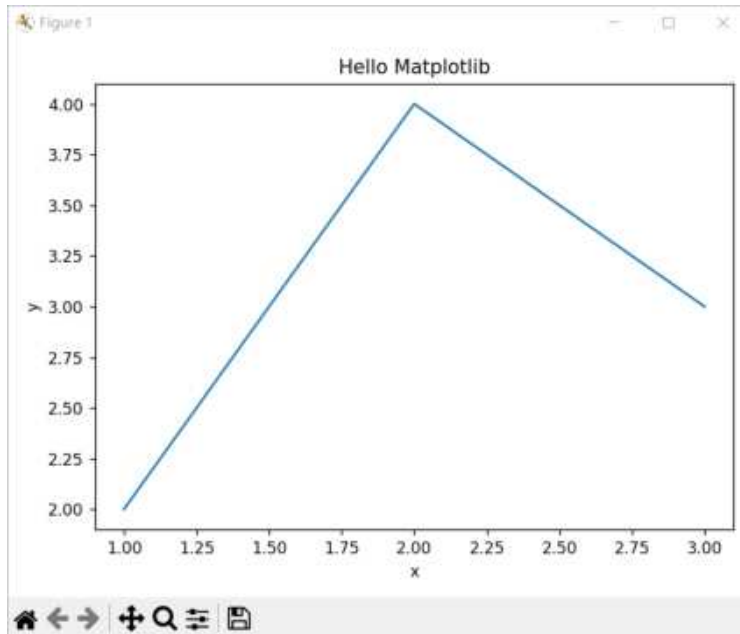
資料讀取與檢視

- 1、使用 Pandas 讀取 student_scores_100.csv 檔案。
- 2、顯示資料前 5 筆內容。
- 3、顯示資料的欄位名稱與資料型態。

➤ 折線圖

Matplotlib 基本操作

- 1、使用 Matplotlib 繪製一張簡單的折線圖。
 - 設定圖表標題為 Hello Matplotlib
 - 設定 x 軸與 y 軸標籤
 - 繪製資料點 [1,2,3] 與 [2,4,3]



```

matplotlib_01.py X matplotlib_01_2.py ESP-IDF: Search Error
matplotlib_01.py > ...
1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 df = pd.read_csv("./data/student_scores_100.csv") # 或 "student_scores_100.csv"
5 print(df.head())
6 print(df.info())
7
8 # 折線圖
9 fig, ax = plt.subplots()
10 ax.set_title("Hello Matplotlib")
11 ax.set_xlabel("x")
12 ax.set_ylabel("y")
13 ax.plot([1,2,3], [2,4,3])
14 plt.tight_layout()
15 # 存檔
16 plt.savefig("line_chart.png", dpi=300)
17 # 顯示
18 plt.show()

```

範例：matplotlib_01.py

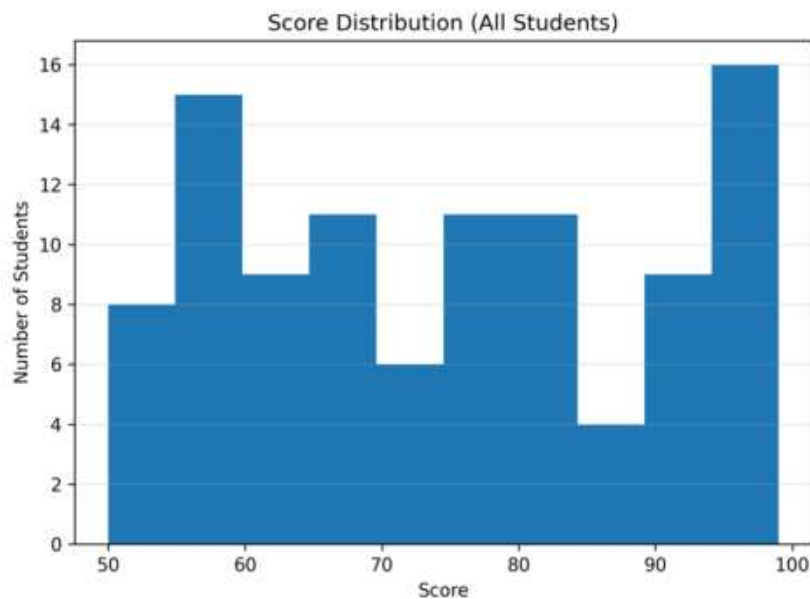
➤ 直方圖

二、用途差異 (最重要)		
項目	直方圖 Histogram	長條圖 Bar Chart
主要用途	看「資料分布情況」	比較「不同類別大小」
常見問題	成績集中在哪？高分多還是低分多？	哪一班平均最高？哪一班人最多？
資料類型	連續數值	類別資料

題目：全班成績分析

1、繪製「全班成績分布直方圖 (Histogram)」。

- 使用 score 欄位
- 設定分組數量為 10
- 顯示 y 軸格線



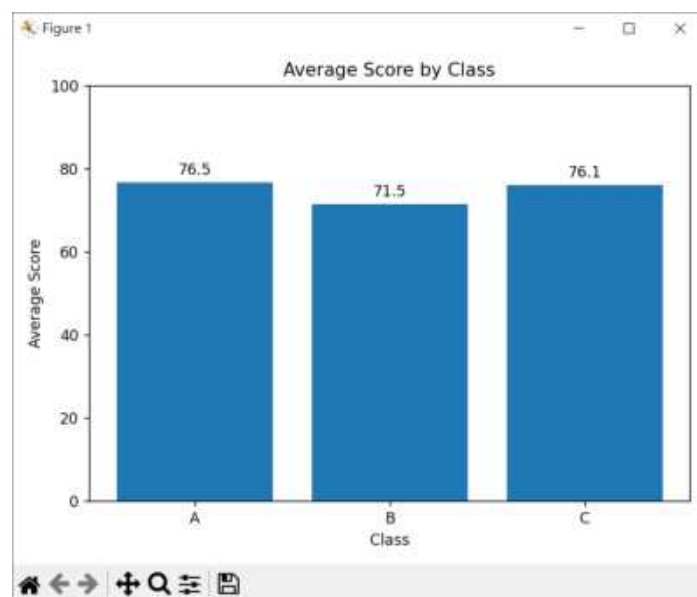
```
matplotlib_02.py X
matplotlib_02.py > ...
1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 df = pd.read_csv("./data/student_scores_100.csv") # 或 "student_scores_100.csv"
5 print(df.head())
6 print(df.info())
7
8 # 直方圖：看全班成績分布 (Histogram)
9 fig, ax = plt.subplots()
10 ax.hist(df["score"], bins=10)
11 ax.set_title("Score Distribution (All Students)")
12 ax.set_xlabel("Score")
13 ax.set_ylabel("Number of Students")
14 ax.grid(True, axis="y", alpha=0.3)
15 plt.tight_layout()
16 # 存檔
17 plt.savefig("./image/histogram.png", dpi=300)
18 # 顯示
19 plt.show()
```

範例：matplotlib_02.py

➤ 長條圖

題目、各班成績統計

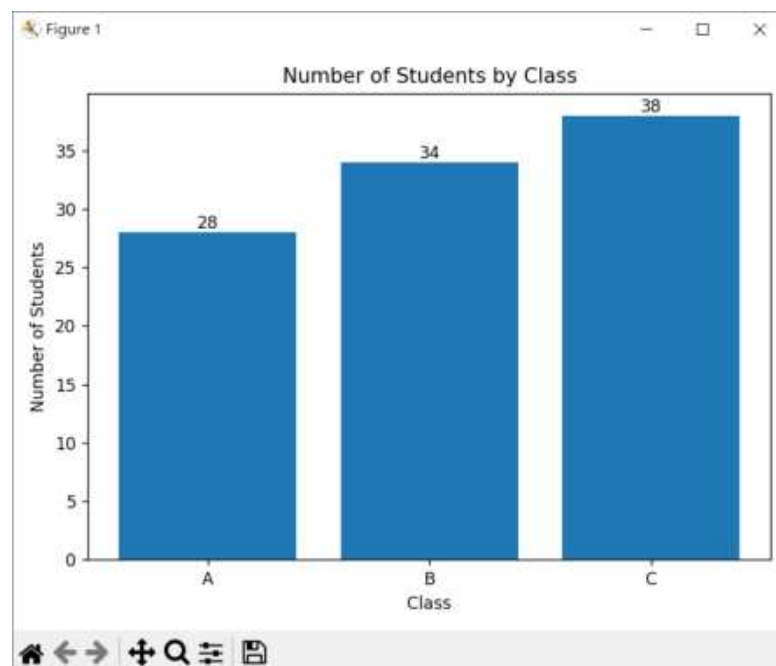
- 1、計算各班的平均成績。
- 2、繪製「各班平均成績長條圖（Bar Chart）」。
- 3、在每個長條上顯示平均分數
- 4、y 軸範圍設定為 0~100
- 5、計算各班學生人數。
- 6、繪製「各班學生人數長條圖」。
- 7、在每個長條上顯示學生人數



```

matplotlib_03.py X
matplotlib_03.py > ...
1  import pandas as pd
2  import matplotlib.pyplot as plt
3
4  df = pd.read_csv("../data/student_scores_100.csv") # 或 "student_scores_100.csv"
5  print(df.head())
6  print(df.info())
7
8  # 各班平均分數長條圖 (Bar chart)
9  mean_by_class = df.groupby("class")["score"].mean().sort_index()
10 # print(mean_by_class)
11 print(mean_by_class.index)
12 print(mean_by_class.values)
13
14 # 建立圖表畫布，建立一個圖表 (figure) 與一個座標軸 (axes)
15 # 之後所有圖形都畫在 ax 上
16 fig, ax = plt.subplots()
17 bars = ax.bar(mean_by_class.index, mean_by_class.values)
18 ax.set_title("Average Score by Class")
19 ax.set_xlabel("Class")
20 ax.set_ylabel("Average Score")
21 ax.bar_label(bars, fmt="%.1f", padding=3) # 在每個長條上顯示數值
22
23 # 在每個長條上顯示數值
24 # for b in bars:
25 #     ax.text(b.get_x() + b.get_width()/2, b.get_height(),
26 #             f"{b.get_height():.1f}", ha="center", va="bottom")
27
28 # # 固定 y 軸範圍
29 ax.set_ylim(0, 100)
30 plt.tight_layout()
31 plt.savefig("../image/bar_chart.png", dpi=300)
32 plt.show()

```



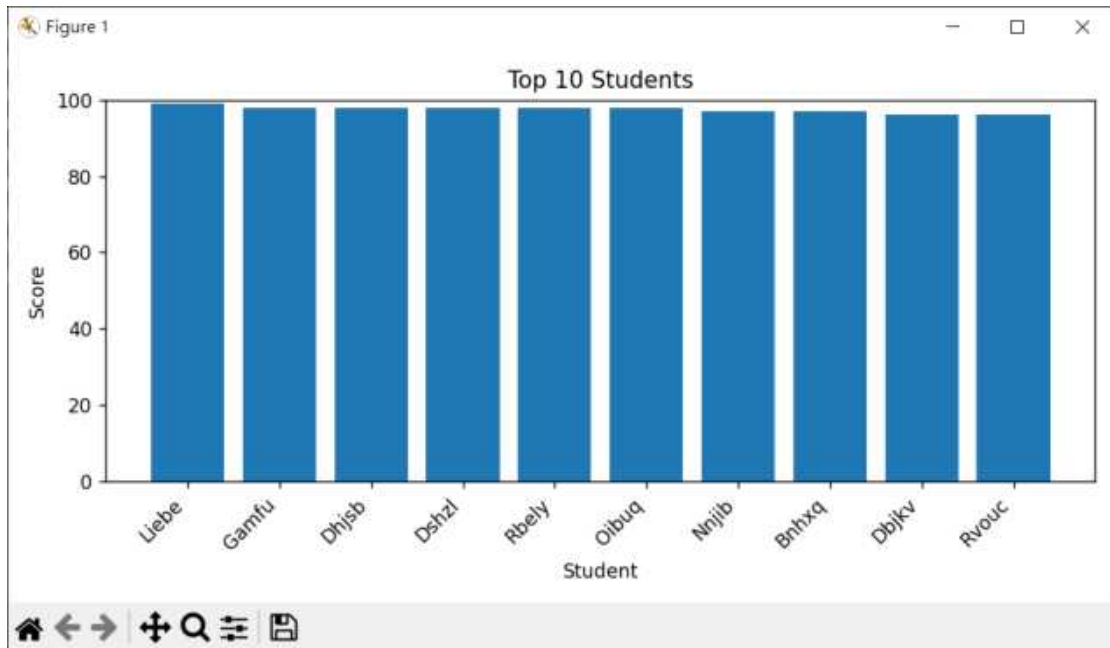
```
matplotlib_03.py X
matplotlib_03.py > ...
34
35 #####
36 # 各班人數長條圖 (Bar chart)
37 count_by_class = df["class"].value_counts().sort_index()
38
39 fig, ax = plt.subplots()
40 bars = ax.bar(count_by_class.index, count_by_class.values)
41 ax.set_title("Number of Students by Class")
42 ax.set_xlabel("Class")
43 ax.set_ylabel("Number of Students")
44
45 # 在每個長條上顯示人數
46 for b in bars:
47     ax.text(
48         b.get_x() + b.get_width() / 2,
49         b.get_height(),
50         f"{int(b.get_height())}",
51         ha="center",
52         va="bottom"
53     )
54
55 plt.tight_layout()
56 plt.savefig("./image/bar_chart_count.png", dpi=300)
57 plt.show()
```

範例：matplotlib_03.py

➤ 長條圖

題目：個人成績表現

- 1、依照成績由高到低排序資料。
- 2、取出成績最高的前 10 位學生。
- 3、繪製「前 10 名學生的成績長條圖」。
- 4、設定 y 軸範圍為 0~100
- 5、將學生名稱旋轉 45 度顯示



```

matplotlib_04.py X
matplotlib_04.py > ...
1  import pandas as pd
2  import matplotlib.pyplot as plt
3
4  df = pd.read_csv("../data/student_scores_100.csv") # 或 "student_scores_100.csv"
5  print(df.head())
6  print(df.info())
7
8  # 個人成績呈現：排序後長條圖 (Top/Bottom)
9  top10 = df.sort_values("score", ascending=False).head(10)
10
11 fig, ax = plt.subplots(figsize=(8,4))
12 ax.bar(top10["student"], top10["score"])
13 ax.set_title("Top 10 Students")
14 ax.set_xlabel("Student")
15 ax.set_ylabel("Score")
16 ax.set_ylim(0, 100)
17 plt.xticks(rotation=45, ha="right")
18 plt.tight_layout()
19 plt.savefig("../image/top10_students.png", dpi=300)
20 plt.show()

```

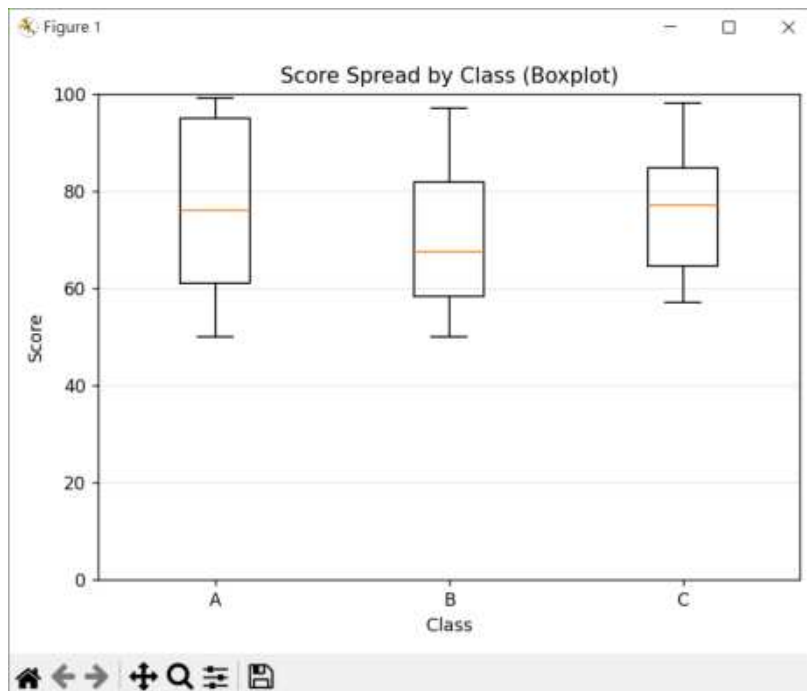
範例：matplotlib_04.py

➤ 箱型圖

題目：各班成績分布比較

- 1、依班級分類成績資料。
 - 2、繪製「各班成績箱型圖 (Boxplot)」。
- 顯示中位數、四分位數與離群值

- y 軸範圍設定為 0~100
- 顯示 y 軸格線



```

matplotlib_05.py X
matplotlib_05.py > ...
1  import pandas as pd
2  import matplotlib.pyplot as plt
3
4  df = pd.read_csv("../data/student_scores_100.csv") # 或 "student_s
5  print(df.head())
6  print(df.info())
7
8  classes = sorted(df["class"].unique())
9  data = [df.loc[df["class"] == c, "score"] for c in classes]
10
11 fig, ax = plt.subplots()
12 ax.boxplot(data, tick_labels=classes)
13 ax.set_title("Score Spread by Class (Boxplot)")
14 ax.set_xlabel("Class")
15 ax.set_ylabel("Score")
16 ax.set_ylim(0, 100)
17 ax.grid(True, axis="y", alpha=0.3)
18 plt.tight_layout()
19 plt.savefig("../image/boxplot_by_class.png", dpi=300)
20 plt.show()

```

範例：matplotlib_05.py

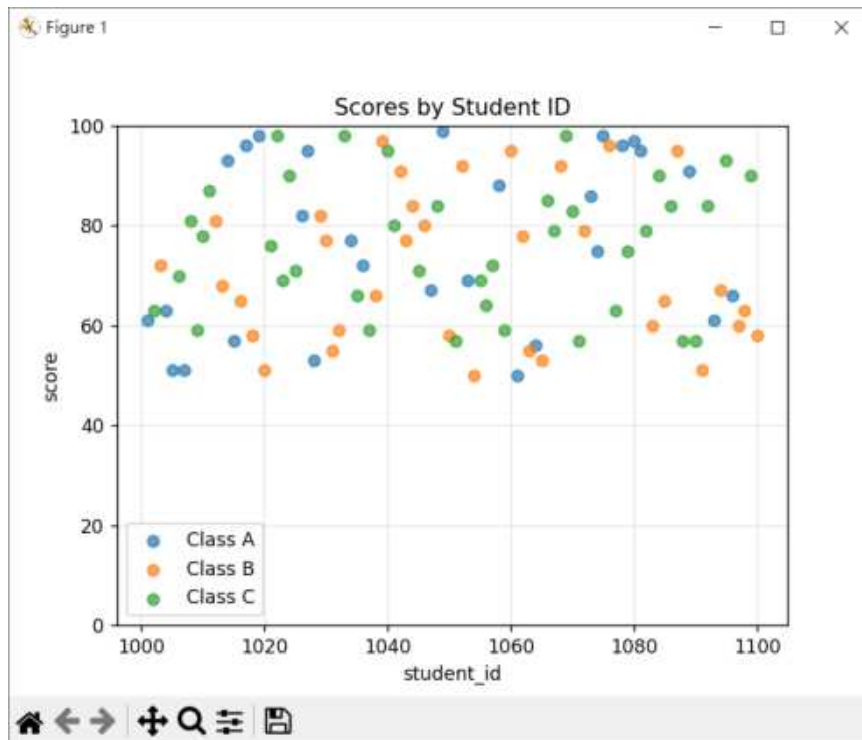
➤ 散點圖

題目：成績與學生編號關係

1、使用 student_id 作為 x 軸。

2、繪製「各班學生編號與成績散點圖 (Scatter)」。

- 不同班級以不同標記顯示
- 顯示圖例 (Legend)



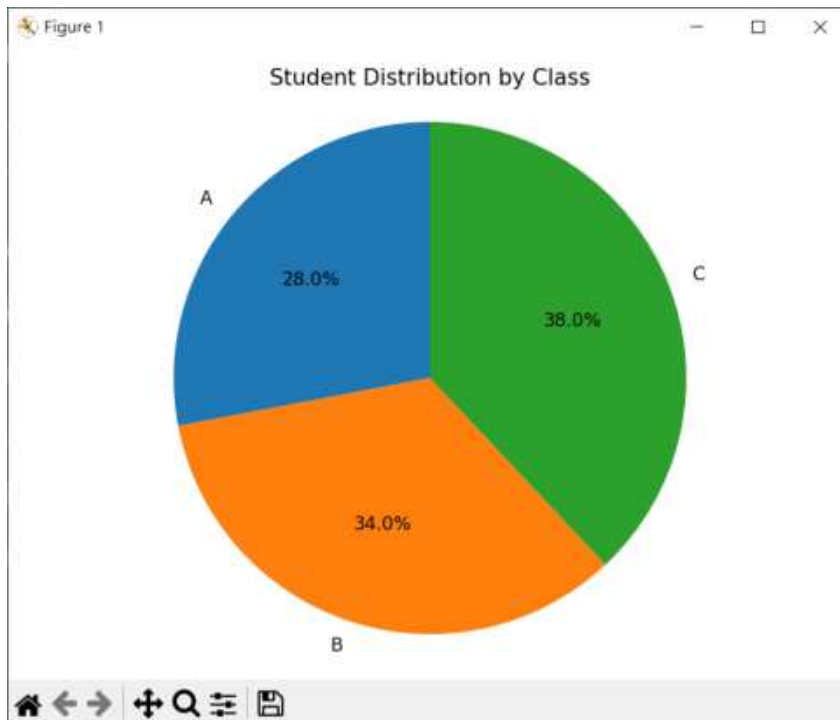
```
matplotlib_06.py X
matplotlib_06.py > ...
1  import pandas as pd
2  import matplotlib.pyplot as plt
3
4  df = pd.read_csv("../data/student_scores_100.csv") # 或 "student_scores_100.csv"
5  print(df.head())
6  print(df.info())
7
8  # 散點圖：用 student_id 當 x (Scatter)
9  fig, ax = plt.subplots()
10
11  for c in sorted(df["class"].unique()):
12      sub = df[df["class"] == c]
13      ax.scatter(sub["student_id"], sub["score"], label=f"Class {c}", alpha=0.7)
14
15  ax.set_title("Scores by Student ID")
16  ax.set_xlabel("student_id")
17  ax.set_ylabel("score")
18  ax.set_ylim(0, 100)
19  ax.legend()
20  ax.grid(True, alpha=0.3)
21  plt.savefig("../image/scatter_by_student_id.png", dpi=300)
22  plt.show()
```

範例：matplotlib_06.py

➤ 圓餅圖

題目：班級人數比例分析

- 1、計算各班學生人數比例。
- 2、繪製「各班學生人數圓餅圖（Pie Chart）」。
 - 顯示百分比
 - 從正上方開始繪製
 - 圖形呈現為正圓



```

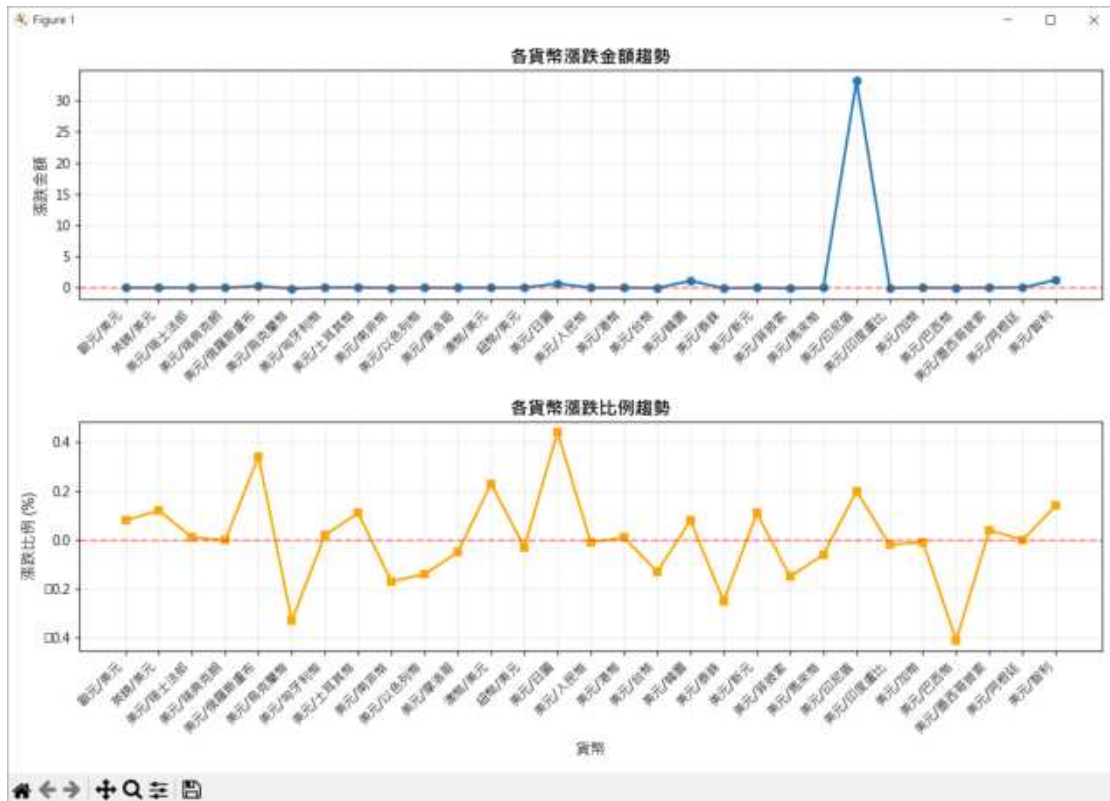
matplotlib_07.py X
matplotlib_07.py > ...
1  import pandas as pd
2  import matplotlib.pyplot as plt
3
4  df = pd.read_csv("../data/student_scores_100.csv") # 或 "student_scores_100.csv"
5  print(df.head())
6  print(df.info())
7
8  # 圓餅圖：各班學生人數比例 (Pie chart)
9
10 count_by_class = df["class"].value_counts().sort_index()
11
12 fig, ax = plt.subplots()
13 ax.pie(
14     count_by_class.values,
15     labels=count_by_class.index,
16     autopct="%1.1f%%", # 顯示百分比
17     startangle=90      # 從正上方開始
18 )
19
20 ax.set_title("Student Distribution by Class")
21 ax.axis("equal") # 讓圓餅圖變成正圓
22 plt.tight_layout()
23 plt.savefig("../image/pie_by_class.png", dpi=300)
24 plt.show()

```

範例：matplotlib_07.py

小專題：

請使用 Python 的 pandas 與 matplotlib 套件，從網路匯率網站抓取全球貨幣即時匯率資料，並分析各貨幣的漲跌情況，最後以折線圖方式呈現貨幣漲跌趨勢。



1. 匯率資料擷取

請使用 `pandas.read_html()` 從下列網址讀取全球貨幣匯率表格：

<http://www.stockq.org/market/currency.php>

2. 資料整理與清理

取得匯率表格後，請完成以下處理：

- 選取正確的匯率資料表
- 移除表格前兩列無效資料
- 修改欄位名稱為：
■ | 貨幣 | 匯率 | 漲跌 | 比例 | 台北 |
- 將「漲跌」與「比例」欄位轉換為數值型態

3. 無效資料過濾

請將無法轉換為數字的資料移除，確保後續分析正確。

4. 資料視覺化

請繪製兩張折線圖（上下排列）：

(1) 各貨幣漲跌金額折線圖

- X 軸：貨幣名稱
- Y 軸：漲跌金額
- 加入 0 基準線（紅色虛線）

(2) 各貨幣漲跌比例折線圖

- X 軸：貨幣名稱
- Y 軸：漲跌比例 (%)
- 加入 0 基準線（紅色虛線）

5. 圖表格式要求

- 圖表需顯示中文（微軟正黑體）
- X 軸標籤需旋轉避免重疊
- 圖表需顯示格線
- 使用 `tight_layout()` 排版

範例：`matplotlib_08.py`

2-5 DataFrame 合併

➤ 檔名與欄位

檔名:aisles.csv

欄位:aisle_id、aisle

檔名:departments.csv

欄位:department_id、department

檔名:order_products__prior.csv

欄位:order_id product_id、add_to_cart_order、reordered

檔名:orders.csv

欄位:order_id、user_id、eval_set、order_number、order_dow、order_hour_of_day、days_since_prior_order

檔名:products.csv

欄位:product_id、product_name、aisle_id、department_id

➤ 說明每個檔案的用途：

檔名:aisles.csv

1. aisle_id:貨架編號
2. aisle:貨架名稱

此檔案提供每個商品所屬的「走道(aisle)分類」名稱，通常用於：分析商品在商店中屬於哪個走道商品分類統計與 products.csv 合併使用（透過 aisle_id）。

=====

檔名:departments.csv

1. department_id:部門編號
2. department:部門名稱

這個檔案提供 每個商品所屬的大分類（department），常用於：探索不同部門的銷售量依部門分析使用者購物偏好與 products.csv 合併使用（透過 department_id）。

=====

檔名:order_products__prior.csv (訂單中每個商品的明細)

1. order_id: 訂單編號
2. product_id: 商品編號
3. add_to_cart_order: 加入購物車順序
4. reordered: 是否為再次購買

「每張訂單有哪些商品、」

這是 Instacart 資料集中最重要的檔案之一，用來描述：每張訂單有哪些商品商品在購物車中加入的順序（購物行為分析）使用者是否習慣重複購買某商品可用於建購物推薦模型、購買行為分析它通常會與：products.csv（商品資料）、orders.csv（訂單資料）合併來使用。

=====

檔名:orders.csv (訂單的主檔)

1. order_id: 訂單編號
2. user_id: 使用者編號
3. eval_set: 訓練/測試集類別(使用者過去所有歷史訂單、被選來訓練模型的訂單、需要模型預測商品的訂單)
4. order_number: 使用者的第幾次訂單
5. order_dow: 星期幾下單
6. order_hour_of_day: 下單時間（小時）
7. days_since_prior_order: 與前一次下單間隔天數

orders.csv 的筆數一定比 order_products__prior.csv 多，orders.csv 包含所有使用者的全部訂單（prior + train + test），而 order_products__prior.csv 只有 prior 訂單的商品明細。

「記錄每筆訂單的屬性與使用者購物習慣，如每個 user 的購物週期」

用途：此檔案記錄 每筆訂單的屬性與使用者購物習慣：每個 user 的購物週期下單時間分析（例如：週末是否較多人購物）訂單排序（使用者的第幾次購物）與 order_products__prior.csv 合併，取得 user 購買哪些商品

=====

檔名:products.csv

1. product_id: 商品編號
2. product_name: 商品名稱
3. aisle_id: 貨架分類編號
4. department_id: 部門分類編號

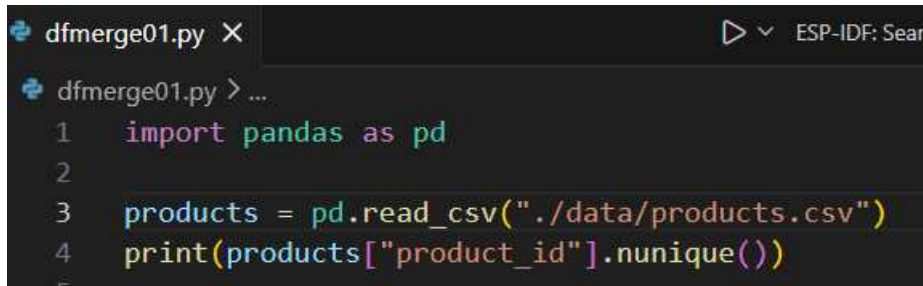
用途：

此檔案提供每個商品的完整資訊：

商品名稱、商品分類（走道、部門）與 aisles.csv、departments.csv 合併以取得完整分類資訊與 order_products__prior.csv 合併分析哪些商品比較熱門

➤ 第一部分：一個資料表分析

題目 1：計算 **products.csv** 中共有多少種商品。



```
dfmerge01.py X
dfmerge01.py > ...
1 import pandas as pd
2
3 products = pd.read_csv("./data/products.csv")
4 print(products["product_id"].nunique())
```

範例: dfmerge-part1-01.py

題目 2：計算 **departments.csv** 中共有多少個部門。

範例: dfmerge-part1-01.py

題目 3：計算 **aisles.csv** 中共有多少個 aisle。

範例: dfmerge-part1-01.py

題目 4：統計 **orders.csv** 中共有多少筆訂單。

範例: dfmerge-part1-01.py

題目 5：統計 不同 eval_set 的訂單數量(eval_set 可能包含 train / test / prior)

繪製：

長條圖 (Bar Chart)

X 軸：eval_set

Y 軸：訂單數量

範例: dfmerge-part1-01.py

題目 6：分析 order_hour_of_day

一天中哪個時間點訂單最多、繪製長條圖、X 軸 0~23 小時、Y 軸訂單數。

範例: dfmerge-part1-02.py

題目 7：分析 order_dow

一週中哪一天訂單最多、繪製長條圖、X 軸：0~6 (星期)、Y 軸：訂單數。

範例: dfmerge-part1-02.py

題目 8：分析 days_since_prior_order

顧客通常隔幾天購買一次、繪製：Histogram (直方圖)、X 軸：days_since_prior_order

Y 軸：次數

範例: dfmerge-part1-02.py

題目 9：統計 reordered

商品是否為回購商品、繪製：圓餅圖、0 = 第一次購買、1 = 回購

範例: dfmerge-part1-02.py

➤ 第二部分：多個資料表分析

題目 1：建立商品完整資料表

情境：

某線上超市希望建立一份完整的商品主檔，讓管理人員可以清楚知道每一項商品屬於哪一個貨架與哪一個部門。

目的：

請將 products.csv、aisles.csv、departments.csv 合併成一張新資料表，內容需包含：

product_id

product_name

aisle

department

延伸要求：

請統計每個 department 的商品數量。

畫圖要求：

請繪製「各 department 商品數量長條圖」。

範例: dfmerge-part2-01.py

題目 2：查詢訂單中的商品明細

情境：

超市想知道每一筆訂單裡實際買了哪些商品，並了解商品加入購物車的順序。

目的：

請將 order_products__prior.csv 與 products.csv 合併，建立訂單商品明細表，內容需包含：

order_id

product_name

add_to_cart_order

reordered

延伸要求：

請找出每筆訂單中最先加入購物車的商品。

畫圖要求：

請繪製「商品加入購物車順序分布圖」或「前 10 名最常第一個加入購物車的商品長條圖」。

範例: dfmerge-part2-01.py

題目 3：建立使用者購物紀錄表

情境：

行銷部門想分析每位顧客的購物行為，因此需要將訂單資料與訂單商品資料整合。

目的：

請將 orders.csv 與 order_products__prior.csv 合併，建立使用者購物紀錄表，內容需包含：

user_id

order_id

product_id

order_number

order_dow

order_hour_of_day

延伸要求：

請統計每位使用者的總購買次數。

畫圖要求：

請繪製「不同使用者的訂單數量分布圖」或「前 10 名訂單最多的使用者長條圖」。

範例: dfmerge-part2-02.py

題目 4：分析顧客最常購買的商品

情境：

超市希望了解哪些商品最受歡迎，以便安排促銷活動與庫存補貨。

目的：

請合併 orders.csv、order_products__prior.csv、products.csv，找出被購買次數最多的前 10 項商品。

輸出欄位建議：

product_name

purchase_count

畫圖要求：

請繪製「前 10 名熱銷商品長條圖」。

範例: dfmerge-part2-02.py

題目 5：分析最常被再次購買的商品

情境：

商品若經常被再次購買，代表顧客忠誠度高，適合作為主力商品。

目的：

請合併 order_products__prior.csv 與 products.csv，找出 reordered = 1 次數最多的前 10 項商品。

輸出欄位建議：

product_name

reordered_count

畫圖要求：

請繪製「前 10 名再次購買商品長條圖」。

範例: dfmerge-part2-03.py

題目 6：分析各部門的銷售情況

情境：

管理者想了解哪一個部門的商品最常被購買，作為商品陳列與採購依據。

目的：

請合併以下資料表：

order_products__prior.csv

products.csv

departments.csv

計算各 department 的商品被購買總次數。

輸出欄位建議：

department

sales_count

畫圖要求：

請繪製「各 department 銷售次數長條圖」或「圓餅圖」。

範例: dfmerge-part2-03.py

題目 7：分析不同星期幾的購物情形

情境：

超市想知道顧客最常在哪一天購物，以安排人力與促銷時段。

目的：

請合併 orders.csv 與 order_products__prior.csv，統計不同 order_dow 的訂單數量或購買商品數量。

延伸要求：

可再加入 products.csv、departments.csv，找出不同星期最常購買的商品部門。

畫圖要求：

請繪製：

「星期幾 vs 訂單數量」長條圖

與「星期幾 vs 購買商品數量」折線圖

範例: dfmerge-part2-04.py

題目 8：分析一天中哪個時段最常下單

情境：

營運團隊想知道顧客通常在幾點下單，以安排客服與物流資源。

目的：

請使用 orders.csv，統計每個 order_hour_of_day 的訂單數量。

延伸要求：

再合併 order_products__prior.csv，分析每個時段的商品購買總量。

畫圖要求：

請繪製「下單時段分布折線圖」或「長條圖」。

範例: dfmerge-part2-04.py

題目 9：分析各 aisle 的熱門商品

情境：

超市陳列人員想知道每一個貨架中最受歡迎的商品，以調整擺設位置。

目的：

請合併：

order_products__prior.csv

products.csv

aisles.csv

統計每個 aisle 中購買次數最高的商品。

輸出欄位建議：

aisle

product_name

purchase_count

畫圖要求：

請選出前 10 個 aisle，繪製「各 aisle 熱門商品銷量比較圖」。

範例: dfmerge-part2-05.py

題目 10：建立完整顧客購物分析資料表

情境：

資料分析師要建立一份完整的購物分析資料，提供給行銷、營運與商品部門共同使用。

目的：

請將以下五個資料表全部合併：

orders.csv

order_products__prior.csv

products.csv

aisles.csv

departments.csv

建立完整分析表，至少包含：

user_id

order_id

order_number

order_dow

order_hour_of_day

product_name

aisle

department

reordered

延伸要求：

利用此完整資料表，自行提出至少 4 個分析問題並完成統計。

畫圖要求：

製作 4 張圖，例如：

熱銷商品圖

部門銷售圖

下單時段圖

再購商品圖

範例: dfmerge-part2-05.py

第三部分：KMeans + PCA

3.1 機器學習流程(非監督式學習(Unsupervised Learning))

取得資料→資料前處理→特徵選擇→資料標準化→PCA 降維→K-Means 分群
→尋找最佳 K 值→模型訓練→分群結果分析→PCA 2D 視覺化

➤ 取得資料

1. 取得資料

首先準備一份資料集。

例如：

學生	Python	Java	SQL	Linux	網頁
A	90	85	80	75	90
B	30	40	35	30	45
C	80	90	85	88	82
D	40	35	50	45	38

這裡的：

- Python
- Java
- SQL
- Linux
- 網頁

就是特徵 (Features) 。

➤ 資料前處理

2. 資料前處理

檢查資料是否有：

- 缺失值
- 異常值
- 重複資料
- 不合理資料

例如：

```
Python = 95  
Java = 88  
SQL = 缺失
```

需要先處理缺失值，才能進行後續機器學習。

➤ 特徵選擇

3. 資料標準化

這一步對 PCA 與 K-Means 都非常重要。

例如：

年收入：30,000 ~ 1,000,000
 年齡：20 ~ 70
 消費次數：1 ~ 100

不同特徵的尺度差異很大。

因此通常使用：

```
Python
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

標準化後，各特徵大致會轉換成：

平均值 = 0
 標準差 = 1

例如：

實例：學生的「身高、體重、年收入」

假設有 5 個人的資料：

學生	身高(cm)	體重(kg)	年收入(萬元)
A	160	50	30
B	165	55	40
C	170	60	50
D	175	65	60
E	180	70	70

可以看到：

- 身高：160~180
- 體重：50~70
- 年收入：30~70

雖然三個特徵都是數字，但尺度不同，如果直接做 PCA，年收入可能因為數值範圍較大而影響 PCA 結果。

1. 先計算平均值

例如身高：

$$\mu_{\text{身高}} = \frac{160 + 165 + 170 + 175 + 180}{5} = 170$$

體重：

$$\mu_{\text{體重}} = 60$$

年收入：

$$\mu_{\text{收入}} = 50$$

所以平均值：

身高 = 170
體重 = 60
年收入 = 50



② 每個數值減掉平均值

身高 X	$X - \mu$
160	-10
165	-5
170	0
175	5
180	10

③ 將差值平方

身高 X	$X - 170$	$(X - 170)^2$
160	-10	100
165	-5	25
170	0	0
175	5	25
180	10	100

平方加總：

$$100 + 25 + 0 + 25 + 100 = 250$$

④ 除以資料筆數 5

因為 `StandardScaler` 使用母體標準差：

$$\frac{250}{5} = 50$$

這個就是變異數 (Variance) 。

⑤ 開根號得到標準差

$$\sigma = \sqrt{50}$$

$\sigma \approx 7.071$

所以身高的標準差就是：

7.071 cm

特徵	資料	平均值	標準差 
身高	160,165,170,175,180	170	7.071
體重	50,55,60,65,70	60	7.071
年收入	30,40,50,60,70	50	14.142

➤ PCA 降維

三、PCA 降維

接下來進入你的主題第一個核心：

PCA (Principal Component Analysis , 主成分分析)

假設原本有：

10 個特徵

透過 PCA 可以將它轉換成：

PC1
PC2
PC3
...

例如：

原始資料
10 個維度
↓
PCA
↓
2 個維度
PC1、PC2

程式：

```
</> Python  
  
from sklearn.decomposition import PCA  
  
pca = PCA(n_components=2)  
  
X_pca = pca.fit_transform(X_scaled)
```

最後：

X_pca
├─ PC1
└─ PC2

就可以拿來畫 2D 圖。

假設資料有：

```
Python
Java
C++
SQL
Linux
HTML
CSS
JavaScript
Vue
Django
```

總共有 10 個維度。

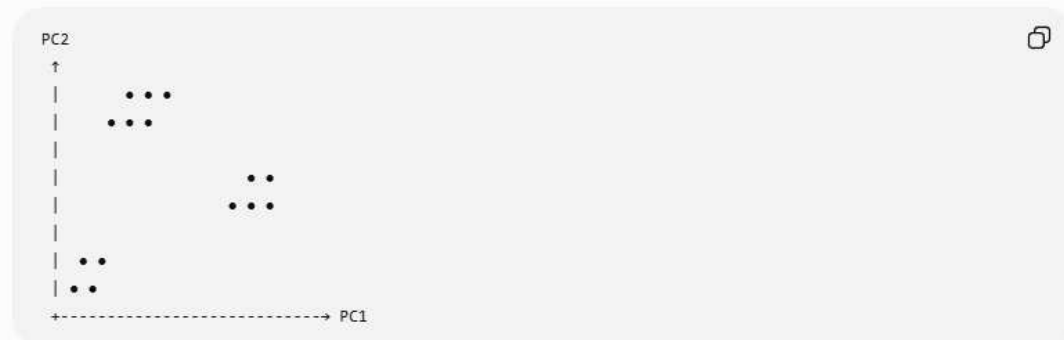
我們很難直接畫出 10 維資料。

PCA 可以將：

```
10維
↓
2維
```

讓我們可以在平面圖上觀察資料。

例如：



因此：

PCA 主要是為了降低資料維度，同時盡量保留資料的重要資訊。

➤ **K-Means 分群** ➔ **尋找最佳 K 值** ➔ **分群結果分析** ➔ **PCA 2D 視覺化**

五、K-Means 分群

PCA 完成後，進入第二個核心：

K-Means Clustering

K-Means 是一種非監督式學習（Unsupervised Learning）。

它不需要事先知道答案。

例如我們不知道學生可以分成：

第1群：Python能力強
第2群：Java能力強
第3群：整體能力較弱



K-Means 可以自行找出資料中的群組。

六、K-Means 如何運作？

假設：

K = 3



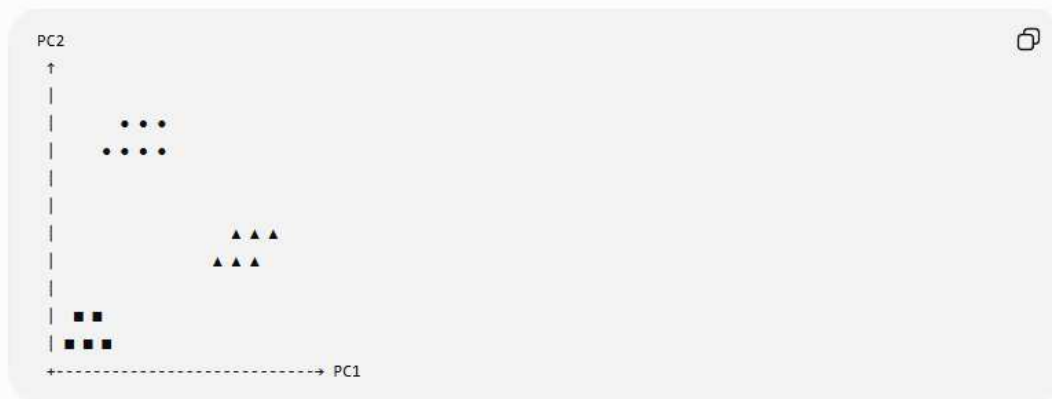
代表我們希望將資料分成 3 群。

K-Means 大致流程：

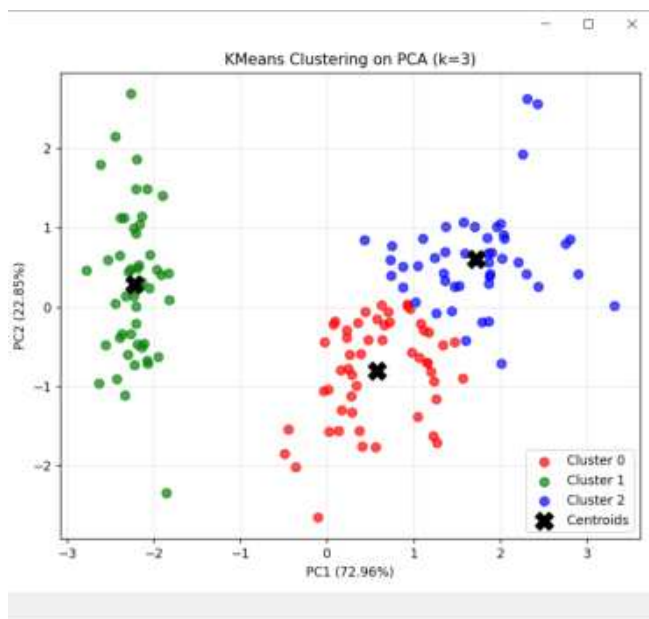
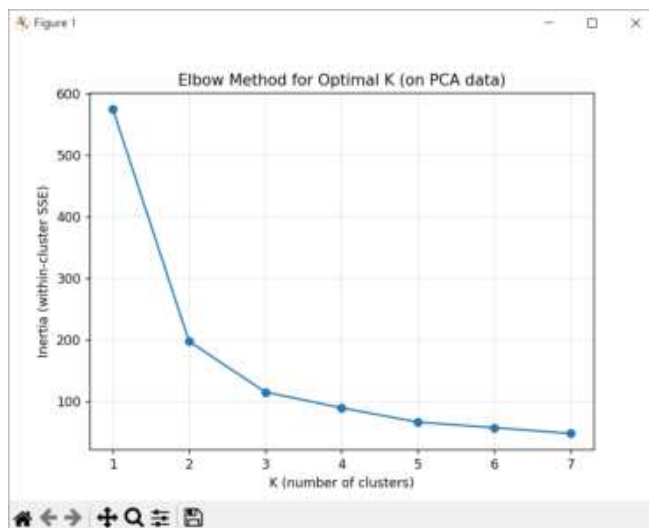
- ① 決定 K = 3
- ↓
- ② 隨機產生 3 個中心點
- ↓
- ③ 計算每個資料點距離哪個中心最近
- ↓
- ④ 將資料分配到最近的群
- ↓
- ⑤ 重新計算每一群的中心
- ↓
- ⑥ 重複 ③~⑤
- ↓
- ⑦ 中心點不再明顯變化
- ↓
- 分群完成



最後可能看到：



不同符號 / 顏色代表不同群組。



3.2 主成分分析(Principal Component Analysis , PCA)

➤ 目的

- A. 抑制雜訊
- B. 降低推論時間與降低記憶體容量

```
# 3. 資料前處理
# 處理缺失值
# 移除 user_id
# 使用 StandardScaler 對特徵進行標準化

scaler = StandardScaler()
X_scaled = scaler.fit_transform(user_features)

#####
# 4. PCA 降維
# 使用 PCA 將資料降為 2 維
# 輸出 PC1 與 PC2 的解釋變異比例
# 簡單說明 PC1、PC2 主要代表的購物特徵

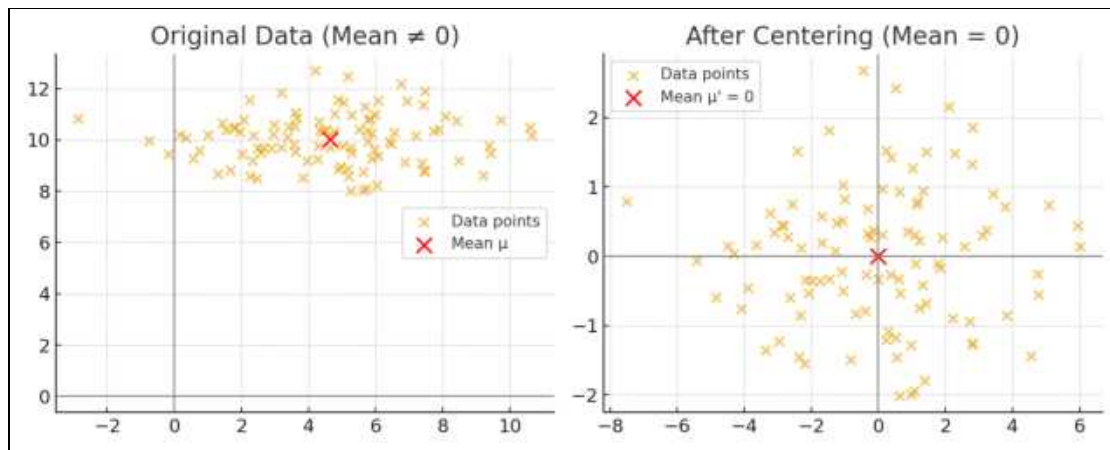
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
```

➤ 步驟如下

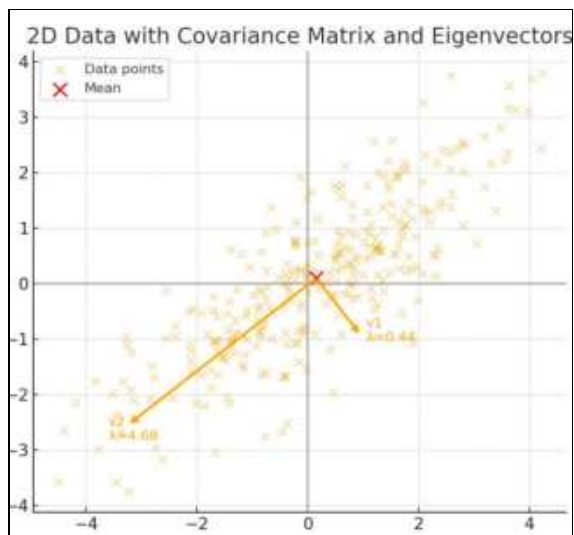
1. 中心化:
 - 將每個特徵減去該特徵的平均值，使資料的平均值變為 0
2. 計算協方差矩陣/共變異數矩陣:
 - 計算所有特徵兩兩之間的共變異程度
3. 特徵分解
 - 對協方差矩陣做：特徵值與特徵向量分解
4. 特徵值排序 (Sort Eigenvalues)
 - 選擇前 k 個主成分 (選擇特徵向量)
5. 投影 (Projection)
 - 將原始資料投影到所選的主成分空間

➤ 圖形說明

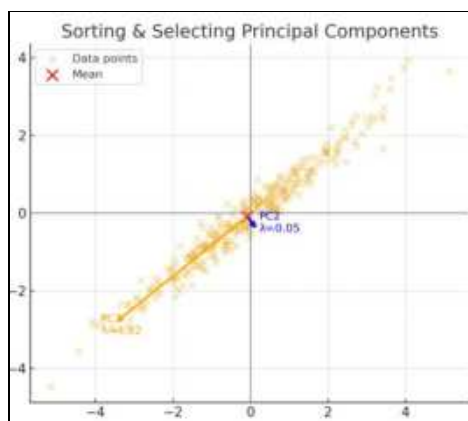
中心化：



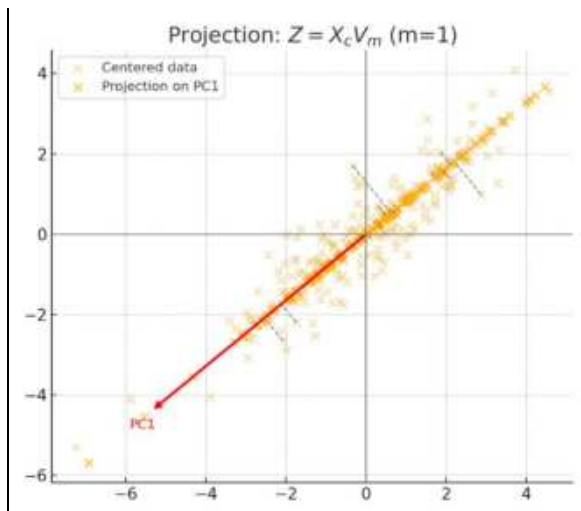
計算協方差矩陣後，找到特徵值與特徵向量



特徵值大小排序



投影後的結果



➤ 細部說明如下：

● 中心化:

將每個特徵減去該特徵的平均值，使資料的平均值變為 0

尋問 ChatGPT:

1. 中心化:
將每個特徵減去該特徵的平均值，使資料的平均值變為 0
請說明與範例?

「中心化 (Centering)」是資料前處理中很常見的步驟，特別是在：

- 機器學習
- PCA (主成分分析)
- 線性回歸
- 統計分析

都會先做中心化。

一、什麼是中心化？

中心化的意思：

將每個資料減去「該欄位的平均值」

讓資料的平均數變成 0。

公式：

$$x' = x - \bar{x}$$

其中：

- x ：原始資料
- $\bar{x}$ ：平均值
- x' ：中心化後的資料

這個公式可視覺化如下：

$$x' = x - \bar{x}$$

二、簡單範例

假設有一組學生身高：

學生	身高(cm)
A	160
B	170
C	180

Step 1：計算平均值

平均：

$$\bar{x} = \frac{160 + 170 + 180}{3}$$

$$\bar{x} = \frac{160+170+180}{3} = 170$$

所以平均值 = 170

Step 2：每個值減去平均

$$160 - 170 = -10$$

$$170 - 170 = 0$$

$$180 - 170 = 10$$

中心化後結果

學生	原始身高	中心化後
A	160	-10
B	170	0
C	180	10

三、驗證平均值是否為 0

新的平均：

$$\frac{-10 + 0 + 10}{3} = 0$$

$$\frac{-10+0+10}{3} = 0$$

所以：

✅ 中心化後資料平均值會變成 0

四、幾何上的意義

中心化其實就是：

把整個資料「平移」

讓資料以「0」為中心。

原本：

160 170 180



中心化後：

-10 0 10



資料形狀不變，只是移動位置。

NumPy 寫法

```
import numpy as np
x = np.array([160,170,180])
mean = np.mean(x)
centered = x - mean
print(centered)
```

七、二維資料範例

假設：

身高	體重
160	50
170	60
180	70

每一欄都要各自減平均：

身高平均：

$$\frac{160 + 170 + 180}{3} = 170$$

體重平均：

$$\frac{50 + 60 + 70}{3} = 60$$

中心化後：

身高	體重
-10	-10
0	0
10	10

八、中心化 vs 標準化

很多人會混淆：

方法	做法
中心化	減平均值
標準化	減平均後再除以標準差

標準化公式：

$$z = \frac{x - \bar{x}}{\sigma}$$

$$z = \frac{x - \bar{x}}{\sigma}$$

一、最核心差異

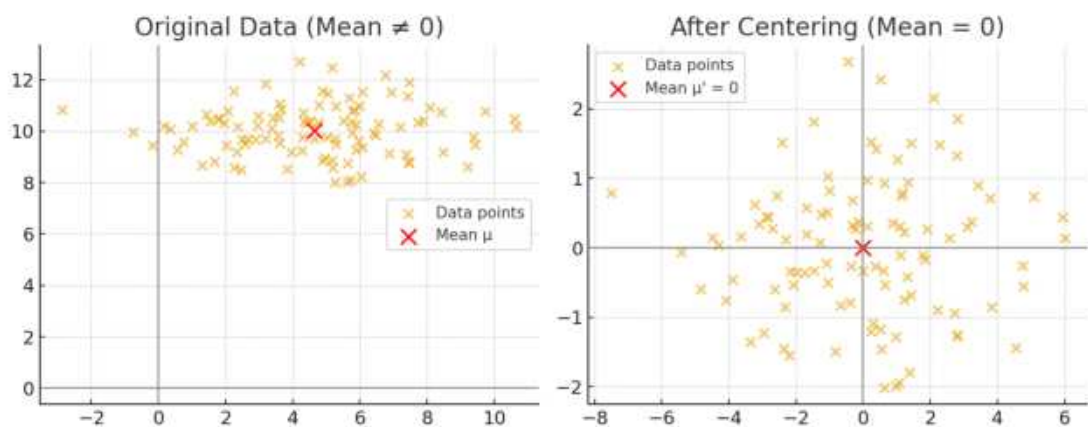
方法	做什麼	結果
中心化	減去平均值	平均變成 0
標準化	減平均再除標準差	平均=0，標準差=1

八、最大的差別

比較	中心化	標準化
平均值	0	0
標準差	不變	1
改變尺度	不會	會
改變分布寬度	不會	會
是否縮放	否	是

生成式 AI 範例: 中心化概念與範例.mhtml

中心化(前與後):



➤ 計算協方差矩陣/共變異數矩陣(Covariance Matrix):

$$\text{cov}(X, Y) = \begin{bmatrix} \text{cov}(x_1, y_1) & \text{cov}(x_1, y_2) & \cdots & \text{cov}(x_1, y_n) \\ \text{cov}(x_2, y_1) & \text{cov}(x_2, y_2) & \cdots & \text{cov}(x_2, y_n) \\ \vdots & \vdots & \ddots & \vdots \\ \text{cov}(x_m, y_1) & \text{cov}(x_m, y_2) & \cdots & \text{cov}(x_m, y_n) \end{bmatrix}$$

● 計算所有特徵兩兩之間的共變異程度

尋問 ChatGPT:

生成式 AI 範例: 協方差矩陣說明.mhtml

2. 計算協方差矩陣/共變異數矩陣:
計算所有特徵兩兩之間的共變異程度
請說明與提供範例

協方差矩陣 (Covariance Matrix, 也稱共變異數矩陣) 是統計與機器學習中非常重要的工具, 用來表示:

「各個特徵之間如何一起變動」

例如:

- 身高增加時, 體重是否也增加?
- 溫度升高時, 冰品銷量是否增加?

這種「一起變化」的程度, 就是共變異數。

一、什麼是共變異數 (Covariance)

若有兩個變數:

- X
- Y

則共變異數公式:

$$\text{Cov}(X, Y) = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$$

其中:

- x_i : 第 i 筆 X 資料
- y_i : 第 i 筆 Y 資料
- $\bar{x}$: X 平均值
- $\bar{y}$: Y 平均值
- n : 資料數量

二、共變異數的意義

1. 共變異數 > 0

代表:

- X 增加時
- Y 通常也增加

稱為:

✔ 正相關

例如:

- 身高 ↑
- 體重 ↑

2. 共變異數 < 0

代表：

- X 增加時
- Y 減少

稱為：

✔ 負相關

例如：

- 運動時間 $\uparrow$
- 體脂肪 $\downarrow$

3. 共變異數 ≈ 0

代表：

- 兩者幾乎沒有線性關係

三、什麼是協方差矩陣 (Covariance Matrix)

當資料有很多特徵：

例如：

身高	體重	年齡
----	----	----

我們需要：

- 身高與體重的共變異數
- 身高與年齡的共變異數
- 體重與年齡的共變異數

全部整理成矩陣：

$$\begin{bmatrix} Cov(X, X) & Cov(X, Y) & Cov(X, Z) \\ Cov(Y, X) & Cov(Y, Y) & Cov(Y, Z) \\ Cov(Z, X) & Cov(Z, Y) & Cov(Z, Z) \end{bmatrix}$$

這就是：

協方差矩陣

四、範例說明

假設有以下資料：

人員	身高(X)	體重(Y)
A	160	55
B	165	60
C	170	65

Step 1：計算平均值

身高平均：

$$\bar{x} = \frac{160 + 165 + 170}{3} = 165$$

體重平均：

$$\bar{y} = \frac{55 + 60 + 65}{3} = 60$$

Step 2：計算各差值

人員	$x_i - \bar{x}$	$y_i - \bar{y}$
A	-5	-5
B	0	0
C	5	5

Step 3：相乘

人員	乘積
A	25
B	0
C	25

總和：

$$25 + 0 + 25 = 50$$

Step 4：除以 $n - 1$

因為：

$$n = 3$$

所以：

$$Cov(X, Y) = \frac{50}{2} = 25$$

補充：其中相乘指的是公式中如下：

$$\text{Cov}(X, Y) = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$$

五、建立協方差矩陣

先計算：

- $\text{Cov}(X, X)$
- $\text{Cov}(Y, Y)$
- $\text{Cov}(X, Y)$

結果：

$$\begin{bmatrix} 25 & 25 \\ 25 & 25 \end{bmatrix}$$

補充：

$$x_i - \bar{x} \quad , \quad y_i - \bar{y}$$

```
import numpy as np

# 身高與體重
data = np.array([
    [160, 55],
    [165, 60],
    [170, 65]
])

# 計算協方差矩陣
cov_matrix = np.cov(data.T)

print(cov_matrix)
```

執行結果

Python

```
[[25, 25.]  
 [25, 25.]]
```

七、協方差矩陣用途

協方差矩陣常用於：

領域	用途
PCA 主成分分析	找出最重要特徵
機器學習	特徵分析
資料分析	判斷變數關係
財務分析	股票風險分析
AI	降維與特徵抽取

八、與相關係數的差異

協方差

- 有方向
- 但數值大小受單位影響

例如：

- 公分
- 公斤

會影響結果

相關係數 (Correlation)

會標準化到：

$-1 \sim 1$

更容易比較。

九、重點整理

名稱	說明
共變異數	兩變數一起變化程度
正值	正相關
負值	負相關
0	幾乎無關
協方差矩陣	所有特徵間共變異數集合
常見用途	PCA、機器學習、資料分析

● 特徵分解(Eigendecomposition)

尋問 ChatGPT:

生成式 AI 範例:特徵分解簡介.mhtml

我是一個國中的學生，想要了解特徵分解，說明的例子若有公式，請詳細說明過程，其中矩陣相乘也不熟

我會用「國中程度可以接住」的方式說明：先補矩陣相乘，再用一個 2×2 矩陣一步一步做特徵值、特徵向量，最後說明「特徵分解」在做什麼。

已思考 6s >

特徵分解可以先理解成一句話：

把一個矩陣，拆成「方向」和「放大倍率」。

1. 先懂矩陣相乘

假設：

$$A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}, \quad x = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$$

矩陣乘法是「每一列」去乘「向量」：

$$Ax = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} 3 \\ 1 \end{bmatrix}$$

第一個數：

$$2 \times 3 + 1 \times 1 = 7$$

第二個數：

$$1 \times 3 + 2 \times 1 = 5$$

所以：

$$Ax = \begin{bmatrix} 7 \\ 5 \end{bmatrix}$$

2. 什麼是特徵向量？

有些向量被矩陣乘完後，方向不變，只是變長或變短。

例如：

$$A = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}, \quad v = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

計算：

$$Av = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \times 1 + 1 \times 1 \\ 1 \times 1 + 2 \times 1 \end{bmatrix} = \begin{bmatrix} 3 \\ 3 \end{bmatrix}$$

而：

$$\begin{bmatrix} 3 \\ 3 \end{bmatrix} = 3 \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

所以：

$$Av = 3v$$

這代表：

$$\begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

是特徵向量，而 3 是它的特徵值。

意思是：這個方向被矩陣作用後，方向不變，只放大 3 倍。

3. 特徵值與特徵向量的公式

核心公式是：

$$Av = \lambda v$$

其中：

$$A$$

是矩陣，

$$v$$

是特徵向量，

$$\lambda$$

是特徵值。

讀法是：

矩陣 A 乘上一個特殊向量 v ，結果等於把 v 放大 λ 倍。

4. 再找另一個特徵向量

取：

$$v = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

計算：

$$Av = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

第一個數：

$$2 \times 1 + 1 \times (-1) = 1$$

第二個數：

$$1 \times 1 + 2 \times (-1) = -1$$

所以：

$$Av = \begin{bmatrix} 1 \\ -1 \end{bmatrix} = 1v$$

因此：

$$\begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

也是特徵向量，特徵值是 1。

5. 什麼是特徵分解？

我們剛剛找到：

$$v_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad \lambda_1 = 3$$

$$v_2 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}, \quad \lambda_2 = 1$$

特徵分解就是把矩陣寫成：

$$A = PDP^{-1}$$

其中：

$$P$$

放特徵向量：

$$P = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

$$D$$

放特徵值：

$$D = \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix}$$

所以：

$$\begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} = PDP^{-1}$$

其中p-1是什麼意思

很好，你問到特徵分解最重要的部分之一。

在

$$A = PDP^{-1}$$

裡面的

$$P^{-1}$$

念作：

▮ P 的反矩陣 (Inverse Matrix)

它不是

$$\frac{1}{P}$$

而是矩陣世界中的「倒數」。

先想數字的倒數

例如：

$$5 \times \frac{1}{5} = 1$$

因為

$$\frac{1}{5}$$

是 5 的倒數。

矩陣也有倒數

例如某個矩陣

$$P = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

它的反矩陣寫成

$$P^{-1}$$

並且滿足：

$$PP^{-1} = I$$

其中

$$I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

稱為 單位矩陣。

就像數字世界的 1。

單位矩陣有什麼作用？

任何向量乘上它都不變：

$$I \begin{bmatrix} 3 \\ 5 \end{bmatrix} = \begin{bmatrix} 3 \\ 5 \end{bmatrix}$$

所以它相當於數字裡的 1。

算出剛剛例子的反矩陣

對於

$$P = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$$

2×2矩陣的反矩陣公式：

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1} = \frac{1}{ad-bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

代入：

$$a = 1, \quad b = 1, \quad c = 1, \quad d = -1$$

先算：

$$\begin{aligned} & ad - bc \\ &= (1)(-1) - (1)(1) \\ &= -2 \end{aligned}$$

因此：

$$P^{-1} = \frac{1}{-2} \begin{bmatrix} -1 & -1 \\ -1 & 1 \end{bmatrix}$$

整理：

$$P^{-1} = \begin{bmatrix} \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & -\frac{1}{2} \end{bmatrix}$$

驗證看看

$$PP^{-1} = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & -\frac{1}{2} \end{bmatrix}$$

第一列第一行：

$$1 \times \frac{1}{2} + 1 \times \frac{1}{2} = 1$$

第一列第二行：

$$1 \times \frac{1}{2} + 1 \times \left(-\frac{1}{2}\right) = 0$$

第二列第一行：

$$1 \times \frac{1}{2} + (-1) \times \frac{1}{2} = 0$$

第二列第二行：

$$1 \times \frac{1}{2} + (-1) \times \left(-\frac{1}{2}\right) = 1$$

得到：

$$PP^{-1} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = I$$

成功！

為什麼特徵分解需要 P^{-1} ?

把

$$A = PDP^{-1}$$

想成三個步驟：

第一步

$$P^{-1}$$

把原本的座標系統轉換到「特徵向量座標系」。

第二步

$$D$$

直接做縮放：

$$D = \begin{bmatrix} 3 & 0 \\ 0 & 1 \end{bmatrix}$$

代表：

- 第一個方向放大 3 倍
- 第二個方向放大 1 倍

第三步

$$P$$

再把結果轉回原本座標系。

可以把它想成：

中文 → 英文 → 處理資料 → 再翻回中文

其中：

- P^{-1} ：翻成英文
- D ：在英文環境做最簡單的運算
- P ：翻回中文



這就是特徵分解最核心的想法。

➤ 以 5 位同學的身高與體重為例

尋問 ChatGPT:

步驟如下：

1. 中心化:

□ 將每個特徵減去該特徵的平均值，使資料的平均值變為 0

2. 計算協方差矩陣/共變異數矩陣:

□ 計算所有特徵兩兩之間的共變異程度

3. 特徵分解

□ 對協方差矩陣做：特徵值與特徵向量分解

4. 特徵值排序 (Sort Eigenvalues)

□ 選擇前 k 個主成分 (選擇特徵向量)

5. 投影 (Projection)

□ 將原始資料投影到所選的主成分空間

請用同學身高、體重當例子，說明每個階段的公式細節

顯示較少 ^

生成式 AI 範例:PCA 步驟解析.mhtml

以 5 位同學的身高與體重 為例，完整示範 PCA (Principal Component Analysis，主成分分析) 的每個步驟。

原始資料

同學	身高(cm)	體重(kg)
A	160	55
B	165	60
C	170	65
D	175	70
E	180	75

用矩陣表示：

$$X = \begin{bmatrix} 160 & 55 \\ 165 & 60 \\ 170 & 65 \\ 175 & 70 \\ 180 & 75 \end{bmatrix}$$

其中：

- 第1個特徵：身高
- 第2個特徵：體重

Step 1：中心化 (Centering)

計算每個特徵平均值

身高平均

$$\bar{x}_1 = \frac{160 + 165 + 170 + 175 + 180}{5} = 170$$

體重平均

$$\bar{x}_2 = \frac{55 + 60 + 65 + 70 + 75}{5} = 65$$

因此平均向量：

$$\mu = \begin{bmatrix} 170 \\ 65 \end{bmatrix}$$

每筆資料減去平均值

公式：

$$X_c = X - \mu$$

得到：

$$X_c = \begin{bmatrix} -10 & -10 \\ -5 & -5 \\ 0 & 0 \\ 5 & 5 \\ 10 & 10 \end{bmatrix}$$

此時：

$$\text{平均值} = 0$$

Step 2 : 計算協方差矩陣

協方差公式

兩個特徵 x 與 y :

$$Cov(x, y) = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})$$

這裡 :

$$n = 5$$

所以分母 :

$$n - 1 = 4$$

計算 Height 與 Height

$$\begin{aligned} Cov(H, H) &= \frac{(-10)^2 + (-5)^2 + 0^2 + 5^2 + 10^2}{4} \\ &= \frac{100 + 25 + 0 + 25 + 100}{4} \\ &= \frac{250}{4} = 62.5 \end{aligned}$$

計算 Weight 與 Weight

$$Cov(W, W) = \frac{250}{4} = 62.5$$

計算 Height 與 Weight

$$\begin{aligned} Cov(H, W) &= \frac{(-10)(-10) + (-5)(-5) + 0(0) + 5(5) + 10(10)}{4} \\ &= \frac{250}{4} = 62.5 \end{aligned}$$

建立協方差矩陣

$$C = \begin{bmatrix} 62.5 & 62.5 \\ 62.5 & 62.5 \end{bmatrix}$$

Step 3：特徵分解 (Eigen Decomposition)

對協方差矩陣：

$$C = \begin{bmatrix} 62.5 & 62.5 \\ 62.5 & 62.5 \end{bmatrix}$$

求：

$$Cv = \lambda v$$

求特徵值

公式：

$$|C - \lambda I| = 0$$

即：

$$\begin{vmatrix} 62.5 - \lambda & 62.5 \\ 62.5 & 62.5 - \lambda \end{vmatrix} = 0$$

展開：

$$(62.5 - \lambda)^2 - (62.5)^2 = 0$$

利用：

$$a^2 - b^2 = (a + b)(a - b)$$

得到：

$$\lambda_1 = 125$$

$$\lambda_2 = 0$$

求第一個特徵向量

代入：

$$(C - 125I)v = 0$$

$$\begin{bmatrix} -62.5 & 62.5 \\ 62.5 & -62.5 \end{bmatrix}$$

得到：

$$v_1 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

正規化：

$$\begin{aligned} v_1 &= \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} \\ &= \begin{bmatrix} 0.707 \\ 0.707 \end{bmatrix} \end{aligned}$$

求第二個特徵向量

$$v_2 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

正規化：

$$\begin{aligned} v_2 &= \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ -1 \end{bmatrix} \\ &= \begin{bmatrix} 0.707 \\ -0.707 \end{bmatrix} \end{aligned}$$

Step 4：特徵值排序

特徵值：

$$\lambda_1 = 125$$

$$\lambda_2 = 0$$

排序後：

主成分	特徵值
PC1	125
PC2	0

解釋變異量比例

公式：

$$Explained\ Variance = \frac{\lambda_i}{\sum \lambda}$$

總變異量：

$$125 + 0 = 125$$

因此：

$$PC1 = \frac{125}{125} = 100\%$$

$$PC2 = 0\%$$

代表：

身高與體重完全線性相關，所以全部資訊都集中在第一個主成分。

Step 5 : 投影 (Projection)

選擇第一主成分：

$$W = \begin{bmatrix} 0.707 \\ 0.707 \end{bmatrix}$$

PCA投影公式

$$Z = X_c W$$

第一位同學

$$\begin{aligned} [-10, -10] \begin{bmatrix} 0.707 \\ 0.707 \end{bmatrix} \\ = -14.14 \end{aligned}$$

第二位同學

$$[-5, -5] \begin{bmatrix} 0.707 \\ 0.707 \end{bmatrix} = -7.07$$

第三位同學

$$0$$

第四位同學

$$7.07$$

第五位同學

$$14.14$$

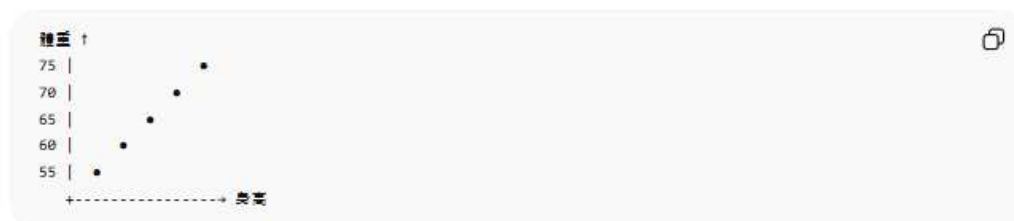


投影後資料

$$Z = \begin{bmatrix} -14.14 \\ -7.07 \\ 0 \\ 7.07 \\ 14.14 \end{bmatrix}$$

幾何意義

原本資料在二維空間：



所有點都落在同一直線上。

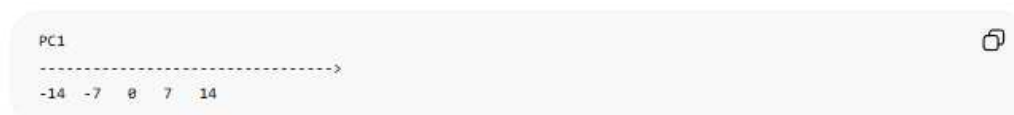
PCA 找到變異最大的方向：

$$v_1 = \begin{bmatrix} 0.707 \\ 0.707 \end{bmatrix}$$

也就是 45° 方向。

$$v_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

將二維資料投影到這條線後：



原本的 2 個特徵（身高、體重）就被壓縮成 1 個主成分（PC1），但仍保留 100% 的資訊。

PCA 五步驟總結

1. 中心化

$$X_c = X - \mu$$

2. 協方差矩陣

$$C = \frac{1}{n-1} X_c^T X_c$$

3. 特徵分解

$$Cv = \lambda v$$

4. 依特徵值大小排序

$$\lambda_1 > \lambda_2 > \dots$$

5. 投影

$$Z = X_c W$$

其中：

- W ：選出的主成分（特徵向量）
- Z ：降維後的新資料

這就是 PCA 從「身高、體重」資料中找出最重要方向並完成降維的完整數學過程。

➤ PCA 實作

題目：使用 Python 實作 PCA 主成分分析

某班級收集了 5 位學生的「身高」與「體重」資料：

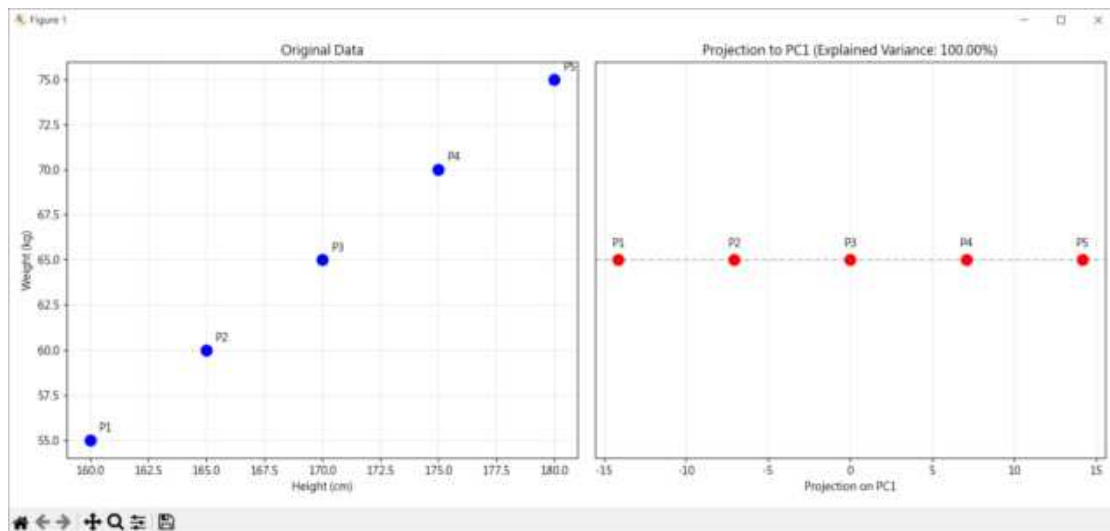
學生	身高 (cm)	體重 (kg)
A	160	55
B	165	60
C	170	65
D	175	70
E	180	75

請使用 Python 與 NumPy 完成 PCA (Principal Component Analysis · 主成分分析)，將原本的「身高、體重」兩個特徵，降維成 1 個主成分。

作答要求：

1. 建立原始資料矩陣 X
2. 計算每個特徵的平均值
3. 對資料進行中心化 (Centering)
4. 計算協方差矩陣

5. 計算協方差矩陣的特徵值與特徵向量
6. 將特徵值由大到小排序
7. 選擇最大的特徵值所對應的特徵向量
8. 將原始資料投影到第一主成分
9. 計算每個主成分的解釋變異量比例
10. 使用 Matplotlib 將 PCA 結果繪製成圖形



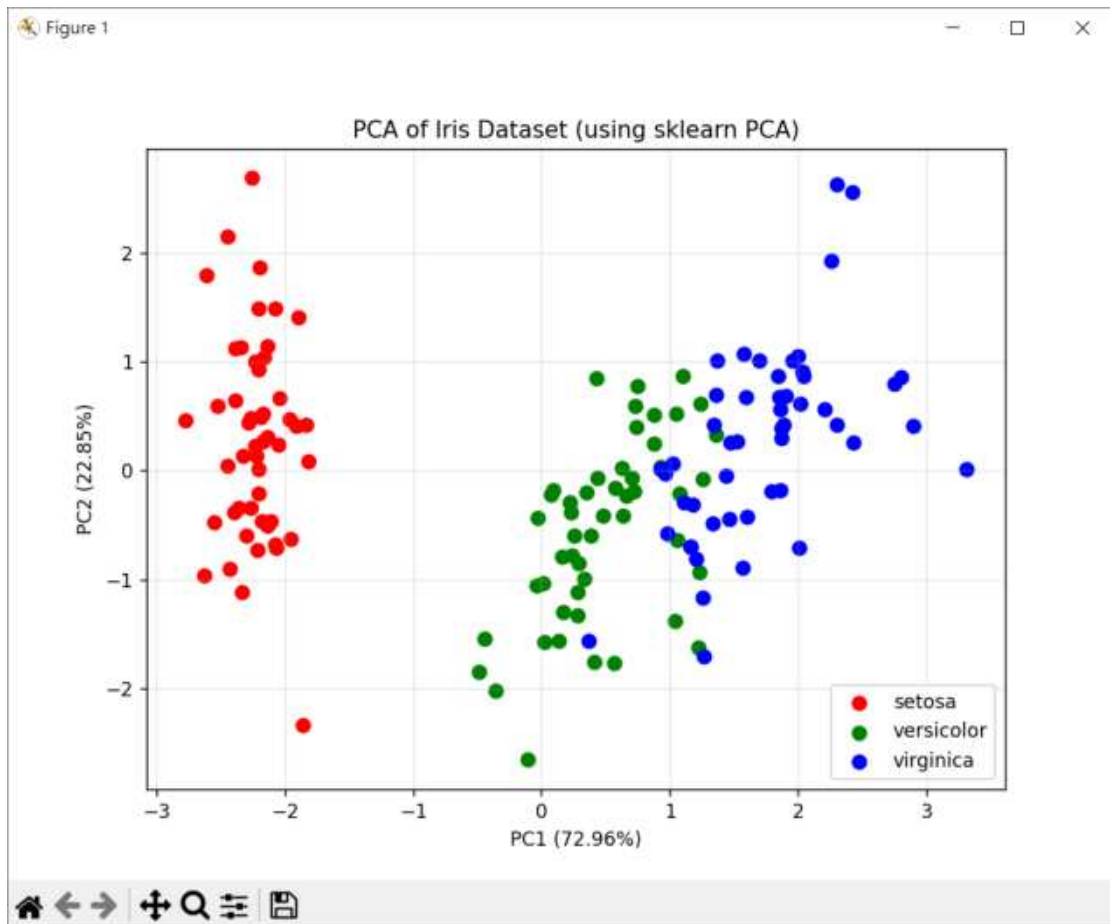
範例:pca1.py

範例:pca2.py

題目：Iris（鳶尾花）資料集是機器學習中的經典範例，包含 150 筆樣本、4 個特徵（花萼長度、花萼寬度、花瓣長度、花瓣寬度）與 3 個類別（Setosa、Versicolor、Virginica）。利用 scikit-learn 提供的 PCA 函式，將 Iris 資料集從 4 維降到 2 維，並完成以下工作：

1. 載入資料：使用 `sklearn.datasets.load_iris()` 讀取特徵矩陣 X 與類別標籤 y 。
2. 資料標準化：使用 `StandardScaler` 將每個特徵轉為平均值 0、標準差 1，避免不同單位造成偏誤。
3. PCA 降維：使用 `PCA(n_components=2)` 將標準化後的資料投影到前 2 個主成分。
4. 分析結果，輸出以下數值：
 - 標準化後資料的共變異數矩陣
 - 特徵值（`explained_variance_`）
 - 特徵向量（`components_`）

- 每個主成分的解釋變異比例 (explained_variance_ratio_)
 - 前 2 個主成分的累積解釋比例
5. 視覺化：以散布圖繪製降維後結果，將三種鳶尾花以不同顏色標示，並在座標軸上標註各主成分所解釋的變異百分比。



範例:pca3.py

3.3 K-Means(K 平均值分群法)

➤ 介紹

尋問 ChatGPT:

生成式 AI 範例: K-Means 分群法介紹.mhtml

請說明kmean

K-Means (K 平均值分群法) 是一種**非監督式學習 (Unsupervised Learning) **的機器學習演算法，主要用來將資料自動分成 K 個群組 (Cluster) 。

一、K-Means 的概念

假設有一群學生的身高和體重資料：

學生	身高	體重
A	160	50
B	162	52
C	165	55
D	180	75
E	182	78
F	185	80

如果沒有告訴電腦誰屬於哪一組，K-Means 可以自動找出：

群組1

- A
- B
- C

群組2

- D
- E
- F

因為它們彼此比較相似。

二、K-Means 的工作流程

假設 $K=2$ (分成兩群)

Step 1：隨機選擇兩個中心點

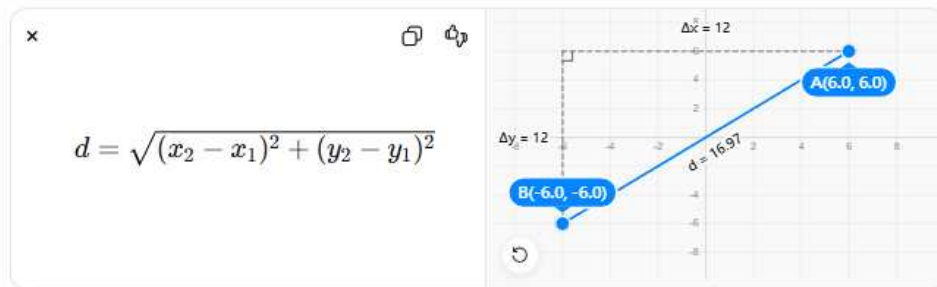
例如：

中心1 = A(160,50)

中心2 = E(182,78)

Step 2：計算每個資料到中心點的距離

距離公式：



例如：

學生 C(165,55)

到中心1：

$$\sqrt{(165 - 160)^2 + (55 - 50)^2}$$

到中心2：

$$\sqrt{(165 - 182)^2 + (55 - 78)^2}$$

比較哪個距離較近。

Step 3：分配群組

距離中心1較近 → 群組1

距離中心2較近 → 群組2

Step 4：重新計算群中心

例如群組1：

身高	體重
160	50
162	52
165	55

新的中心：

$$\left(\frac{160 + 162 + 165}{3}, \frac{50 + 52 + 55}{3} \right)$$

=

$$(162.33, 52.33)$$

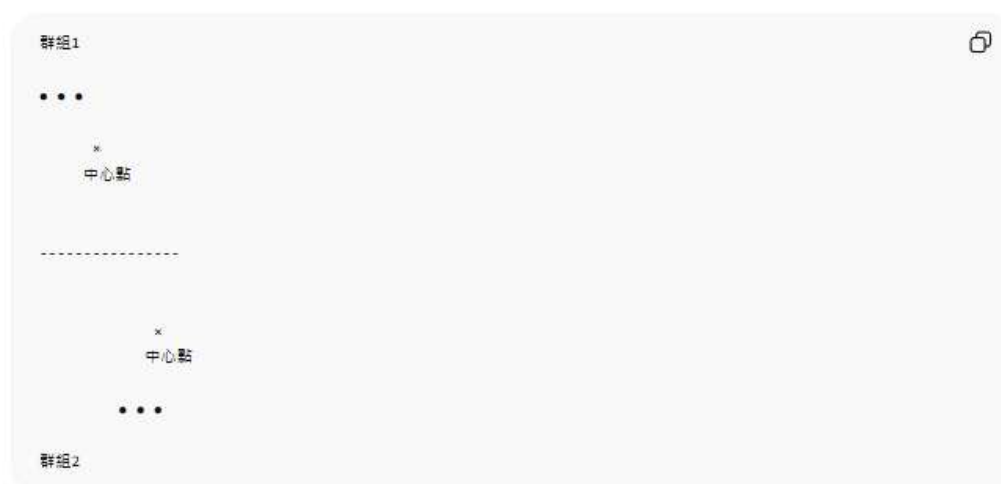
Step 5：重複計算

再次：

1. 算距離
2. 分群
3. 更新中心

直到中心不再改變。

三、K-Means 圖形概念



其中：

- ● = 資料點
- × = 群中心(Centroid)

四、K-Means 的目標

讓同群資料距離越近越好。

目標函數：

$$J = \sum_{i=1}^k \sum_{x_j \in C_i} \|x_j - \mu_i\|^2$$

$$J = \sum_{i=1}^k \sum_{x_j \in C_i} \|x_j - \mu_i\|^2$$

其中：

- k = 群數
- C_i = 第 i 群
- μ_i = 群中心

目的是讓 J 最小。

五、K-Means 優點

- ✓ 簡單易懂
- ✓ 執行速度快
- ✓ 適合大量資料
- ✓ 容易實作

六、K-Means 缺點

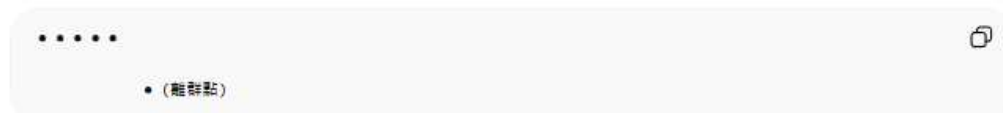
1. 必須先知道 K 值

例如：

- $K=2$
- $K=3$
- $K=4$

結果可能完全不同。

2. 容易受到離群值影響

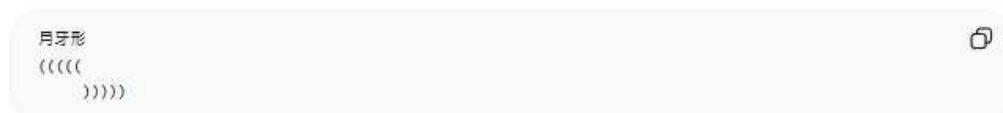


離群點可能拉動中心位置。

3. 不適合非圓形群集

K-Means 假設資料呈現球狀或圓形分布。

例如：



效果通常不好。

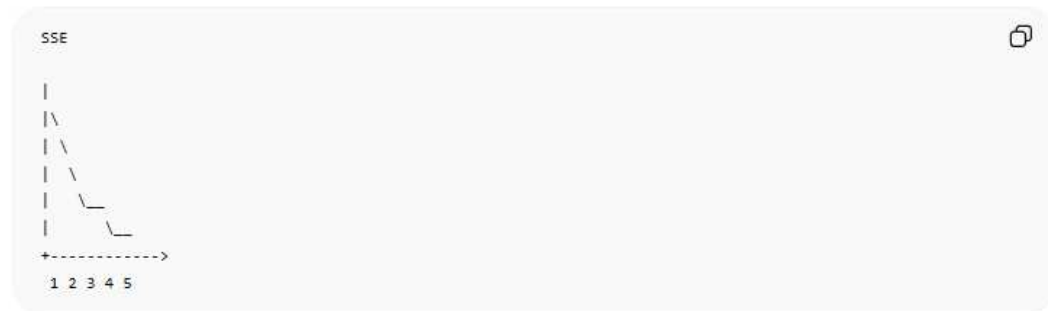
七、如何決定 K 值

常用：

Elbow Method (肘部法)

繪製：

- X軸：K值
- Y軸：誤差(SSE)



轉折點 (肘部) 就是較佳的 K 。

九、實際應用

客戶分群

依據：

- 年齡
- 消費金額
- 購買頻率

找出不同客群。

商品推薦

將相似使用者分群。

影像分割

把相近顏色像素分為同一群。

IoT 感測器分析

例如 DHT22：

- 溫度
- 濕度

利用 K-Means 找出：

- 正常狀態
 - 高溫狀態
 - 異常狀態
-

一句話總結：

K-Means 是利用距離計算，反覆更新群中心（Centroid），將相似資料自動分成 K 個群組的分群演算法。

➤ K-Means 實作

題目：使用 K-Means 分群演算法將身高體重資料分為兩群

■ 版本一：基礎練習題

題目：使用 K-Means 分群演算法將身高體重資料分為兩群

給定 5 筆身高 (cm) 與體重 (kg) 的資料如下：

編號	身高	體重
1	160	55
2	165	60
3	170	65
4	175	70
5	180	75

請使用 `sklearn` 的 `KMeans` 演算法，將資料分為 2 群，並完成以下要求：

1. 印出每筆資料所屬的群組編號。
2. 印出每個群組的中心點座標。
3. 使用 `matplotlib` 繪製散佈圖，將不同群組以不同顏色顯示，並標示出中心點。

■ 版本二：情境應用題

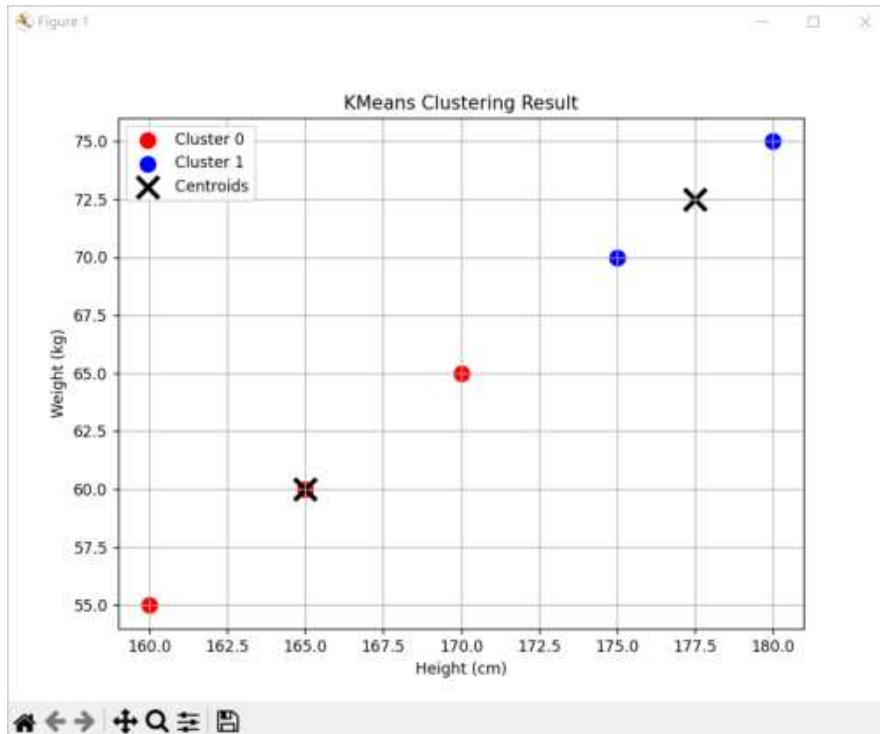
題目：服飾店客戶身材分群分析

某服飾店想根據顧客的身高與體重，將顧客分為兩種身材類型（例如：標準型、高大型），以便推薦適合的服裝尺寸。已收集 5 位顧客的資料：

```
Code (160, 55), (165, 60), (170, 65), (175, 70), (180, 75)
```

請利用 K-Means 分群演算法 ($K=2$) 完成以下任務：

1. 找出兩種身材類型的分群結果。
2. 求出各群的代表身材（中心點）。
3. 以視覺化圖形呈現分群結果，方便向店長報告。



範例:kmean1.py

題目：Iris（鳶尾花）資料集是機器學習中的經典範例，包含 150 筆樣本、4 個特徵（花萼長度、花萼寬度、花瓣長度、花瓣寬度）與 3 個類別（Setosa、Versicolor、Virginica）。利用 scikit-learn 提供的 PCA 函式，將 Iris 資料集從 4 維降到 2 維，並完成以下工作：

請依下列步驟完成程式與分析：

1、資料載入與匯出

- 使用 `sklearn.datasets.load_iris()` 載入資料。
- 將資料整理為 `DataFrame`，包含 4 個特徵、target 與 species 名稱。
- 匯出為 CSV 檔（`./output/iris_dataset.csv`）。

2、資料前處理

- 使用 `StandardScaler` 對特徵進行標準化。

3、PCA 降維

- 將標準化後的資料降至 2 維。
- 輸出每個主成分的解釋變異比例（explained variance ratio）與累積解釋比例。

4、KMeans 分群

- 以 PCA 降維後的資料 進行 KMeans 分群（ $k=3$ ）。
- 取得每群的中心點。

5、分群結果評估

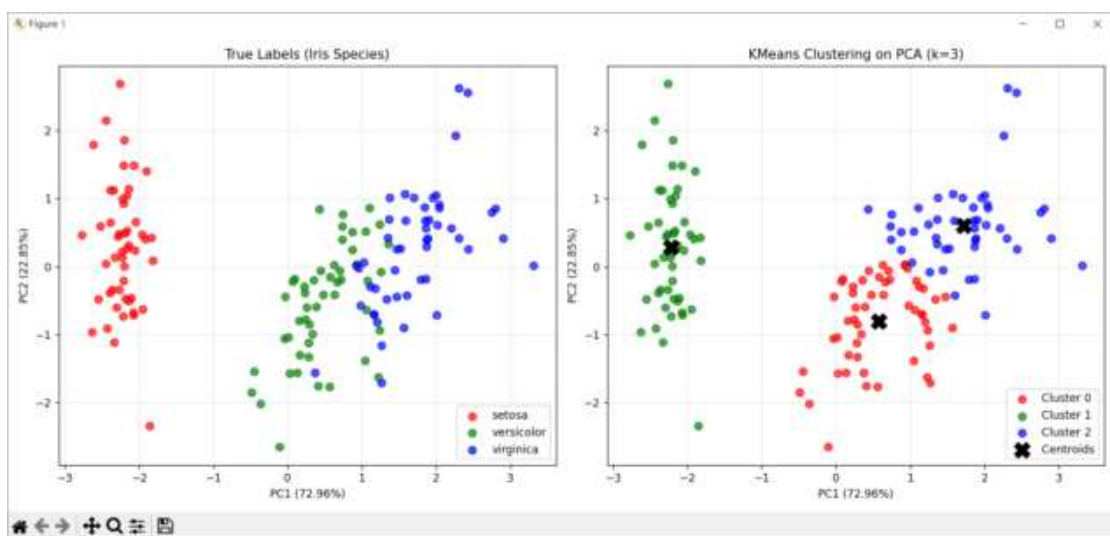
- 輸出 Confusion Matrix (真實類別 vs 預測群)。
- 計算 Adjusted Rand Index (ARI)。
- 顯示 Inertia (群內平方和)。

6、視覺化 (子圖並排)

- 左圖：以 PC1、PC2 為軸繪製 真實類別 散佈圖。
- 右圖：繪製 KMeans 分群結果，並標出群中心 (Centroids)。
- 兩張圖之座標軸須標示主成了解釋比例 (%)。

7、加分題：Elbow Method

- 對 $K = 1 \sim 7$ 分別計算 KMeans 的 Inertia。
- 繪製 Elbow 曲線，觀察最佳分群數 K 。



➤ K-Means 手肘法 (Elbow Method)

手肘法是用來決定 K-Means 分群中最佳 K 值 (群數) 的一種常用方法。

K-Means 需要事先指定分幾群 (K)，但實務上我們往往不知道最佳的 K 是多少。手肘法透過觀察「群內誤差平方和」隨 K 變化的趨勢，找出一個轉折點作為建議值。

關鍵指標：WCSS / SSE / Inertia

WCSS (Within-Cluster Sum of Squares)，也稱為 SSE 或 Inertia：

$$WCSS = \sum_{i=1}^K \sum_{x \in C_i} |x - \mu_i|^2$$

- C_i ：第 i 群
- μ_i ：第 i 群的質心
- 意義：每個點到其所屬群質心的距離平方和



K 越大 → WCSS 越小 (極端情況： $K =$ 樣本數時， $WCSS = 0$)

執行步驟

- 1. 設定一個 K 的範圍，例如 $K = 1, 2, 3, \dots, 10$
- 2. 對每個 K 執行 K-Means，計算對應的 WCSS
- 3. 以 K 為 X 軸、WCSS 為 Y 軸繪圖
- 4. 找出曲線的「手肘點 (elbow) 」——也就是斜率突然變平緩的轉折點
- 5. 該點對應的 K 即為建議的最佳群數

最佳 K = 3

判讀方式：Elbow Method (手肘法)

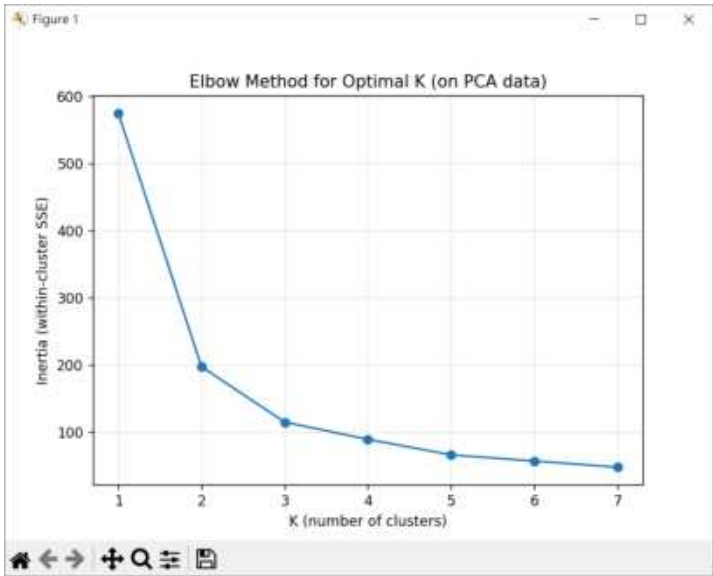
Elbow Method 是看 inertia (群內平方和 SSE) 隨 K 增加而下降的曲線，尋找「下降速度突然變緩」的轉折點，也就是像手肘彎曲的地方。

以 Iris 資料集 (標準化後) 跑出來的 inertia 大約會是這樣：

K	Inertia (約略值)	與前一個 K 的下降量
1	~596	—
2	~223	↓ 373 ← 大幅下降
3	~140	↓ 83 ← 仍明顯下降
4	~114	↓ 26 ← 下降變緩 (手肘！)
5	~91	↓ 23
6	~81	↓ 10
7	~72	↓ 9

👉 從 $K=3 \rightarrow K=4$ ，下降幅度從 83 驟降到 26，曲線在 $K=3$ 之後明顯變平，這就是「手肘」的位置。

↓



➤ 輪廓係數(Silhouette Coefficient)說明

輪廓係數由 Peter J. Rousseeuw 於 1987 年提出，用來衡量一個資料點被分到某一群的合適程度。

它同時考慮兩件事：

凝聚度 (Cohesion)：點與同群其他點的相似程度 → 越相似越好

分離度 (Separation)：點與其他群的差異程度 → 差異越大越好

最終將兩者結合成一個 $[-1, 1]$ 的分數。

二、符號定義

假設資料被分成 K 個群集 $C_1, C_2, \dots, C_K$ ：

符號	意義
i	某個資料點
C_l	點 i 所屬的群
$ C_l $	群 C_l 中的點數
$d(i, j)$	點 i 與點 j 的距離 (通常為歐氏距離)

三、核心公式

📌 公式 1：群內平均距離 $a(i)$

點 i 到同群其他所有點的平均距離：

$$a(i) = \frac{1}{|C_l| - 1} \sum_{j \in C_l, j \neq i} d(i, j)$$

說明：

- 分母為 $|C_l| - 1$ 因為排除點 i 自己
- $a(i)$ 越小 → 群內越緊密 → 分群品質越好
- 若群內只有 1 個點 ($|C_l| = 1$)，定義 $s(i) = 0$

🔗 公式 2：最近鄰群平均距離 $b(i)$

點 i 到其他每一群的平均距離中，取最小值：

$$b(i) = \min_{K \neq I} \left[\frac{1}{|C_K|} \sum_{j \in C_K} d(i, j) \right]$$

說明：

- 先算點 i 到「每一個其他群」的平均距離
- 取當中最小的那個 → 也就是「最像鄰居」的那群
- 該群稱為點 i 的鄰群 (neighboring cluster)
- $b(i)$ 越大 → 群與群分離越好

🔗 公式 3：單點輪廓係數 $s(i)$

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))}$$

🔗 公式 4：群平均輪廓係數

某一群 C_k 的平均分數：

$$S_{C_k} = \frac{1}{|C_k|} \sum_{i \in C_k} s(i)$$

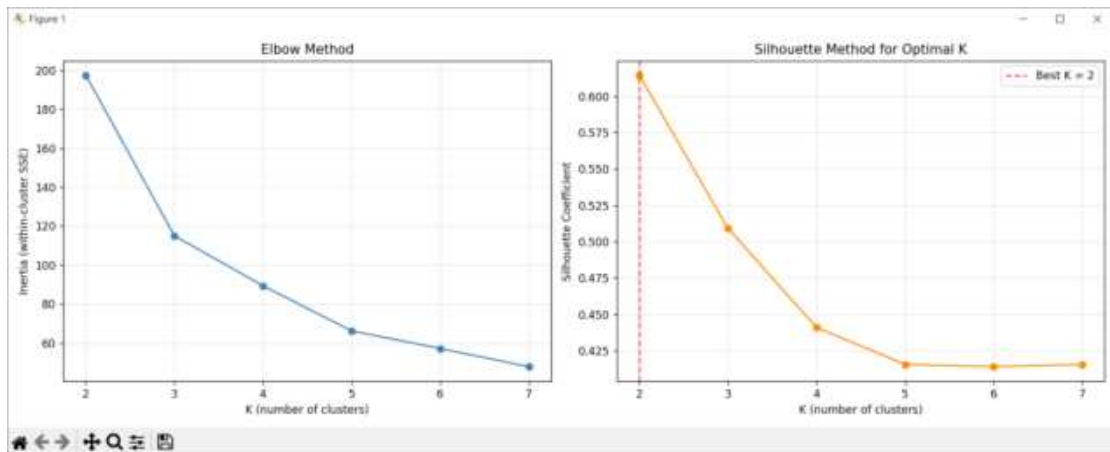
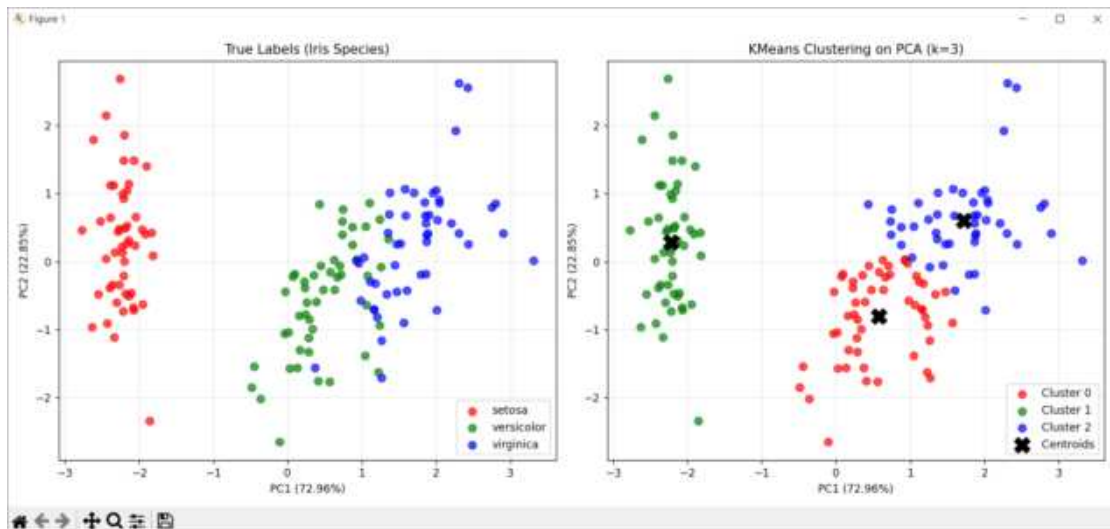
用途：檢視每一群的分群品質，找出品質差的群。

🔗 公式 5：整體平均輪廓係數

所有點的平均：

$$\bar{S} = \frac{1}{N} \sum_{i=1}^N s(i)$$

- N 為總樣本數
- 這是 sklearn 中 `silhouette_score()` 回傳的值
- 選 K 時：對不同 K 計算 $\bar{S}$ ，取最大者



綜合三個指標判斷:

- 1、Elbow Method 在 $K=3$ 出現明顯轉折(Inertia 由 197.40 驟降至 115.02,之後趨緩)。
- 2、Silhouette Coefficient 雖在 $K=2$ 時最高(0.6145),但 $K=3$ 仍有 0.5092 的良好水準。
- 3、Silhouette 偏好 $K=2$ 的原因為 setosa 與其他兩類在特徵空間分離明顯,而 versicolor 與 virginica 部分重疊,導致分離度指標低估了 $K=3$ 。綜合考量,本實驗採用 $K=3$ 。

範例:kmean2.py

3.4 PCA+Kmean 實際範例 1(線上購物資料分析)

Instacart 線上購物資料：

資料表如下：

- 檔名:aisles.csv
 - 欄位:aisle_id、aisle
- 檔名:departments.csv
 - 欄位:department_id、department
- 檔名:order_products__prior.csv
 - 欄位:order_id product_id、add_to_cart_order、reordered
- 檔名:orders.csv
 - 欄位:order_id、user_id、eval_set、order_number、order_dow、
order_hour_of_day、days_since_prior_order
- 檔名:products.csv
 - 欄位:product_id、product_name、aisle_id、department_id

流程如下：

讀取資料→合併資料→建立顧客特徵→ StandardScaler→ PCA→ PCA 畫圖→
K-Means→ K-Means 畫圖→ Cluster 商業分析

➤ 第 1 階段：資料載入與合併

Instacart 線上購物資料包含多個 CSV 檔案，請使用 Python pandas 讀取以下資料：

- orders.csv
- order_products__prior.csv
- products.csv
- departments.csv
- aisles.csv

要求：

1. 使用 `pandas.read_csv()` 讀取資料。
2. 顯示每個資料表的前 5 筆資料。
3. 顯示每個資料表的資料筆數與欄位數。
4. 確認後續 PCA + K-Means 分析所需要的資料是否成功載入。

```
===== orders =====
  order_id  user_id  eval_set  order_number  order_dow  order_hour_of_day  days_since_prior_order
0    2539329         1    prior             1           2              8                NaN
1    2398795         1    prior             2           3              7                15.0
2     473747         1    prior             3           3             12                21.0
3    2254736         1    prior             4           4              7                29.0
4     431534         1    prior             5           4             15                28.0

===== order_products__prior =====
  order_id  product_id  add_to_cart_order  reordered
0         2         33120                 1          1
1         2         28985                 2          1
2         2         9327                  3          0
3         2         45918                 4          1
4         2         30035                 5          0

===== products =====
  product_id  product_name  aisle_id  department_id
0           1  Chocolate Sandwich Cookies        61         19
1           2    All-Seasons Salt       104         13
2           3  Robust Golden Unsweetened Oolong Tea    94          7
3           4  Smart Ones Classic Favorites Mini Rigatoni Wit...   38          1
4           5    Green Chile Anytime Sauce          5         13

===== departments =====
  department_id  department
0              1    frozen
1              2    other
2              3    bakery
3              4    produce
4              5    alcohol
```

```

===== aisles =====
  aisle_id      aisle
0         1  prepared soups salads
1         2    specialty cheeses
2         3    energy granola bars
3         4      instant foods
4         5  marinades meat preparation

===== 資料大小 =====
orders : (998566, 7)
order_products : (3239827, 4)
products : (49688, 4)
departments : (21, 2)
aisles : (134, 2)

```

範例：pca+kmean01_01.py

➤ 第 2 階段：合併資料

找出每筆訂單購買的商品部門，Instacart 的商品資料分散在不同資料表中。

請利用資料表之間的關聯欄位，將：

orders、order_products__prior、products、departments，進行合併。

最後產生一個新的資料表，至少包含：

user_id、order_id、product_id、product_name、department

```

查看 orders 表：
  order_id  product_id  add_to_cart_order  reordered  user_id  eval_set  order_number  order_dow  order_hour_of_day  days_since_prior_order
0         6         40462                1           0      22352    prior         4           1           11           30.0
1         6         15873                2           0      22352    prior         4           1           11           30.0
2         6         41897                3           0      22352    prior         4           1           11           30.0
3         8         23421                1           1       3187    prior         3           4           15           17.0
4        13         17330                1           0      45082    prior         2           6           17           1.0

加入 products 表：
  order_id  product_id  add_to_cart_order  reordered  user_id  ...  order_hour_of_day  days_since_prior_order  product_name  aisle_id  department_id
0         6         40462                1           0      22352    ...           11           30.0          Cleanse          31           7
1         6         15873                2           0      22352    ...           11           30.0  Dyer Sheets Geranium Scent          75          17
2         6         41897                3           0      22352    ...           11           30.0  Clean Day Lavender Scent Room Freshener Spray  101          17
3         8         23421                1           1       3187    ...           4           17.0  Original Hawaiian Sweet Rolls          44           3
4        13         17330                1           0      45082    ...           6           1.0          Light          27           5

[5 rows x 13 columns]

加入 departments 表：
  order_id  product_id  add_to_cart_order  reordered  user_id  ...  days_since_prior_order  product_name  aisle_id  department_id  department
0         6         40462                1           0      22352    ...           30.0          Cleanse          31           7  beverages
1         6         15873                2           0      22352    ...           30.0  Dyer Sheets Geranium Scent          75          17  household
2         6         41897                3           0      22352    ...           30.0  Clean Day Lavender Scent Room Freshener Spray  101          17  household
3         8         23421                1           1       3187    ...           17.0  Original Hawaiian Sweet Rolls          44           3    bakery
4        13         17330                1           0      45082    ...           1.0          Light          27           5    alcohol

[5 rows x 14 columns]

===== 最終資料 =====
  user_id  order_id  product_id  product_name  department
0      22352         6         40462          Cleanse  beverages
1      22352         6         15873  Dyer Sheets Geranium Scent  household
2      22352         6         41897  Clean Day Lavender Scent Room Freshener Spray  household
3       3187         8         23421  Original Hawaiian Sweet Rolls    bakery
4      45082        13         17330          Light    alcohol

資料大小：
(500771, 5)

```

範例：pca+kmean01_02.py

➤ 第 3 階段：統計每位顧客購買各部門的數量

分析顧客的商品部門購買行為

請統計每一位顧客在不同商品部門的購買數量。

例如：

user_id	snacks	beverages	dairy eggs	frozen
1	15	3	8	2
2	2	12	3	10
3	20	1	2	3

將「商品部門」轉換成欄位，作為後續 PCA + K-Means 的特徵。

```
==== 顧客 * 商品部門 ====
department alcohol babies bakery beverages breakfast bulk canned goods dairy eggs ... meat seafood missing other pantry personal care pets produce snacks
user_id
4      1      0      0      1      0      0      0      1      0 ...      0      0      0      0      0      0      0      0
5      0      0      0      0      0      0      1      4 ...      0      0      0      0      0      0      5      0
6      0      0      0      0      0      0      2      1 ...      1      0      0      1      0      0      2      0
7      0      0      1      5      0      0      2      6 ...      2      0      0      0      0      0      6      4
9      0      5      0      1      1      0      1      7 ...      1      0      0      3      0      0      2      7

[5 rows x 21 columns]
資料大小:
(1703, 21)
```

範例：pca+kmean01_03.py

➤ 第 4 階段：資料標準化

由於不同商品部門的購買數量差異可能很大，因此在進行 PCA 前，請使用 StandardScaler 將資料標準化。

要求：

1. 使用 StandardScaler。
2. 將顧客購物資料進行標準化。
3. 顯示標準化前後的資料。
4. 檢查標準化後各特徵的平均值是否接近 0。

```
==== 標準化前 ====
department alcohol babies bakery beverages breakfast
user_id
4      1      0      0      1      0
5      0      0      0      0      0
6      0      0      0      0      0
7      0      0      1      9      0
9      0      5      0      1      1

==== 標準化後 ====
[[ 1.0032016 -0.17797607 -0.51912762 -0.28961408 -0.39007038]
 [-0.12510213 -0.17797607 -0.51912762 -0.55920123 -0.39007038]
 [-0.12510213 -0.17797607 -0.51912762 -0.55920123 -0.39007038]
 [-0.12510213 -0.17797607  0.05245788  1.86708316 -0.39007038]
 [-0.12510213  2.62697358 -0.51912762 -0.28961408  0.32075834]]

==== 標準化後平均值 ====
[-7.32073196e-18  1.03241092e-17  9.76097595e-18  1.67062858e-17
  1.50168861e-18]
```

範例：pca+kmean01_04.py

➤ 第 5 階段：PCA 降成 2 維

利用 PCA 將顧客購物行為降維

顧客購物資料具有許多商品部門特徵，直接觀察不容易。

請使用 PCA 將原本多維度的資料降低為 2 個主成分（PC1、PC2）。

要求：

1. 使用 PCA(n_components=2)。
2. 將標準化資料進行 PCA。
3. 顯示每位顧客的 PC1、PC2。
4. 顯示兩個主成分的解釋變異比例。

```
===== PCA 結果 =====
      PC1      PC2
user_id
4   -1.932570  0.280526
5   -1.053232 -0.605963
6   -1.283803 -0.527888
7    0.909218 -0.271884
9    0.400297 -0.049321
10   5.061243 -3.408202
12  -0.425939  1.185178
13  -1.332048 -0.339571
15  -1.685012  0.113675
19   0.391876 -0.823837

===== 解釋變異比例 =====
[0.25262415  0.06579349]

總解釋變異比例：
0.3184176390803849
```

範例：pca+kmean01_05.py

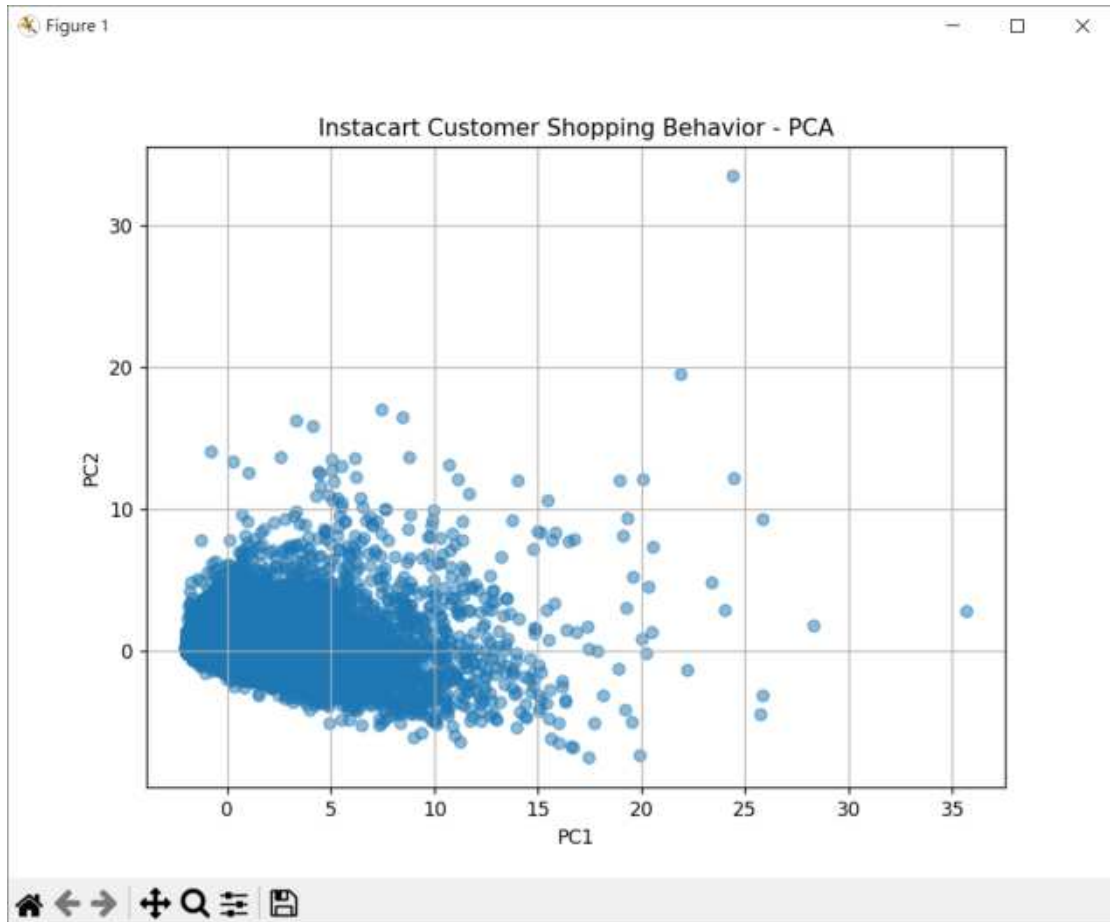
➤ 第 6 階段：畫 PCA 散佈圖

將顧客購物行為視覺化

請將 PCA 降維後的資料繪製成散佈圖。

- X 軸：PC1
- Y 軸：PC2
- 每一個點代表一位顧客

觀察顧客是否自然形成不同的群體。



範例：`pca+kmean01_06.py`

➤ 第 7 階段：K-Means 分群

利用 K-Means 將顧客分群

根據 PCA 降維後的顧客購物行為，使用 K-Means 將顧客分成 3 群。

要求：

使用 KMeans。

設定 `n_clusters=3`。

將每位顧客標記 Cluster。

顯示每一群的顧客數量。

```

===== 分群結果 =====
      PC1      PC2  Cluster
user_id
4      -1.932570  0.280526      1
5      -1.053232 -0.605963      1
6      -1.283803 -0.527888      1
7       0.909218 -0.271884      0
9       0.400297 -0.049321      1
10      5.061243 -3.408202      0
12     -0.425939  1.185178      1
13     -1.332048 -0.339571      1
15     -1.685012  0.113675      1
19      0.391876 -0.823837      1

===== 每群顧客數量 =====
Cluster
0      8077
1     28241
2      1535
Name: count, dtype: int64
範例：pca+kmean01_07.py

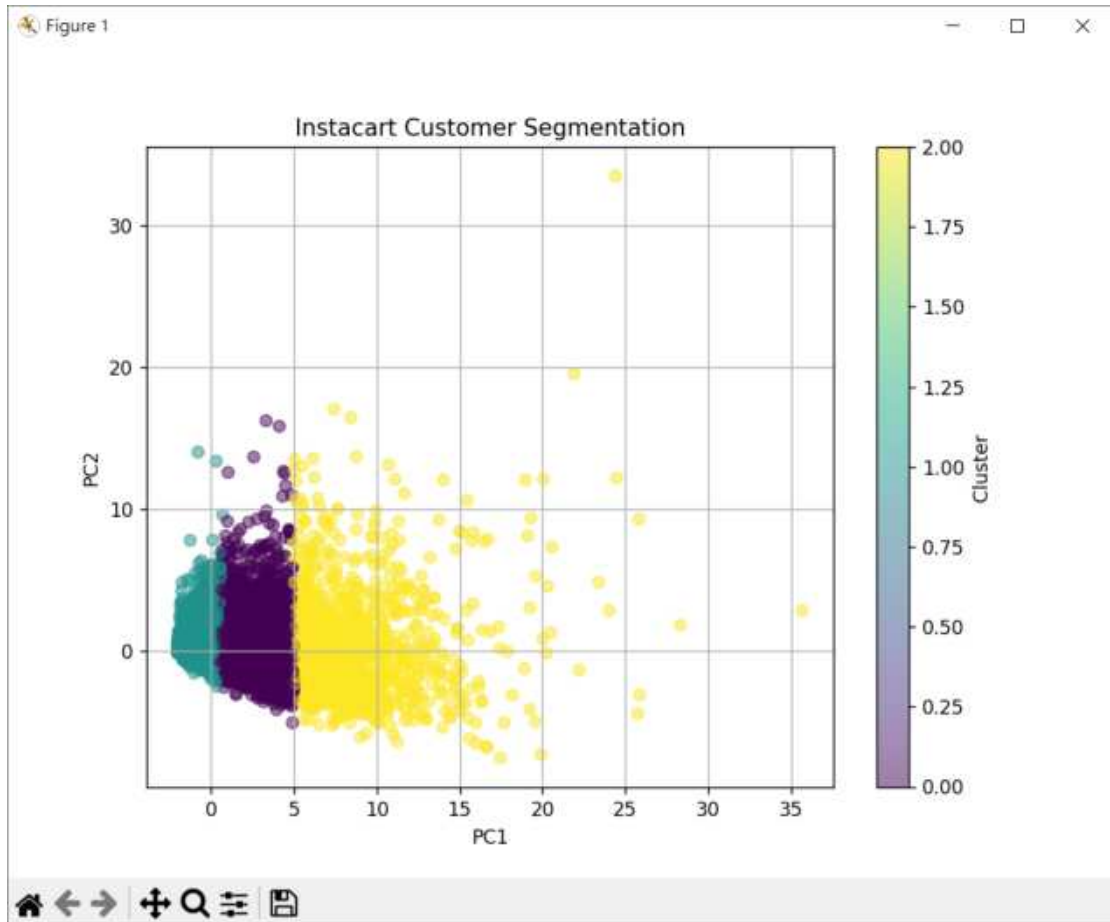
```

➤ 第 8 階段：畫出 K-Means 分群結果

視覺化 K-Means 顧客分群

將 K-Means 分群結果繪製在 PCA 二維座標上。

觀察三個 Cluster 的分布情況。



範例：`pca+kmean01_08.py`

➤ 第 9 階段：分析每一群到底是什麼顧客

請計算每一個 Cluster 在各商品部門的平均購買數量。

根據分析結果，嘗試替每個 Cluster 命名。

例如：

```
Cluster 0 → 零食型  
Cluster 1 → 家庭採購型  
Cluster 2 → 飲料型
```

```
[3 rows x 21 columns]

===== Cluster 0 =====
department
produce      14.307911
dairy eggs  8.256531
snacks      4.348644
beverages   3.799307
frozen      3.443605
Name: 0, dtype: float64

===== Cluster 1 =====
department
produce      3.923799
dairy eggs  2.079530
beverages   1.205269
snacks      1.105450
frozen      0.855635
Name: 1, dtype: float64

===== Cluster 2 =====
department
produce      32.400651
dairy eggs  19.994137
snacks     11.776547
beverages   8.985668
frozen      7.863192
Name: 2, dtype: float64
```

將結果貼給 ChatGPT 解讀：

你的結果可以這樣解讀

Cluster	Produce	Dairy Eggs	Snacks	Beverages	Frozen	顧客特徵
0	14.31	8.26	4.35	3.80	3.44	中度購物型
1	3.92	2.08	1.11	1.21	0.86	低購買量型
2	32.40	19.99	11.78	8.99	7.86	高購買量型

Cluster 0

主要購買：

produce
dairy eggs
snacks
beverages
frozen

可以命名為：

Cluster 0：一般 / 中度購物型顧客

特徵是購買量中等，而且以蔬果、乳製品及蛋類為主。

Cluster 1

所有部門的購買量都比較低：

produce	3.92
dairy eggs	2.08
beverages	1.21
snacks	1.11
frozen	0.86

可以命名：

Cluster 1：低購買量型顧客

可能是購物頻率較低，或每次購買商品較少的顧客。

Cluster 2

各部門購買量都非常高：

produce	32.40
dairy eggs	19.99
snacks	11.78
beverages	8.99
frozen	7.86

可以命名：

Cluster 2：高購買量型顧客

這群顧客明顯比 Cluster 0、1 購買更多商品。

範例：pca+kmean01_09.py

➤ 第 10 題：分析「購買比例」

```
===== Cluster 0 =====
department
produce      0.272368
dairy eggs  0.164205
snacks      0.087436
beverages   0.077636
frozen      0.072975
Name: 0, dtype: float64

===== Cluster 1 =====
department
produce      0.284553
dairy eggs  0.158294
beverages   0.110023
snacks      0.088526
frozen      0.067772
Name: 1, dtype: float64

===== Cluster 2 =====
department
produce      0.262384
dairy eggs  0.165836
snacks      0.096864
beverages   0.074122
frozen      0.067873
Name: 2, dtype: float64
```

將結果貼給 ChatGPT 再分析：

這個結果就很有意思了。和上一題的「購買數量」相比，現在改成「購買比例」後，可以發現三個 Cluster 的購物偏好其實非常接近。

1. 整體結果

前三大部門幾乎都是：

1. produce (蔬果)
2. dairy eggs (乳製品與蛋)
3. snacks (零食)

因此可以先得一個重要結論：

三個顧客群的主要購物結構相似，差異主要不在「買什麼」，而比較可能在「買多少」。

2. Cluster 0 分析

produce	27.24%
dairy eggs	16.42%
snacks	8.74%
beverages	7.76%
frozen	7.30%



特徵

Cluster 0 最主要購買：

- Produce：約 27.2%
- Dairy Eggs：約 16.4%
- Snacks：約 8.7%
- Beverages：約 7.8%
- Frozen：約 7.3%

可以稱為：

Cluster 0：一般型顧客

整體購物結構比較平均，沒有特別突出某一個非主要部門。

3. Cluster 1 分析

produce	28.46%
dairy_eggs	15.83%
beverages	11.00%
snacks	8.85%
frozen	6.78%



這一群比較特殊的是：

beverages = 11.00%

明顯高於：

- Cluster 0 : 7.76%
- Cluster 2 : 7.41%

所以可以稱為：

Cluster 1：飲料偏好型顧客

尤其可以注意：

Cluster 0 → beverages 7.76%
Cluster 1 → beverages 11.00%
Cluster 2 → beverages 7.41%



Cluster 1 的飲料比例明顯比較高。

4. Cluster 2 分析

produce	26.24%
dairy_eggs	16.58%
snacks	9.69%
beverages	7.41%
frozen	6.79%



這一群：

- Produce : 26.24%
- Dairy Eggs : 16.58%
- Snacks : 9.69%

整體比例和 Cluster 0 很接近。

比較值得注意的是：

snacks 比例最高

三群：

Cluster	Snacks
0	8.74%
1	8.85%
2	9.69%

所以可以稱為：

Cluster 2：零食偏好型顧客

但這個差異其實不是非常大，因此不建議下太強的結論。



5. 三群比較

把最重要的差異整理：

部門	Cluster 0	Cluster 1	Cluster 2	最高	
Produce	27.24%	28.46%	26.24%	C1	
Dairy Eggs	16.42%	15.83%	16.58%	C2	
Snacks	8.74%	8.85%	9.69%	C2	
Beverages	7.76%	11.00%	7.41%	C1	
Frozen	7.30%	6.78%	6.79%	C0	

最明顯的差異其實是：

Beverages

C0 = 7.76%
C1 = 11.00%
C2 = 7.41%

Cluster 1 特別高。

Snacks

C0 = 8.74%
C1 = 8.85%
C2 = 9.69%

Cluster 2 稍微高一些，但差距不大。

6. 和上一題合在一起看，才是最有價值的

你上一題的結果：

Cluster	購買量
Cluster 0	中等
Cluster 1	低
Cluster 2	高

現在的購買比例：

Cluster	購物偏好
Cluster 0	一般型
Cluster 1	飲料比例較高
Cluster 2	零食比例稍高

所以可以得到一個非常漂亮的結論：

Cluster 0、1、2 的「購物結構」其實相近，但「購買總量」有明顯差異。Cluster 1 購買量較低，但其中飲料所占比例較高；Cluster 2 購買量最高，而且零食比例略高。

如果你是超市行銷人員，針對 Cluster 1 和 Cluster 2，你會設計什麼促銷活動？

Cluster 1 : 飲料偏好型

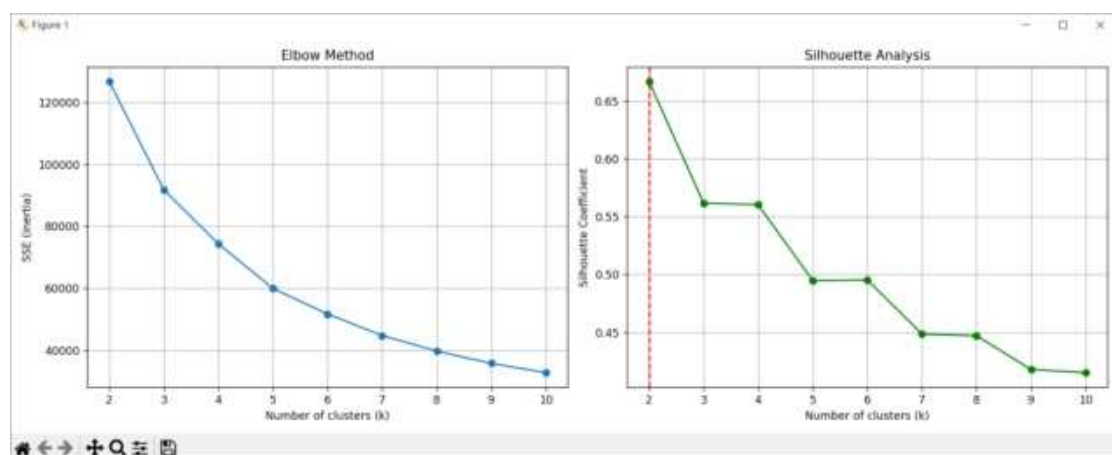
→ 飲料買二送一、飲料 + 零食組合

Cluster 2 : 高購買量 / 零食比例較高

→ 大包裝家庭優惠、零食組合包、滿額折扣

範例：pca+kmean01_10.py

➤ 第 11 題：使用 Elbow Method 與 Silhouette Coefficient 決定最佳分群數



```

(myExam) PS D:\backup\myExam\analyze> d:; cd 'd:\backup\myExam\analy
-x64\bundled\libs\debugpy\launcher' '55585' '--' 'D:\backup\myExam\ar
k=2 | SSE=126,771 | Silhouette=0.6669
k=3 | SSE=91,780 | Silhouette=0.5617
k=4 | SSE=74,383 | Silhouette=0.5604
k=5 | SSE=59,984 | Silhouette=0.4948
k=6 | SSE=51,753 | Silhouette=0.4954
k=7 | SSE=44,813 | Silhouette=0.4486
k=8 | SSE=39,811 | Silhouette=0.4472
k=9 | SSE=35,847 | Silhouette=0.4181
k=10 | SSE=32,847 | Silhouette=0.4153

最佳 k ( Silhouette 最大 ) : 2
      PC1      PC2  cluster
user_id
4      -1.932570  0.280526      1
5      -1.053232 -0.605963      1
6      -1.283803 -0.527888      1
7       0.909218 -0.271884      1
9       0.400297 -0.049321      1

各群人數：
cluster
1      32694
0       5159
Name: count, dtype: int64

```

範例：pca+kmean01_11.py

第四章 機器學習分類器

4.1 機器學習演算法分類

- 監督式學習 (Supervised Learning)
- 非監督式學習 (Unsupervised Learning)
- 強化式學習 (Reinforcement Learning)

➤ 監督式學習 (Supervised Learning)

定義：

資料已經具有正確答案(Label)，模型利用大量已知答案的資料進行學習，之後預測新的資料。

圖片	答案(Label)
貓圖片	貓
狗圖片	狗
手寫數字	5
房屋資料	800萬元

模型知道答案，因此稱為監督式。

流程：

資料→特徵(X)→模型訓練→建立模型→新資料預測

監督式主要工作：

(1) 分類(Classification)，預測是哪一類。

例如：

垃圾郵件、正常郵件，貓、狗，數字 0~9，好瓜、壞瓜

常見演算法：

- KNN
- Decision Tree
- Random Forest
- Logistic Regression
- SVM
- Naive Bayes

- XGBoost
- LightGBM
- CNN（影像）
- RNN/LSTM（時間序列）
- Transformer（文字）

(2) 回歸(Regression)，預測數值

例如：

房價、股票價格、溫度、銷售額

常見演算法：

- Linear Regression
- Polynomial Regression
- Ridge
- Lasso
- Elastic Net
- SVR
- Random Forest Regression
- XGBoost Regression

➤ 非監督式學習（Unsupervised Learning）

定義：

沒有 Label，只有大量資料，希望讓電腦自己找出規律。

例如：10000 位客戶資料、沒有標示 VIP、沒有標示一般客戶，模型自己分群。

流程：

資料(X)→沒有答案(Label)→找規律→得到群組

常見工作：

(1) 分群(Clustering)

例如：

電商，學生 A、學生 B、學生 C...，自動分成
第一群、第二群、第三群、

常見演算法：

- K-Means
- GMM
- DBSCAN
- Hierarchical Clustering
- Mean Shift

(2) 降維

目的：將很多特徵，例如 1000 個特徵，變成 20 個特徵

方便：視覺化、降低運算量、去除雜訊

常見演算法：

- PCA
- t-SNE
- UMAP

(3) 異常偵測

例如：

信用卡盜刷、設備故障、網路攻擊

演算法：

- Isolation Forest
- One-Class SVM
- LOF

➤ 線性預測 (Linear Prediction)

線性預測通常指線性模型 (Linear Models)，屬於監督式學習，主要用於建立輸入與輸出之間的線性關係。

定義：

假設資料之間可以用一直線 (或超平面) 表示：

$$y = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

其中：x：特徵 (Feature)、w：權重 (Weight)、b：偏差 (Bias)、y：預測值

模型的目標是找到最合適的權重與偏差，使預測結果最接近真實值。

常見線性模型：

(1) 線性迴歸 (Linear Regression)

用途：預測連續數值

例如：房價預測、氣溫預測、銷售量預測

(2) Logistic Regression (邏輯斯迴歸)

名稱雖然有「Regression」，實際上是分類演算法。

用途：

- 是否違約
- 是否生病
- 是否為垃圾郵件

輸出通常是機率 (0~1)，再依門檻值分類。

➤ 線性模型特色

優點：

- 容易理解與解釋
- 訓練速度快
- 適合作為機器學習入門
- 可分析各特徵的重要影響方向

限制：

- 假設資料呈線性關係
- 對複雜的非線性問題表現有限
- 需要適當的特徵工程才能提升效果

各類型比較

類型	是否有 Label	主要目的	常見演算法	應用範例
監督式學習	✔ 有	分類、回歸	Linear Regression、Logistic Regression、KNN、Decision Tree、Random Forest、SVM、CNN、XGBoost	手寫數字辨識、垃圾郵件分類、房價預測
非監督式學習	✗ 無	分群、降維、異常偵測	K-Means、GMM、DBSCAN、PCA、t-SNE、UMAP	客戶分群、資料探索、異常交易偵測
強化式學習	回饋獎勵	學習最佳策略	Q-Learning、Deep Q Network (DQN)、Policy Gradient	遊戲 AI、機器人控制、自動駕駛

➤ 建議的教學順序（適合初學者）

1. 機器學習概觀：監督式、非監督式、強化式學習的差異。
2. 監督式學習：從線性迴歸開始，再介紹分類（Logistic Regression、KNN、Decision Tree、Random Forest、CNN）。
3. 非監督式學習：介紹 K-Means、GMM、PCA，讓學生理解資料探索與分群。
4. 實作案例：
 - **Linear Regression**：房價預測(Boston Housing 或 California Housing)
 - **KNN**：手寫數字辨識 (MNIST)
 - **Random Forest**：鳶尾花 (Iris) 分類
 - **CNN**：MNIST 手寫數字辨識
 - **K-Means**：客戶分群
 - **PCA**：資料降維與視覺化

這樣的安排能讓學生先理解機器學習的基本概念，再透過代表性的演算法建立完整的知識架構，並銜接到深度學習（如 CNN）與實際應用。

4.2 Linear Regression：

➤ 說明

Linear Regression（線性迴歸）是一種監督式學習（Supervised Learning）演算法。

目的：找出 X 與 Y 的線性關係，建立一條最佳直線，用來預測未知資料。

二、數學公式

Linear Regression 最重要公式：

$$\hat{y} = b_0 + b_1x$$

其中：

符號	意思
x	輸入(特徵)
$\hat{y}$	預測值
b_0	截距(Intercept)
b_1	斜率(Coefficient)

例如

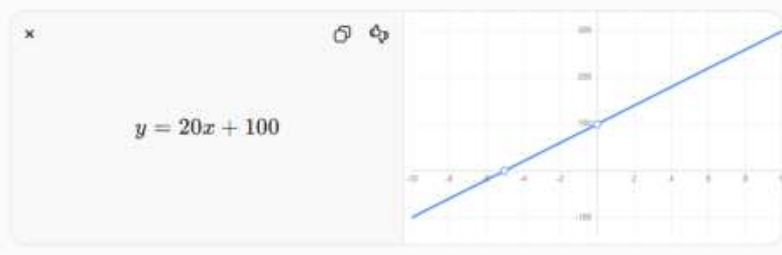
$$\hat{y} = 100 + 20x$$

三、圖形說明

下面就是公式

$$\hat{y} = 100 + 20x$$

所代表的直線。

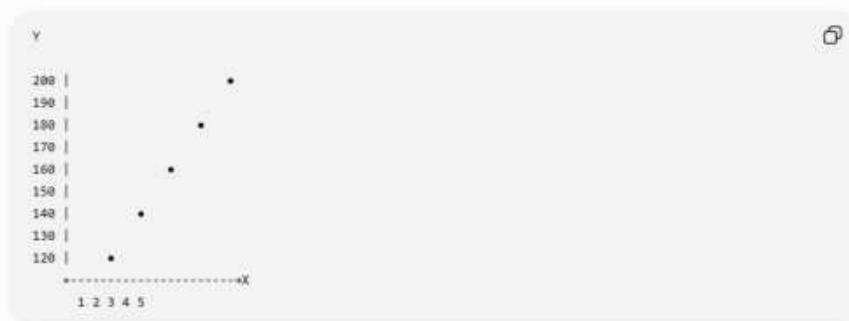


六、如果資料不是一直線？

例如

X	Y
1	118
2	141
3	159
4	182
5	198

畫面



這些點並沒有完全落在同一直線上。

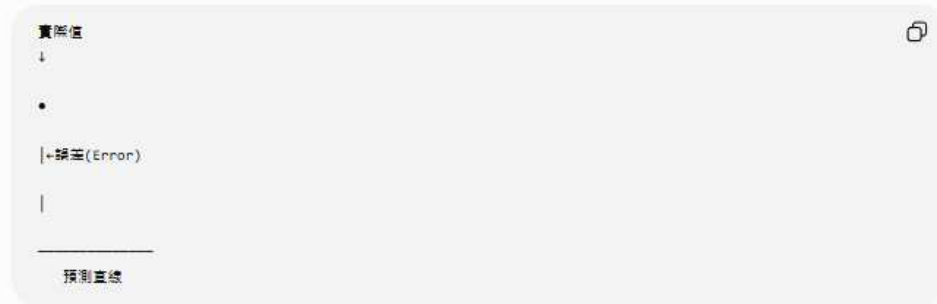
Linear Regression 會找到

最接近所有點的那條直線。

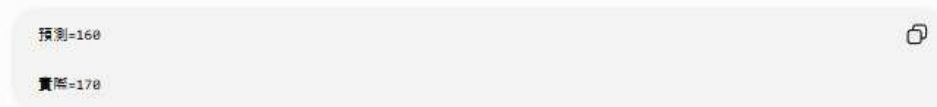


七、什麼叫 Least Squares (最小平方法) ?

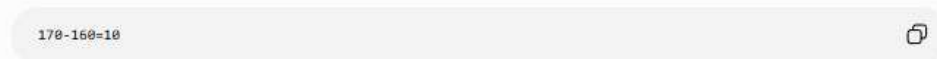
每個點都有誤差



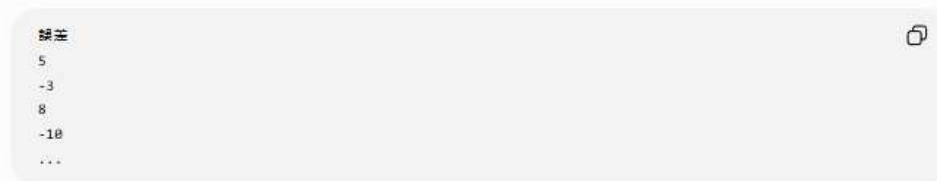
例如



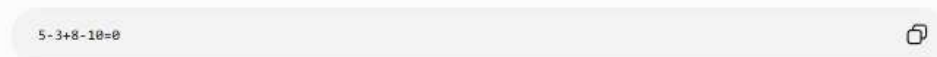
誤差



如果有很多點



直接相加



正負會互相抵消。

因此改用

平方

變成

$$5^2=25$$

$$(-3)^2=9$$

$$8^2=64$$

$$(-10)^2=100$$

全部加總

$$25+9+64+100$$

這就是

SSE (Sum of Squared Errors)

$$SSE = \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Linear Regression 的目的就是

找到一條直線，使 SSE 最小。

公式如下：

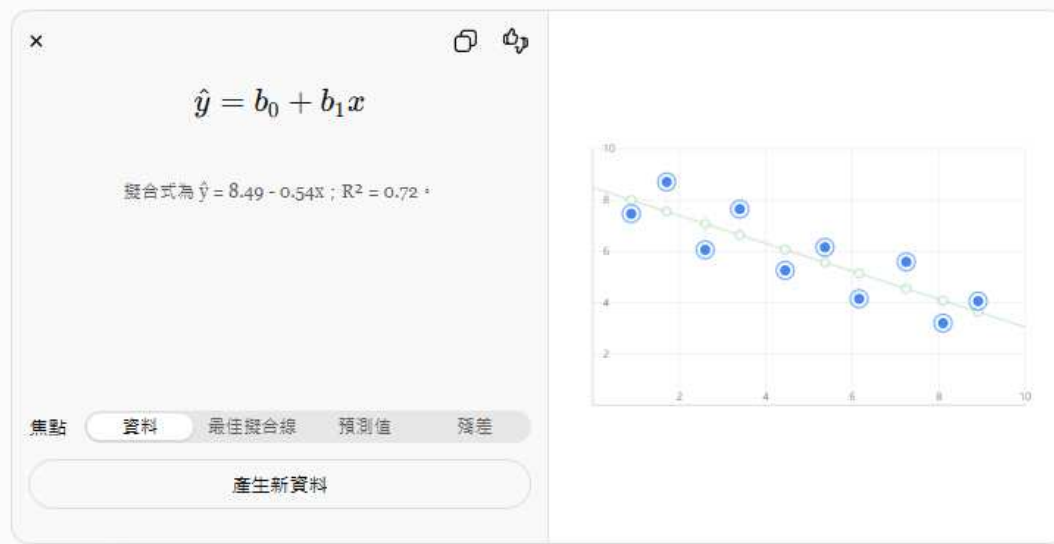
1. 單一變數線性迴歸

模型：

$$\hat{y}_i = b_0 + b_1 x_i$$

其中：

- b_0 ：截距
- b_1 ：斜率
- y_i ：第 i 筆實際值
- $\hat{y}_i$ ：第 i 筆預測值



最小平方誤差

最佳的 b_0, b_1 必須讓以下函數最小：

$$SSE(b_0, b_1) = \sum_{i=1}^n [y_i - (b_0 + b_1 x_i)]^2$$

因此最佳解可以寫成：

$$(\hat{b}_0, \hat{b}_1) = \arg \min_{b_0, b_1} \sum_{i=1}^n [y_i - (b_0 + b_1 x_i)]^2$$

4. 公式推導概念

平方誤差函數：

$$SSE = \sum_{i=1}^n (y_i - b_0 - b_1 x_i)^2$$

分別對 b_0 、 b_1 做偏微分，並令其等於 0：

$$\frac{\partial SSE}{\partial b_0} = 0$$

$$\frac{\partial SSE}{\partial b_1} = 0$$

得到常態方程式：

$$\sum_{i=1}^n (y_i - b_0 - b_1 x_i) = 0$$

$$\sum_{i=1}^n x_i (y_i - b_0 - b_1 x_i) = 0$$

整理後，即可得到：

$$\hat{b}_1 = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$$

$$\hat{b}_0 = \bar{y} - \hat{b}_1 \bar{x}$$

➤ 房價預測 1 (California Housing)

範例：利用房屋坪數預測房價

假設有以下資料：

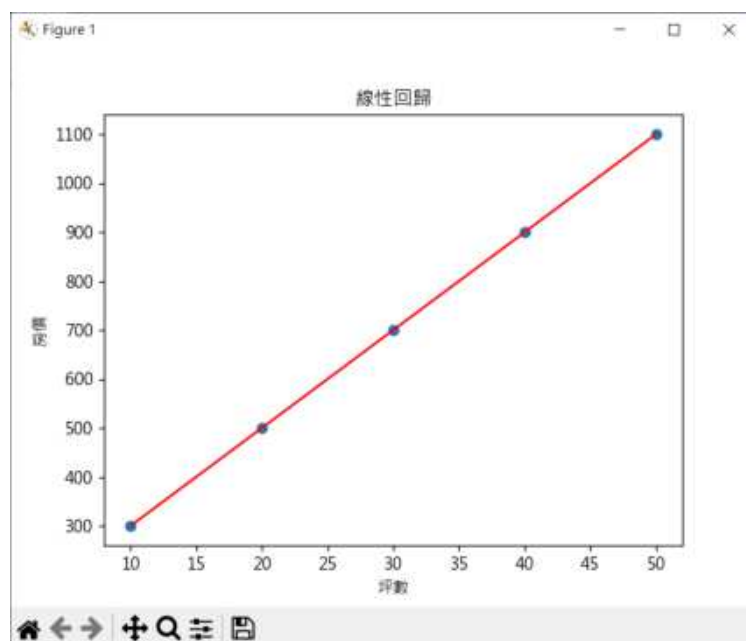
房屋坪數(X)	房價(Y, 萬元)
10	300
20	500
30	700
40	900
50	1100

可以畫成散佈圖：

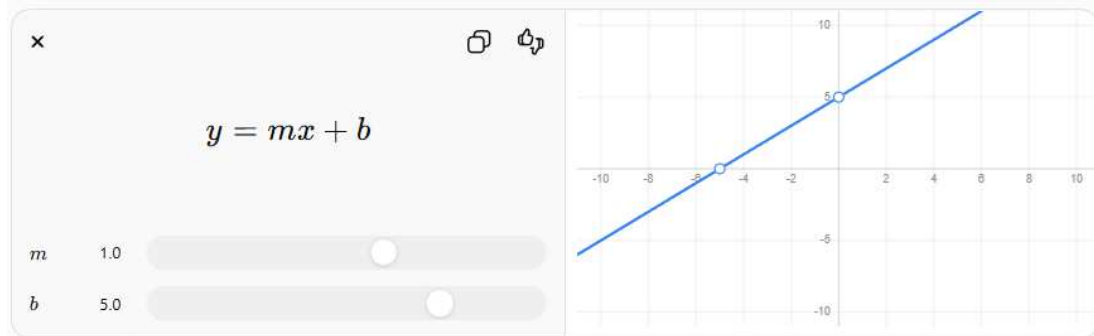


Linear Regression 就是在找一條最符合所有資料的直線。

結果如下：



在線性迴歸中，公式為：



其中：

- Y ：預測值（例如房價）
- X ：輸入值（例如坪數）
- b_0 ：截距（Intercept）
- b_1 ：斜率（Slope）

流程如下：

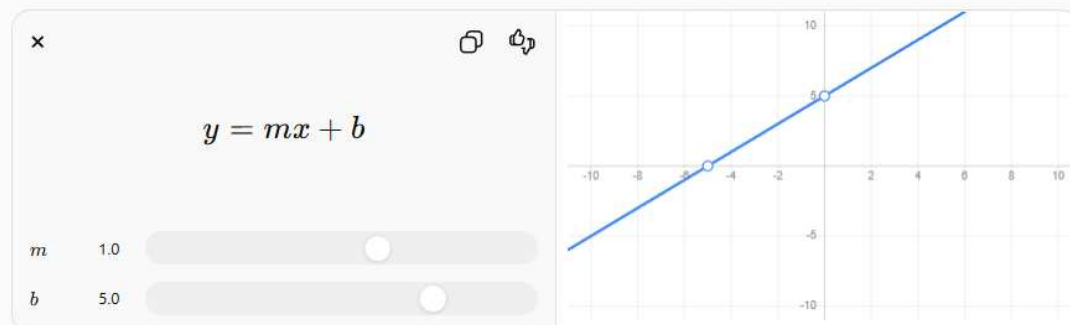
房屋資料(坪數、房價)→建立 X 、 y →Linear Regression→模型訓練 $\text{fit}()$ →得到方程式 $Y = 100 + 20X$ → $\text{predict}()$ →預測新房價

```
截距：100.00000000000011  
斜率：19.99999999999996  
35坪預測房價：800.0
```

其實就是因為浮點數(Floating Point) 的精度誤差，可以直接視為：

- 截距 = 100
- 斜率 = 20

因此，Linear Regression 得到的模型公式為：



也就是：

$$y = 20x + 100$$

其中：

- x ：房屋坪數
- y ：預測房價（萬元）

範例:LE01.py

➤ 房價預測 2 (California Housing)

原始資料：

LinearR

檔案 常用 插入 版面配置 公式 資料 校閱 檢視 開發人員 告訴我您想要執行的動作...

剪下 複製 貼上 複製格式 剪貼簿

新細明體 12 A A B I U 字型

自動換列 對齊方式 數值

K14

	A	B	C	D	E	F	G	H	I	J	K	L
	編號	行政區	坪數	屋齡	房間數	樓層	捷運距離	學校距離	有車位	房價萬元		
1												
2	1	台北市信義區	19.2	17	2	5	1103	1485	0	1534		
3	2	台南市東區	19.5	5	2	17	995	154	1	580		
4	3	新北市新店區	51.7	34	4	15	1818	669	1	2500		
5	4	新北市板橋區	50.8	21	4	7	1238	309	1	2776		
6	5	台北市信義區	35.9	22	3	2	1508	1198	1	3347		
7	6	台北市信義區	43.9	40	4	23	1581	193	0	3902		
8	7	新北市新店區	54.3	5	4	13	1053	1028	1	2779		
9	8	台中市西屯區	34.7	17	3	18	392	434	0	1328		
10	9	高雄市左營區	35.8	40	3	22	2381	1673	1	1087		
11	10	台北市大安區	55.8	25	4	7	1196	1570	1	5159		
12	11	台中市西屯區	28	31	3	5	1720	385	1	1061		
13	12	桃園市中壢區	53.1	27	5	12	2490	383	0	1473		
14	13	高雄市左營區	22.3	3	2	21	549	1722	0	727		
15	14	台南市東區	35.9	29	3	18	2267	1493	0	598		
16	15	台北市信義區	50	34	4	4	1192	990	1	4380		
17	16	桃園市中壢區	63.7	11	4	21	2179	1408	1	1989		
18	17	新北市新店區	25.2	10	1	20	2309	1100	1	1603		
19	18	桃園市中壢區	29.3	15	1	3	2423	1771	1	825		
20	19	台北市信義區	64	34	5	22	615	1225	1	5847		

共 120 筆，80%用於訓練，20%用於測試。

使用訓練模型：

Linear Regression

目標：

#預測新房子

例如：

#中壢區

#35 坪

#屋齡 8 年

#3 房

#12 樓

#捷運 400 公尺

#學校 600 公尺

#有車位

預測房價： 1022.9 萬元

流程如下：

CSV 房價資料 → 讀取資料(pandas) → 資料前處理(文字轉數字) → 建立 X/y
 → 切分 Train/Test → 建立 Linear Regression → 模型訓練 fit() → 模型預測
 predict() → 模型評估(MAE、RMSE、 R^2) → 預測新房價

步驟如下：

1、讀取 csv 資料

2、將文字轉成數字：

```
df = pd.get_dummies(df, columns=["行政區"], drop_first=True)
```

將行政區欄位轉換為虛擬變數，並刪除第一個欄位以避免多重共線性

例如：行政區_中壢區、行政區_桃園區、行政區_平鎮區、行政區_八德區
 全部變成 0、1，方便模型學習。

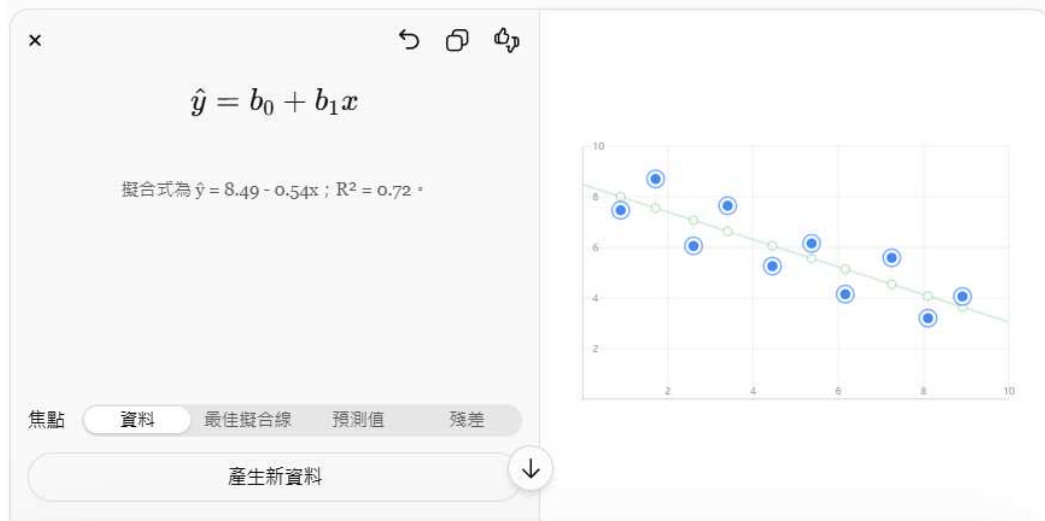
	A	B	C	D	E	F	G	H	I	J	K
1	編號	行政區	坪數	屋齡	房間數	樓層	捷運距離	學校距離	有車位	房價萬元	
2	1	台北市信義區	19.2	17	2	5	1103	1485	0	1534	
3	2	台南市東區	19.5	5	2	17	995	154	1	580	
4	3	新北市新店區	51.7	34	4	15	1818	669	1	2500	
5	4	新北市板橋區	50.8	21	4	7	1238	309	1	2776	
6	5	台北市信義區	35.9	22	3	2	1508	1198	1	3347	
7	6	台北市信義區	43.9	40	4	23	1581	193	0	3902	
8	7	新北市新店區	54.3	5	4	13	1053	1028	1	2779	
9	8	台中市西屯區	34.7	17	3	18	392	434	0	1328	
10	9	高雄市左營區	35.8	40	3	22	2381	1673	1	1087	
11	10	台北市大安區	55.8	25	4	7	1196	1570	1	5159	
12	11	台中市西屯區	28	31	3	5	1720	385	1	1061	
13	12	桃園市中壢區	53.1	27	5	12	2490	383	0	1473	
14	13	高雄市左營區	22.3	3	2	21	549	1722	0	727	
15	14	台南市東區	35.9	29	3	18	2267	1493	0	598	

轉換後如下：

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
1	序號	屋齡	坪數	樓層	樓層數	坪數	學校距離(公尺)	有車位	房價萬元	行政區_台北市信義區	行政區_台北市大安區	行政區_台北市板橋區	行政區_台北市新店區	行政區_台北市板橋區	行政區_桃園市中壢區	行政區_桃園市平鎮區
2	19.2	2	5	1103	1485	0	1534	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
3	19.5	2	17	995	154	1	580	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
4	34.7	4	15	1818	669	1	2500	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE	FALSE
5	50.8	4	7	1238	309	1	2776	FALSE	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE	FALSE
6	55.9	3	2	1508	1198	1	3347	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE	FALSE
7	43.9	4	23	1581	193	0	3902	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	TRUE	FALSE	FALSE
8	21.5	4	7	1196	1570	1	5159	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
9	22	3	5	1720	385	1	1061	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
10	27	5	12	2490	383	0	1473	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
11	31	3	21	549	1722	0	727	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE
12	29	3	18	2267	1493	0	598	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE	FALSE

Linear Regression 公式：

Linear Regression 公式



3、建立 X、y

X (Features, 特徵、輸入資料): 模型拿來學習的資料

坪數	屋齡	海墘距離	樓層	房屋距離山尺	學校距離山尺	海邊位	行政區_台北市信義區	行政區_台北市大安區	行政區_台北市東區	行政區_台北市松山區	行政區_台北市中山區	行政區_台北市中正區	行政區_台北市南港區
0	28.2	17	2	5	1883	9489	0	True	False	False	False	False	False
1	18.5	5	2	17	995	154	1	False	False	True	False	False	False
2	31.7	34	4	15	1818	869	1	False	False	True	False	False	False
3	18.8	21	4	2	1218	389	1	False	False	False	True	False	False
4	15.9	12	1	2	1969	1798	1	True	False	False	False	False	False

y (Target, Label、目標值): 模型要預測的答案

0	1534
1	580
2	2500
3	2776
4	3347

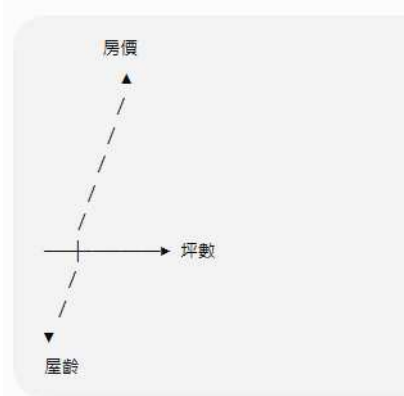
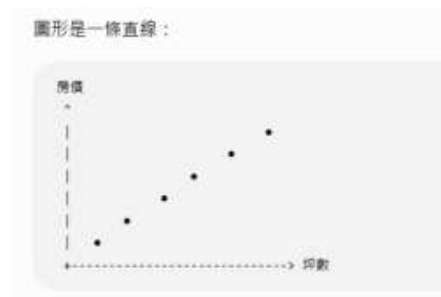
4、切分 Train/Test

20%的數據用於測試, 80%的數據用於訓練

5、建立 Linear Regression 與模型訓練 fit()

若只有一個特徵(坪數), 模型就是圖形是一條直線。

如果特徵超過兩個(例如你的模型有 10 個特徵), 就變成超平面 (Hyperplane), 已經無法直接畫在三維空間中。



這是一個平面（如果只有兩個特徵）：



6、模型預測與模型預測 predict()

預測資料如下：

	A	B	C	D	E	F	G	H	I	J	K	L	M	N							
1	坪數	屋齡	房間數	樓層	建築距離公尺	學校距離公尺	和車位	行啟區	台北市信義區	行啟區	台北市大安區	行啟區	台南市東區	行啟區	新北市新店區	行啟區	新北市板橋區	行啟區	桃園市中原區	行啟區	桃園市五股區
2	61.6	12	5	6	183	1080	0	FALSE		TRUE		FALSE	FALSE	FALSE	FALSE		FALSE	FALSE	FALSE		FALSE
3	29.6	12	3	2	301	1073	0	FALSE		FALSE		FALSE	FALSE	FALSE	FALSE		FALSE	FALSE	FALSE		FALSE
4	33.6	36	3	7	962	1133	1	FALSE		FALSE		FALSE	FALSE	FALSE	FALSE		FALSE	FALSE	FALSE		FALSE
5	34.5	32	3	1	1731	834	0	FALSE		FALSE		FALSE	FALSE	FALSE	FALSE		FALSE	FALSE	FALSE		FALSE
6	34.2	35	4	15	2525	486	1	FALSE		FALSE		FALSE	FALSE	FALSE	FALSE		FALSE	FALSE	FALSE		FALSE
7	36.8	35	2	21	201	130	1	TRUE		FALSE		FALSE	FALSE	FALSE	FALSE		FALSE	FALSE	FALSE		FALSE
8	39.1	29	4	3	1276	807	1	FALSE		FALSE		FALSE	FALSE	FALSE	FALSE		FALSE	FALSE	FALSE		FALSE
9	44.2	7	4	15	1969	1244	1	FALSE		TRUE		FALSE	FALSE	FALSE	FALSE		FALSE	FALSE	FALSE		FALSE
10	51.1	3	3	2	1037	245	1	TRUE		FALSE		FALSE	FALSE	FALSE	FALSE		FALSE	FALSE	FALSE		FALSE
11	29.2	19	2	25	1072	139	0	FALSE		FALSE		TRUE	FALSE	FALSE	FALSE		FALSE	FALSE	FALSE		FALSE

結果如下：

	實際房價	預測房價
0	5870	4651.396271
1	565	411.858522
2	1821	1640.955176
3	834	279.166959
4	1127	890.420510
5	3789	3926.570905
6	1482	1353.482006
7	4239	4139.871969
8	4805	4370.453261
9	487	222.584098

7、模型評估

- MAE: mean absolute error(平均絕對誤差)

1. MAE (Mean Absolute Error · 平均絕對誤差)

MAE 表示預測值與真實值誤差的平均值，不考慮誤差方向（全部取絕對值）。

公式

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

其中：

- n ：資料筆數
- y_i ：第 i 筆真實值 (Actual)
- $\hat{y}_i$ ：第 i 筆預測值 (Predicted)
- $||$ ：取絕對值

範例

真實值	預測值	誤差	絕對誤差
100	95	5	5
120	125	-5	5
90	85	5	5

$$MAE = \frac{5 + 5 + 5}{3} = 5$$

代表平均每筆資料預測誤差約 5 單位。

- RMSE: mean squared error(中文:均方誤差)

2. RMSE (Root Mean Squared Error , 均方根誤差)

RMSE 是先將誤差平方、求平均，再開根號，因此對較大的誤差會給予較大的懲罰。

公式

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

其中：

- n ：資料筆數
- y_i ：真實值
- $\hat{y}_i$ ：預測值

注意：RMSE 是「均方根誤差 (Root Mean Squared Error)」，而**均方誤差 (Mean Squared Error, MSE) **則是：

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

也就是 $RMSE = \sqrt{MSE}$ 。

範例

真實值	預測值	誤差	誤差平方
100	95	5	25
120	125	-5	25
90	85	5	25

$$RMSE = \sqrt{\frac{25 + 25 + 25}{3}} = \sqrt{25} = 5$$

- “MAE：每個誤差權重相同，容易理解，代表平均誤差。”
- “RMSE：大誤差會被放大，能更有效反映模型是否存在嚴重預測錯誤。”
- “兩者皆是越小越好，表示模型預測越準確。”

8、模型評估(MAE、RMSE、R²)

R² 公式

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

其中：

- y_i ：真實值
- $\hat{y}_i$ ：模型預測值
- $\bar{y}$ ：所有真實值的平均值

公式意義

公式可拆成兩部分：

分子 (Residual Sum of Squares, RSS)

$$\sum (y_i - \hat{y}_i)^2$$

表示：

模型預測的誤差總和

越小越好。

分母 (Total Sum of Squares, TSS)

$$\sum (y_i - \bar{y})^2$$

表示：

資料本身的總變異

代表資料原本有多少變化。

因此

$$R^2 = 1 - \frac{\text{模型誤差}}{\text{資料總變異}}$$

表示：

模型解釋了多少比例的資料變異。

R² 的範圍

通常：

$$-\infty < R^2 \leq 1$$

R ²	意義
1.0	完全預測正確
0.9	非常好
0.8	良好
0.6	尚可
0	與直接用平均值預測一樣
<0	比直接預測平均值還差

範例

假設有 5 筆房價資料：

房屋	真實值 y	預測值 $\hat{y}$
A	500	490
B	600	610
C	700	690
D	800	790
E	900	910

第一步：求平均值

真實房價平均：

$$\bar{y} = \frac{500 + 600 + 700 + 800 + 900}{5} = 700$$

第二步：計算 RSS (Residual Sum of Squares · 殘差平方和)

公式：

$$RSS = \sum (y_i - \hat{y}_i)^2$$

逐筆計算：

真實值	預測值	誤差	誤差平方
500	490	10	100
600	610	-10	100
700	690	10	100
800	790	10	100
900	910	-10	100

因此

$$RSS = 100 + 100 + 100 + 100 + 100 = 500$$

第三步：計算 TSS (Total Sum of Squares · 總平方和)

公式：

$$TSS = \sum (y_i - \bar{y})^2$$

因為平均值為 700：

真實值	平均值	差值	差值平方
500	700	-200	40000
600	700	-100	10000
700	700	0	0
800	700	100	10000
900	700	200	40000

因此

$$TSS = 40000 + 10000 + 0 + 10000 + 40000 = 100000$$

第四步：代入 R² 公式

公式：

$$R^2 = 1 - \frac{RSS}{TSS}$$

代入數值：

$$\begin{aligned} R^2 &= 1 - \frac{500}{100000} \\ &= 1 - 0.005 \\ &= 0.995 \end{aligned}$$

結果

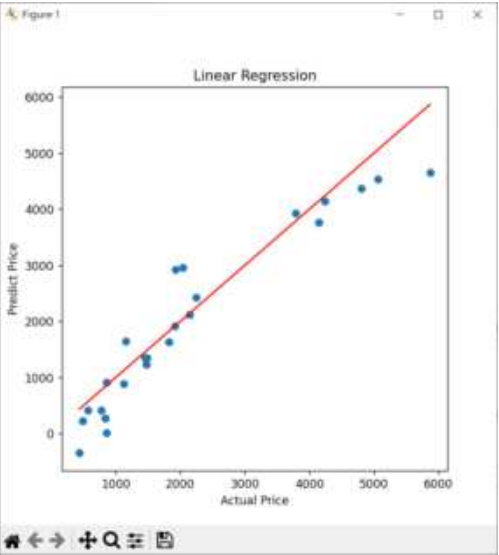
$R^2 = 0.995$

表示

模型可以解釋 99.5% 的房價變異。



實際房價與預測房價的圖表結果如下：



如果預測效果好，大部分點會靠近紅色對角線。

9、預測新房價：



範例:LE02.py

➤ 房價預測(已訓練好模型+Django Web)

```
# pip install Django
# Django-admin startproject Leweb
# cd Leweb
# python manage.py startapp myapp
# mkdir templates
# mkdir static
修改 settings.py

ALLOWED_HOSTS = ['*']

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'myapp',
]

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [BASE_DIR / 'templates'],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]
```

```
LANGUAGE_CODE = 'zh-Hant'

TIME_ZONE = 'Asia/Taipei'

STATIC_URL = 'static/'
STATICFILES_DIRS = [
    BASE_DIR / 'static',
]

# python manage.py makemigrations (建立 migration 資料檔)
# python manage.py migrate (模型與資料庫同步)
# python manage.py runserver 0.0.0.0:8080
```

尋問 ChaptGPT:

欄位名稱:

坪數,屋齡,房間數,樓層,捷運距離公尺,學校距離公尺,有車位,行政區_台北市信義區,行政區_台北市大安區,行政區_台南市東區,行政區_新北市新店區,行政區_新北市板橋區,行政區_桃園市中壢區,行政區_高雄市左營區

資料:

59.3,23,5,15,1267,1540,1,True,False,False,False,False,False,False

利用 Linear Regression 建立模型，
LinearRegressionModel.pkl

我想建立一個 django 網站使用初學者的程式寫法，
專案名為 Lewweb，app 名為 myapp
讓使用者輸入如下的資料:

```
new_house = pd.DataFrame({
    "坪數":[35],
    "屋齡":[8],
    "房間數":[3],
    "樓層":[12],
    "捷運距離公尺":[400],
    "學校距離公尺":[600],
    "有車位":[1],
```

```
"行政區_台北市信義區":[0],  
"行政區_台北市大安區":[0],  
"行政區_台南市東區":[0],  
"行政區_新北市新店區":[0],  
"行政區_新北市板橋區":[0],  
"行政區_桃園市中壢區":[1],  
"行政區_高雄市左營區":[0]  
})
```

其中

```
"行政區_台北市信義區":[0],  
  
"行政區_台北市大安區":[0],  
  
"行政區_台南市東區":[0],  
  
"行政區_新北市新店區":[0],  
  
"行政區_新北市板橋區":[0],  
  
"行政區_桃園市中壢區":[1],  
  
"行政區_高雄市左營區":[0]
```

讓使用者選擇

最後產生預測房價結果

生成式 AI 範例 範例:Django 房價預測網站(LE).mhtml

結果:

房價預測系統

問問 Gemini

192.168.57.246:8080

不安全

應用程式

登入您的帳戶

瀏覽管理系統

桃竹苗分署學員學...

所有書籤

Linear Regression 房價預測系統

坪數
35

樓數
8

樓層數
3

樓層
12

捷運距離 (公尺)
400

學校距離 (公尺)
600

是否有車位
有車位

行政區
桃園市中區區

預測房價

預測結果
預測房價：1049.51 萬元

4.2 KNN (K-Nearest Neighbors, K 最近鄰演算)

➤ 說明

KNN (K-Nearest Neighbors) 是一種監督式學習 (Supervised Learning) 演算法。它的核心概念非常直觀：「物以類聚，人以群分」當有一筆新的資料要判斷時，KNN 會找出距離它最近的 K 個已知資料，再依照這些鄰居的結果進行預測。KNN 不會先建立數學公式，而是：看附近有哪些資料，再投票決定答案。KNN 最重要的是計算距離，最常用的是歐氏距離 (Euclidean Distance)：

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

例如

A

(150, 45)

B

(160, 50)

距離

$$\begin{aligned}
 d &= \sqrt{(160 - 150)^2 + (50 - 45)^2} \\
 &= \sqrt{100 + 25} \\
 &= \sqrt{125} \\
 &= 11.18
 \end{aligned}$$

距離越小，代表兩筆資料越相似。

KNN 的優點：

- 演算法簡單，容易理解。
- 不需要建立複雜模型。
- 適合小型資料集。
- 可用於分類與迴歸。

KNN 的缺點：

- 資料量大時，預測速度較慢 (需要與所有資料比較距離)。
- 對雜訊 (Noise) 較敏感。
- K 值選擇會影響結果。
- 不同特徵尺度差異大時，需先進行資料標準化 (Standardization) 或正規化 (Normalization)。

➤ 利用身高來判斷男生或女生，使用 knn 演算法

舉例：假設要判斷一個人是男生還是女生。

編號	身高(cm)	性別
1	150	女
2	155	女
3	160	女
4	170	男
5	175	男
6	180	男

現在來一位新的人、身高 = 165 cm、想知道是男還是女。

步驟如下：

第一步：計算距離

KNN就是先算距離。

距離公式（一維）

$$\text{距離} = |A - B|$$

例如

與150距離

$$|165 - 150| = 15$$

全部算出來		
身高	性別	距離
150	女	15
155	女	10
160	女	5
170	男	5
175	男	10
180	男	15

第二步：選 K 個最近

假設

K=3

最近三位

身高	性別	距離
160	女	5
170	男	5
155	女	10



160	女	+5
170	男	+5
155	女	+10
175	男	+10
150	女	+15
180	男	+15

第三步：投票

女生：
2票
男生：
1票
因此
預測結果：女生

只取最近三位

165		
160	女	✓
170	男	✓
155	女	✓
女生：2票		
男生：1票		

如果 $K=5$

最近五位

身高	性別
160	女
170	男
155	女
175	男
150	女

女生
3票

男生
2票

結果仍然是

女生

流程：

訓練資料 → 不建立公式 → 輸入新資料 → 計算與所有資料的距離 → 找最近 K 個鄰居 → 投票 → 輸出分類結果

平手問題：

方法一： K 選奇數（就沒有平手問題）

方法二：看誰距離最近

例如

身高	性別	距離
166	男	1
164	女	1

仍然平手。

如果改成

身高	性別	距離
166	男	1
160	女	5

則可以選距離最近的男生。

方法三：距離加權投票（Weighted KNN）

例如

身高	性別	距離	權重 $1/d$
164	女	1	1.00
170	男	5	0.20

雖然都是一票，
但是

女生權重：1.00

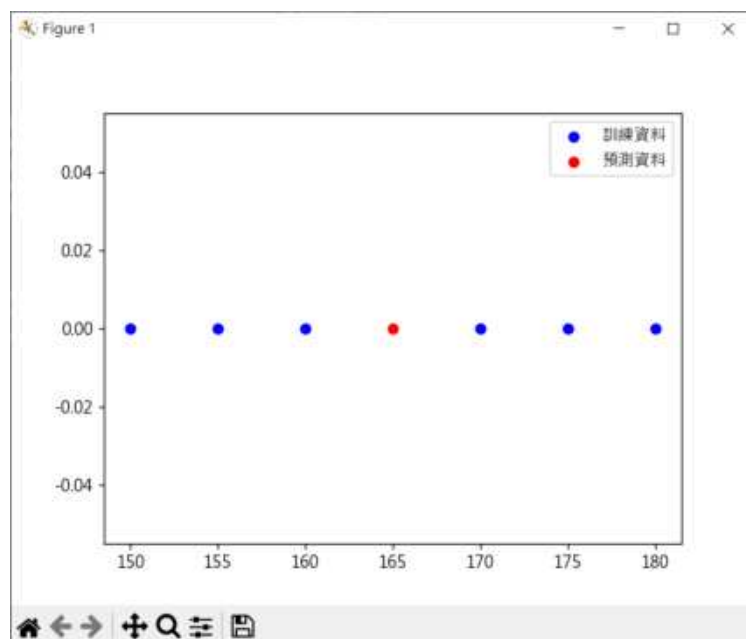
男生權重：0.20

因此

預測女生

↓

程式結果如下：



範例：knn01.py

KNN 與 Linear Regression 的比較		
比較項目	Linear Regression	KNN
模型類型	建立一條最佳迴歸線	不建立公式，直接比鄰居
學習方式	訓練時求係數	只記住所有訓練資料
預測方式	代入公式計算	找最近的 K 個資料投票
適合用途	連續數值預測（如房價、溫度）	分類（如是否為垃圾郵件、手寫數字、是否生病）
是否需要距離	否	是（通常使用歐氏距離）

➤ 利用 knn 的手寫數字辨識系統(使用學者的資料庫與方法)

Reference:

<https://archive.ics.uci.edu/ml/datasets/Optical+Recognition+of+Handwritten+Digits>

Reference paper:

Methods of Combining Multiple Classifiers and Their Applications to Handwritten Digit Recognition

Class Labels

Class: No of examples in training set

0: 376
1: 389
2: 380
3: 389
4: 387
5: 376
6: 377
7: 387
8: 380
9: 382

Class: No of examples in testing set

0: 178
1: 182
2: 177
3: 183
4: 181
5: 182
6: 181
7: 179
8: 174
9: 180



```
(myExam) PS D:\backup\myExam\machineLearning> d:; cd 'd:\backup\myExam\machineLearning'; & 'd:\backup\myE
2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '52060' '--' 'D:\backup\myExam\machineLearning\knn02.py'
Accuracy: 0.9833333333333333
```

範例:knn02.py

➤ 利用 knn 的手寫數字辨識系統（匯出資料庫成圖片、匯出模型、
測試圖片）

1、將現有資料庫匯出成圖片：

尋問 chatGPT:

我是一位初階 python 程師

我的訓練來源為: `digits = load_digits()` #共有 1797 筆資料，每筆資料是 8x8 的灰階
圖片，標籤是 0~9 的數字。

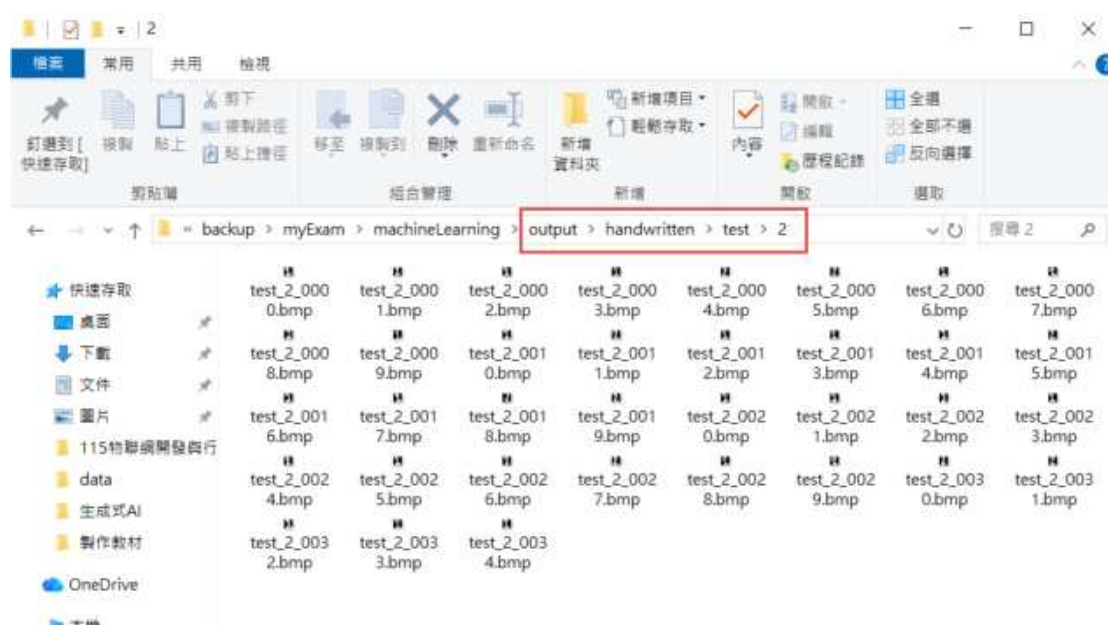
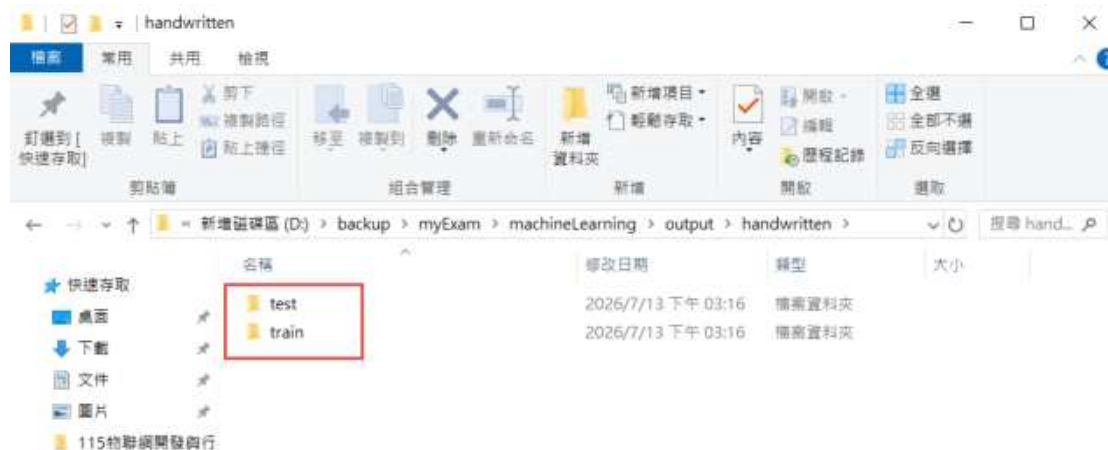
我想要將資料匯出成圖片檔案，並依照標籤分類存放。

將全部資料存成圖片 bmp 檔到 output 中的 handwritten 資料夾，並分類別存放
「train」、「test」兩類，數量為 80%與 20%。

另外，要顯示執行完成度、程式寫法 mac 也要能使用，請給我程式 python 碼。

生成式 AI 範例: Python 圖片匯出分類.mhtml

結果如下:



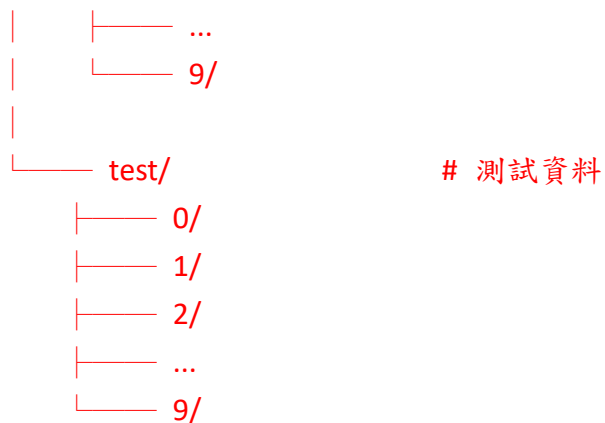
範例:knn03.py

2、讀取現有的資料庫，再利用 KNN 訓練與測試，最後匯出模型「knn_model.pkl」尋問 chatGPT：

我的訓練與測試資料夾如下：

```
data/
├── handwritten/
│   ├── train/
│   │   ├── 0/
│   │   ├── 1/
│   │   └── 2/
│   └── test/
│       ├── 0/
│       ├── 1/
│       └── 2/
```

訓練資料
數字 0
數字 1



內的圖片為.bmp，名稱無固定

請給我簡單好理解的 python 程式，將這些資料讀取。

利用 knn 訓練，再進行測式。

生成式 AI 範例: KNN 訓練與測試.mhtml

結果如下：

```
PS D:\backup\myExam\machineLearning> (Set-ExecutionPolicy -Scope Process -ExecutionPolicy RemoteSigned) PS D:\backup\myExam\machineLearning> & 'd:\backup\myExam\Scripts\python.exe' ncher' '64733' '--' 'D:\backup\myExam\machineLearning\knn04.py'
```

```
=====
```

```
讀取訓練資料
```

```
=====
```

```
數字 0: 讀取 142 張圖片
```

```
數字 1: 讀取 146 張圖片
```

```
數字 2: 讀取 142 張圖片
```

```
數字 3: 讀取 146 張圖片
```

```
數字 4: 讀取 145 張圖片
```

```
數字 5: 讀取 145 張圖片
```

```
數字 6: 讀取 145 張圖片
```

```
數字 7: 讀取 143 張圖片
```

```
數字 8: 讀取 139 張圖片
```

```
數字 9: 讀取 144 張圖片
```

```
=====
```

```
讀取測試資料
```

```
=====
```

```
數字 0: 讀取 36 張圖片
```

```
數字 1: 讀取 36 張圖片
```

```
數字 2: 讀取 35 張圖片
```

```
數字 3: 讀取 37 張圖片
```

```
數字 4: 讀取 36 張圖片
```

```
數字 5: 讀取 37 張圖片
```

```
數字 6: 讀取 36 張圖片
```

```
數字 7: 讀取 36 張圖片
```

```
數字 8: 讀取 35 張圖片
```

```
數字 9: 讀取 36 張圖片
```

```

=====
資料資訊
=====
訓練圖片數量： 1437
測試圖片數量： 360
每張圖片的特徵數量： 64
訓練資料形狀： (1437, 64)
訓練標籤形狀： (1437,)
測試資料形狀： (360, 64)
測試標籤形狀： (360,)

=====
開始訓練 KNN 模型
=====
KNN 模型訓練完成

=====
開始測試
=====
測試正確率：93.89%

```

```

=====
分類報告
=====

```

	precision	recall	f1-score	support
0	0.95	0.97	0.96	36
1	0.77	1.00	0.87	36
2	0.95	1.00	0.97	35
3	0.93	1.00	0.96	37
4	1.00	0.97	0.99	36
5	0.97	1.00	0.99	37
6	0.97	0.94	0.96	36
7	0.97	0.97	0.97	36
8	0.96	0.66	0.78	35
9	1.00	0.86	0.93	36
accuracy			0.94	360
macro avg	0.95	0.94	0.94	360
weighted avg	0.95	0.94	0.94	360

範例:knn04.py

其中讀取圖片的前處理流程如下：

開始



讀取 0~9 資料夾

- ▶ 讀取 BMP 圖片
- ▶ 灰階化
- ▶ 縮放成 8×8
- ▶ 二值化
- ▶ 正規化 (0~1)
- ▶ 攤平成 64 維特徵
- ▶ 加入訓練資料 (X)
- ▶ 加入標籤 (y)
- ▶ 全部圖片完成
- ▶ 回傳 X、y、file_paths
- ▶ 結束

3、讀取「knn_model.pkl」，再利用一樣的前處理方式，預測手繪文字。

將 knn04.py 程式所產生出來的模型，將 output 資料夾內的「knn_model.pkl」，放到 data 資料夾內「knn_model.pkl」。

使用小畫家寫出圖片（64x64）並存檔成「hand_sample01.png」

一定要用圖形大約：8x8（不然縮小後會變形）、背景為黑色、前景白色





尋問 chatGPT:

```
def load_images(folder_path):
```

```
    """
```

```
    讀取資料夾中的 BMP 圖片。
```

```
    資料夾格式：
```

```
    folder_path/
```

```
    |—— 0/
```

```
    |—— 1/
```

```
    |—— ...
```

```
    |—— 9/
```

```
    回傳：
```

```
    X：圖片資料
```

```
    y：圖片標籤
```

```
    file_paths：圖片路徑
```

```
    """
```

```
    images = []
```

```
    labels = []
```

```
    file_paths = []
```

```
    # 依序讀取數字 0 到 9
```

```
    for label in range(10):
```

```
        # 例如：data/handwritten/train/0
```

```
        number_folder = os.path.join(folder_path, str(label))
```

```
        # 檢查資料夾是否存在
```

```
if not os.path.exists(number_folder):
    print(f"找不到資料夾：{number_folder}")
    continue

# 取得資料夾內所有檔案
filenames = os.listdir(number_folder)

# 計算成功讀取的圖片數量
image_count = 0

for filename in filenames:

    # 只處理 BMP 圖片
    if not filename.lower().endswith(".bmp"):
        continue

    image_path = os.path.join(number_folder, filename)

    # 以灰階方式讀取圖片
    image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

    # 如果讀取失敗
    if image is None:
        print(f"圖片讀取失敗：{image_path}")
        continue

    # 將圖片統一成 28 x 28
    image = cv2.resize(
        image,
        (IMAGE_WIDTH, IMAGE_HEIGHT)
    )

    # 二值化
    # 大於 127 的像素變成 255
    # 小於或等於 127 的像素變成 0
    _, image = cv2.threshold(
        image,
```

```

        127,
        255,
        cv2.THRESH_BINARY
    )

    # 將像素由 0~255 轉成 0~1
    image = image.astype("float32") / 255.0

    # 將 28 x 28 圖片攤平成 784 個數字
    image_data = image.flatten()

    # 儲存圖片資料
    images.append(image_data)

    # 儲存正確答案
    labels.append(label)

    # 儲存檔案路徑
    file_paths.append(image_path)

    image_count += 1

print(f"數字 {label}：讀取 {image_count} 張圖片")

# 轉成 NumPy 陣列
X = np.array(images)
y = np.array(labels)

return X, y, file_paths

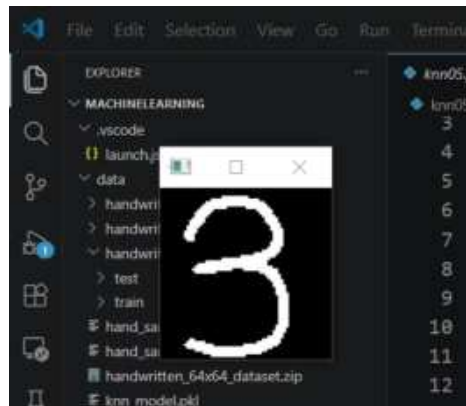
```

讀取文字與前處理的程式如下，請產生一個新程式，假設我已建立好的 knn 模式為「./data/knn_model.pkl」

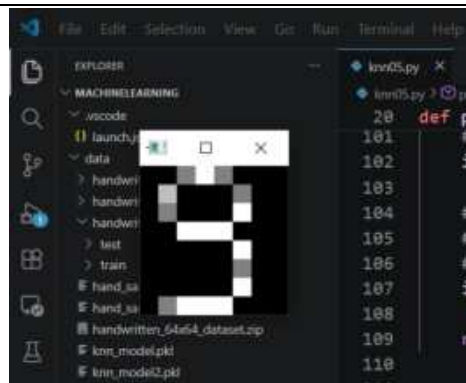
我新手寫的圖片檔為 hand_sample01.png，請讀取此檔並做前處理，再預測

生成式 AI 範例: 手寫數字預測程式.mhtml

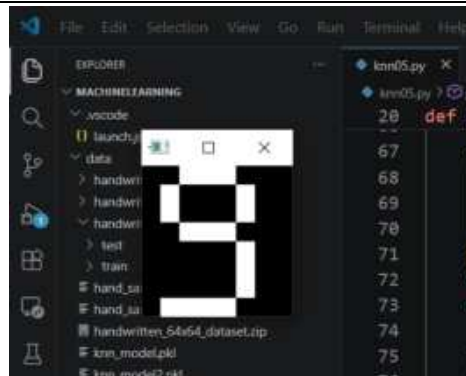
顯示原圖，並放大成 64 x 64，方便觀看，並顯示視窗 1



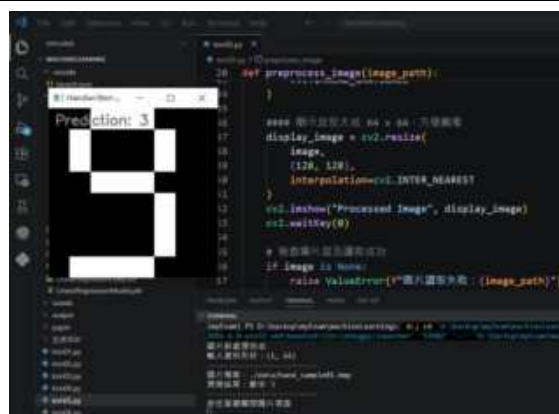
將圖片調整成 8x8，並顯示視窗 2



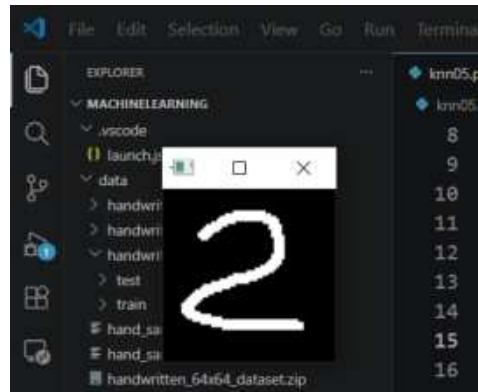
二值化，並顯示視窗 3



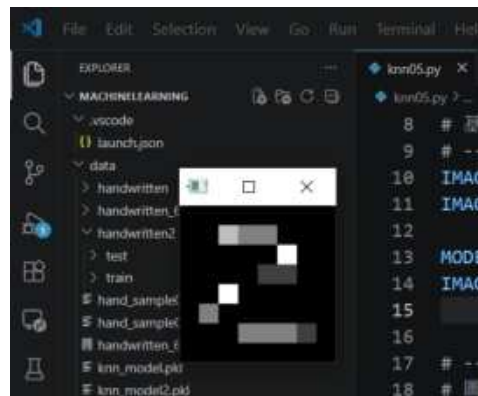
預測結果，並顯示視窗 4



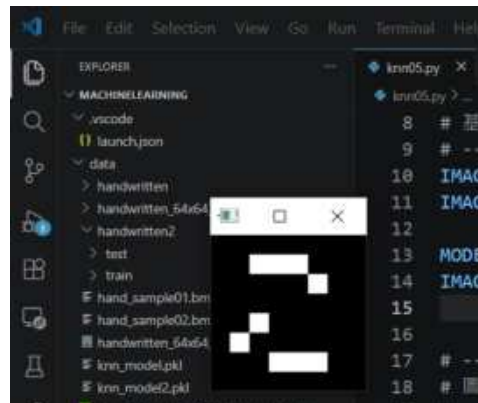
顯示原圖，並放大成 64 x 64，方便觀看，並顯示視窗 1



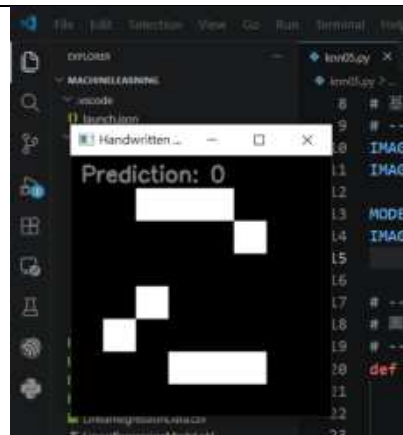
將圖片調整成 8x8，並顯示視窗 2



二值化，並顯示視窗 3



預測結果，並顯示視窗 4



問題：

1、讀取訓練資料庫圖片太小 8x8，容易失真

範例:knn05.py

➤ 利用 knn 的手寫數字辨識系統修改 1(重新產生與訓練資料庫)

尋問 chatGPT:

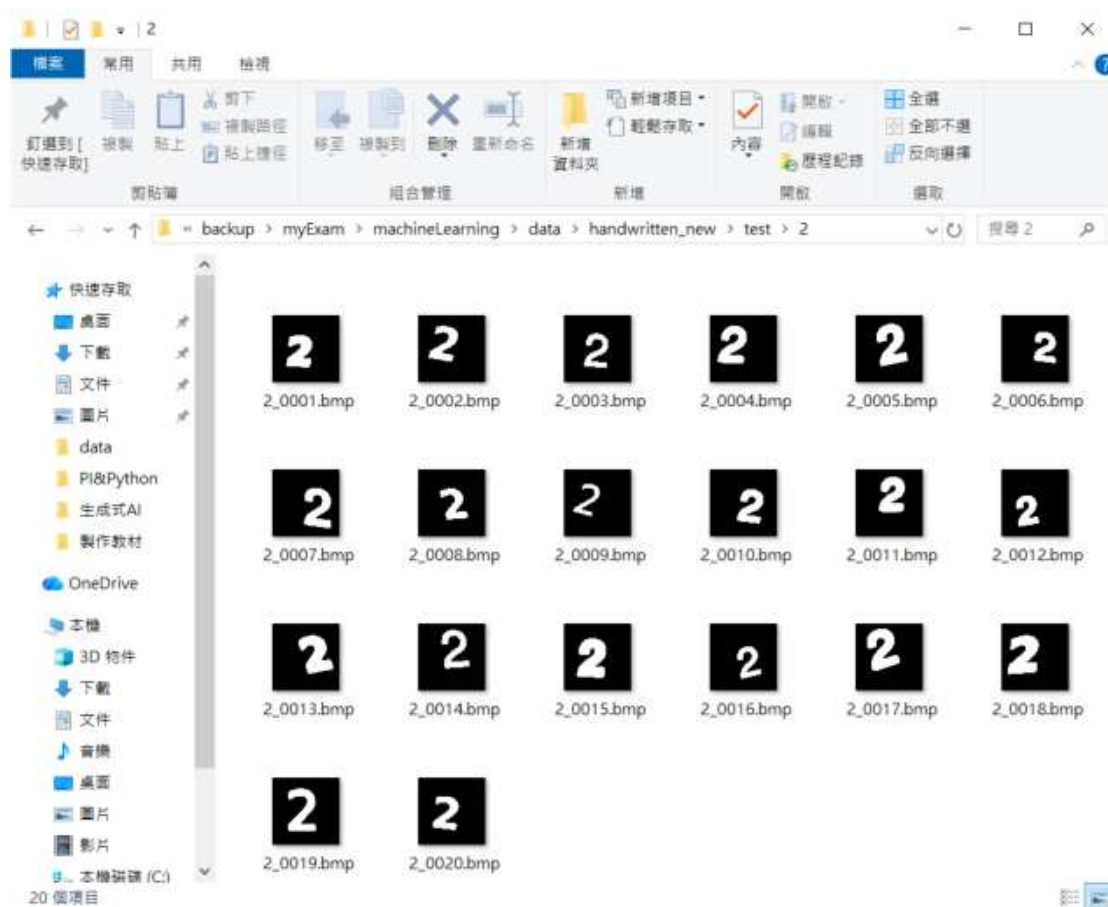
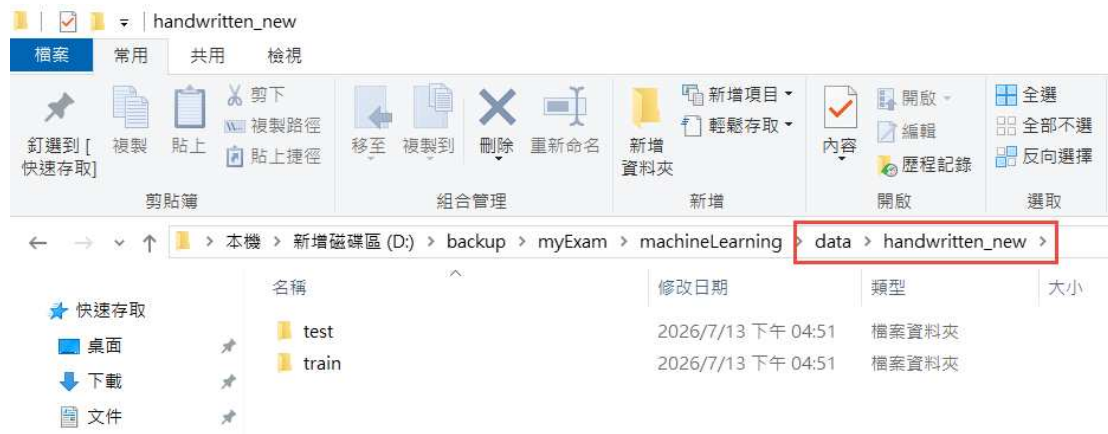
請幫我產生 64x64 的手寫資料庫，train1000 張

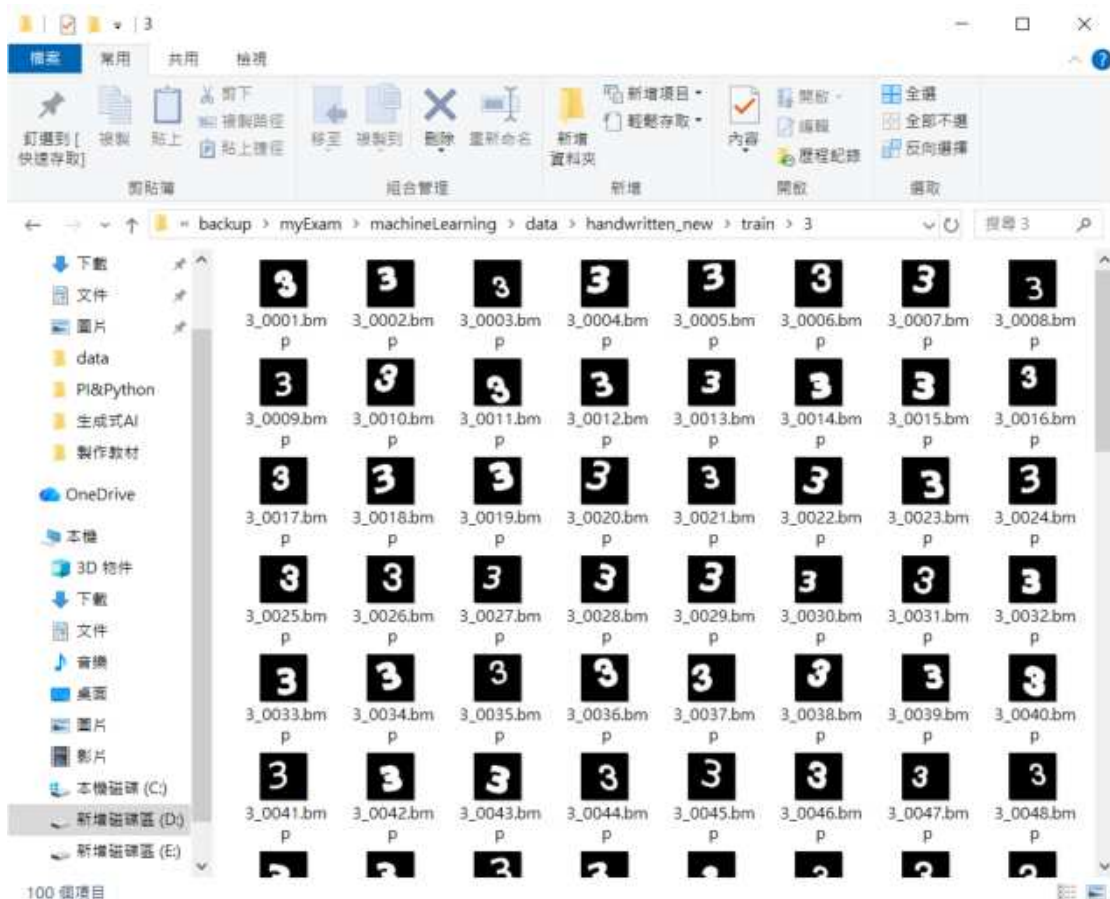
```
data/
├── handwritten/
│   ├── train/                # 訓練資料
│   │   ├── 0/                # 數字 0
│   │   ├── 1/                # 數字 1
│   │   ├── 2/
│   │   ├── ...
│   │   └── 9/
│   └── test/                 # 測試資料
│       ├── 0/
│       ├── 1/
│       ├── 2/
│       ├── ...
│       └── 9/
```

生成式 AI 範例:手寫資料庫生成.mhtml

檔案: handwritten_64x64_dataset.zip

將產生的資料庫複製並重新命名為「handwritten_new」

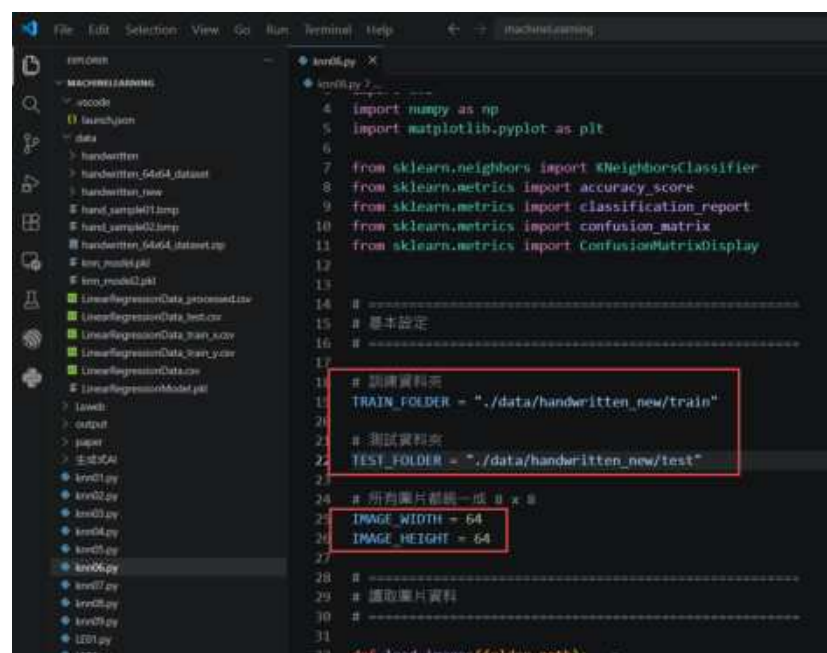




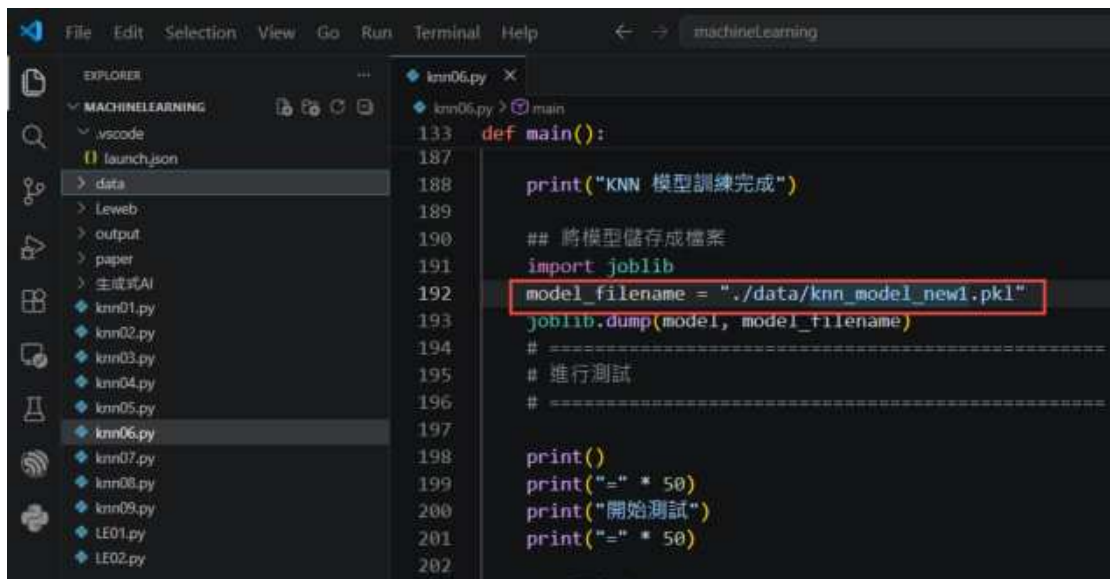
將 knn04.py 程式複製並更名為 knn06.py

將 knn05.py 程式複製並更名為 knn07.py

更改訓練資料與圖片大小



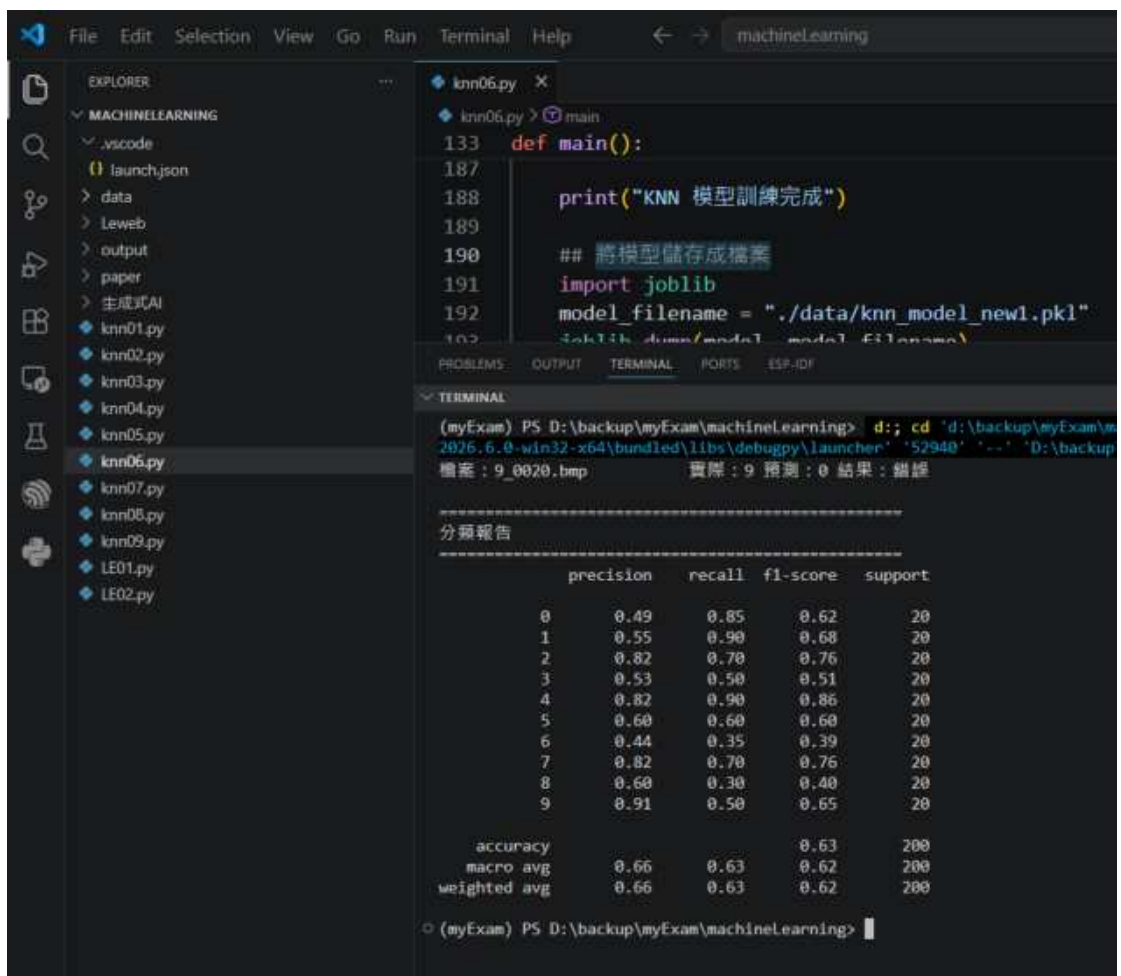
更改模型儲存成檔案



The screenshot shows the VS Code editor with the file `knn06.py` open. The `main()` function is visible, and the line `model_filename = "./data/knn_model_new1.pkl"` is highlighted with a red box. The Explorer sidebar on the left shows the project structure with folders like `data`, `LeWeb`, `output`, `paper`, and `生成式AI`, and files like `knn01.py` through `LE02.py`.

```
133 def main():
187
188     print("KNN 模型訓練完成")
189
190     ## 將模型儲存成檔案
191     import joblib
192     model_filename = "./data/knn_model_new1.pkl"
193     joblib.dump(model, model_filename)
194     # =====
195     # 進行測試
196     # =====
197
198     print()
199     print("=" * 50)
200     print("開始測試")
201     print("=" * 50)
202
```

訓練與測試結果:



The screenshot shows the VS Code editor with the `Terminal` panel active. The terminal displays the output of the `knn06.py` script, including the training completion message, the model filename, and the confusion matrix report.

```
(myExam) PS D:\backup\myExam\machineLearning> d:; cd 'd:\backup\myExam\2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '52940' '-' 'D:\backup
檔案: 9_0020.bmp          實際: 9  預測: 0  結果: 錯誤

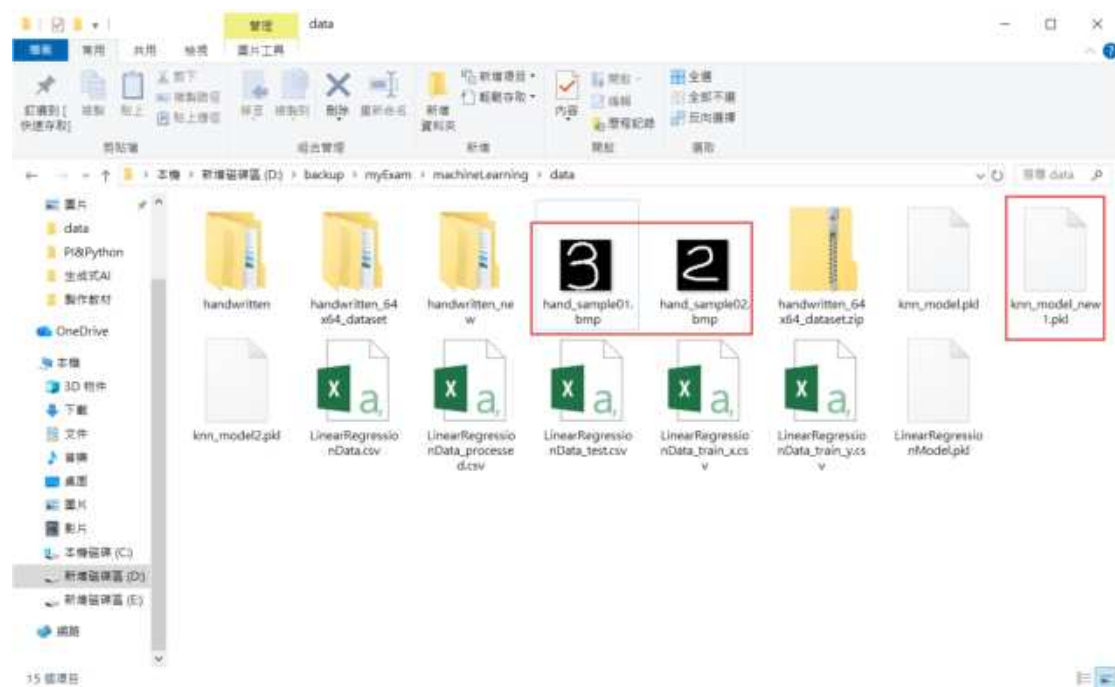
=====
分算報告
=====
```

	precision	recall	f1-score	support
0	0.49	0.85	0.62	20
1	0.55	0.90	0.68	20
2	0.82	0.70	0.76	20
3	0.53	0.50	0.51	20
4	0.82	0.90	0.86	20
5	0.60	0.60	0.60	20
6	0.44	0.35	0.39	20
7	0.82	0.70	0.76	20
8	0.60	0.30	0.40	20
9	0.91	0.50	0.65	20
accuracy			0.63	200
macro avg	0.66	0.63	0.62	200
weighted avg	0.66	0.63	0.62	200

```
(myExam) PS D:\backup\myExam\machineLearning>
```

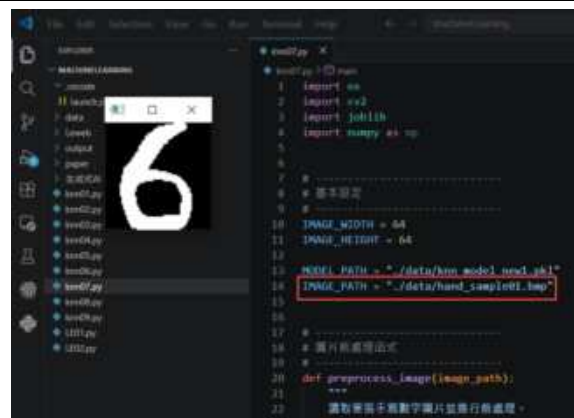
範例:knn06.py

開啟 knn07.py，修改模組來源、測試檔與大小參數



手寫文字測試結果如下：

顯示原圖並放大成 64 x 64，方便觀看，並顯示視窗 1



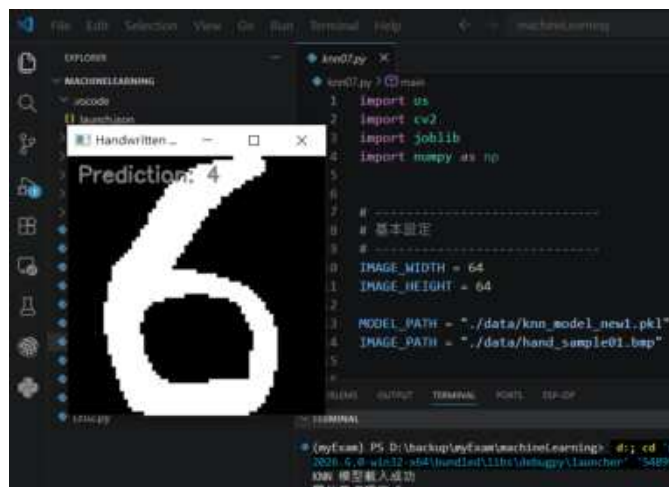
將圖片調整成 64x64，並顯示視窗 2



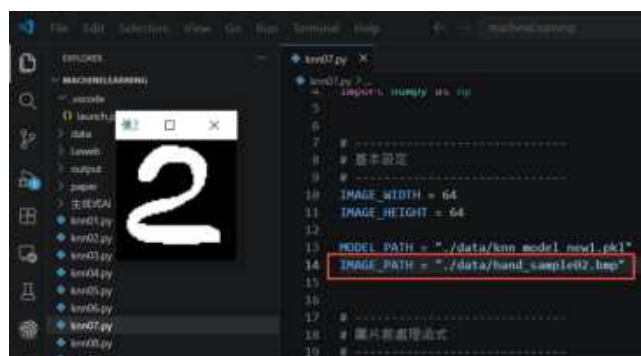
二值化，並顯示視窗 3



預測結果，並顯示視窗 4



顯示原圖，並放大成 64 x 64，方便觀看，並顯示視窗 1



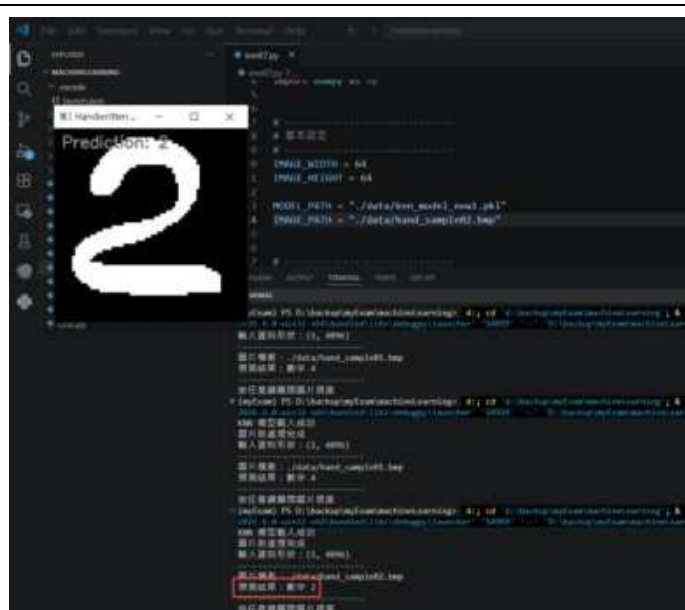
將圖片調整成 64x64，並顯示視窗 2

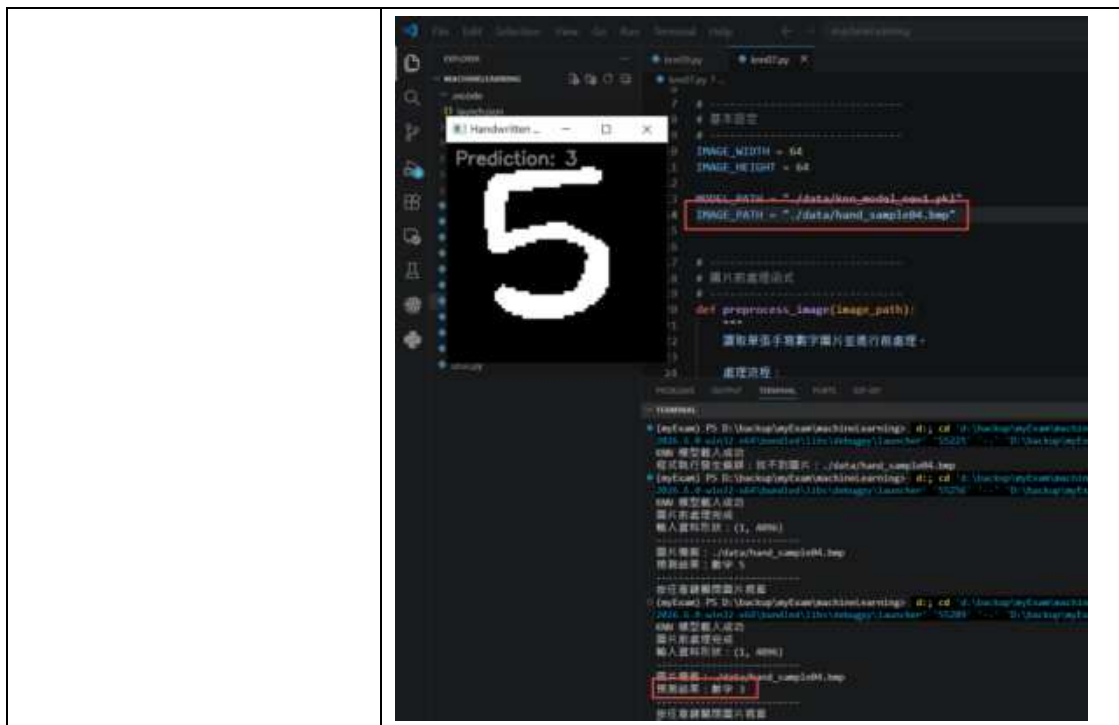


二值化，並顯示視窗 3



預測結果，並顯示視窗 4





範例:knn07.py

問題:

1、使用的特徵為相對位置，因此背景的資訊容易讓訓練產生錯誤資訊。

➤ 利用 knn 的手寫數字辨識系統修改 2

問題:

使用的特徵為相對位置，因此背景的資訊容易讓訓練產生錯誤資訊。

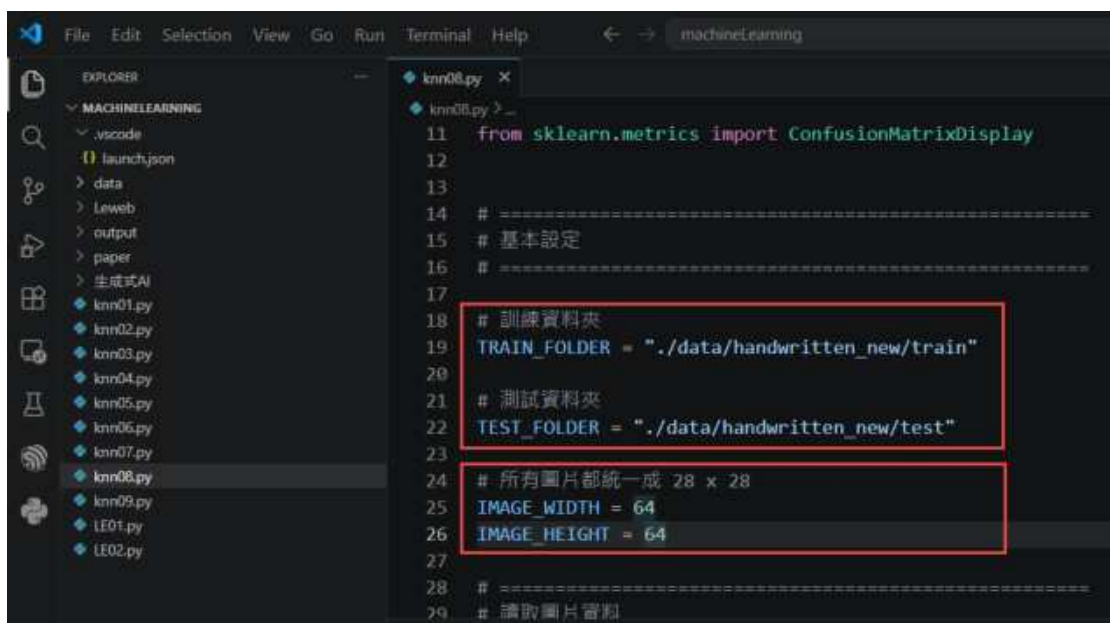
解決:

將前景字找到邊界並載取出來，再放大成 64x64。

將 knn04.py 程式複製並更名為 knn08.py

將 knn05.py 程式複製並更名為 knn09.py

修改成式如下:



```
11 from sklearn.metrics import ConfusionMatrixDisplay
12
13
14 # =====
15 # 基本設定
16 # =====
17
18 # 訓練資料夾
19 TRAIN_FOLDER = "./data/handwritten_new/train"
20
21 # 測試資料夾
22 TEST_FOLDER = "./data/handwritten_new/test"
23
24 # 所有圖片都統一成 28 x 28
25 IMAGE_WIDTH = 64
26 IMAGE_HEIGHT = 64
27
28 # =====
29 # 讀取圖片資料
```

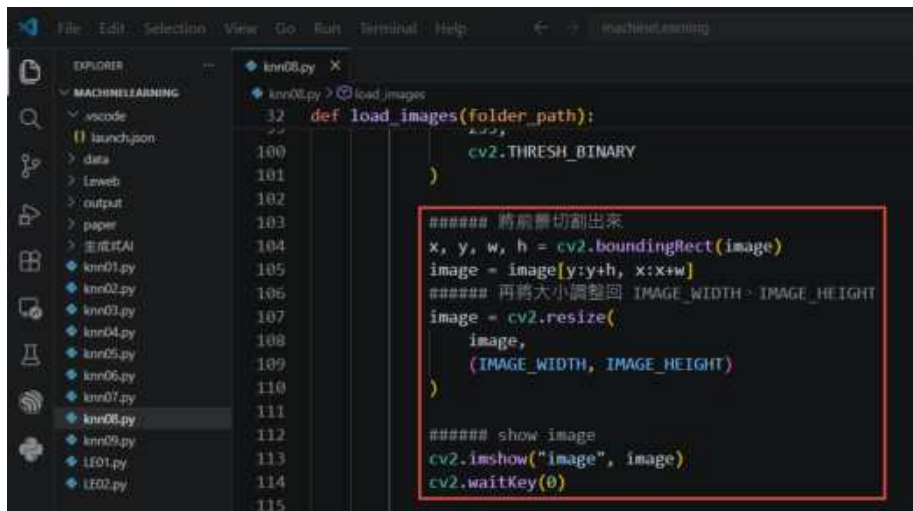
利用 github copilot 修改程式:

寫註解如下:

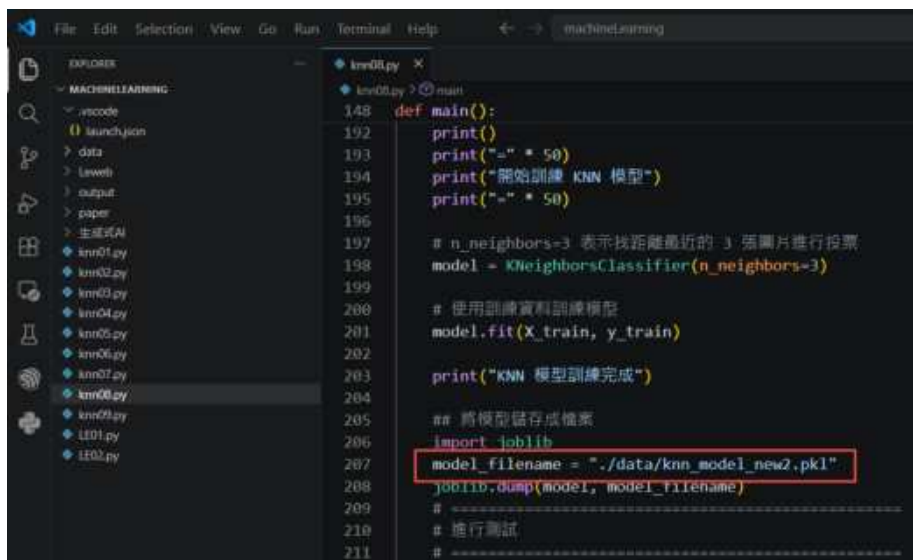
#將前景切割出來，自動產生程式

#再將大小調整回 IMAGE_WIDTH、IMAGE_HEIGHT

#show image

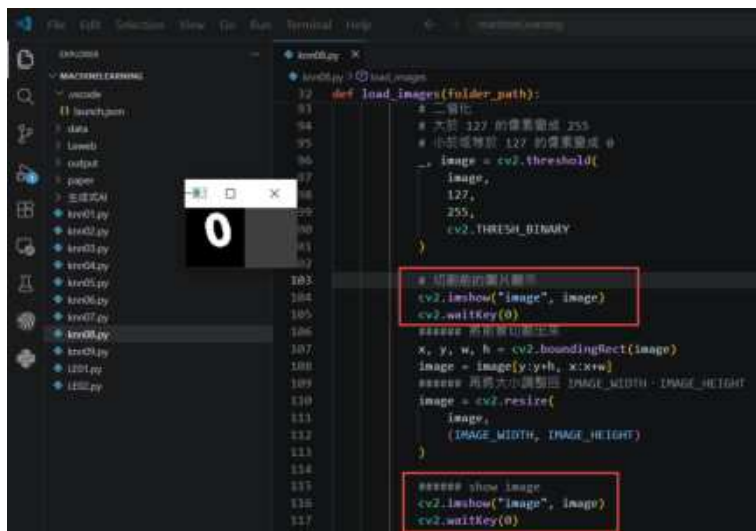


```
File Edit Selection View Go Run Terminal Help
knn08.py X
knn08.py > load_images
32 def load_images(folder_path):
100     cv2.THRESH_BINARY
101 )
102
103 ##### 將圖片切割出來
104 x, y, w, h = cv2.boundingRect(image)
105 image = image[y:y+h, x:x+w]
106 ##### 再將大小調整回 IMAGE_WIDTH * IMAGE_HEIGHT
107 image = cv2.resize(
108     image,
109     (IMAGE_WIDTH, IMAGE_HEIGHT)
110 )
111
112 ##### show image
113 cv2.imshow("image", image)
114 cv2.waitKey(0)
115
```

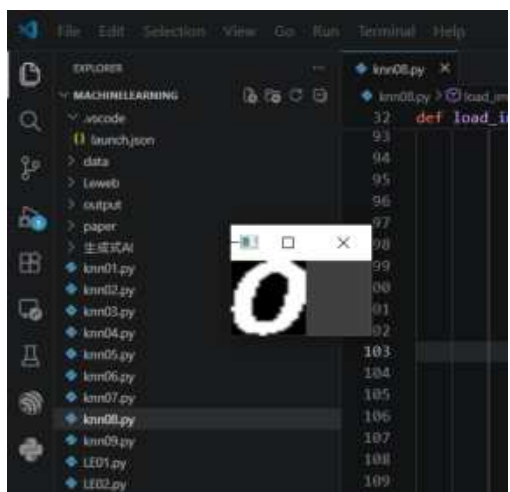
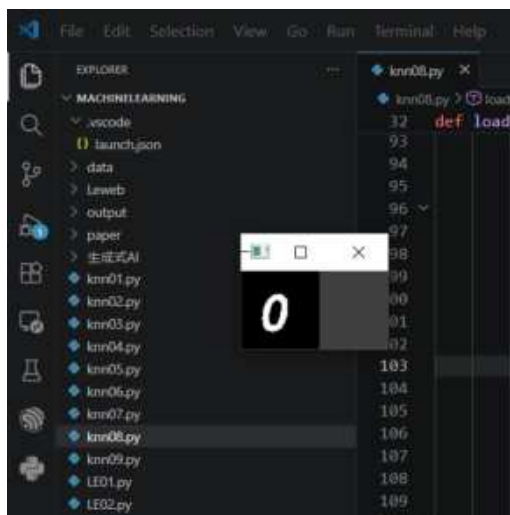
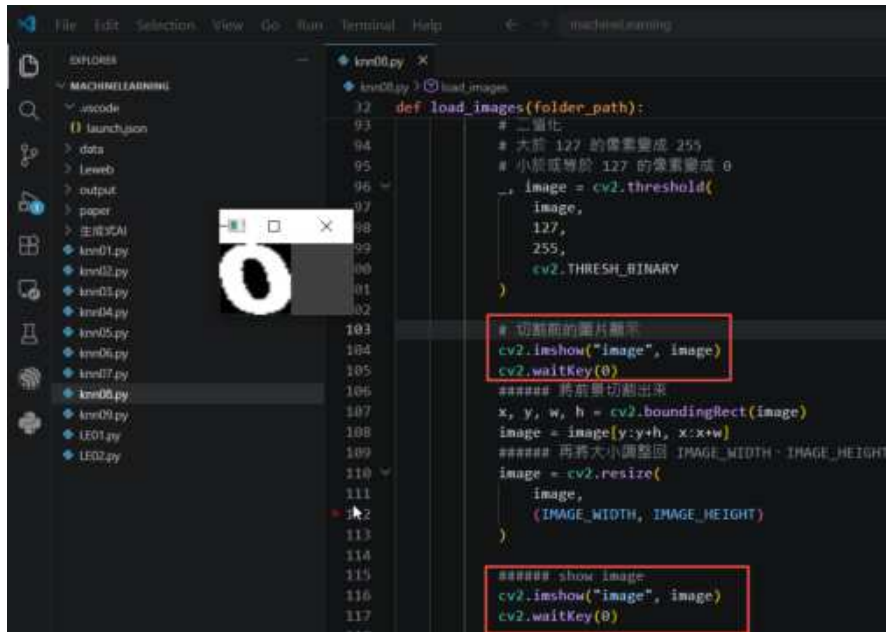


```
File Edit Selection View Go Run Terminal Help
knn08.py X
knn08.py > main
148 def main():
192     print()
193     print("-" * 50)
194     print("開始訓練 KNN 模型")
195     print("-" * 50)
196
197     # n_neighbors=3 表示找距離最近的 3 張圖片進行投票
198     model = KNeighborsClassifier(n_neighbors=3)
199
200     # 使用訓練資料訓練模型
201     model.fit(X_train, y_train)
202
203     print("KNN 模型訓練完成")
204
205     ## 將模型儲存成檔案
206     import joblib
207     model_filename = "./data/knn_model_new2.pkl"
208     joblib.dump(model, model_filename)
209     # -----
210     # 進行測試
211     # -----
```

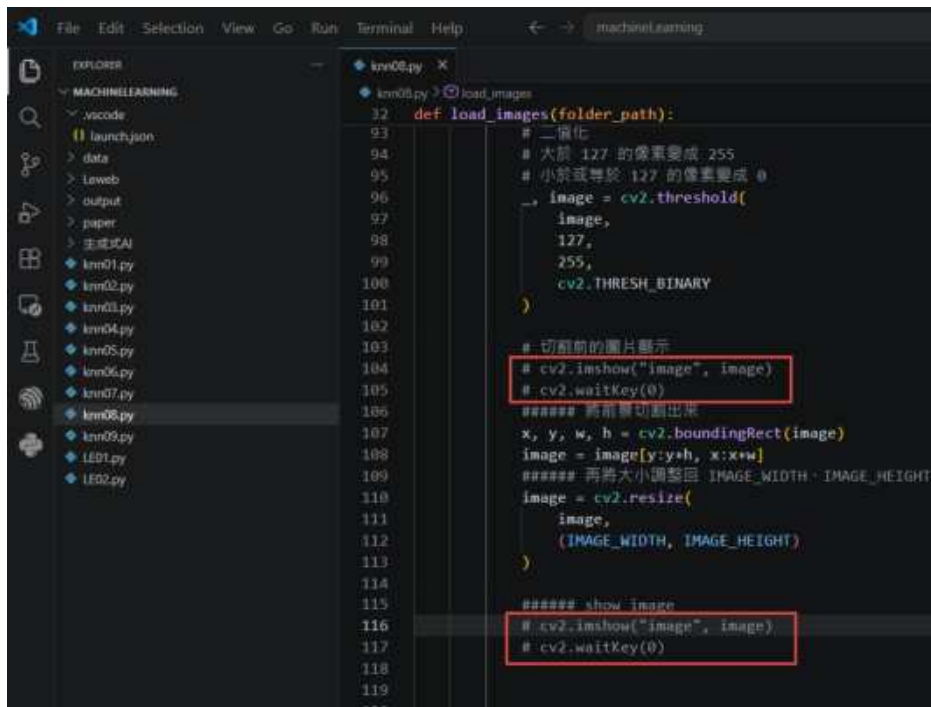
測試時先跑幾張圖，比較切割前與切割後差異



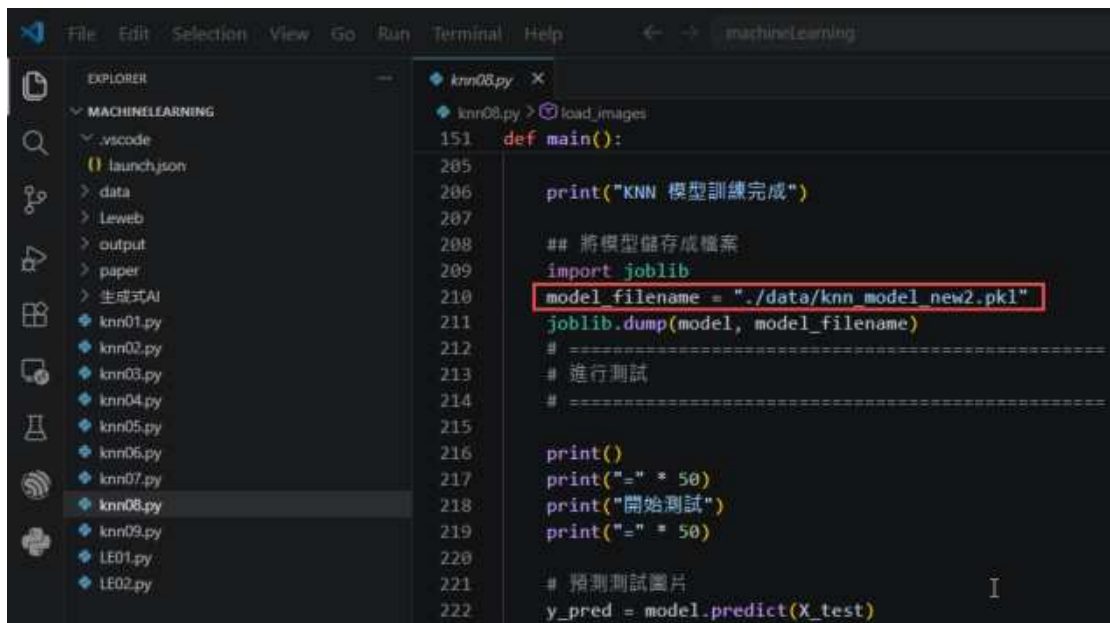
```
File Edit Selection View Go Run Terminal Help
knn08.py X
knn08.py > load_images
32 def load_images(folder_path):
33     # 二值化
34     # 大於 127 的像素變成 255
35     # 小於等於 127 的像素變成 0
36     image = cv2.threshold(
37         image,
38         127,
39         255,
40         cv2.THRESH_BINARY
41     )
42
43     # 切割前的圖片顯示
44     cv2.imshow("image", image)
45     cv2.waitKey(0)
46     ##### 切割後圖片顯示
47     x, y, w, h = cv2.boundingRect(image)
48     image = image[y:y+h, x:x+w]
49     ##### 再將大小調整回 IMAGE_WIDTH * IMAGE_HEIGHT
50     image = cv2.resize(
51         image,
52         (IMAGE_WIDTH, IMAGE_HEIGHT)
53     )
54
55     ##### show image
56     cv2.imshow("image", image)
57     cv2.waitKey(0)
58
```



再註解顯示程式，並更改模型輸出名稱，並執行：



```
File Edit Selection View Go Run Terminal Help machineLearning
EXPLORER
MACHINELEARNING
  .vscode
  launch.json
  data
  Leweb
  output
  paper
  生成式AI
  knn01.py
  knn02.py
  knn03.py
  knn04.py
  knn05.py
  knn06.py
  knn07.py
  knn08.py
  knn09.py
  LE01.py
  LE02.py
knn08.py
32 def load_images(folder_path):
93     # 二值化
94     # 大於 127 的像素變成 255
95     # 小於或等於 127 的像素變成 0
96     image = cv2.threshold(
97         image,
98         127,
99         255,
100         cv2.THRESH_BINARY
101     )
102
103     # 切割前的圖片顯示
104     cv2.imshow("image", image)
105     cv2.waitKey(0)
106     ##### 將前景切割出來
107     x, y, w, h = cv2.boundingRect(image)
108     image = image[y:y+h, x:x+w]
109     ##### 再將大小調整回 IMAGE_WIDTH * IMAGE_HEIGHT
110     image = cv2.resize(
111         image,
112         (IMAGE_WIDTH, IMAGE_HEIGHT)
113     )
114
115     ##### show image
116     cv2.imshow("image", image)
117     cv2.waitKey(0)
118
119
```



```
File Edit Selection View Go Run Terminal Help machineLearning
EXPLORER
MACHINELEARNING
  .vscode
  launch.json
  data
  Leweb
  output
  paper
  生成式AI
  knn01.py
  knn02.py
  knn03.py
  knn04.py
  knn05.py
  knn06.py
  knn07.py
  knn08.py
  knn09.py
  LE01.py
  LE02.py
knn08.py
151 def main():
205
206     print("KNN 模型訓練完成")
207
208     ## 將模型儲存成檔案
209     import joblib
210     model_filename = "./data/knn_model_new2.pkl"
211     joblib.dump(model, model_filename)
212     # =====
213     # 進行測試
214     # =====
215
216     print()
217     print("=" * 50)
218     print("開始測試")
219     print("=" * 50)
220
221     # 預測測試圖片
222     y_pred = model.predict(X_test)
```

訓練與測試結果：

The screenshot shows a VS Code editor with a file explorer on the left containing a project named 'MACHINELEARNING'. The main editor displays a Python script 'knn08.py' with a function 'load_images' that processes images. The terminal window shows the command to run the script and its output, which is a confusion matrix and performance metrics. The 'accuracy' row is highlighted with a red box.

```

def load_images(folder_path):
    # 二值化
    # 大於 127 的像素變成 255
    # 小於或等於 127 的像素變成 0
    image = cv2.threshold(
        image,
        127,
        255,
        cv2.THRESH_BINARY
    )
    return image

```

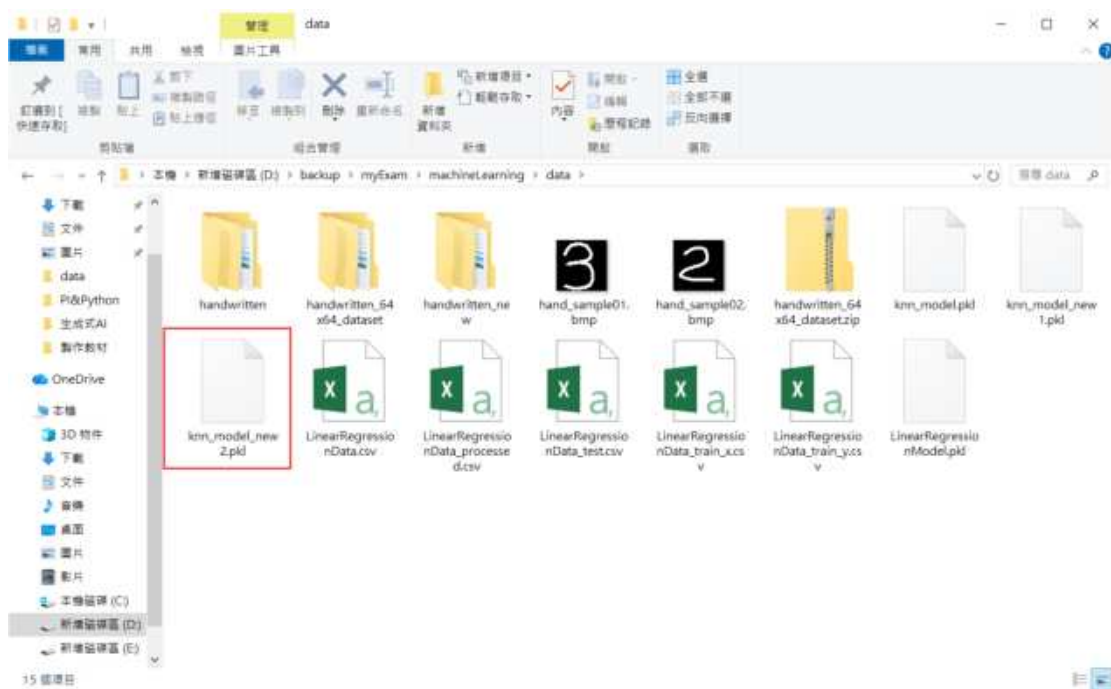
```

(myExam) PS D:\backup\myExam\machineLearning> .\knn08.py
分數報告
=====
              precision    recall  f1-score   support

      0           1.00        1.00        1.00         20
      1           0.95        1.00        0.98         20
      2           1.00        1.00        1.00         20
      3           1.00        1.00        1.00         20
      4           1.00        1.00        1.00         20
      5           1.00        0.95        0.97         20
      6           0.95        0.95        0.95         20
      7           1.00        0.95        0.97         20
      8           0.95        1.00        0.98         20
      9           1.00        1.00        1.00         20

 accuracy          0.98         0.98         0.98         200
 macro-avg         0.99         0.98         0.98         200
 weighted-avg      0.99         0.98         0.98         200

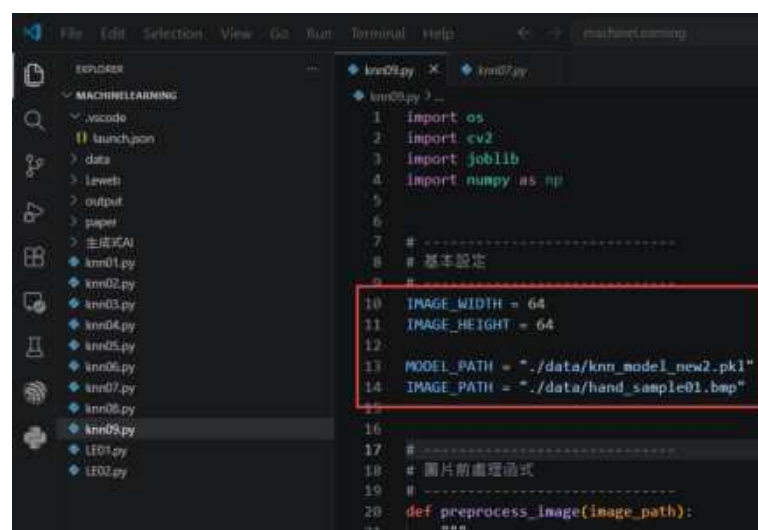
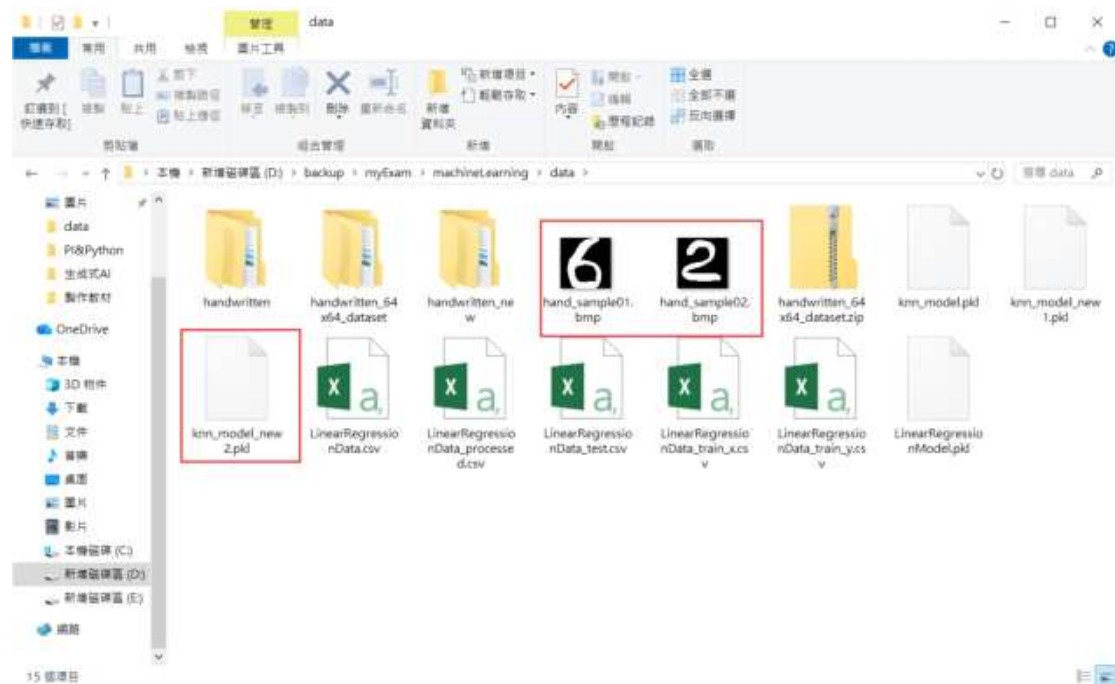
```



範例:knn08.py

結果：明顯提高辨識率

開啟 knn09.py，修改模組來源、測試檔、大小參數、取出前景，方法與 knn08 相同，如下：



```

def preprocess_image(image_path):
    20
    21     255,
    22     cv2.THRESH_BINARY
    23 )
    24
    25 # 將原圖切割出來
    26 x, y, w, h = cv2.boundingRect(image)
    27 image = image[y:y+h, x:x+w]
    28 # 將大小調整成 IMAGE_WIDTH, IMAGE_HEIGHT
    29 image = cv2.resize(
    30     image,
    31     (IMAGE_WIDTH, IMAGE_HEIGHT)
    32 )
    33
    34 # show image
    35 cv2.imshow("image", image)
    36 cv2.waitKey(0)
    37
    38 # 將第一份處理後的圖片，為模型中
    39 processed_image = image.copy()
    40

```

```

def preprocess_image(image_path):
    20
    100
    101
    102 # 將像素由 0~255 轉成 0~1
    103 image = image.astype("float32") / 255.0
    104
    105 # 顯示並放大成 64 x 64，方便觀看
    106 display_image = cv2.resize(
    107     image,
    108     (128, 128),
    109     interpolation=cv2.INTER_NEAREST
    110 )
    111 cv2.imshow("Processed image", display_image)
    112 cv2.waitKey(0)
    113
    114 # 將 28 x 28 陣平成 784 個數字
    115 image_data = image.flatten()
    116
    117 # 模型要求二維資料
    118 # 陣平: (784,)
    119 # 轉成: (1, 784)
    120 image_data = image_data.reshape(1, -1)
    121
    122 return image_data, processed_image
    123

```

手寫文字測試結果如下：

顯示原圖並放大成 64 x 64，方便觀看，並顯示視窗 1

```

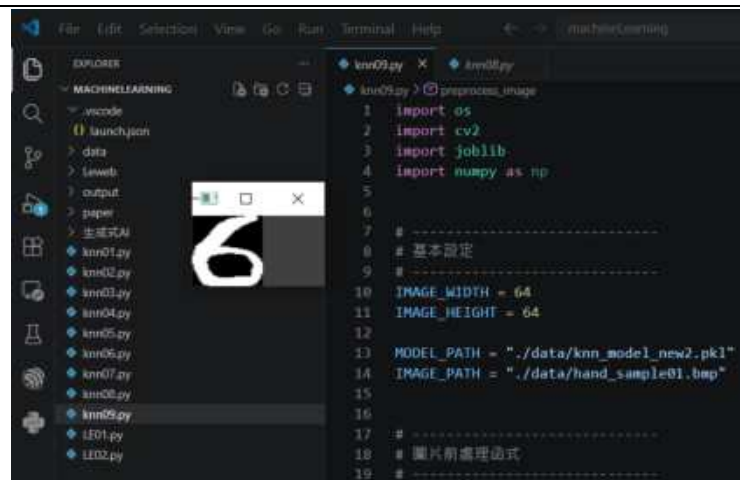
1  import os
2  import cv2
3  import joblib
4  import numpy as np
5
6
7  # -----
8  # 基本設定
9  # -----
10 IMAGE_WIDTH = 64
11 IMAGE_HEIGHT = 64
12
13 MODEL_PATH = "/data/knn_model_new2.pk1"
14 IMAGE_PATH = "/data/hand_sample01.bmp"
15
16
17 # -----
18 # 圖片處理過程式
19 # -----

```

將圖片調整成 64x64，並顯示視窗 2



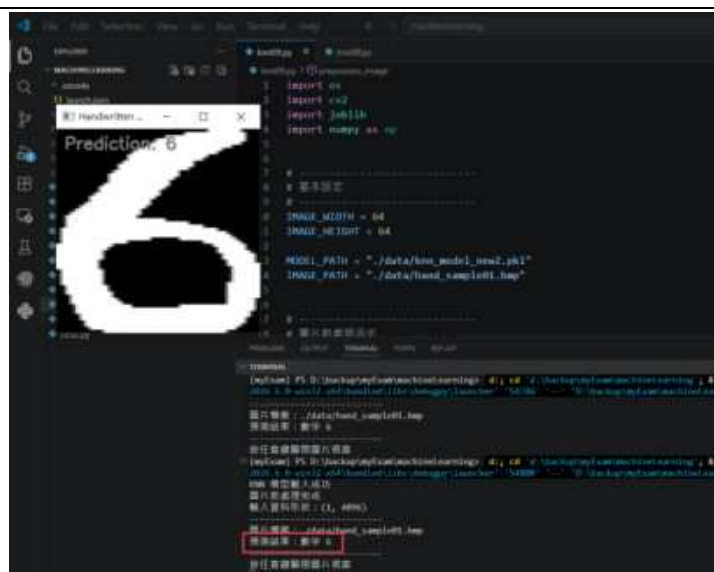
二值化，並顯示視窗 3



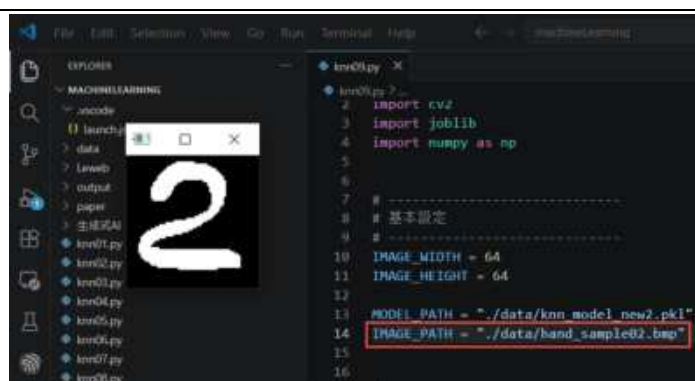
切割並大小正規化到 64x64，並顯示視窗 4



預測結果，並顯示視窗 5



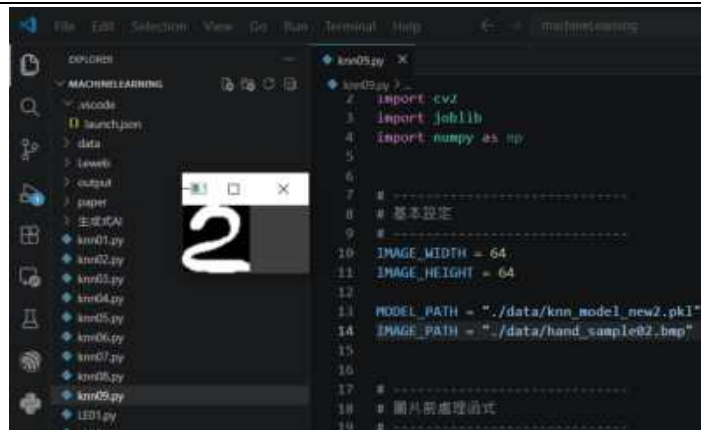
顯示原圖並放大成 64 x 64，方便觀看，並顯示視窗 1



將圖片調整成 64x64，並顯示視窗 2



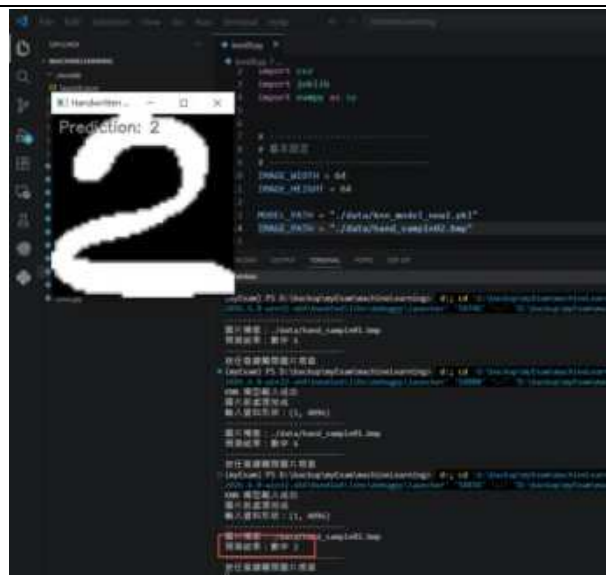
二值化，並顯示視窗 3

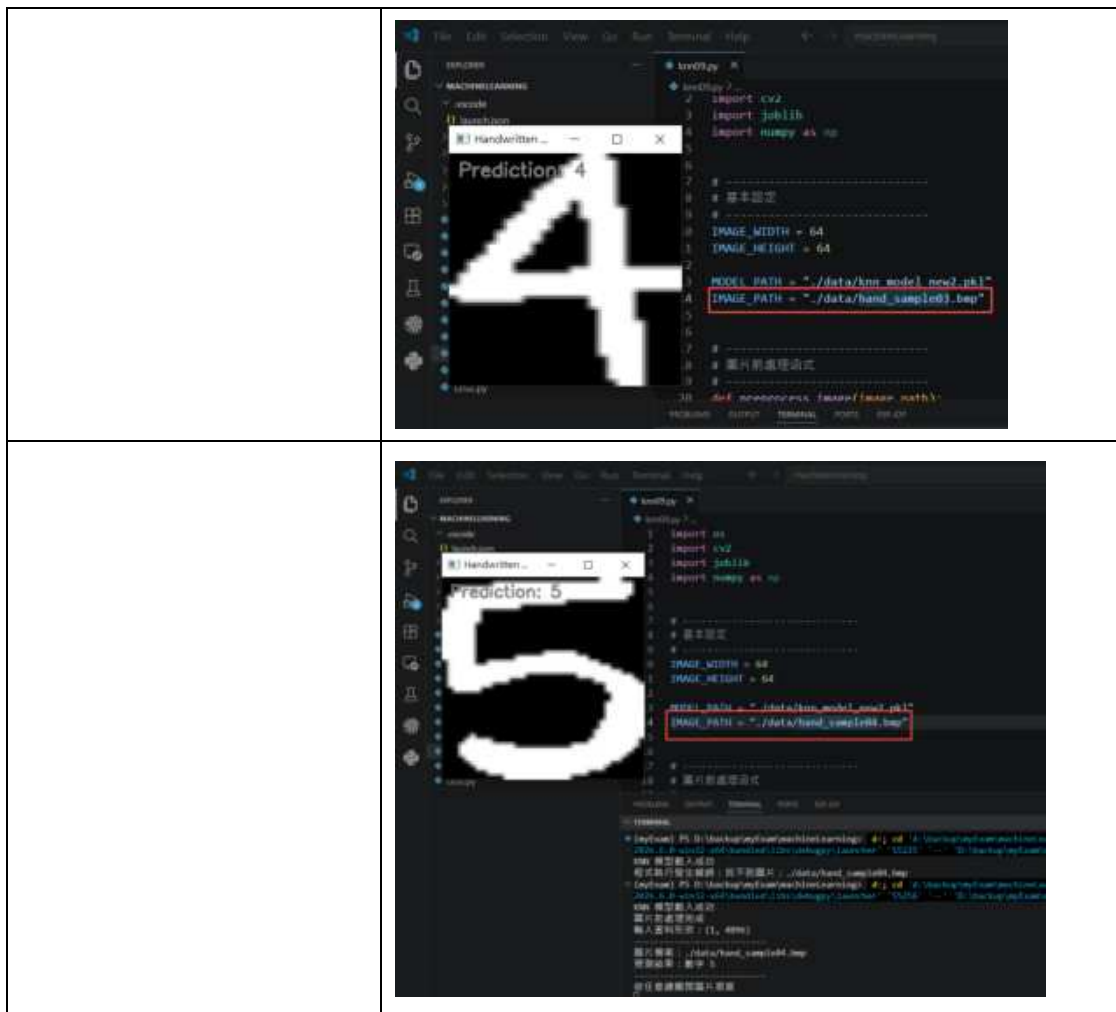


切割並大小正規化到 64x64，並顯示視窗 4



預測結果，並顯示視窗 5





範例:knn09.py

➤ 利用 knn 的手寫數字辨識系統 by Django Web

尋問 chaptGPT:

我利用 knn 演算法，產生出辨識手寫數字的模型，模型名稱為「knn_model_new2.pkl」，其中前景為白色、背景為黑色。

輸入圖片前的前處理程式如下，我想建立一個 django 網站使用初學者的程式寫法，專案名為 knnweb，app 名為 myapp，讓使用者可以用滑鼠寫數字，利用模型預測 0-9 的數字。

再將程式貼入，如下：

```
def preprocess_image(image_path):  
    """  
    讀取單張手寫數字圖片並進行前處理。  
  
    處理流程：  
    1. 灰階讀取  
    2. 調整成 28 x 28  
    3. 二值化  
    4. 像素轉成 0~1  
    5. 攤平成 784 個數字  
  
    回傳：  
    image_data：提供模型預測的資料  
    processed_image：處理完成的 28 x 28 圖片  
    """  
  
    # 檢查圖片是否存在  
    if not os.path.exists(image_path):  
        raise FileNotFoundError(f"找不到圖片：{image_path}")  
  
    # 以灰階方式讀取圖片  
    image = cv2.imread(  
        image_path,  
        cv2.IMREAD_GRAYSCALE  
    )  
  
    ## 顯示並放大成 64 x 64，方便觀看  
    display_image = cv2.resize(
```

```
        image,
        (128, 128),
        interpolation=cv2.INTER_NEAREST
    )
    cv2.imshow("Processed Image1", display_image)
    cv2.waitKey(0)

# 檢查圖片是否讀取成功
if image is None:
    raise ValueError(f"圖片讀取失敗：{image_path}")

# 將圖片調整成 64 x 64

image = cv2.resize(
    image,
    (IMAGE_WIDTH, IMAGE_HEIGHT)
)

## 顯示並放大成 64 x 64，方便觀看
display_image = cv2.resize(
    image,
    (128, 128),
    interpolation=cv2.INTER_NEAREST
)
cv2.imshow("Processed Image2", display_image)
cv2.waitKey(0)

# 二值化
# 大於 127 的像素變成 255
# 小於或等於 127 的像素變成 0
_, image = cv2.threshold(
    image,
    127,
    255,
    cv2.THRESH_BINARY
)

## 將前景切割出來
```

```

x, y, w, h = cv2.boundingRect(image)
image = image[y:y+h, x:x+w]
# 再將大小調整回 IMAGE_WIDTH、IMAGE_HEIGHT
image = cv2.resize(
    image,
    (IMAGE_WIDTH, IMAGE_HEIGHT)
)

# show image
cv2.imshow("Processed Image3", image)
cv2.waitKey(0)

# 保留一份處理後的圖片，方便顯示
processed_image = image.copy()

# 將像素由 0~255 轉成 0~1
image = image.astype("float32") / 255.0

## 顯示並放大成 64 x 64，方便觀看
display_image = cv2.resize(
    image,
    (128, 128),
    interpolation=cv2.INTER_NEAREST
)
cv2.imshow("Processed Image4", display_image)
cv2.waitKey(0)

# 將 64 x 64 攤平成 4096 個數字
image_data = image.flatten()

# 模型要求二維資料
# 原本：(4096,)
# 轉成：(1, 4096)

image_data = image_data.reshape(1, -1)

return image_data, processed_image

```

```

14 IMAGE_PATH = "../data/hand_sample04.bmp"
15
16
17 # -----
18 # 圖片前處理函式
19 # -----
20 def preprocess_image(image_path):
21     """
22     讀取單張手寫數字圖片並進行前處理。
23
24     處理流程：
25     1. 灰階讀取
26     2. 調整成 28 x 28
27     3. 二值化
28     4. 像素轉成 0~1
29     5. 攤平成 784 個數字
30
31     回傳：
32     image_data: 提供模型預測的資料
33     processed_image: 處理完成的 28 x 28 圖片
34     """
35
36     # 檢查圖片是否存在
37     if not os.path.exists(image_path):
38         raise FileNotFoundError(f"找不到圖片: {image_path}")
39
40     # 以灰階方式讀取圖片
41     image = cv2.imread(
42         image_path,
43         cv2.IMREAD_GRAYSCALE
44     )
45
46     ## 顯示並放大成 64 x 64，方便觀看
47     display_image = cv2.resize(
48         image,
49         (128, 128),
50         interpolation=cv2.INTER_NEAREST
51     )
52     cv2.imshow("Processed Image1", display_image)
53     cv2.waitKey(0)
54
55     # 檢查圖片是否讀取成功

```

生成式 ai 範例：Django 手寫數字辨識網站設置/Django 手寫數字辨識網站設置.mhtml

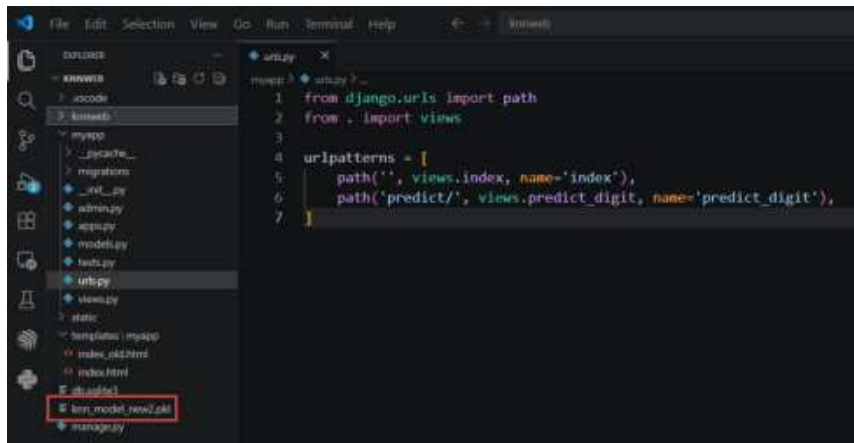
生成式 ai 範例：第一次修改程式

步驟如下：

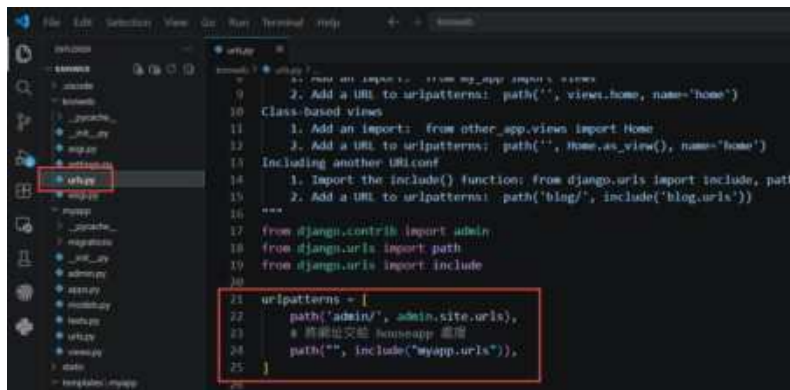
pip install django opencv-python numpy scikit-learn joblib pillow

1、

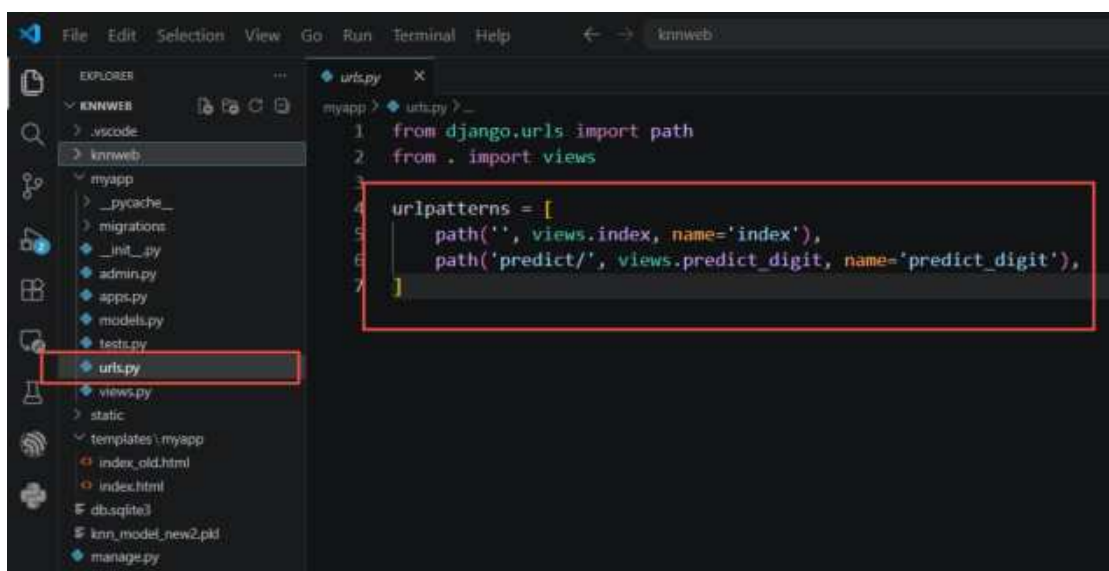
請把模型檔案：knn_model_new2.pkl 放在跟 manage.py 同一層。



2、修改 knnweb/urls.py



3、建立 myapp/urls.py



4、撰寫 myapp/views.py

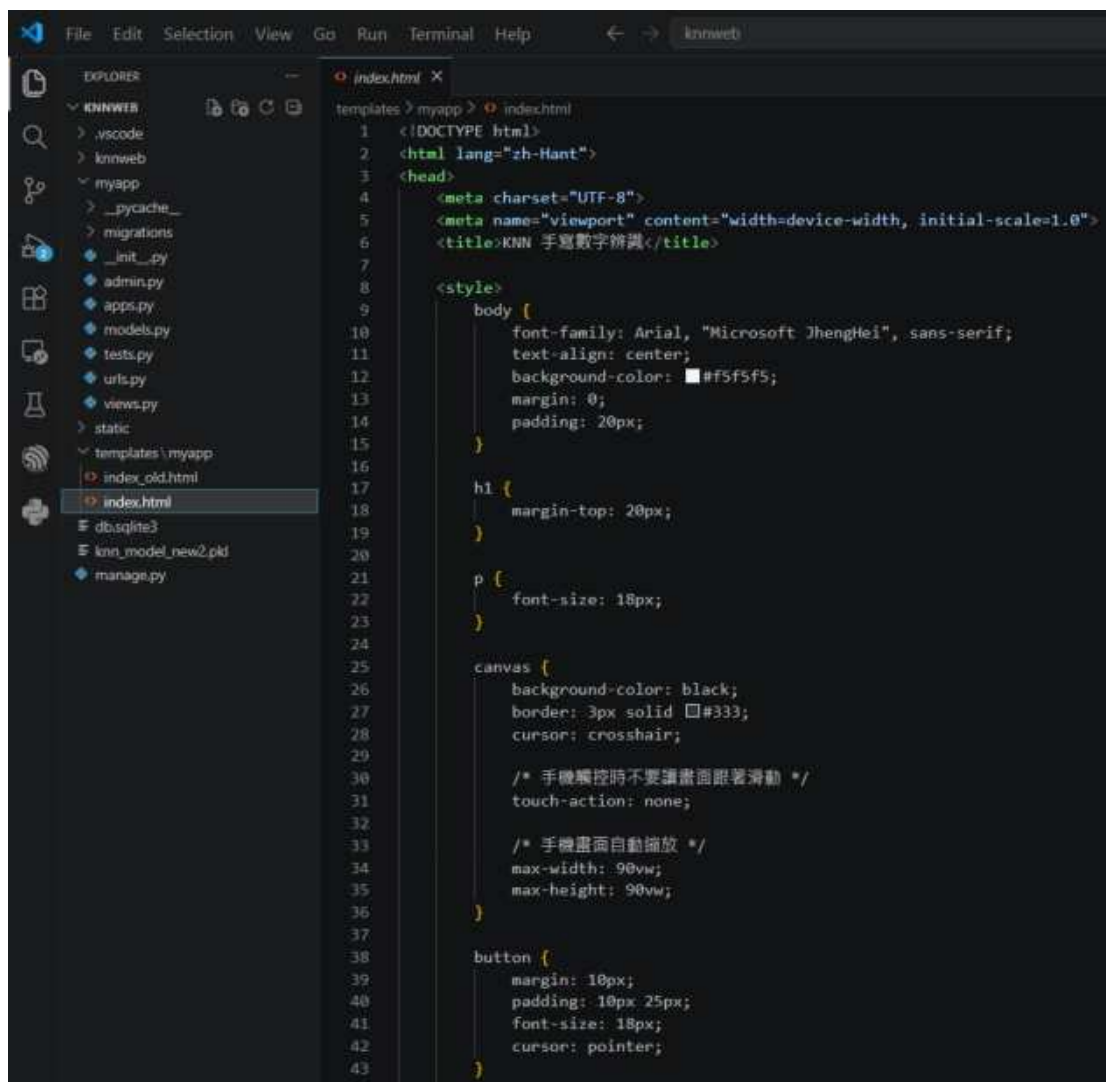
The screenshot shows a code editor with a file explorer on the left and a Python file named `views.py` open in the main editor. The file explorer shows a project structure with folders like `static`, `templates`, and `myapp`. The `views.py` file contains the following code:

```

1 from django.shortcuts import render
2 from django.http import JsonResponse
3
4 import os
5 import base64
6 import joblib
7 import cv2
8 import numpy as np
9
10 from django.conf import settings
11
12
13 IMAGE_WIDTH = 64
14 IMAGE_HEIGHT = 64
15
16
17 # 載入模型
18 MODEL_PATH = os.path.join(settings.BASE_DIR, 'knn_model_new2.pkl')
19 model = joblib.load(MODEL_PATH)
20
21 from django.views.decorators.csrf import csrf_exempt
22
23 @csrf_exempt
24 def index(request):
25     return render(request, 'myapp/index.html')
26
27
28 def preprocess_canvas_image(image_data):
29     """
30     處理從 Canvas 傳來的圖片。
31
32     流程：
33     1. 解開 base64 圖片資料
34     2. 轉成 OpenCV 圖片
35     3. 灰階處理
36     4. 二值化
37     5. 找出數字範圍
38     6. 調整成 28 x 28
39     7. 轉成 0-1
40     8. 扁平化成模型需要的格式
41     """
42
43     # 去掉 base64 前面的資料格式說明
44     image_data = image_data.split(',')[1]
45
46     # base64 解碼
47     image_bytes = base64.b64decode(image_data)

```

5、建立網頁 index.html



```
1 <!DOCTYPE html>
2 <html lang="zh-Hant">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>KNN 手寫數字辨識</title>
7
8   <style>
9     body {
10       font-family: Arial, "Microsoft JhengHei", sans-serif;
11       text-align: center;
12       background-color: #f5f5f5;
13       margin: 0;
14       padding: 20px;
15     }
16
17     h1 {
18       margin-top: 20px;
19     }
20
21     p {
22       font-size: 18px;
23     }
24
25     canvas {
26       background-color: black;
27       border: 3px solid #333;
28       cursor: crosshair;
29
30       /* 手機觸控時不要讓畫面跟著滑動 */
31       touch-action: none;
32
33       /* 手機畫面自動縮放 */
34       max-width: 90vw;
35       max-height: 90vw;
36     }
37
38     button {
39       margin: 10px;
40       padding: 10px 25px;
41       font-size: 18px;
42       cursor: pointer;
43     }
44
```

6、

如果預測不準，最常見原因

如果你的 KNN 模型訓練時使用的是 $28 \times 28 = 784$ 個特徵，那這裡必須是：



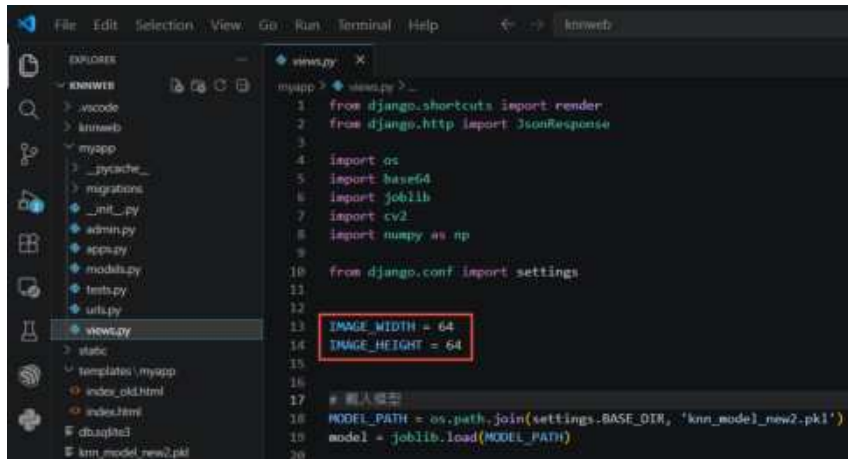
```
Python
IMAGE_WIDTH = 28
IMAGE_HEIGHT = 28
```

如果你訓練時是 $64 \times 64 = 4096$ 個特徵，就要改成：



```
Python
IMAGE_WIDTH = 64
IMAGE_HEIGHT = 64
```

也就是說，網站前處理後的輸入形狀一定要跟模型訓練時完全一樣。



```
1 from django.shortcuts import render
2 from django.http import JsonResponse
3
4 import os
5 import base64
6 import joblib
7 import cv2
8 import numpy as np
9
10 from django.conf import settings
11
12
13 IMAGE_WIDTH = 64
14 IMAGE_HEIGHT = 64
15
16 # 載入模型
17 MODEL_PATH = os.path.join(settings.BASE_DIR, 'knn_model_new2.pkl')
18 model = joblib.load(MODEL_PATH)
```

尋問 chatGPT:

若此網站換成手機瀏覽，我也希望可以用手機觸碰畫面寫字。

生成式 ai 範例：Django 手寫數字辨識網站設置/Django 手寫數字辨識網站設置.mhtml

生成式 ai 範例：第二次修改程式

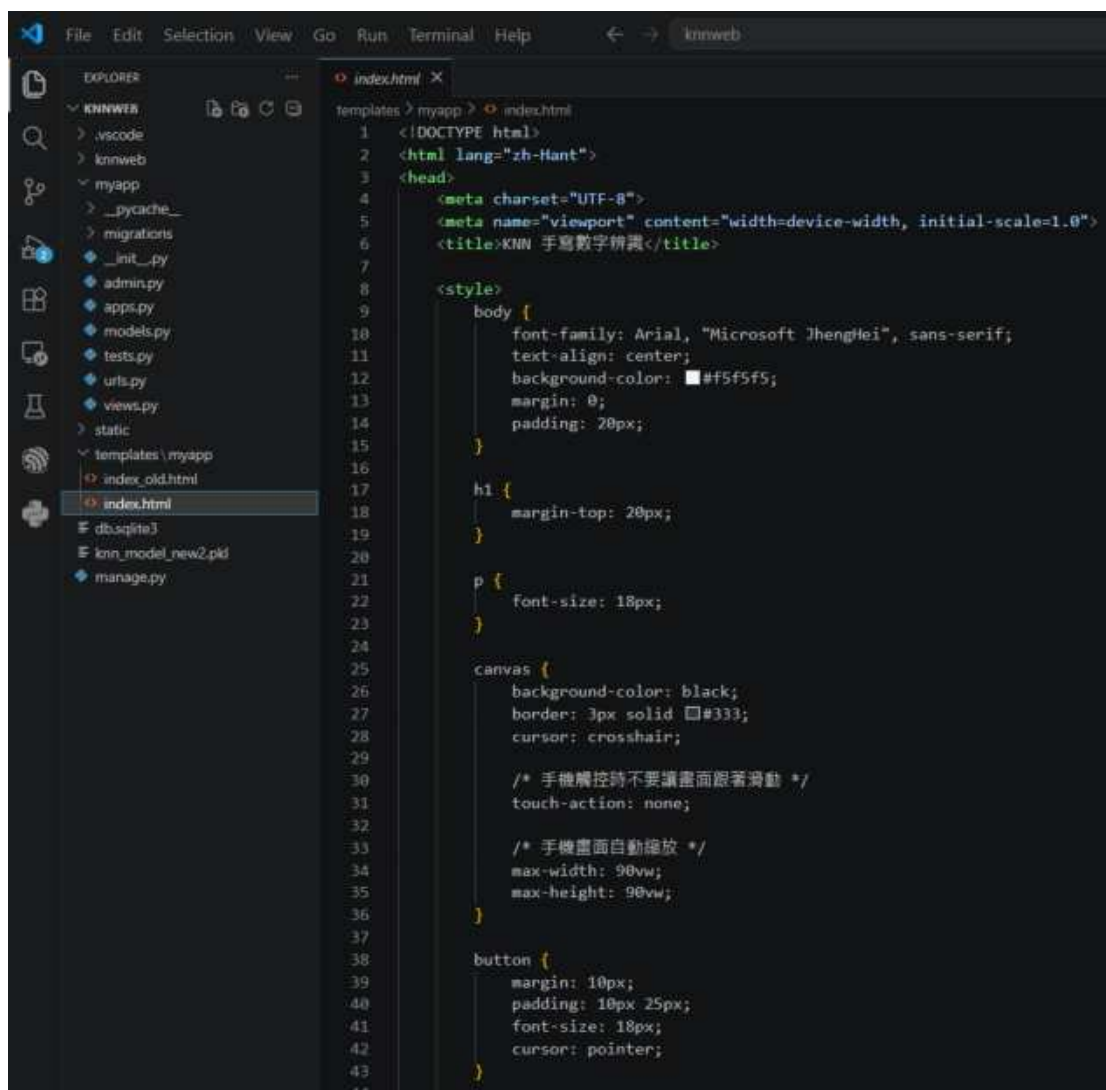
尋問 chatGPT:

給我 index.html 完整程式

生成式 ai 範例：Django 手寫數字辨識網站設置/Django 手寫數字辨識網站設置.mhtml

生成式 ai 範例：第三次修改程式

1、下面是支援「電腦滑鼠」與「手機觸控」的完整 index.html



```
1 <!DOCTYPE html>
2 <html lang="zh-Hant">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>KNN 手寫數字辨識</title>
7
8   <style>
9     body {
10       font-family: Arial, "Microsoft JhengHei", sans-serif;
11       text-align: center;
12       background-color: #f5f5f5;
13       margin: 0;
14       padding: 20px;
15     }
16
17     h1 {
18       margin-top: 20px;
19     }
20
21     p {
22       font-size: 18px;
23     }
24
25     canvas {
26       background-color: black;
27       border: 3px solid #333;
28       cursor: crosshair;
29
30       /* 手機觸控時不要讓畫面跟著滑動 */
31       touch-action: none;
32
33       /* 手機畫面自動縮放 */
34       max-width: 90vw;
35       max-height: 90vw;
36     }
37
38     button {
39       margin: 10px;
40       padding: 10px 25px;
41       font-size: 18px;
42       cursor: pointer;
43     }
44
```

出現錯誤如下：



尋問 chatGPT:

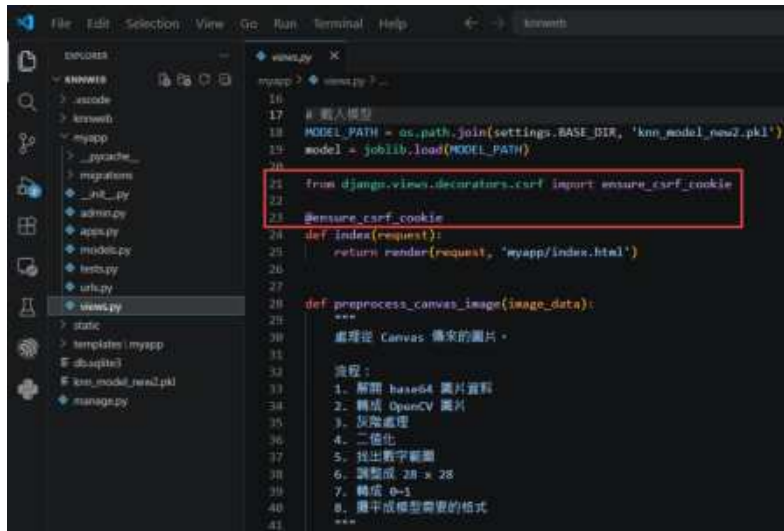
使用手機出現 發生錯誤：SyntaxError: Unexpected token '<', "<!DOCTYPE "... is not valid JSON

1、依照文 chatGPT 文件都無錯誤

2、views.py 的 index 建議加上：

7. views.py 的 index 建議加上：





```
16
17 # 載入模型
18 MODEL_PATH = os.path.join(settings.BASE_DIR, 'knn_model_new2.pkl')
19 model = joblib.load(MODEL_PATH)
20
21 from django.views.decorators.csrf import csrf_exempt
22
23 @csrf_exempt
24 def index(request):
25     return render(request, 'myapp/index.html')
26
27
28 def preprocess_canvas_image(image_data):
29     """
30     處理從 Canvas 傳來的圖片。
31
32     流程：
33     1. 解碼 base64 圖片資料
34     2. 轉換 OpenCV 圖片
35     3. 灰階處理
36     4. 二值化
37     5. 找出數字範圍
38     6. 調整成 28 x 28
39     7. 轉成 0-3
40     8. 扁平成模型需要的格式
41     """
```

結果如下：



4.3 CNN(Convolutional Neural Network，卷積神經網路)

CNN 和 DNN 都屬於深度學習(Deep Learning)神經網路，但最大的差異在於：CNN 特別適合處理具有空間結構的資料，例如圖片；DNN 則是比較一般化的全連接神經網路。DNN 可以想成是「很多層神經元連接起來」。



因此 DNN 很適合：

- 房價預測
- 客戶流失預測
- 信用評分
- 銷售預測
- 表格資料分類

CNN 是專門為影像等具有空間結構的資料設計的神經網路。

例如一張：

例如一張：

貓的圖片
↓
CNN
↓
貓



CNN 不會一開始就把整張圖片所有像素全部連接起來，而是利用**卷積 (Convolution)** 去尋找局部特徵。

例如：

圖片
↓
卷積層
↓
邊緣
↓
形狀
↓
眼睛、耳朵
↓
臉部
↓
貓



所以 CNN 可以逐層學習：

低階特徵
邊緣、線條
↓
中階特徵
眼睛、鼻子、耳朵
↓
高階特徵
臉、身體
↓
分類
貓

這也是 CNN 非常適合圖片辨識的原因。

5. CNN 與 DNN 的關係

其實不要把 CNN 和 DNN 想成完全不同的東西。

CNN 本身就是深度神經網路的一種。

可以理解成：



所以比較精確地說：

DNN 是一個比較廣泛的概念，而 CNN 是深度神經網路的一種特殊架構。

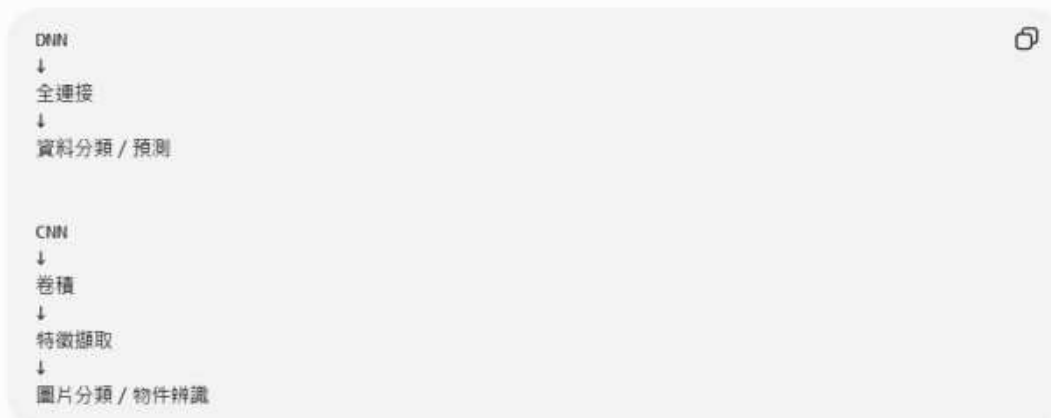
7. 用一句話記憶

如果你是在教 Python / 機器學習與深度學習，可以這樣讓學生記：

DNN：一般型的深度神經網路，適合數值、表格等資料。

CNN：專門處理具有空間結構的資料，尤其是圖片與影像。

再進一步：



另外要注意，現在的 CNN 不只用於圖片，也可以應用在影片、語音、時間序列等具有局部結構的資料。

➤ 安裝套件

```
# pip install torch torchvision matplotlib
```

➤ CNN 說明

CNN 常用在影像任務，例如：

1. 手寫數字辨識
2. 圖片分類
3. 人臉辨識
4. 醫學影像分析

CNN 的核心概念是：

層	功能
Convolution 卷積層	從圖片中抓特徵，例如邊緣、線條、形狀
ReLU	加入非線性，讓模型能學更複雜的模式
Pooling 池化層	縮小圖片尺寸，保留重要資訊
Fully Connected 全連接層	根據特徵做分類

➤ 基本知識(捲積、邊緣偵測 Prewitt、Sobel)

1、捲積 (convolution)

- 捲積：
一種運算方式，把一個小矩陣 filter/kernel 在影像上滑動，計算加權和。
- filter/kernel：
決定你想偵測或強調什麼特徵。
- 邊緣偵測 filter：
是一類常見的捲積 kernel，用來找亮度變化大的地方，也就是邊緣。

例如常見的邊緣偵測 kernel：

它主要偵測垂直邊緣，也就是水平方向亮度變化明顯的地方。

Sobel X：

```
[-1  0  1  
 -2  0  2  
 -1  0  1]
```

另一個方向：

它主要偵測水平邊緣。

Sobel Y：

```
[-1 -2 -1  
  0  0  0  
  1  2  1]
```

但捲積 filter 不只用於邊緣偵測，也可以用於：

- 模糊 blur
- 銳化 sharpen
- 降噪
- 特徵擷取
- CNN 裡的影像特徵學習

所以更精確地說：

邊緣偵測 filter 是捲積 filter 的一種。

捲積是方法，filter/kernel 是工具，邊緣偵測是其中一種用途。

公式：

1. 連續時間捲積

用於連續訊號，例如類比訊號・系統響應。

$$y(t) = x(t) * h(t) = \int_{-\infty}^{\infty} x(\tau)h(t-\tau)d\tau$$

也常寫成：

$$y(t) = \int_{-\infty}^{\infty} x(t-\tau)h(\tau)d\tau$$

用途：

- 線性時不變系統 LTI
- 訊號濾波
- 微分方程系統響應

2. 離散時間捲積

用於離散訊號，例如數位訊號處理 DSP。

$$y[n] = x[n] * h[n] = \sum_{k=-\infty}^{\infty} x[k]h[n-k]$$

或：

$$y[n] = \sum_{k=-\infty}^{\infty} x[n-k]h[k]$$

用途：

- 數位濾波器
- FIR/IIR 系統分析
- 音訊、影像處理

5. 二維捲積

常用於影像處理與卷積神經網路 CNN。

$$Y[i, j] = \sum_m \sum_n X[m, n] K[i - m, j - n]$$

在影像處理中，也常寫成：

$$Y[i, j] = \sum_m \sum_n X[i - m, j - n] K[m, n]$$

用途：

- 影像模糊
- 邊緣偵測
- 銳化
- CNN 特徵提取



6. CNN 中的捲積

深度學習中常用的捲積通常是「互相關」形式，不一定會翻轉 kernel：

$$Y[i, j] = \sum_m \sum_n X[i + m, j + n] K[m, n]$$

若有多通道輸入：

$$Y[o, i, j] = b_o + \sum_c \sum_m \sum_n X[c, i + m, j + n] K[o, c, m, n]$$

其中：

- (X)：輸入特徵圖
- (K)：卷積核
- (b_o)：偏置
- (o)：輸出通道
- (c)：輸入通道



尋問 chatGPT：

此圖形利用 3*3filter 做捲積，請用圖形說明

生成式 AI 範例：重疊滑動的卷積說明 1.mhtml

將說明：

1. 原始圖像與 3×3 filter 假設原始圖像灰階值如下： <table><tr><td>1</td><td>2</td><td>3</td><td>0</td><td>1</td></tr><tr><td>4</td><td>5</td><td>6</td><td>1</td><td>2</td></tr><tr><td>7</td><td>8</td><td>9</td><td>2</td><td>3</td></tr><tr><td>1</td><td>1</td><td>1</td><td>0</td><td>0</td></tr><tr><td>2</td><td>2</td><td>2</td><td>1</td><td>1</td></tr></table>	1	2	3	0	1	4	5	6	1	2	7	8	9	2	3	1	1	1	0	0	2	2	2	1	1	使用一個 3×3 filter： <table><tr><td>1</td><td>0</td><td>-1</td></tr><tr><td>1</td><td>0</td><td>-1</td></tr><tr><td>1</td><td>0</td><td>-1</td></tr></table> 這是一個常見的邊緣偵測濾波器，用來偵測垂直邊緣。	1	0	-1	1	0	-1	1	0	-1					
1	2	3	0	1																																				
4	5	6	1	2																																				
7	8	9	2	3																																				
1	1	1	0	0																																				
2	2	2	1	1																																				
1	0	-1																																						
1	0	-1																																						
1	0	-1																																						
2. 第一次捲積：filter 放在左上角 取圖像左上角的 3×3 區域： <table><tr><td>1</td><td>2</td><td>3</td></tr><tr><td>4</td><td>5</td><td>6</td></tr><tr><td>7</td><td>8</td><td>9</td></tr></table>	1	2	3	4	5	6	7	8	9	與 filter 對應相乘： <table><tr><th>圖像區域</th><th>filter</th><th>相乘結果</th></tr><tr><td>1</td><td>1</td><td>1</td></tr><tr><td>2</td><td>0</td><td>0</td></tr><tr><td>3</td><td>-1</td><td>-3</td></tr><tr><td>4</td><td>1</td><td>4</td></tr><tr><td>5</td><td>0</td><td>0</td></tr><tr><td>6</td><td>-1</td><td>-6</td></tr><tr><td>7</td><td>1</td><td>7</td></tr><tr><td>8</td><td>0</td><td>0</td></tr><tr><td>9</td><td>-1</td><td>-9</td></tr></table> 全部加總： $1 + 0 - 3 + 4 + 0 - 6 + 7 + 0 - 9 = -6$ 所以輸出圖的第一格是： -6 ↓	圖像區域	filter	相乘結果	1	1	1	2	0	0	3	-1	-3	4	1	4	5	0	0	6	-1	-6	7	1	7	8	0	0	9	-1	-9
1	2	3																																						
4	5	6																																						
7	8	9																																						
圖像區域	filter	相乘結果																																						
1	1	1																																						
2	0	0																																						
3	-1	-3																																						
4	1	4																																						
5	0	0																																						
6	-1	-6																																						
7	1	7																																						
8	0	0																																						
9	-1	-9																																						
3. filter 向右滑動一格 現在取下一個 3×3 區域： <table><tr><td>2</td><td>3</td><td>0</td></tr><tr><td>5</td><td>6</td><td>1</td></tr><tr><td>8</td><td>9</td><td>2</td></tr></table>	2	3	0	5	6	1	8	9	2	與 filter 相乘： $2(1) + 3(0) + 0(-1) + 5(1) + 6(0) + 1(-1) + 8(1) + 9(0) + 2(-1)$ $= 2 + 0 + 0 + 5 + 0 - 1 + 8 + 0 - 2 = 12$ 所以輸出圖第二格是： -6 12 ↓																														
2	3	0																																						
5	6	1																																						
8	9	2																																						

4. filter 依序滑過整張圖

3×3 filter 會像這樣在圖像上滑動：

第 1 列位置

位置 1	位置 2	位置 3
覆蓋左上 3×3	向右一格	再向右一格

第 2 列位置

位置 4	位置 5	位置 6
向下移一格	向右一格	再向右一格

第 3 列位置

位置 7	位置 8	位置 9
再向下移一格	向右一格	再向右一格

因為原圖是 5×5，filter 是 3×3，若不補零 padding = 1，輸出大小為：

$$(5 - 3 + 1) \times (5 - 3 + 1) = 3 \times 3$$

所以輸出會是一個 3×3 的特徵圖。

5. 完整捲積結果

計算完所有位置後，輸出 feature map 為：

-6	12	9
-6	14	10
0	9	8

6. 圖形化概念總結

可以想成：

Text

原始圖像		3×3 filter		輸出特徵圖
$\begin{bmatrix} 1 & 2 & 3 & 0 & 1 \\ 4 & 5 & 6 & 1 & 2 \\ 7 & 8 & 9 & 2 & 3 \\ 1 & 1 & 1 & 0 & 0 \\ 2 & 2 & 2 & 1 & 1 \end{bmatrix}$	*	$\begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$	→	$\begin{bmatrix} -6 & 12 & 9 \\ -6 & 14 & 10 \\ 0 & 9 & 8 \end{bmatrix}$

2、以 python 程式碼實做

尋問 chatGPT：

```

1  2  3  0  1
4  5  6  1  2
7  8  9  2  3
1  1  1  0  0
2  2  2  1  1

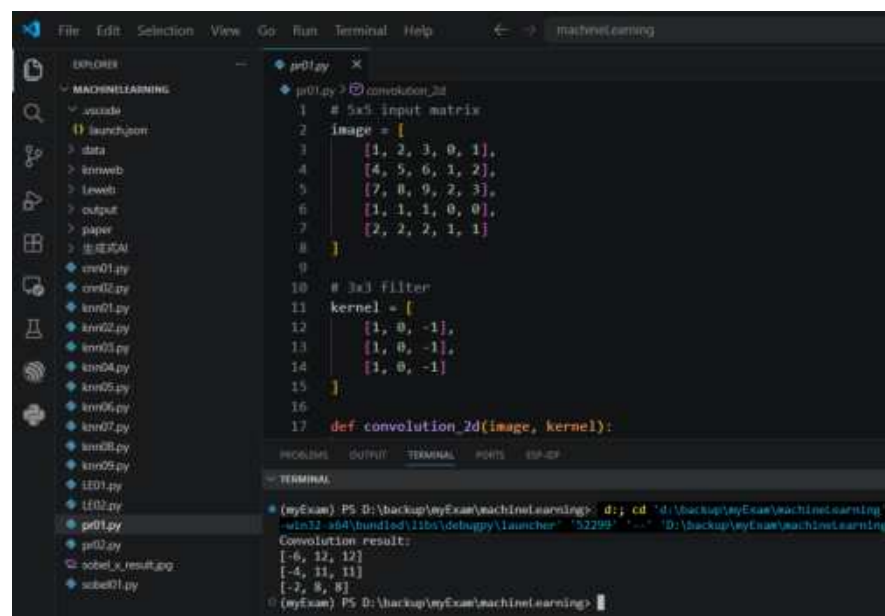
使用一個 3x3 filter：

1  0 -1
1  0 -1
1  0 -1

做convolution
請用python原始程式寫

```

生成式 AI 範例：重疊滑動的卷積說明 2.mhtml



```

File Edit Selection View Go Run Terminal Help
EXPLORER
  MACHINELEARNING
    vscode
    launch.json
    data
    knnweb
    knnweb
    output
    paper
    生成式AI
    cov01.py
    cov02.py
    knn01.py
    knn02.py
    knn03.py
    knn04.py
    knn05.py
    knn06.py
    knn07.py
    knn08.py
    knn09.py
    LED1.py
    LED2.py
    pr01.py
    pr02.py
    sobel_x_result.jpg
    sobel01.py

pr01.py
  pr01.py 2 convolution_2d
  1 # 5x5 input matrix
  2 image = [
  3     [1, 2, 3, 0, 1],
  4     [4, 5, 0, 1, 2],
  5     [7, 0, 9, 2, 3],
  6     [1, 1, 1, 0, 0],
  7     [2, 2, 2, 1, 1]
  8 ]
  9
 10 # 3x3 filter
 11 kernel = [
 12     [1, 0, -1],
 13     [1, 0, -1],
 14     [1, 0, -1]
 15 ]
 16
 17 def convolution_2d(image, kernel):
 18     # ... (implementation details) ...
 19     return result

TERMINAL
  (myExam) PS D:\backup\myExam\machinelearning> cd "d:\backup\myExam\machinelearning"
  .\win12-664\bundled\libs\debugpy\launcher "52299" "... "D:\backup\myExam\machinelearning\
  Convolution result:
  [-6, 12, 12]
  [-4, 11, 11]
  [-2, 8, 8]
  (myExam) PS D:\backup\myExam\machinelearning>

```

範例:cov01.py

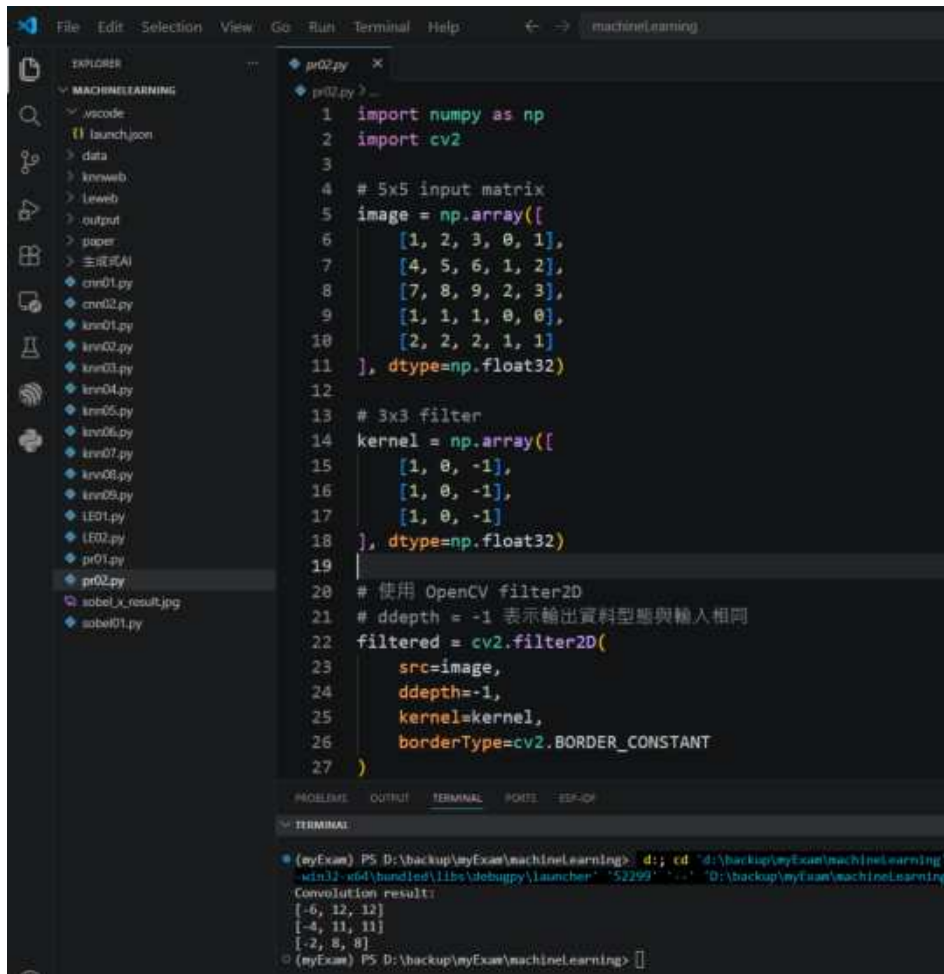
尋問 chatGPT：

請將程式改寫成filter2D function

可以使用numpy

可以使用numpy與filter2D

生成式 AI 範例: 重疊滑動的卷積說明 2.mhtml



```
1 import numpy as np
2 import cv2
3
4 # 5x5 input matrix
5 image = np.array([
6     [1, 2, 3, 0, 1],
7     [4, 5, 6, 1, 2],
8     [7, 8, 9, 2, 3],
9     [1, 1, 1, 0, 0],
10    [2, 2, 2, 1, 1]
11 ], dtype=np.float32)
12
13 # 3x3 filter
14 kernel = np.array([
15     [1, 0, -1],
16     [1, 0, -1],
17     [1, 0, -1]
18 ], dtype=np.float32)
19
20 # 使用 OpenCV filter2D
21 # ddepth = -1 表示輸出資料型態與輸入相同
22 filtered = cv2.filter2D(
23     src=image,
24     ddepth=-1,
25     kernel=kernel,
26     borderType=cv2.BORDER_CONSTANT
27 )
```

Terminal Output:

```
(myExam) PS D:\backup\myExam\machinelearning> cd "D:\backup\myExam\machinelearning"
win32-x64\bundled\libs\debugpy\launcher "52299" ... "D:\backup\myExam\machinelearning"
Convolution result:
[-6, 12, 12]
[-4, 11, 11]
[-2, 8, 8]
(myExam) PS D:\backup\myExam\machinelearning>
```

範例:cov02.py

尋問 chatGPT:

將 cov02.py 貼入，

將此程式改成輸入圖片./data/input1.jpeg，此 3*3 filter 偵測邊緣，並輸出結果

生成式 AI 範例: 重疊滑動的卷積說明 3/重疊滑動的卷積說明 3.mhtml

結果如下:



垂直邊緣偵測：

```

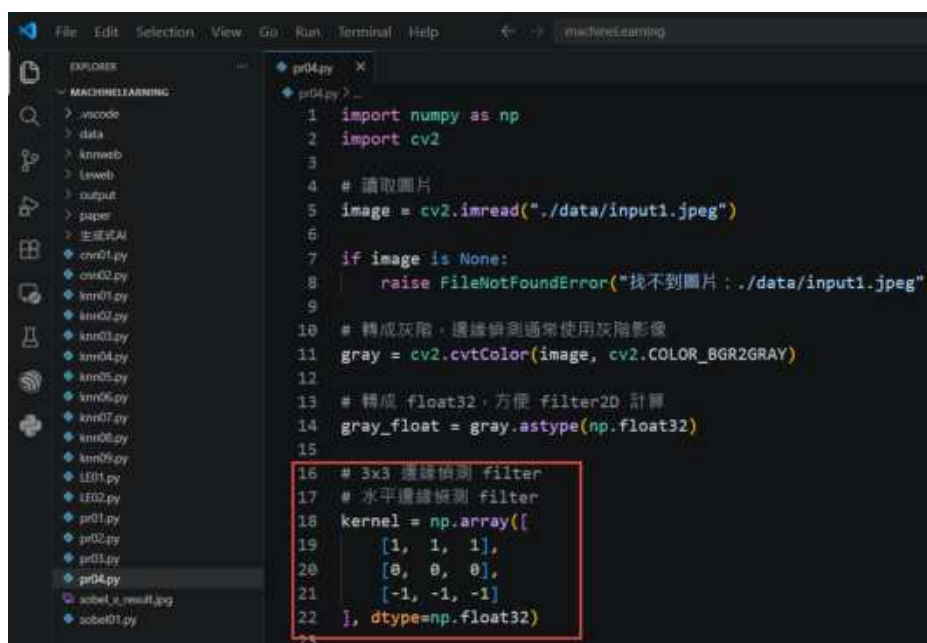
16 # 3x3 邊緣偵測 filter
17 kernel = np.array([
18     [1, 0, -1],
19     [1, 0, -1],
20     [1, 0, -1]
21 ], dtype=np.float32)
22

```

範例:pr01.py

水平邊緣偵測：

程式修正如下：



```

File Edit Selection View Go Run Terminal Help machineLearning
EXPLORER
MACHINELEARNING
  .uncode
  data
  knnweb
  leweb
  output
  paper
  生成式AI
  cnn01.py
  cnn02.py
  knn01.py
  knn02.py
  knn03.py
  knn04.py
  knn05.py
  knn06.py
  knn07.py
  knn08.py
  knn09.py
  LE01.py
  LE02.py
  pr01.py
  pr02.py
  pr03.py
  pr04.py
  sobel_x_result.jpg
  sobel01.py
pr04.py
1 import numpy as np
2 import cv2
3
4 # 讀取圖片
5 image = cv2.imread("../data/input1.jpeg")
6
7 if image is None:
8     raise FileNotFoundError("找不到圖片：../data/input1.jpeg")
9
10 # 轉成灰階，邊緣偵測通常使用灰階影像
11 gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
12
13 # 轉成 float32，方便 filter2D 計算
14 gray_float = gray.astype(np.float32)
15
16 # 3x3 邊緣偵測 filter
17 # 水平邊緣偵測 filter
18 kernel = np.array([
19     [1, 1, 1],
20     [0, 0, 0],
21     [-1, -1, -1]
22 ], dtype=np.float32)
23

```

範例:pr02.py

結果如下：



3、邊緣偵測 Prewitt 算子

Reference:

https://en.wikipedia.org/wiki/Prewitt_operator

説明：

Prewitt operator

Article Talk

From Wikipedia, the free encyclopedia

The **Prewitt operator** is used in [image processing](#), particularly within [edge detection](#) algorithms. Technically, it is a [discrete differentiation operator](#), computing an approximation of the [gradient](#) of the image intensity function. At each point in the image the result of the Prewitt operator is either the corresponding gradient vector or the [norm](#) of this vector. The Prewitt operator is based on convolving the image with a small, separable, and integer valued filter in horizontal and vertical directions and is therefore relatively inexpensive in terms of computations like [Sobel](#) and [Kayyali](#)^[1] operators. On the other hand, the gradient approximation which it produces is relatively crude, in particular for high frequency variations in the image. The Prewitt operator was developed by [Judith M. S. Prewitt](#).^[2]

Formulation | [edit](#) |

Mathematically, the operator uses two 3×3 kernels which are [convolved](#) with the original image to calculate approximations of the derivatives - one for horizontal changes, and one for vertical. If we define **A** as the source image, and **G_x** and **G_y** are two images which at each point contain the horizontal and vertical derivative approximations, the latter are computed as:

$$\mathbf{G}_y = \begin{bmatrix} +1 & +1 & +1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix} * \mathbf{A} \quad \text{and} \quad \mathbf{G}_x = \begin{bmatrix} +1 & 0 & -1 \\ +1 & 0 & -1 \\ +1 & 0 & -1 \end{bmatrix} * \mathbf{A}$$

where * here denotes the 2-dimensional [convolution](#) operation.

Since the Prewitt kernels can be decomposed as the products of an averaging and a differentiation kernel, they compute the gradient with smoothing. Therefore, it is a [separable filter](#). For example, **G_x** can be written as

$$\begin{bmatrix} +1 & 0 & -1 \\ +1 & 0 & -1 \\ +1 & 0 & -1 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \begin{bmatrix} +1 & 0 & -1 \end{bmatrix}$$

The x-coordinate is defined here as increasing in the "left"-direction, and the y-coordinate is defined as increasing in the "up"-direction. At each point in the image, the resulting gradient approximations can be combined to give the gradient magnitude, using:

$$\mathbf{G} = \sqrt{\mathbf{G}_x^2 + \mathbf{G}_y^2}$$

Using this information, we can also calculate the gradient's direction:

$$\Theta = \text{atan2}(\mathbf{G}_y, \mathbf{G}_x)$$

where, for example, Θ is 0 for a vertical edge which is darker on the right side.

Example [\[edit\]](#)



Grayscale image of a brick wall and a bike rack



Gradient with Prewitt operator of grayscale image of a brick wall and a bike rack

尋問 chatGPT:

我是一個初學 python 工程師，我想要產生一個利用 Prewitt operator 偵測邊緣的程式，輸入資料來源為「./data/input1.jpeg」，輸出結果./output/output3_edges.jpeg。其中利用 numpy 與 filter2D 方法工具。

生成式 AI 範例: Prewitt edge detection in Python/ Prewitt edge detection in Python.mhtml

範例:pr03.py

結果如下:



3、邊緣偵測 Sobel 算子

Reference:

https://en.wikipedia.org/wiki/Sobel_operator

說明：

Sobel operator

[Article](#) [Talk](#)

From Wikipedia, the free encyclopedia

The **Sobel operator**, sometimes called the **Sobel–Feldman operator** or **Sobel filter**, is used in [image processing](#) and [computer vision](#), particularly within [edge detection](#) algorithms where it creates an image emphasizing edges. It is named after [Irwin Sobel](#) and Gary M. Feldman, colleagues at the [Stanford Artificial Intelligence Laboratory](#) (SAIL). Sobel and Feldman presented the idea of an "Isotropic 3 × 3 Image Gradient Operator" at a talk at SAIL in 1968.^[1] Technically, it is a [discrete differentiation operator](#), computing an approximation of the [gradient](#) of the image intensity function. At each point in the image, the result of the Sobel–Feldman operator is either the corresponding gradient vector or the [norm](#) of this vector. The Sobel–Feldman operator is based on convolving the image with a small, separable, and integer-valued filter in the horizontal and vertical directions and is therefore relatively inexpensive in terms of computations. On the other hand, the gradient approximation that it produces is relatively crude, in particular for high-frequency variations in the image.

Formulation [\[edit \]](#)

The operator uses two 3×3 kernels which are [convolved](#) with the original image to calculate approximations of the [derivatives](#) – one for horizontal changes, and one for vertical. If we define **A** as the source image, and **G_x** and **G_y** are two images which at each point contain the horizontal and vertical derivative approximations respectively, the computations are as follows:^[1]

$$\mathbf{G}_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix} * \mathbf{A}$$
$$\mathbf{G}_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix} * \mathbf{A}$$

where * here denotes the 2-dimensional [signal processing convolution](#) operation.

In his text describing the origin of the operator,^[1] Sobel shows different signs for these kernels. He defined the operators as neighborhood masks (i.e. correlation kernels), and therefore are mirrored from what described here as convolution kernels. He also assumed the vertical axis increasing upwards instead of downwards as is common in image processing nowadays, and hence the vertical kernel is flipped.

Since the Sobel kernels can be decomposed as the products of an averaging and a differentiation kernel, they compute the gradient with smoothing. For example, G_x and G_y can be written as

$$G_x = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} * ([1 \ 0 \ -1] * A)$$

$$G_y = \begin{bmatrix} +1 \\ 0 \\ -1 \end{bmatrix} * ([1 \ 2 \ 1] * A)$$

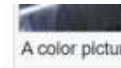
The x-coordinate is defined here as increasing in the "right"-direction, and the y-coordinate is defined as increasing in the "down"-direction. At each point in the image, the resulting gradient approximations can be combined to give the gradient magnitude, using [Pythagorean addition](#):

$$G = \sqrt{G_x^2 + G_y^2}$$

Using this information, we can also calculate the gradient's direction:

$$\Theta = \text{atan2}(G_y, G_x)$$

where, for example, Θ is 0 for a vertical edge which is lighter on the right side (for [atan2](#) see [atan2](#)).



尋問 chatGPT:

我是一個初學 python 工程師，我想要產生一個利用 Sobel operator 偵測邊緣的程式，輸入資料來源為「./data/input1.jpeg」，輸出結果./output/output4_edges.jpeg。其中利用 numpy 與 filter2D 方法工具。

生成式 AI 範例: Sobel edge detection in Python/ Sobel edge detection in Python.mhtml

範例:sobel01.py

結果如下:



➤ 利用 CNN 判斷圖片裡是直線還是橫線

目標：

輸入一張很小的黑白圖片，CNN 判斷它是，橫線或直線
如圖片大小是 5x5。

例子 1：橫線圖片

用 1 表示黑色，0 表示白色；



這是一條橫線。

例子 2：直線圖片



這是一條直線。

CNN 在做什麼？

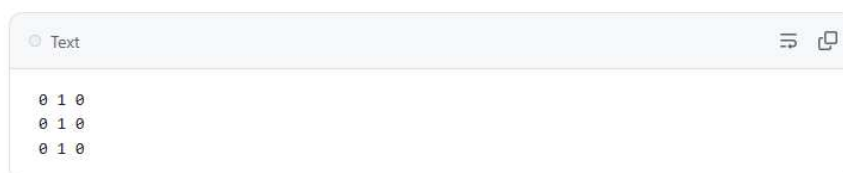
CNN 會用一個小濾鏡，例如 3×3 ，去圖片上掃描。

偵測橫線的濾鏡



這個濾鏡看到橫線時，分數會很高。

偵測直線的濾鏡



這個濾鏡看到直線時，分數會很高。

輸出大概是



重點理解

CNN 的核心概念其實很簡單：

用小濾鏡在圖片上滑動，找出特定形狀。

在這個例子中：

Filter	功能
橫線 filter	找橫線
直線 filter	找直線

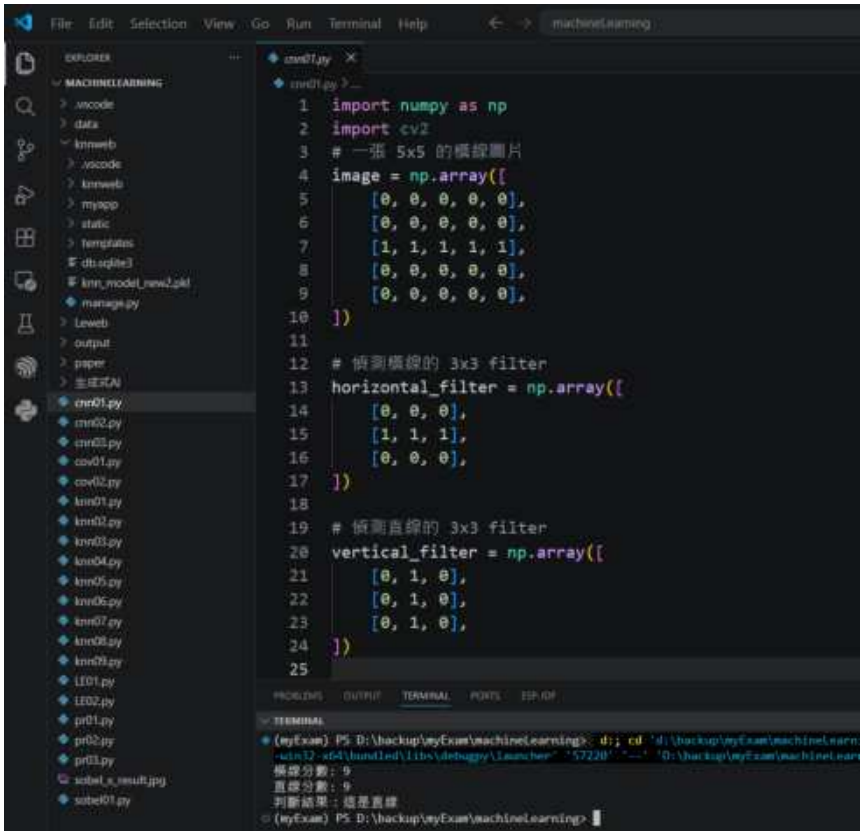
所以 CNN 可以想成：

一堆會找特徵的小偵探。

手寫數字辨識只是更複雜的版本，因為它要找的是：

- 邊緣
- 圓弧
- 轉角
- 筆畫
- 數字形狀

但最基本的原理，就是這個「直線 vs 橫線」例子。 ↓



```
1 import numpy as np
2 import cv2
3 # 一張 5x5 的橫線圖片
4 image = np.array([
5     [0, 0, 0, 0, 0],
6     [0, 0, 0, 0, 0],
7     [1, 1, 1, 1, 1],
8     [0, 0, 0, 0, 0],
9     [0, 0, 0, 0, 0],
10 ])
11
12 # 偵測橫線的 3x3 filter
13 horizontal_filter = np.array([
14     [0, 0, 0],
15     [1, 1, 1],
16     [0, 0, 0],
17 ])
18
19 # 偵測直線的 3x3 filter
20 vertical_filter = np.array([
21     [0, 1, 0],
22     [0, 1, 0],
23     [0, 1, 0],
24 ])
25
```

Terminal Output:

```
(myExam) P5 D:\hacup\myExam\machinelearning> .\cnn01.py
橫線分數: 9
直線分數: 9
判斷結果: 這是直線
```

範例:cnn01.py

範例:cnn02.py

➤ 利用 CNN 手寫數字辨識

卷積神經網路 (Convolutional Neural Networks, CNN)說明：

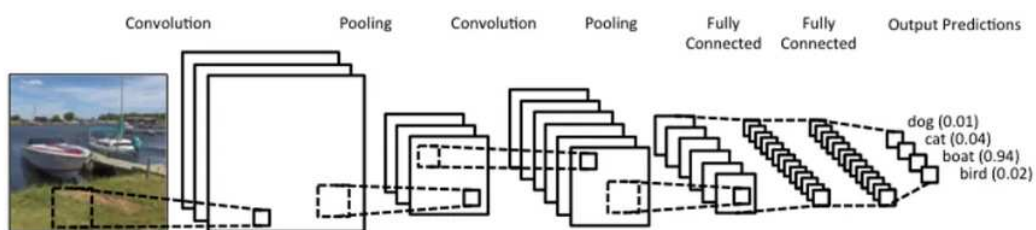
Reference:

<https://www.youtube.com/watch?v=OP5HcXJg2Aw>

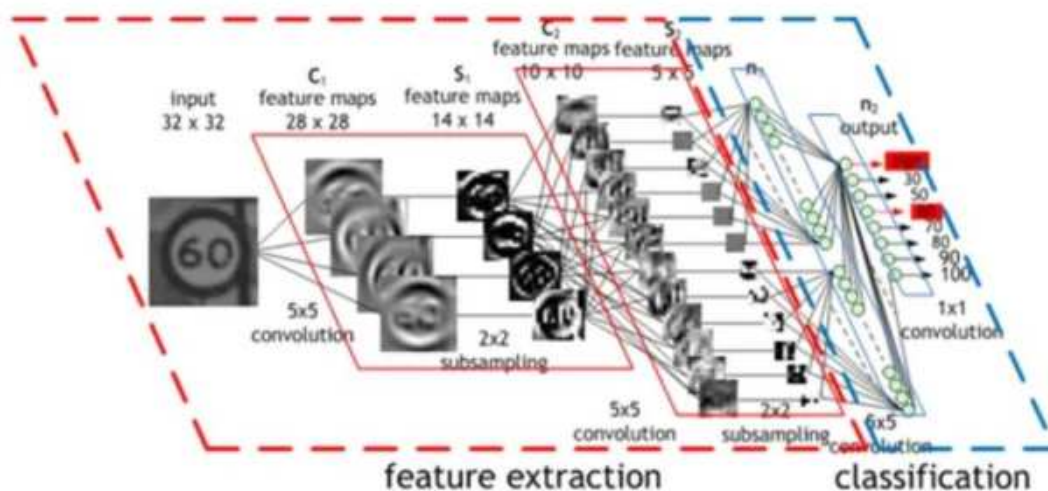
<https://medium.com/jameslearningnote/%E8%B3%87%E6%96%99%E5%88%86%E6%9E%90-%E6%A9%9F%E5%99%A8%E5%AD%B8%E7%BF%92-%E7%AC%AC5-1%E8%AC%9B-%E5%8D%B7%E7%A9%8D%E7%A5%9E%E7%B6%93%E7%B6%B2%E7%B5%A1%E4%BB%8B%E7%B4%B9-convolutional-neural-network-4f8249d65d4f>

CNN 在影像識別方面的威力非常強大，許多影樣辨識的模型也都是以 CNN 的架構為基礎去做延伸。另外值得一提的是 CNN 模型也是少數參考人的大腦視覺組織來建立的深度學習模型，學會 CNN 之後，對於學習其他深度學習的模型也會很有幫助

CNN的概念圖如下：



CNN概念圖



CNN概念圖2

簡單來說，圖片經過各兩次的 Convolution, Pooling, Fully Connected 就是 CNN 的架構了，因此只要搞懂 Convolution, Pooling, Fully Connected 三個部分的內容就可以完全掌握了 CNN！

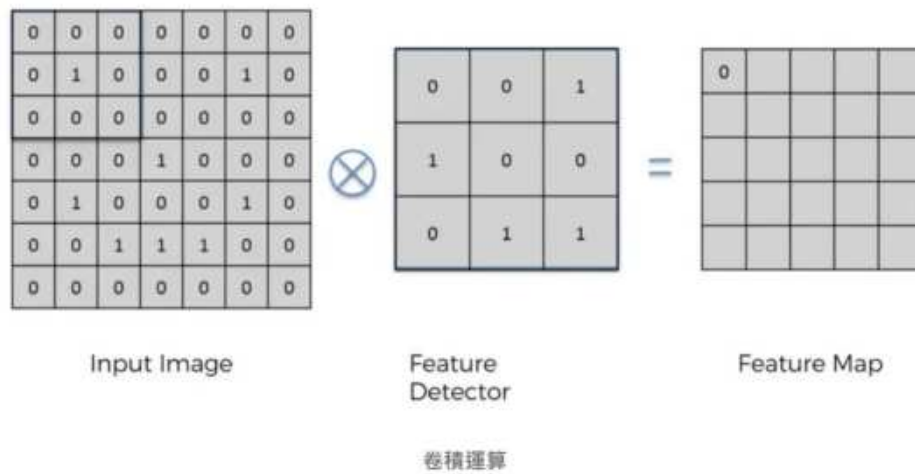
典型 CNN 架構如下：



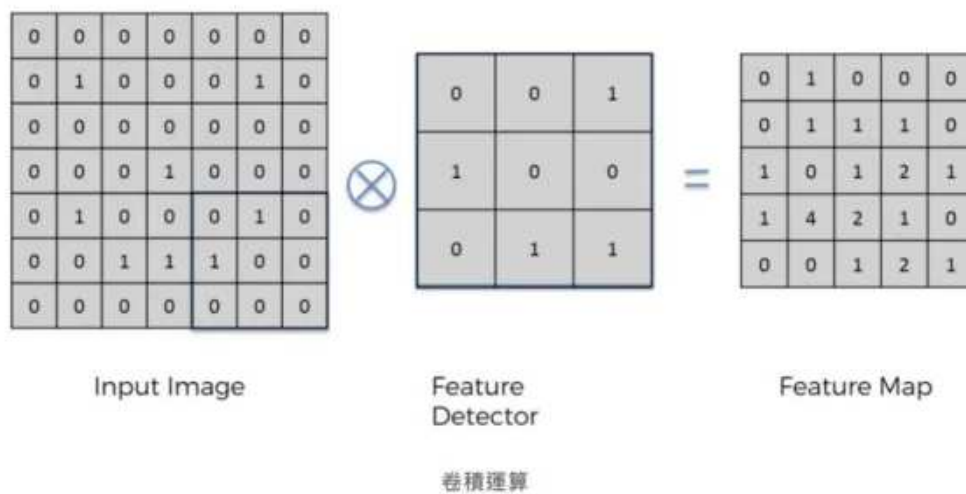
1. Convolution Layer 卷積層

卷積運算就是將原始圖片的與特定的 Feature Detector(filter)做卷積運算(符號 $\otimes$)，

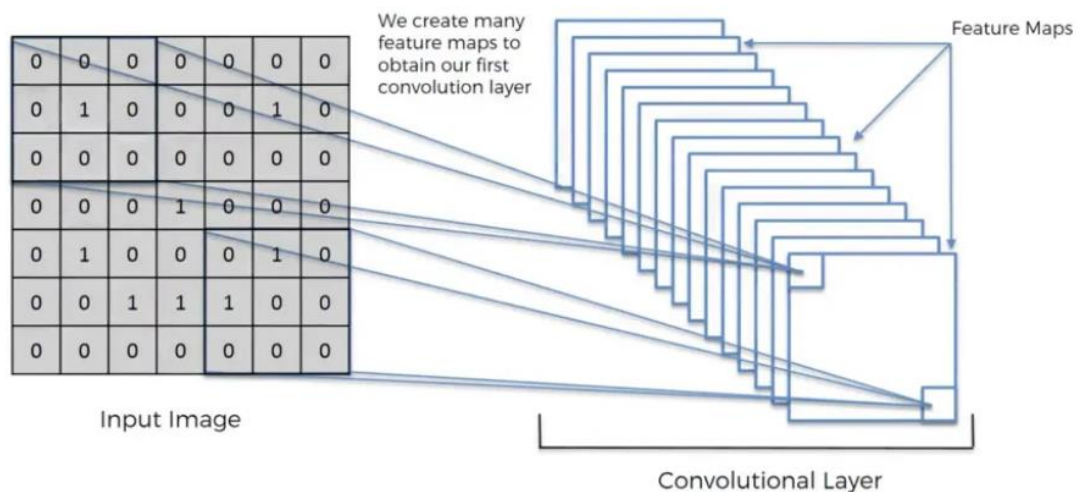
卷積運算就是將下圖兩個 3x3 的矩陣作相乘後再相加，以下圖為例 $0*0 + 0*0 + 0*1 + 0*1 + 1*0 + 0*0 + 0*0 + 0*1 + 0*1 = 0$



依序做完整張表

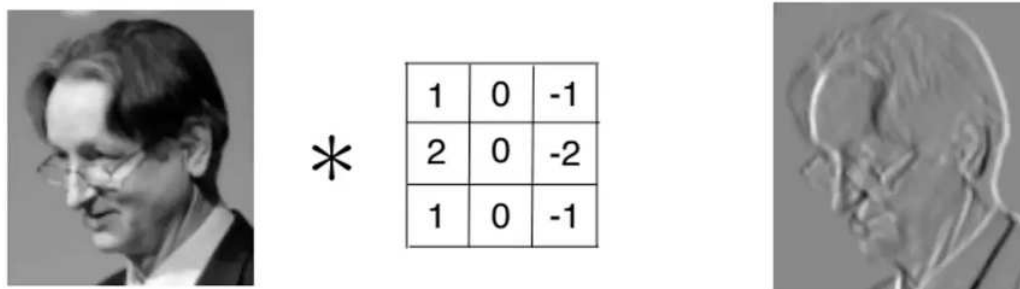


中間的 Feature Detector(Filter)會隨機產生好幾種(ex:16 種)，Feature Detector 的目的就是幫助我們萃取出圖片當中的一些特徵(ex:形狀)，就像人的大腦在判斷這個圖片是什麼東西也是根據形狀來推測



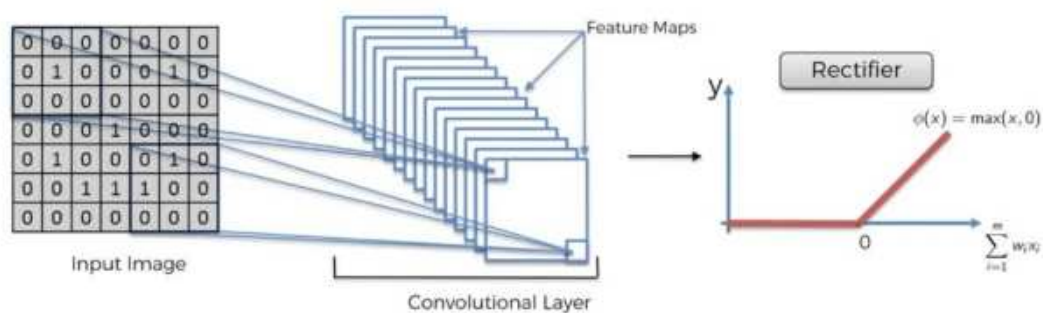
16種不同的Feature Detector

利用Feature Detector萃取出物體的邊界



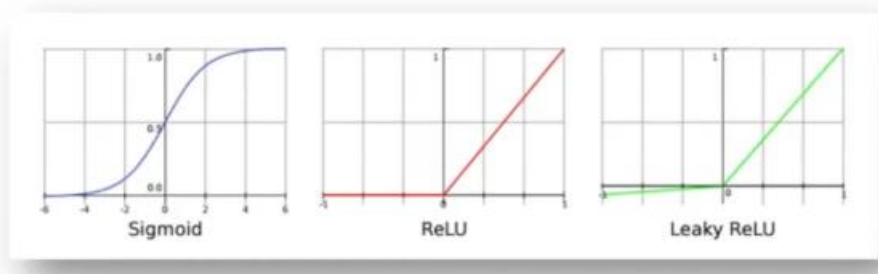
利用Feature Detector萃取出物體的邊界

使用Relu函數去掉負值，更能淬煉出物體的形狀





其他函數



尋問 chatGPT:

生成式 ai 範例: 激活函数在 CNN 中的比较.mhtml

1. Sigmoid

公式

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

輸出範圍

$$0 \sim 1$$

特性

Sigmoid 會把輸入壓縮到 0 到 1 之間，因此常用在二元分類的輸出層，例如判斷「是貓」或「不是貓」。

優點

- 輸出可以解釋成機率
- 適合二分類輸出層

缺點

- 容易發生 梯度消失
- 當輸入很大或很小時，梯度接近 0，模型學習變慢
- 輸出不是以 0 為中心，訓練可能較不穩定

圖像概念

輸入很小時接近 0，輸入很大時接近 1，中間平滑上升。

2. ReLU

公式

$$f(x) = \max(0, x)$$

也就是：

$$f(x) = \begin{cases} x, & x > 0 \\ 0, & x \leq 0 \end{cases}$$

輸出範圍

$$0 \sim \infty$$

特性

ReLU 是 CNN 中最常用的激活函數之一，當輸入大於 0 時直接輸出原值，小於等於 0 時輸出 0。

優點

- 計算簡單
- 收斂速度通常比 Sigmoid 快
- 較不容易有梯度消失問題
- 適合深層 CNN 的隱藏層

缺點

- 可能發生 Dead ReLU 問題
- 如果某些神經元長期輸出 0，梯度也會是 0，該神經元可能不再更新

圖像概念

負數區域全是 0，正數區域是一條斜率為 1 的直線。

3. Leaky ReLU

公式

$$f(x) = \begin{cases} x, & x > 0 \\ \alpha x, & x \leq 0 \end{cases}$$

其中 (α) 通常是很小的數，例如 0.01。

輸出範圍

$$-\infty \sim \infty$$

但負數部分斜率很小。

特性

Leaky ReLU 是 ReLU 的改良版，它在負數區域不直接輸出 0，而是保留一點很小的斜率。

優點

- 減少 Dead ReLU 問題
- 負數區域仍然有梯度，可以繼續學習
- 適合較深的網路或 ReLU 表現不穩時使用

缺點

- 多了一個超參數 (α)
- 不一定永遠比 ReLU 好，需要實驗比較

圖像概念

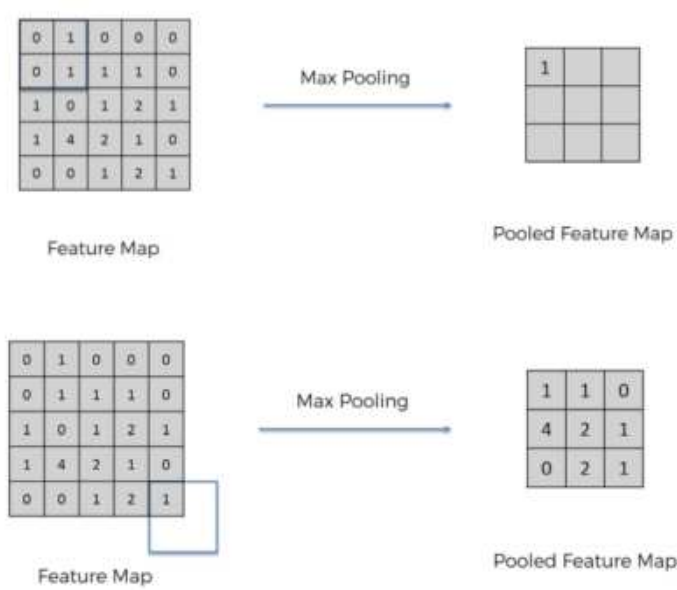
正數區域和 ReLU 一樣，負數區域不是 0，而是一條很平緩的斜線。

差異比較表

激活函數	公式	輸出範圍	優點	缺點	常見用途
Sigmoid	$(1/(1+e^{-x}))$	0 到 1	可表示機率	梯度消失、訓練慢	二分類輸出層
ReLU	$(\max(0,x))$	0 到 ∞	計算快、收斂快	Dead ReLU	CNN 隱藏層
Leaky ReLU	x 或 (αx)	$-\infty$ 到 ∞	減少 Dead ReLU	需設定 (α)	CNN 隱藏層、深層網路

2. Pooling Layer 池化層

在 Pooling Layer 這邊主要是採用 **Max Pooling**，Max Pooling 的概念很簡單只要挑出矩陣當中的最大值就好，Max Pooling 主要的好處是當圖片整個平移幾個 Pixel 的話對判斷上完全不會造成影響，以及有很好的抗雜訊功能。



3. Fully Connected Layer 全連接層

在 CNN 中通常用在最後的分類或回歸階段，負責把前面卷積層萃取出的特徵轉換成最終輸出，例如判斷圖片是哪一類。全連接層的意思是：前一層的每一個神經元，都會連接到下一層的每一個神經元。

全連接層主要有三個功能：

1. 整合特徵

卷積層提取的是局部特徵，全連接層會把所有特徵綜合起來。

例如在辨識貓的圖片時，卷積層可能偵測到：

耳朵、眼睛、鬍鬚、毛髮紋理

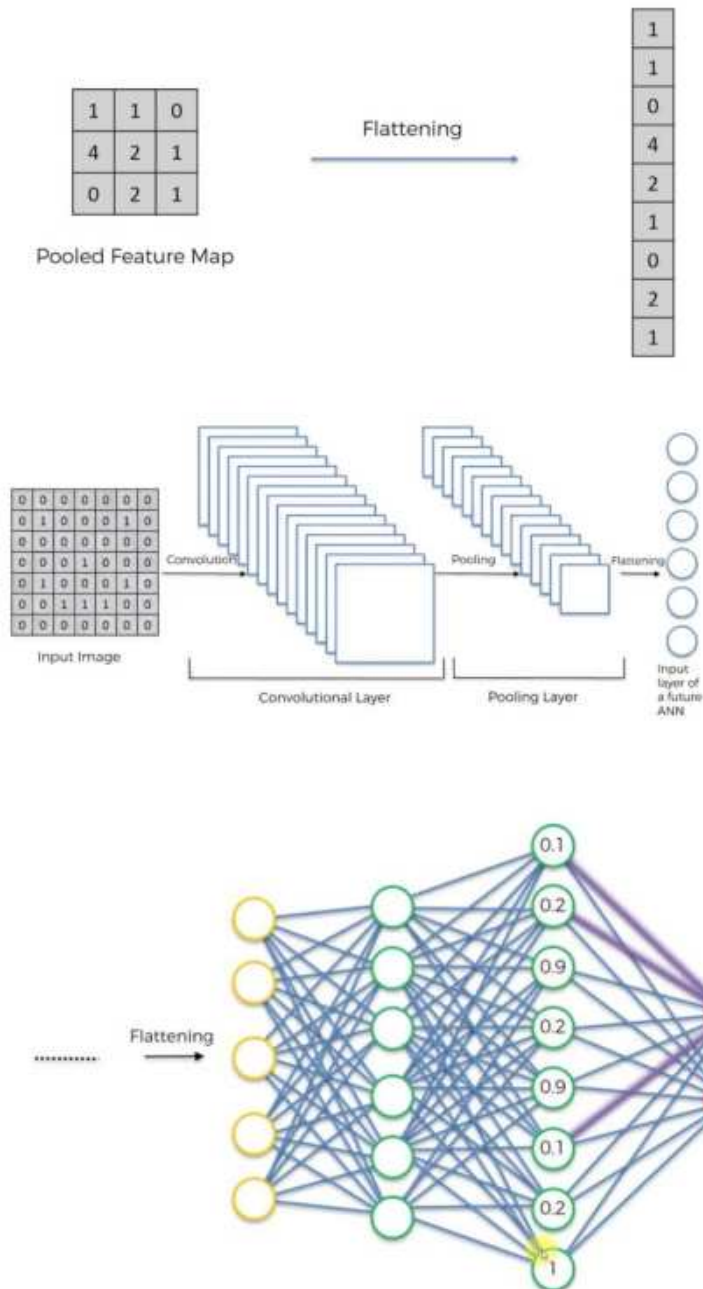
全連接層會綜合這些特徵，判斷這張圖是不是貓。

2. 進行分類

如果任務是分類 10 種物件，最後一層通常會有 10 個神經元：

[狗, 貓, 車, 鳥, 飛機, ...]

每個神經元代表一個類別的分



尋問 chatGPT:

生成式 ai 範例: CNN 手寫辨識/CNN 手寫辨識測試與訓練.mhtml

Prompt:

我想要教 cnn 的手寫辨識程式例子，程式為 python，程式等級要適合教學，訓練與測試資料庫，請將訓練與測試資料庫匯出到資料夾存成 png，再匯入執行，訓練後也將模式匯出，再將此模式匯入訓練。匯出的資料都存到 output 中的 cnn_handwritten 資料夾，再分別存到指定資料夾內。

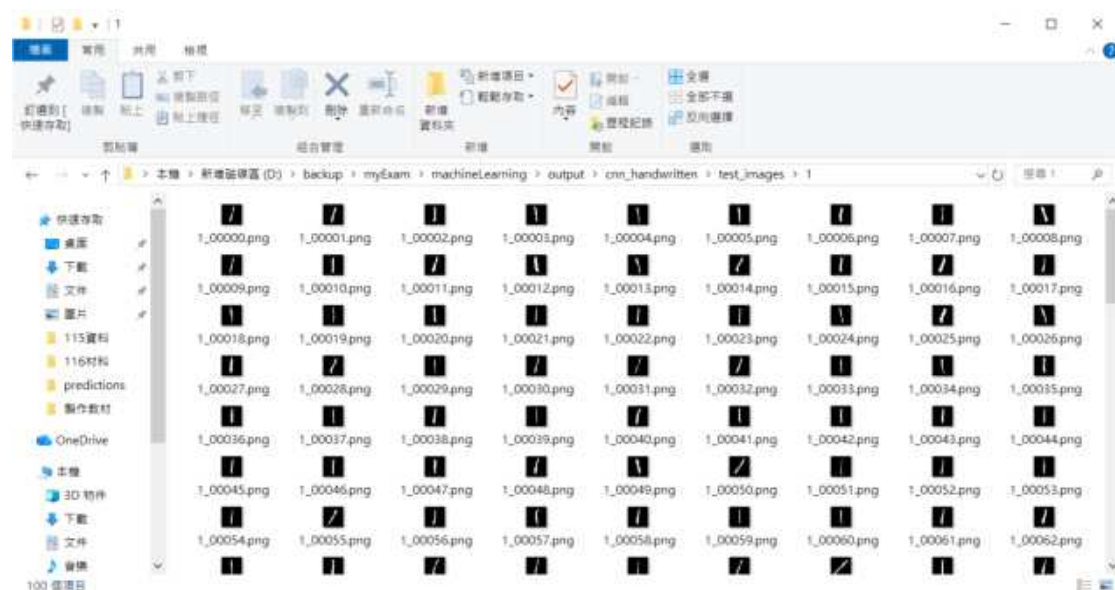
Prompt:

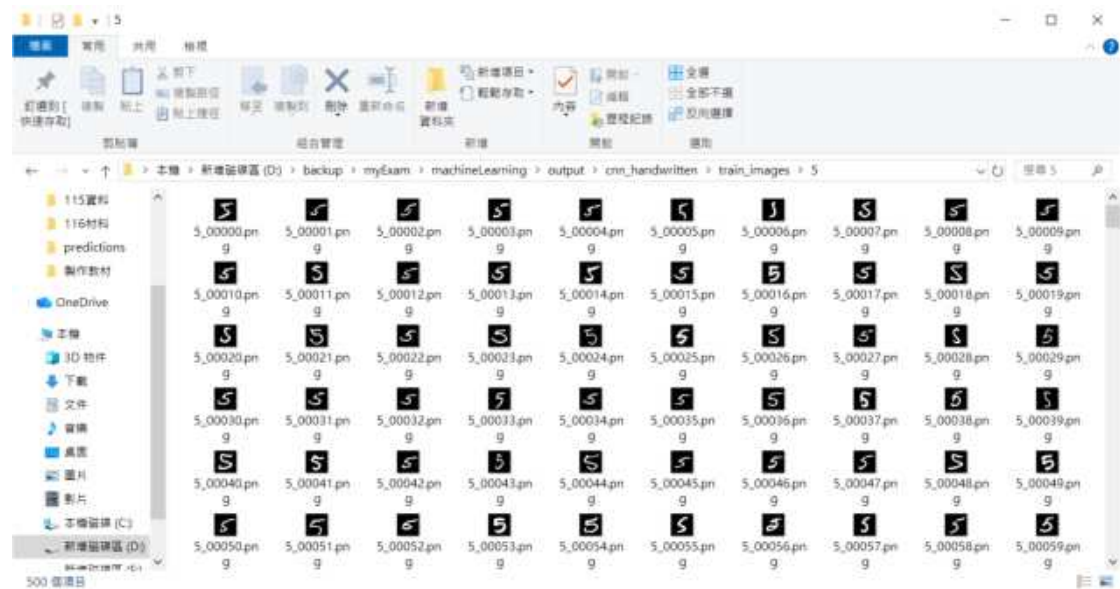
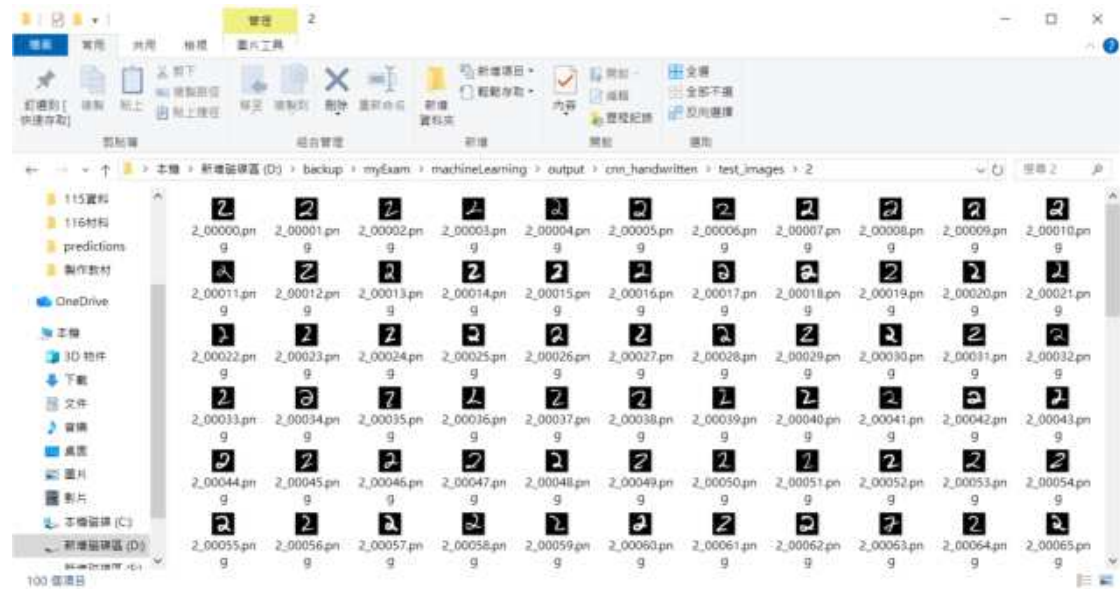
我想要自己使用小畫家畫一個手寫圖片，圖片在 data/hand_sample01.bmp，再使用 cnn_handwritten\model\cnn_mnist_model.h5 進行測試，請給我程式

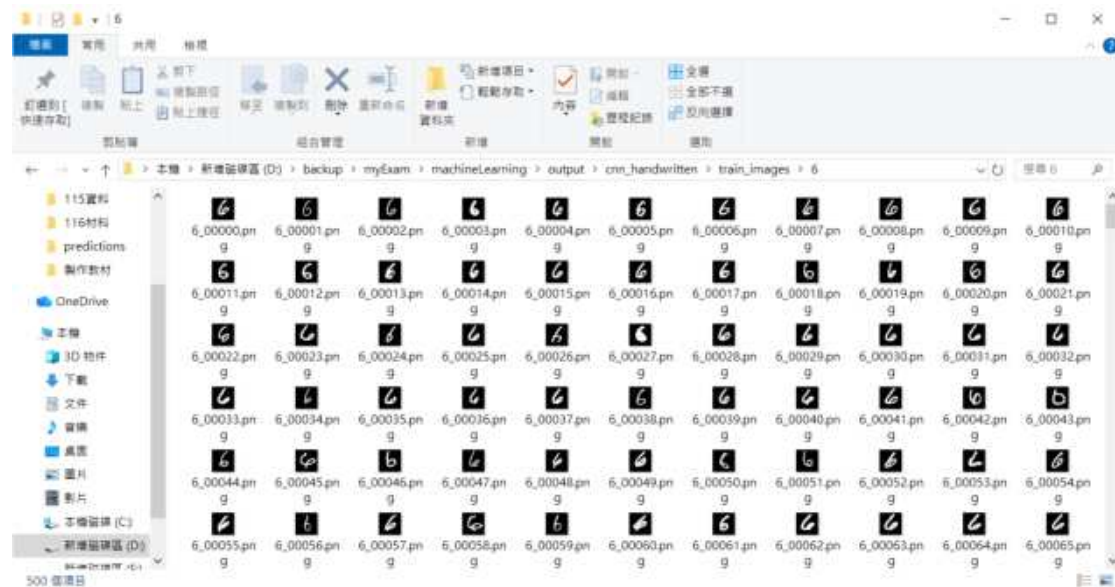
範例：

1、匯出 MNIST 資料集為 PNG 圖片（訓練集 + 測試集）

MNIST 資料集是一個大型的手寫數字資料庫，常被視為機器學習與深度學習領域的「Hello World」範例。它包含 60,000 張訓練圖片與 10,000 張測試圖片，每張圖片皆為 28x28 像素的灰階數字（0 到 9）。







2、從 PNG 圖片重新載入資料

```

80 # =====
81 # 步驟 2: 從 PNG 圖片重新載入資料
82 # =====
83 def load_images_from_folder(folder):
84     """從資料夾讀取 PNG 圖片，回傳 (X, y) numpy 陣列。"""
85     X, y = [], []
86     for digit in range(10):
87         digit_dir = os.path.join(folder, str(digit))
88         if not os.path.isdir(digit_dir):
89             continue
90         for fname in os.listdir(digit_dir):
91             if fname.lower().endswith(".png"):
92                 img_path = os.path.join(digit_dir, fname)
93                 img = Image.open(img_path).convert("L") # 灰階
94                 X.append(np.array(img, dtype=np.float32))
95                 y.append(digit)
96
97     X = np.array(X)
98     y = np.array(y)
99
100     # 正規化到 [0, 1]，並增加 channel 維度 (28,28) -> (28,28,1)
101     X = X / 255.0
102     X = X.reshape(-1, 28, 28, 1)
103     return X, y

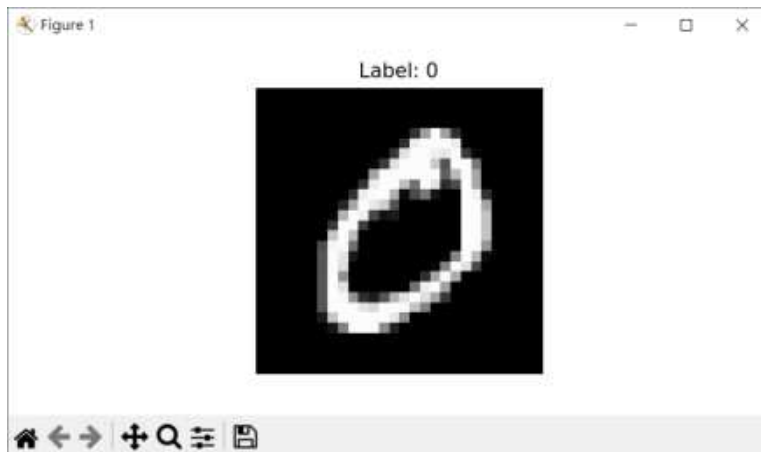
```

增加以下程式：

```

plt.imshow(x_train[0].reshape(28, 28), cmap="gray")
plt.title(f"Label: {y_train[0]}")
plt.axis("off")
plt.show()

```



```

80 # =====
81 # 步驟 2: 從 PNG 圖片重新載入資料
82 # =====
83 # 目的: 將 PNG 圖片讀回成 numpy 陣列, 方便後續訓練 CNN 模型。
84 def load_images_from_folder(folder):
85     """從資料夾讀取 PNG 圖片, 回傳 (X, y) numpy 陣列。"""
86     X, y = [], []
87     for digit in range(10):
88         digit_dir = os.path.join(folder, str(digit))
89         if not os.path.isdir(digit_dir):
90             continue
91         for fname in os.listdir(digit_dir):
92             if fname.lower().endswith(".png"):
93                 img_path = os.path.join(digit_dir, fname)
94                 img = Image.open(img_path).convert("L") # 灰階
95                 X.append(np.array(img, dtype=np.float32))
96                 y.append(digit)
97
98     X = np.array(X)
99     y = np.array(y)
100
101     # 正規化到 [0, 1], 並增加 channel 維度 (28,28) -> (28,28,1)
102     X = X / 255.0
103     X = X.reshape(-1, 28, 28, 1)
104     return X, y
105
106 # 目的: 將訓練集與測試集分別讀回, 並回傳 (x_train, y_train), (x_test, y_test)
107 def load_datasets_from_png():
108     print("\n[步驟 2] 從 PNG 圖片載入資料集...")
109     x_train, y_train = load_images_from_folder(TRAIN_DIR)
110     x_test, y_test = load_images_from_folder(TEST_DIR)
111     print(f" 訓練資料: {x_train.shape}, 標籤: {y_train.shape}")
112     print(f" 測試資料: {x_test.shape}, 標籤: {y_test.shape}")
113     print("[步驟 2] 完成!")
114     return (x_train, y_train), (x_test, y_test)

```

3、建立並訓練 CNN 模型

```

117 # =====
118 # 步驟 3：建立 CNN 模型
119 # =====
120 def build_cnn_model():
121     """建立一個簡單易懂的 CNN 模型 (適合教學)"""
122     model = models.Sequential([
123         # 輸入層：28x28 灰階圖
124         layers.Input(shape=(28, 28, 1)),
125
126         # 第 1 個卷积區塊：找出邊緣、線條等基本特徵
127         layers.Conv2D(32, kernel_size=(3, 3), activation="relu"), # 32 個 3x3 濾波器, relu 激活函數
128         layers.MaxPooling2D(pool_size=(2, 2)), # 2x2 最大池化, 降低特徵圖尺寸
129
130         # 第 2 個卷积區塊：組合出更複雜的形狀特徵
131         layers.Conv2D(64, kernel_size=(3, 3), activation="relu"), # 64 個 3x3 濾波器, relu 激活函數
132         layers.MaxPooling2D(pool_size=(2, 2)), # 2x2 最大池化, 降低特徵圖尺寸
133
134         # 展平後接全連接層做分類
135         layers.Flatten(), # 將多維特徵圖展平成一維向量
136         layers.Dropout(0.5), # 防止過擬合, 隨機丟棄 50% 神經元
137         layers.Dense(128, activation="relu"), # 全連接層, 128 個神經元, relu 激活函數
138         layers.Dense(10, activation="softmax"), # 輸出 0~9 的機率, softmax 激活函數
139     ])
140
141     model.compile(
142         optimizer="adam",
143         loss="sparse_categorical_crossentropy", # 標籤是整數不需 one-hot
144         metrics=["accuracy"],
145     )
146     return model

```

4、匯出訓練好的模型

```

149 # =====
150 # 步驟 4：訓練模型並匯出
151 # =====
152 def train_and_save_model(x_train, y_train, x_test, y_test, epochs=5, batch_size=128):
153     print("\n[步驟 3-4] 建立 CNN、開始訓練...")
154     model = build_cnn_model()
155     model.summary()
156
157     history = model.fit(
158         x_train, y_train,
159         validation_data=(x_test, y_test),
160         epochs=epochs,
161         batch_size=batch_size,
162         verbose=2,
163     )
164
165     os.makedirs(MODEL_DIR, exist_ok=True)
166     model.save(MODEL_PATH)
167     print(f"[步驟 4] 模型已儲存到: {MODEL_PATH}")
168     return model, history

```

```
[步驟 3-4] 建立 CNN 開始訓練...
2025-08-18 11:23:02.632112: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
Model: "sequential"

Layer (type)                 Output Shape                 Param #
conv2d (Conv2D)              (None, 28, 28, 32)          320
max_pooling2d (MaxPooling2D) (None, 14, 14, 32)          0
conv2d_1 (Conv2D)            (None, 14, 14, 64)          16,400
max_pooling2d_1 (MaxPooling2D) (None, 7, 7, 64)            0
flatten (Flatten)            (None, 3136)                 0
dropout (Dropout)            (None, 3136)                 0
dense (Dense)                (None, 128)                  404,416
dense_1 (Dense)              (None, 10)                   1,290

Total params: 225,014 (879.04 KB)
Trainable params: 225,014 (879.04 KB)
Non-trainable params: 0 (0.00 B)
Epoch 1/5
40/40 - 2s - 48ms/step - accuracy: 0.6154 - loss: 1.2252 - val_accuracy: 0.8540 - val_loss: 0.4460
Epoch 2/5
40/40 - 1s - 19ms/step - accuracy: 0.8958 - loss: 0.3464 - val_accuracy: 0.9330 - val_loss: 0.2387
Epoch 3/5
40/40 - 1s - 18ms/step - accuracy: 0.9326 - loss: 0.2184 - val_accuracy: 0.9398 - val_loss: 0.1784
Epoch 4/5
40/40 - 1s - 22ms/step - accuracy: 0.9476 - loss: 0.1820 - val_accuracy: 0.9370 - val_loss: 0.1733
Epoch 5/5
40/40 - 1s - 18ms/step - accuracy: 0.9548 - loss: 0.1505 - val_accuracy: 0.9610 - val_loss: 0.1269
WARNING:absl:You are saving your model as an HDF5 file via 'model.save()' or 'keras.saving.save_model(model)'. This file format is considered legacy. We recommend using instead the native Keras format, e.g. 'model.save("my_model.keras")' or 'keras.saving.save_model(model, "my_model.keras")'.
[步驟 4] 模型已儲存到: output\cnn_handwritten\model\cnn_mnist_model.h5
```



5、匯入模型並進行預測測試

```

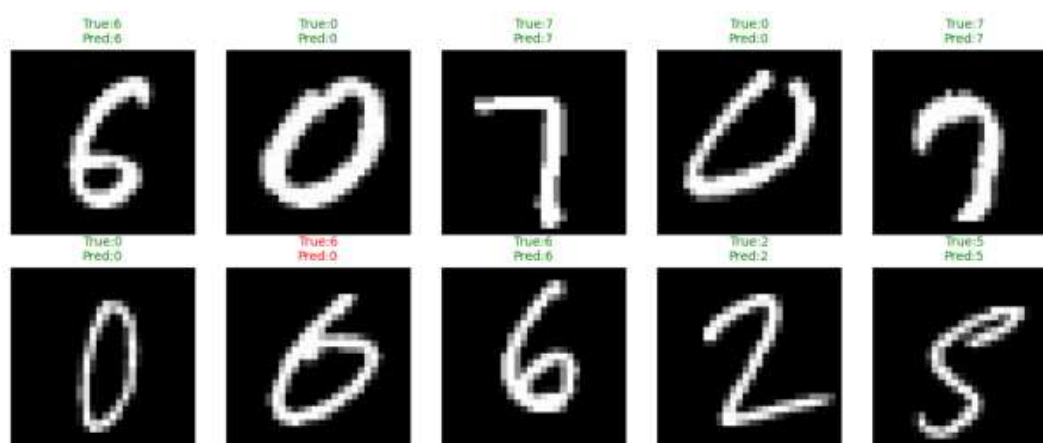
171 # =====
172 # 步驟 5：載入模型並進行預測
173 # =====
174 def load_and_predict(x_test, y_test, num_samples=10):
175     print("\n[步驟 5] 從檔案載入模型並進行預測...")
176     model = tf.keras.models.load_model(MODEL_PATH)
177
178     # 整體評估
179     loss, acc = model.evaluate(x_test, y_test, verbose=0)
180     print(f" 測試集正確率: {acc * 100:.2f}%")
181
182     # 隨機選幾張圖看預測結果
183     os.makedirs(PRED_DIR, exist_ok=True)
184     indices = np.random.choice(len(x_test), size=num_samples, replace=False)
185
186     plt.figure(figsize=(12, 5))
187     for i, idx in enumerate(indices):
188         img = x_test[idx].reshape(28, 28)
189         true_label = int(y_test[idx])
190         pred = model.predict(x_test[idx:idx + 1], verbose=0)
191         pred_label = int(np.argmax(pred))
192
193         plt.subplot(2, 5, i + 1)
194         plt.imshow(img, cmap="gray")
195         color = "green" if pred_label == true_label else "red"
196         plt.title(f"True:{true_label}\nPred:{pred_label}", color=color, fontsize=10)
197         plt.axis("off")
198
199     plt.tight_layout()
200     result_path = os.path.join(PRED_DIR, "prediction_result.png")
201     plt.savefig(result_path)
202     plt.close()
203     print(f" 預測結果圖已存到: {result_path}")
204

```

```

[步驟 5] 從檔案載入模型並進行預測...
WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. 'model.compile_metrics' will be empty until you train or evaluate the model.
測試集正確率: 96.18%
預測結果圖已存到: output/cnn_handwritten/predictions/prediction_result.png
✔ 教學範例執行完畢！請到 output/cnn_handwritten/ 檢視結果。

```



範例:cnn03_01.py

範例：

1、圖片前處理：把小畫家的圖轉成 MNIST 格式

A. 開啟圖片並轉灰階



B. 反轉顏色：小畫家白底黑字 -> MNIST 黑底白字

C. 二值化並找出邊界

```
np_img = np.array(img)
np_img = np.where(np_img > 50, np_img, 0) # 亮度 > 50 視為筆跡

coords = np.argwhere(np_img > 0)
if coords.size == 0:
    raise ValueError(f"圖片 {img_path} 找不到任何筆跡，請確認畫的是深色數字。")
```



D. 等比例縮到 20x20，貼到 28x28 中央

```
y0, x0 = coords.min(axis=0)
y1, x1 = coords.max(axis=0) + 1
cropped = np_img[y0:y1, x0:x1]
```



E. 正規化 + reshape

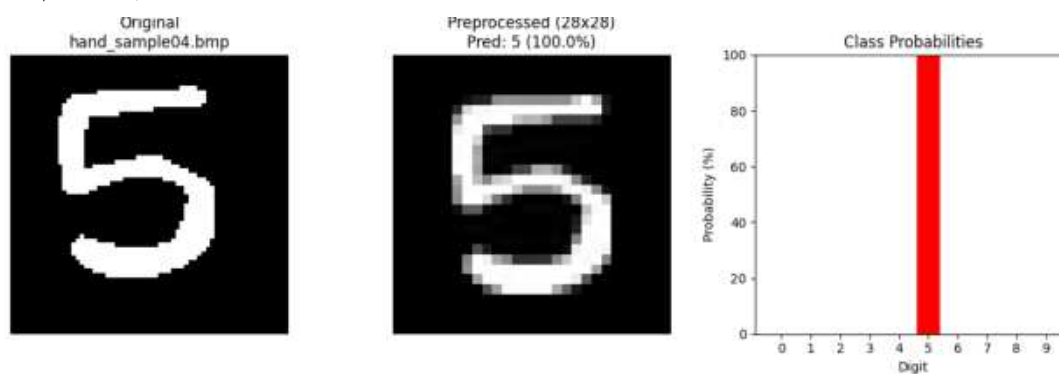
```
# 5. 正規化 + reshape
final = np.array(canvas, dtype=np.float32) / 255.0
final = final.reshape(1, 28, 28, 1)
```

2、預測

```
(pyfcom) PS D:\Backup\pyfcom\machinelearning> python .\cnn03_02.py
2026-08-18 13:13:15.059010: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
2026-08-18 13:13:18.518130: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
[ 载入模型: output\cnn_handwritten\model\cnn_mnist_model.h5
2026-08-18 13:13:19.779929: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. 'model.compile_metrics' will be empty until you train or evaluate the model.
[ 模型载入完成
[ 读取图片: data\hand_sample04.bmp
[ 预处理后 shape: (1, 28, 28, 1)

=====
[ 预测数字: 5
[ 信心度: 99.96%
=====
各类别概率:
0: 0.00%
1: 0.00%
2: 0.00%
3: 0.00%
4: 0.00%
5: 99.96%
6: 0.00%
7: 0.00%
8: 0.00%
9: 0.00%
<-- 预测
```

3、印出結果



範例:cnn03_02.py

➤ 利用 CNN 人臉辨識系統

Prompt:

我想要教 cnn 的人臉辨識程式例子，程式為 python，程式等級要適合教學，訓練與測試資料庫，請將訓練與測試資料庫匯出到資料夾存成 png，再匯入執行，訓練後也將模式匯出，再將此模式匯入訓練。匯出的資料都存到 output 中的 face_recognition 資料夾，再分別存到指定資料夾內。 cnn 使用 tensorflow.keras

尋問 chatGPT:

生成式 ai 範例: CNN 人臉辨識/ CNN 人臉辨識教學範例.mhtml

範例：

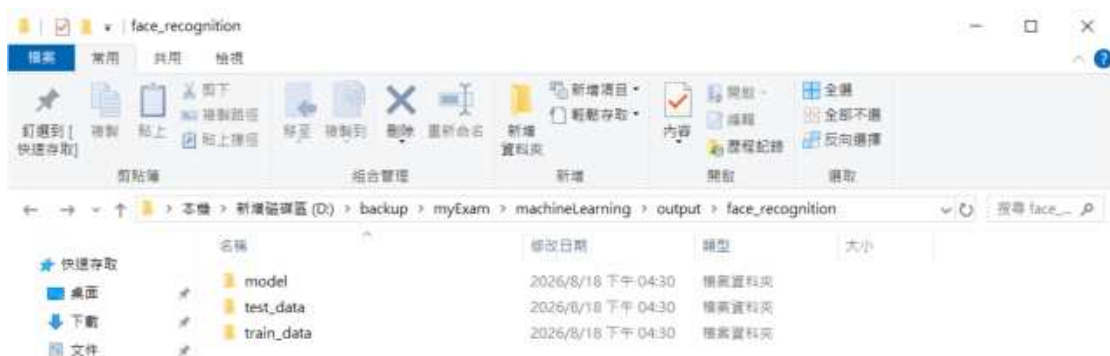
1、資料匯出：

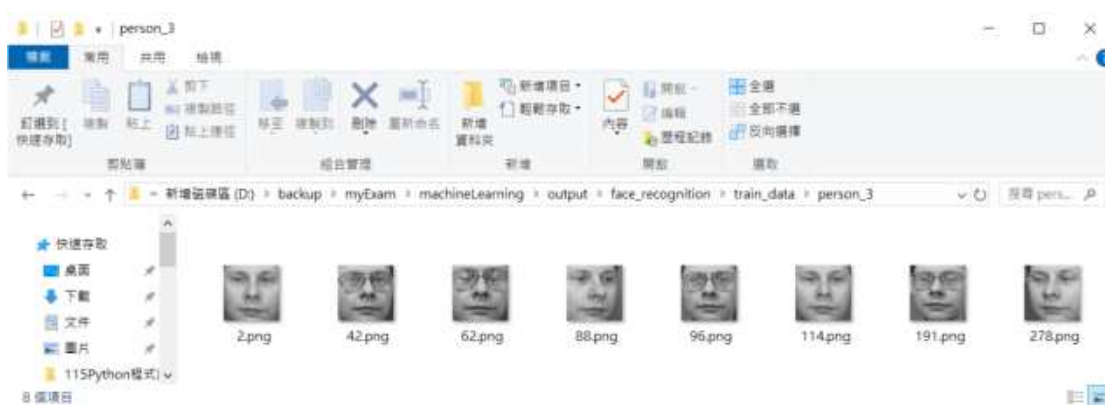
使用 sklearn 內建 Olivetti Faces (40 人 × 10 張)，依標籤分資料夾存 PNG。

Olivetti 人臉資料集的函式，常用於電腦視覺中的人臉識別、機器學習降維（如 PCA）以及無監督分群演算法的基準測試。

資料集核心規格：

- 受試人數：共有 40 個不同人物（Subject）的臉部影像。
- 總樣本數：每人包含 10 張照片，總計 400 張灰階影像。
- 影像解析度：每張影像已調整為 64×64 像素（原始 AT&T 實驗室版本為 92×112 ）。
- 特徵維度：攤平後（Flattened）的資料維度為 4096 維（ 64×64 ）。
- 資料型態：像素值已自動正規化為 0.0 至 1.0 之間的浮點數。
- 影像變異：照片包含不同的拍攝時間、光線變化、面部表情（微笑/不微笑、睜眼/閉眼）及面部細節（戴眼鏡/不戴眼鏡）。所有照片皆在黑暗背景下以直立、正面角度拍攝。





```

39 # 1. 下載資料集並匯出成 PNG
40 # =====
41 def export_dataset_to_png():
42     """把 Olivetti Faces 資料集分成訓練/測試 - 並存成 PNG"""
43     print("[Step 1] 下載 Olivetti Faces 資料集並匯出成 PNG...")
44
45     data = fetch_olivetti_faces(shuffle=True, random_state=42)
46     X = data.images          # shape: (400, 64, 64), 值域 0~1
47     y = data.target          # shape: (400,)
48
49     # 分成訓練 (80%) / 測試 (20%)
50     X_train, X_test, y_train, y_test = train_test_split(
51         X, y, test_size=0.2, random_state=42, stratify=y
52     )
53
54     # 分別建立資料夾並存 PNG
55     for folder, imgs, labels in [
56         (TRAIN_DIR, X_train, y_train),
57         (TEST_DIR, X_test, y_test),
58     ]:
59         for label in range(NUM_CLASSES):
60             os.makedirs(os.path.join(folder, f"person_{label}"), exist_ok=True)
61
62             for i, (img, label) in enumerate(zip(imgs, labels)):
63                 img_uint8 = (img * 255).astype(np.uint8) # 轉回 0~255
64                 save_path = os.path.join(folder, f"person_{label}", f"{i}.png")
65                 Image.fromarray(img_uint8).save(save_path)
66
67     print(f"訓練資料存到: {TRAIN_DIR} (共 {len(X_train)} 張)")
68     print(f"測試資料存到: {TEST_DIR} (共 {len(X_test)} 張)")
69

```

2、資料匯入

從資料夾讀 PNG → 正規化 → reshape 成 (N, 64, 64, 1)

```

71 # =====
72 # 2. 從 PNG 資料夾匯入資料
73 # =====
74 def load_dataset_from_png(folder):
75     """從資料夾讀取 PNG 圖片，回傳 (X, y)"""
76     X, y = [], []
77     for class_name in sorted(os.listdir(folder)):
78         class_path = os.path.join(folder, class_name)
79         if not os.path.isdir(class_path):
80             continue
81         label = int(class_name.split("_")[1]) # person_0 → 0
82         for fname in os.listdir(class_path):
83             if fname.endswith(".png"):
84                 img = Image.open(os.path.join(class_path, fname)).convert("L")
85                 X.append(np.array(img, dtype=np.float32) / 255.0) # 正規化
86                 y.append(label)
87
88     X = np.array(X).reshape(-1, IMG_SIZE, IMG_SIZE, 1) # CNN 需要 channel 維度
89     y = to_categorical(np.array(y), num_classes=NUM_CLASSES)
90     return X, y

```

3、CNN 建模

3 組 Conv2D + MaxPooling + Dense.

```

93 # =====
94 # 3. 建立 CNN 模型 (簡單、適合教學)
95 # =====
96 def build_cnn_model():
97     """建立一個簡單的 CNN 模型: Conv → Pool → Conv → Pool → Dense"""
98     model = models.Sequential([
99         layers.Input(shape=(IMG_SIZE, IMG_SIZE, 1)),
100
101         # 第一組卷積 + 池化
102         layers.Conv2D(32, (3, 3), activation="relu", padding="same"),
103         layers.MaxPooling2D((2, 2)),
104
105         # 第二組卷積 + 池化
106         layers.Conv2D(64, (3, 3), activation="relu", padding="same"),
107         layers.MaxPooling2D((2, 2)),
108
109         # 第三組卷積 + 池化
110         layers.Conv2D(128, (3, 3), activation="relu", padding="same"),
111         layers.MaxPooling2D((2, 2)),
112
113         # 全連接層
114         layers.Flatten(),
115         layers.Dense(128, activation="relu"),
116         layers.Dropout(0.5),
117         layers.Dense(NUM_CLASSES, activation="softmax"),
118     ])
119
120     model.compile(
121         optimizer="adam",
122         loss="categorical_crossentropy",
123         metrics=["accuracy"],
124     )
125     return model

```

4、訓練+匯出

train_and_export_model，使用 .keras 新格式儲存模型 (Keras 3 推薦格式)

```

128 # =====
129 # 4. 訓練與匯出模型
130 # =====
131 def train_and_export_model():
132     print("\n[Step 2] 從 PNG 匯入訓練/測試資料...")
133     X_train, y_train = load_dataset_from_png(TRAIN_DIR)
134     X_test, y_test = load_dataset_from_png(TEST_DIR)
135     print(f" X_train shape: {X_train.shape}, X_test shape: {X_test.shape}")
136
137     print("\n[Step 3] 建立 CNN 模型並開始訓練...")
138     model = build_cnn_model()
139     model.summary()
140
141     history = model.fit(
142         X_train, y_train,
143         batch_size=BATCH_SIZE,
144         epochs=EPOCHS,
145         validation_data=(X_test, y_test),
146         verbose=2,
147     )
148
149     loss, acc = model.evaluate(X_test, y_test, verbose=0)
150     print(f"\n 測試集 Loss: {loss:.4f}, Accuracy: {acc:.4f}")
151
152     # 匯出模型
153     os.makedirs(MODEL_DIR, exist_ok=True)
154     model.save(MODEL_PATH)
155     print(f"\n[Step 4] 模型已匯出到: {MODEL_PATH}")
156
157     return history

```



```
Problems Output Terminal Ports ESP-IDF
Terminal
(myExam) PS D:\backup\myExam\machineLearning> python .\cnn_face_01.py
Total params: 1,146,536 (4.37 MB)
Trainable params: 1,146,536 (4.37 MB)
Non-trainable params: 0 (0.00 B)
Epoch 1/15
20/20 - 3s - 157ms/step - accuracy: 0.0188 - loss: 3.7085 - val_accuracy: 0.0250 - val_loss: 3.6882
Epoch 2/15
20/20 - 1s - 34ms/step - accuracy: 0.0312 - loss: 3.6891 - val_accuracy: 0.0250 - val_loss: 3.6865
Epoch 3/15
20/20 - 1s - 41ms/step - accuracy: 0.0250 - loss: 3.6929 - val_accuracy: 0.0250 - val_loss: 3.6850
Epoch 4/15
20/20 - 1s - 36ms/step - accuracy: 0.0312 - loss: 3.6866 - val_accuracy: 0.0250 - val_loss: 3.6753
Epoch 5/15
20/20 - 1s - 34ms/step - accuracy: 0.0406 - loss: 3.6725 - val_accuracy: 0.0500 - val_loss: 3.6364
Epoch 6/15
20/20 - 1s - 40ms/step - accuracy: 0.0594 - loss: 3.6006 - val_accuracy: 0.0875 - val_loss: 3.4750
Epoch 7/15
20/20 - 1s - 31ms/step - accuracy: 0.1344 - loss: 3.3597 - val_accuracy: 0.2875 - val_loss: 3.0031
Epoch 8/15
20/20 - 1s - 31ms/step - accuracy: 0.2531 - loss: 2.7349 - val_accuracy: 0.5500 - val_loss: 2.2347
Epoch 9/15
20/20 - 1s - 32ms/step - accuracy: 0.3688 - loss: 2.2165 - val_accuracy: 0.6125 - val_loss: 1.7984
Epoch 10/15
20/20 - 1s - 32ms/step - accuracy: 0.4844 - loss: 1.8189 - val_accuracy: 0.7125 - val_loss: 1.2170
Epoch 11/15
20/20 - 1s - 35ms/step - accuracy: 0.5781 - loss: 1.5205 - val_accuracy: 0.8750 - val_loss: 0.9305
Epoch 12/15
20/20 - 1s - 31ms/step - accuracy: 0.6313 - loss: 1.2641 - val_accuracy: 0.8250 - val_loss: 0.8458
Epoch 13/15
20/20 - 1s - 31ms/step - accuracy: 0.7094 - loss: 1.0023 - val_accuracy: 0.9250 - val_loss: 0.5410
Epoch 14/15
20/20 - 1s - 35ms/step - accuracy: 0.7375 - loss: 0.9212 - val_accuracy: 0.9000 - val_loss: 0.4792
Epoch 15/15
20/20 - 1s - 32ms/step - accuracy: 0.7312 - loss: 0.8343 - val_accuracy: 0.9125 - val_loss: 0.3089
測試集 Loss: 0.3089, Accuracy: 0.9125
[Step 4] 模型已匯出到: output\face_recognition\model\face_cnn_model.keras
```

5、匯入+再訓練

展示 load_model 與 fine-tuning 概念

```
160 # =====
161 # 5. 匯入模型並再次訓練 (示範 fine-tuning)
162 # =====
163 def load_and_retrain_model(extra_epochs=5):
164     print("\n[Step 5] 匯入已儲存的模型，再訓練幾個 epoch...")
165     model = tf.keras.models.load_model(MODEL_PATH)
166     print(" 模型載入成功!")
167
168     X_train, y_train = load_dataset_from_png(TRAIN_DIR)
169     X_test, y_test = load_dataset_from_png(TEST_DIR)
170
171     model.fit(
172         X_train, y_train,
173         batch_size=BATCH_SIZE,
174         epochs=extra_epochs,
175         validation_data=(X_test, y_test),
176         verbose=2,
177     )
178
179     loss, acc = model.evaluate(X_test, y_test, verbose=0)
180     print(f"\n 再訓練後的測試集 Accuracy: {acc:.4f}")
181
182     # 匯入存回
183     model.save(MODEL_PATH)
184     print(f" 更新後的模型已存回: {MODEL_PATH}")
```

[Step 5] 匯入已儲存的模型，再訓練幾個 epoch...

模型載入成功！

```
Epoch 1/5
20/20 - 2s - 93ms/step - accuracy: 0.7906 - loss: 0.6468 - val_accuracy: 0.9375 - val_loss: 0.3318
Epoch 2/5
20/20 - 1s - 33ms/step - accuracy: 0.8188 - loss: 0.6093 - val_accuracy: 0.9250 - val_loss: 0.4074
Epoch 3/5
20/20 - 1s - 32ms/step - accuracy: 0.8188 - loss: 0.5981 - val_accuracy: 0.9500 - val_loss: 0.2680
Epoch 4/5
20/20 - 1s - 32ms/step - accuracy: 0.8469 - loss: 0.4880 - val_accuracy: 0.9250 - val_loss: 0.2453
Epoch 5/5
20/20 - 1s - 31ms/step - accuracy: 0.8469 - loss: 0.4678 - val_accuracy: 0.9625 - val_loss: 0.1654
```

再訓練後的測試集 Accuracy: 0.9625

更新後的模型已存回：output\face_recognition\model\face_cnn_model.keras

6、預測視覺化

show_predictions，用 matplotlib 顯示預測結果

```
187 # =====
188 # 6. 預測示範 (從測試集抓幾張畫出來看)
189 # =====
190 def show_predictions(n=5):
191     print("\n[Step 6] 用模型做預測示範...")
192     model = tf.keras.models.load_model(MODEL_PATH)
193     X_test, y_test = load_dataset_from_png(TEST_DIR)
194
195     idx = np.random.choice(len(X_test), n, replace=False)
196     preds = model.predict(X_test[idx], verbose=0)
197
198     plt.figure(figsize=(12, 3))
199     for i, k in enumerate(idx):
200         plt.subplot(1, n, i + 1)
201         plt.imshow(X_test[k].squeeze(), cmap="gray")
202         plt.title(f"True:{np.argmax(y_test[k])}\nPred:{np.argmax(preds[i])}")
203         plt.axis("off")
204     plt.tight_layout()
205     plt.show()
```



範例:cnn_face_01.py

➤ Fine-tuning (微調)

什麼是 Fine-tuning？

利用預訓練模型（Pre-trained Model，通常在 ImageNet 上訓練）作為基礎，針對新的任務或資料集進行調整，屬於遷移學習（Transfer Learning）的一種。

Fine-tuning 的步驟

步驟 1：載入預訓練模型

- 選擇合適的預訓練模型（如 ResNet50、VGG16、EfficientNet）
- 載入其權重（Weights）

步驟 2：修改網路結構

- 移除原本的輸出層（例如 ImageNet 的 1000 類分類層）
- 替換為適合新任務的輸出層（例如新任務有 10 類，就改為 10 個神經元）

步驟 3：凍結層（Freezing Layers）

- 凍結前面的卷積層：保留通用特徵（邊緣、紋理）
- 只訓練後面幾層：學習任務專屬特徵

步驟 4：設定訓練策略

- 學習率調整：使用較小的學習率（如 1e-4 或 1e-5），避免破壞預訓練權重
- 分層學習率：底層用更小學習率，頂層用較大學習率

步驟 5：訓練與評估

- 用新資料集訓練
- 監控驗證集表現，避免過擬合

步驟 6：漸進式解凍（Progressive Unfreezing，可選）

- 先訓練頂層 → 逐步解凍下層 → 全網路微調

Fine-tuning 的三種策略

策略	適用情境	做法
特徵提取（Feature Extraction）	資料量少、任務類似	凍結所有卷積層，只訓練分類層
部分微調（Partial Fine-tuning）	資料量中等	凍結底層，微調高層 + 分類層
完全微調（Full Fine-tuning）	資料量大、任務差異大	解凍所有層，用小學習率重新訓練

Fine-tuning 的優點

-  節省訓練時間：不需從零開始
-  需要較少資料：預訓練模型已學到通用特徵
-  提升準確率：尤其在小資料集上效果顯著
-  降低運算成本

第 X 章 股票分析

FinMind 的資料查詢 API，可以抓台股的「每日資料」，官方資料集說明中，這個資料集提供個股每日的開、高、低、收、成交量等資料。

來源：

<https://api.finmindtrade.com/api/v4/data>

TPEX 上櫃股票每日收盤行情 API，是櫃買中心提供的 上櫃股票每日收盤行情資料。TPEX 官方的盤後資料 API 顯示，其資料週期就是「每日」。

來源：

https://www.tpex.org.tw/openapi/v1/tpex_mainboard_daily_close_quotes

跟你前面的 FinMind 比較			
項目	TPEX API	FinMind	
資料來源	櫃買中心官方	第三方整理	
上櫃股票	✓	✓	
每日資料	✓	✓	
6763 綠界科技	✓	✓	
開盤價	✓	✓	
最高價	✓	✓	
最低價	✓	✓	
收盤價	✓	✓	
成交量	✓	✓	
即時/逐筆成交	✗	可用其他資料集	
查歷史日期	需看 API 設計	✓	
適合你的每日分析	★★★★★	★★★★★	